JAVA BACKEND FAILURE ATLAS

Java 后端故障图鉴

写给要负责的人——那些不报错的 bug

27 章 · 5 卷 · 从请求进来到故障复盘

初稿 · 2026 年 11 月

CONTENTS

目 录

卷一 · 请求进来

第一章 一个请求的 37 毫秒 1
1.1 现场：接口"慢了"，但所有人都说不是自己慢 1
1.2 六段拆解 4
1.3 应用内部：每一层都可能静默改语义 10
1.4 观测：先建证据，再谈优化 13
1.5 本章的边界 16
第二章 超时与重试：为什么越重试越慢 17
2.1 现场：一次抖动如何变成雪崩 17
2.2 重试风暴的数学 20
2.3 默认值：所有"灵异故障"的源头 21
2.4 超时怎么定：不是越短越好，也不是越长越好 24
2.5 哪些接口能重试：一张判定表 26
2.6 熔断：重试的前置条件 28
2.7 自检清单 30
2.8 一句话总结 31
第三章 序列化陷阱：Long 精度、时区差 8 小时 32
3.1 现场：运营说"这单查不到" 32
3.2 根因：JSON 里没有 long 35
3.3 修法一：Long 全局转字符串 38
3.4 修法二：时区，别只配 Jackson 41
3.5 修法三：BigDecimal 的 0.10 与 0.1 44
3.6 修法四：Feign 的两个静默丢参 46
3.7 接口契约：四个必须写死的类型 48
3.8 自检清单 48
3.9 一句话总结 49
第四章 连接池耗尽：堵死的不是数据库 50
4.1 现场：下游很快，我很慢 50
4.2 罪魁祸首：1990 年代的默认值 53
4.3 池子该多大：一个能算的公式 54
4.4 另一条路径：连接泄漏 57

4.5 第三类：连接被"事务"占着不放 61
4.6 监控：三个必须暴露的指标 63
4.7 自检清单 65
4.8 一句话总结 66
第五章 网关与限流：全局限流为什么没限住 67
5.1 现场：限流规则写了，流量一条没少 67
5.2 维度：单机限流会放大 N 倍 69
5.3 状态：规则不在配置中心，就是在眼泪里 71
5.4 位置：限流要挡在最外层，也要挡在最里层 73
5.5 限流、熔断、降级：三个词别混着用 76
5.6 网关自身的坑：它不是万能的 78
5.7 自检清单 80
5.8 一句话总结 80

卷二 · 业务跑起来

第六章 事务为什么会静默失效 81
6.1 现场：9:04 81
6.2 三次误判 83
6.3 一个事实决定所有失效：你调用的是代理 86
6.4 五问定位法 90
6.5 修复的四个层次 95
6.6 自检清单 99
6.7 版本区间与复现环境 99
6.8 本章的边界 100
第七章 事务里做远程调用：行锁被持有 3 秒之后 101
7.1 现场：库存服务"变慢"了三天 101
7.2 根因：事务里那次 HTTP 调用 103
7.3 为什么很多团队都这么写 104
7.4 修法：把事务收窄到只包数据库 105
7.5 行锁的等待时间：一个必须知道的数字 109
7.6 怎么发现这类问题 110
7.7 自检清单 112
7.8 一句话总结 113
第八章 循环依赖与 Bean 初始化顺序 114
8.1 现场：升级当天起不来 114
8.2 三级缓存到底在解决什么 116

8.3 四种根本无解的循环依赖 120
8.4 四种正确的破坏方式 121
8.5 初始化顺序：另一类"静默错" 124
8.6 自检清单 126
8.7 一句话总结 127
第九章 线程池：任务去哪了 128
9.1 现场：两种"假死" 128
9.2 三个默认值炸弹 129
9.3 事故模型：submit() 吞掉了异常 133
9.4 ThreadLocal：串台比泄漏更可怕 138
9.5 一个可用的池子长什么样 141
9.6 自检清单 144
9.7 一句话总结 145
第十章 并发下的集合与可见性 146
10.1 现场：发布后用户数据"串了" 146
10.2 CME 和并发没有关系 147
10.3 视图陷阱：Arrays.asList 与 subList 150
10.4 最阴险的一种：equals / hashCode 不成对 153
10.5 可见性：volatile 管什么 156
10.6 SimpleDateFormat：共享即出错 160
10.7 自检清单 160
10.8 一句话总结 161
第十一章 全局异常处理吃掉了堆栈 162
11.1 现场：错误率 0，客诉 40 单 162
11.2 根因：三件事被混在一起了 163
11.3 正确写法：分层 + 保留语义 + 保留堆栈 167
11.4 它管不到的五个地方 169
11.5 404 为什么也不进 Advice 172
11.6 异常链：不要断掉堆栈 175
11.7 一个自检用的"事故模拟" 175
11.8 自检清单 178
11.9 一句话总结 179

卷三 · 数据落地

第十二章 索引失效的七种写法 180
12.1 现场：索引建了三个月，一次没生效 180

12.2 失效的七种写法 181
12.3 怎么判读：EXPLAIN 的四个字段 188
12.4 索引不是越多越好 191
12.5 自检清单 194
12.6 一句话总结 195
第十三章 深分页与慢 SQL：从 8 秒到 200 毫秒 196
13.1 现场：导出功能拖垮了主库 196
13.2 三种改法 197
13.3 什么时候该用哪种 202
13.4 其他几类慢 SQL 204
13.5 优化顺序：不要一上来就加索引 207
13.6 自检清单 209
13.7 一句话总结 209
第十四章 锁与死锁：两个事务互等的那 50 秒 211
14.1 现场：一条 UPDATE 锁住了整张表 211
14.2 死锁的三个真实根因 212
14.3 降 RC：最有效的一招 215
14.4 死锁之后：应用看到的什么 217
14.5 锁等待：innodb_lock_wait_timeout 219
14.6 乐观锁：另一种选择 224
14.7 自检清单 226
14.8 一句话总结 226
第十五章 缓存一致性：先删缓存还是先更新库 228
15.1 现场：用户改了昵称，三小时后才生效 228
15.2 正确姿势：Cache Aside（先更库，再删缓存） 229
15.3 删缓存失败怎么办：必须有兜底 233
15.4 延迟双删：一个被高估的方案 237
15.5 五种方案对比 238
15.6 三个常见误区 239
15.7 TTL：最后一道防线 240
15.8 自检清单 241
15.9 一句话总结 241
第十六章 缓存三大问题与热 Key 243
16.1 现场：32 个节点，一个 key 打满 CPU 243
16.2 穿通雪：三个概念，别混着说 244
16.3 热 key：三个解法，按成本排 252

16.4 大 key：比热 key 更常见的杀手 254
16.5 线上禁用命令 256
16.6 Redisson 分布式锁的三个坑 257
16.7 自检清单 260
16.8 一句话总结 260
第十七章 分库分表之后那些"查不到" 261
17.1 现场：列表少了几条，后台数字对不上 261
17.2 分片键选错，代价是什么 262
17.3 基因法：让两个维度都能精准路由 264
17.4 全局唯一 ID：自增必须换掉 265
17.5 跨片查询的三类难题 268
17.6 扩容：唯一安全路径 271
17.7 什么时候不该分表 271
17.8 自检清单 272
17.9 一句话总结 272

卷四 · 异步与峰值

第十八章 消息丢失的三个阶段 278
18.1 现场：少了 47 条支付成功通知 278
18.2 三段防线 279
18.3 最可靠的做法：本地消息表 286
18.4 现实：三阶段全配齐也会丢 290
18.5 自检清单 293
18.6 一句话总结 293
第十九章 重复消费与幂等三板斧 294
19.1 现场：一张订单发了两次货 294
19.2 重复的三类来源 295
19.3 幂等三板斧 296
19.4 幂等的四种层次，选哪个 300
19.5 一个完整的消费模板 303
19.6 自检清单 309
19.7 一句话总结 310
第二十章 积压：从 10 万到 0 的 40 分钟 311
20.1 现场：扩了 8 个消费者，位点不动 311
20.2 先判断：是真积压还是假象 313
20.3 真积压：处置顺序 317

20.4 消费端反压：别把积压变成雪崩 319
20.5 预防：三个必配的监控 320
20.6 自检清单 322
20.7 一句话总结 323
第二十一章 顺序消息被破坏之后 324
21.1 现场：状态倒退了 324
21.2 顺序的两个必要条件 325
21.3 四个隐藏的顺序破坏者 328
21.4 全局有序：一个吞吐量陷阱 332
21.5 一个务实的替代方案：状态机 + 版本号 334
21.6 自检清单 336
21.7 一句话总结 337
第二十二章 秒杀：库存扣减的四种方案 339
22.1 现场：不超卖了，但少了 37 件 339
22.2 四种方案 340
22.3 四种方案对比 347
22.4 无论哪种方案，这三条不能少 347
22.5 对账：少卖问题的唯一解法 348
22.6 自检清单 353
22.7 一句话总结 354

卷五 · 上线之后

第二十三章 CPU 100%：二十分钟内要拿到的三份证据 355
23.1 现场：先别重启 355
23.2 三份证据 358
23.3 容器环境：命令要换 362
23.4 三种高频案例 365
23.5 降级：先保命，再查因 369
23.6 自检清单（排障时逐条过） 371
23.7 一句话总结 371
第二十四章 OOM：加堆之前先取证 372
24.1 现场：加到 16G，第三天还是 OOM 372
24.2 先看冒号后面那几个字 373
24.3 取证：重启之前必须做的五件事 376
24.4 exit code 137：不是 Java 的错 380
24.5 MAT：只看三个视图就够了 383

24.6 三种根因与对策 385
24.7 自检清单 388
24.8 一句话总结 388
第二十五章 GC 停顿与 P99 毛刺 389
25.1 现场：每 40 秒一次 2 秒抖动 389
25.2 P99 毛刺的四种来源 391
25.3 三种 Full GC，三种解法 393
25.4 GC 日志怎么看 394
25.5 调参顺序：别一上来就换 ZGC 395
25.6 一个真实案例的调参过程 398
25.7 自检清单 399
25.8 一句话总结 399
第二十六章 容量与压测：QPS 是怎么估出来的 406
26.1 现场：压测报告很漂亮，大促还是挂了 406
26.2 从业务量估 QPS 407
26.3 压测：找拐点，不是刷峰值 410
26.4 瓶颈在哪：四层排查 414
26.5 容量水位：为什么是 30–50% 414
26.6 监控：容量的先行指标 416
26.7 自检清单 418
26.8 一句话总结 419
第二十七章 一次故障复盘该怎么写 420
27.1 现场：两份复盘 420
27.2 三层原因，必须分开 423
27.3 时间线：写"我们以为"，不只写"发生了什么" 424
27.4 影响面：量化到"钱"和"人" 426
27.5 改进项：三类，且必须可验证 427
27.6 五个反模式 430
27.7 一页纸模板 430
27.8 最后：这本书在讲什么 431

前言　写给要负责的人

这本书为什么存在

我做后端这些年，见过的最贵的故障，没有一个是"服务挂了"。

挂掉的故障有告警、有值班、有预案，处理得再糟也就半小时。真正贵的，是那些**不报错**的：

- 库存多扣了 347 件，订单表干净，日志没有一条 ERROR
- 用户改了昵称，三小时后才生效，因为缓存里是旧值
- 消息 `send success`，但对端从来没收到
- 监控上错误率 0.00%，客诉 40 单

它们的共同形状是：**代码看起来完全正确，框架按文档工作，SQL 也执行了，错的只有数据。**

而数据错了，要等到对账、客诉、第二天早上，才会有人知道。

市面上讲原理的书很多，讲"为什么你的原理没生效"的书很少。这本书想补这个空。

给谁看

工作 3–5 年、已经在线上扛过责任的 Java 后端。

如果下面这些话你说过其中一句，这本书就是写给你的：

- "本地跑得好好的啊。"
- "这个注解我加了呀。"
- "下游说他们没问题，我们也说我们没问题。"
- "重启一下试试？"
- "这个接口以前没这么慢。"

如果你还在准备面试、需要的是知识覆盖面，这本书不合适——它故意不追求完整。

怎么读

不要从头读。 按你正在经历的事跳：

你在经历什么	读哪几章
接口慢，但下游监控正常	第一章、第四章
加了注解不生效	第六章（事务）、第八章（循环依赖）
数据错了，不知道谁改的	第十五章（缓存一致性）、第十九章（重复消费）
消息丢了 / 重复了 / 堆了	第十八、十九、二十章
线上炸了，要先取证	第二十三、二十四章
P99 有周期性毛刺	第二十五章
要过一遍复盘	第二十七章

全书五卷，是一条读者的真实经历线：**请求进来 → 业务跑起来 → 数据落地 → 异步与峰值 → 上线之后出事。** 顺着读也没问题。

每一章的形状

统一六段，你可以直接当检查表用：

1. **现场** —— 什么时间、什么现象、影响多少
2. **误判** —— 当时怎么想的、为什么错（这一段教科书里没有）
3. **根因** —— 原理层解释，一到两层，不发散
4. **修复** —— ✗ 错误写法 → ✓ 修复写法
5. **自检清单** —— 可以直接贴进 CR 模板
6. **延伸** —— 主线之外、但值得知道的支线深挖，以及指向知识库对应原理篇的指针

难度分层：怎么看【必读】【进阶】【选读】

这本书想同时服务两类读者：**工作 3–5 年、正在第一次撞上这类坑的人**，和**已经踩过、想看清机制为什么这么设计的人**。所以每一章按难度分了三层：

- **【必读】** —— 主线。一个故障从现场到修复的完整链路，跳过任何一段都讲不清楚。3 - 5 年读者吃透这一层就够了。
- **【进阶】** —— 深挖。框架为什么这么设计、源码里哪一行走到了这里、公式怎么推出来。想从「会用」到「懂为什么」再看，跳过不影响主线。
- **【选读】** —— 延伸。同一类问题的其它变种、冷门边界、可以直接抄的清单。备查，不要求通读。

每章开头有一行「难度构成」告诉你这章有几处进阶、几处选读；小节标题前的标签让你随时知道自己在哪一层。看到【选读】可以放心跳，看到【进阶】可以放心放一放。

交叉引用索引

书里反复出现几条「跨章才能讲清」的主线。为了避免每章都重讲一遍，约定：**主场章节把原理讲透，其它章节只引用、不重述**。这张表是它们的地图：

跨章主线	主场（讲透）	其它现场（只引用）
Little's law：`L = λ × W`（排队论容量）	第四章（连接池容量公式）	第一、二、七、九、十三、十四、十七、二十一、二十五章
事务里做远程调用 / 行锁持有时间	第七章（整章）	第四、六、十四、二十二章
加线程的负收益（排队型故障加线程只拉长队伍）	第七章	第十三章（加机器同结论）、第二十章（积压场景加线程仍有效，互补）
超时链路：`HTTP < 事务timeout < 锁等待 < 调用方`	第七章、第十四章	第四、二十二章
幂等与去重	第十九章	第四（本地消息表）、第二十二章（防超卖）
缓存一致性	第十五章	第三（序列化）、第十六章（缓存问题）
连接池被事务占满	第四章	第六、七章
对账兜底（丢失/重复的最后防线）	第十八、十九章	第二十二章
全链路减法定位（两处指标相减找差值）	第一章、第四章	第七章

正文里凡出现「见第 X 章」都按这张表走——主场讲了原理，这里只代入本系统的参数。

关于案例

书里的现场由多个真实故障脱敏合成，代码、数字、时间线经过改写。 结论对应真实的排查路径，不是虚构的。

这样做有两个原因：一是保护当事人和相关业务；二是单个真实故障的信息量往往不够支撑一章——把三四个同类故障拼在一起，才能把"这一类问题"讲清楚。

这本书不写什么

明确说出来，省得你找：

- **Java 语法**：网上有更好的，最多在需要时提一句
- **面试题**：那是检索型内容，不该线性读
- **框架怎么用**：官方文档比我写得准
- **架构方法论**：DDD、中台这些，不是这本书的主题

它只讲一件事：**当系统不报错、但结果是错的时候，怎么找到原因。**

版本说明

代码与配置基于：

```
JDK 17 / 21
Spring Boot 3.x（部分章节标注 2.7 的差异）
MySQL 8.0
Redis 7.x
RocketMQ 5.x / Kafka 3.x
```

框架默认值会变，但**"默认值是给 demo 用的"这条判断不会变**。看到具体数值时，请以你手上的版本文档为准；看到判断逻辑时，可以直接用。

> 这本书能给你的不是 27 个答案，是一套提问方式。 五个问题写在最后一章的末尾——如果你已经会问，就不需要这本书了。

第一章　一个请求的 37 毫秒

本章要回答：一个请求从网线进来，到响应出去，中间到底经过了多少个"可以静默出错"的地方？

本章你能带走

- 一张把 37ms 拆成六段的表，每段都标了"证据在哪"
- 一个判定技巧：`$request_time` 与 `$upstream_response_time` 的差值，能立刻区分"网络慢"和"应用慢"
- 一条原则：**没有观测点的链路，等于没有链路**

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

1.1　现场：接口"慢了"，但所有人都说不是自己慢

一次典型的扯皮：

> 前端："接口 3 秒才返回。" 后端："我们服务日志里 RT 只有 80ms。" 运维："Nginx 没报错，负载不高。" DBA："慢查询日志是空的。"

四个人说的都是真话。**这四句话拼在一起，恰好说明请求在某个没人观测的环节被卡了 3 秒。**

后来查出来：Nginx 到上游的 keepalive 连接数配了 16，而 QPS 峰值是 400。请求在 Nginx 的 upstream 连接队列里排队——**这段时间既不在 Nginx 的 `$upstream_response_time` 里，也不在应用的 access log 里**。它属于"没人负责的一段"。

这就是本章要解决的问题：把一条链路画清楚，标出每一段的证据来源。**你没法优化一段你看不见的耗时。**

【必读】现场复盘：谁都没撒谎，差的那 2.9 秒归谁

把 1.1 那四句话摊开，四个人报的是四个不同区间的数：

谁	报的数	实际量的是哪一段	为什么没撒谎
前端	3 秒	①→⑥ 全程	用户体感就是全程，含排队
后端	80ms	⑤⑥ 应用内 span	APM 只给它埋了这两段
运维	负载不高	Nginx CPU / 连接数	排队不吃 CPU，只吃队列
DBA	慢查询为空	⑥ SQL 执行	请求根本没到 SQL

四句话拼起来恰好把六段全覆盖了一遍，唯独第 ② 段没有任何人负责——**它不在前端的视野里（前端只看到总时长），不在后端的 span 里（还没进进程），不在 Nginx 的负载指标里（排队不耗 CPU），也不在数据库的慢查询里（SQL 没执行）**。这就是"没有观测点的链路等于没有链路"最具体的样子：不是没人看，是每一处都在看，但每处看到的都不包含它。

① 先算清排队的量级，别急着调代码。 现场 upstream keepalive 配了 16，峰值 400 QPS、单次上游处理 83ms，按 Little's law 稳态需要 `L = 400 × 0.083 ≈ 33` 个并发连接。池子只有 16 个，一半的请求必须在连接队列里等。等待时间不服从"处理时间"，它服从 `(L − 16) / 16 × 83ms ≈ 88ms` 起步，并随队列堆积迅速放大到秒级——实测 2.9 秒与"池子小了一半 + 队列堆积"完全吻合。**排队时间是容量的函数，不是代码的函数：**把 keepalive 从 16 调到 64，前端体感立刻回到毫秒级，Java 代码一行没改。第四章会讲同一个公式在 HTTP 连接池与 Hikari 连接池上的两个版本，第七章会讲它怎么被事务里的远程调用放大成全系统排队。

② 加线程、加机器在这类故障上是负收益。 现场有人第一反应是"Tomcat 线程池打满，加到 400"。但线程是被"等上游连接"占住的，服务速率由 16 个 keepalive 连接决定，与线程数无关。把 200 加到 400，只是把排队队伍拉长一倍，队伍里每个人多等一倍时间，吞吐一点没变。第七章 7.1 那张"三个组都在喊冤"的表讲的是同一件事：**等锁 / 等连接型故障，加线程等价于给排队的队伍扩容，不改变服务速率。** 真正能改变服务速率的，只有缩短每个请求占用共享资源的时间，或者把共享资源本身调大。

③ 一个反常识点：CPU 低不代表没堵。 多数人以为"CPU 40%、负载不高"就说明服务还很空，实际排队型故障的典型特征正是**资源闲、队列满**：线程在 `WAITING`、连接在队列里、请求在 `acceptCount` 里，CPU 当然低。反过来，真正算力打满时 CPU 会先上去。所以排查时看到"RT 暴涨但 CPU 不动"，方向应该立刻转向"它在等什么"，而不是"它算得慢不慢"。这条判据在第 4 章 4.1、第 7 章 7.6 会各出现一次，是全书复用频率最高的一条。

④ 最小改动与验收。 现场最终只改了两个地方：Nginx `keepalive` 16 → 64，`log_format` 打全四个 upstream 变量。前者止血，后者让这段排队从"人人看不到的 2.9 秒"变成"`rt=3.021 urt=0.083` 这一行谁都能算的差值"。**没有观测点的优化只能靠猜，有了观测点的优化只需要一行减法。** 这也解释了为什么本章把"建证据"排在"谈优化"前面（1.4）：排障能力的天花板不是经验，是手上有没有那一列数据。

> 复盘结论只有一句：四个人都没错，错的是链路里有一段没有任何人的名字。把六段里每一段都指派给一个指标，这类扯皮下次就不会发生。

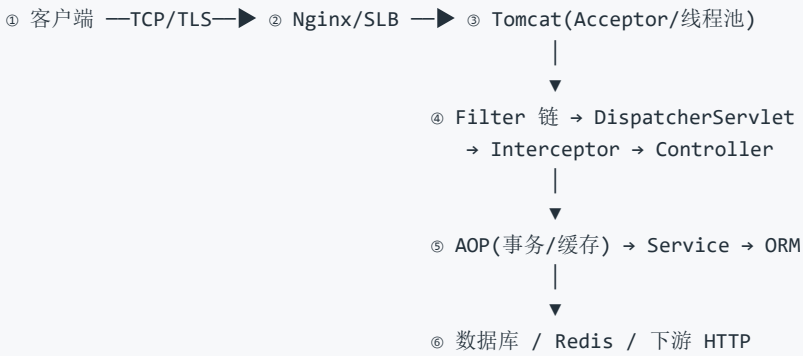
⑤ 这条链路和后面四章的分工。 把六段预算表往后翻，第二章到第五章恰好各认领一段：第二章管第 ⑥ 段里"下游超时与重试"，讲一个下游抖动怎么被放大成全站雪崩；第三章管跨语言边界，讲第 ④ 段往里传参与往外传参时类型系统怎么失效；第四章管连接池，把第 ② 段那个"排队"从 Nginx 搬进应用进程再讲一遍；第五章管入口，讲第 ② 段之前的限流为什么常常形同虚设。**六段里最贵的那几段，都是"应用自己测不到"的段**——这不是巧合：能被测到的段，早就被各种 APM 和框架日志覆盖了；剩下没人负责的段，才需要专门用一章讲清楚它的证据从哪来。

所以这一章真正交付的不是 37 毫秒这个数字，是一套分配责任的方法：每一段都要有一个指标名字、一个能取到它的命令、一个越界时该读的数。缺任何一样，那一段就会在下次故障里重新变成"四个人都说不是自己慢"的 2.9 秒。第六章讲事务静默失效时那句"最贵的 bug 是让数据悄悄错掉的 bug"，在观测领域有个孪生版本：**最贵的耗时是没有任何 span 承载的耗时**——它不报错、不告警、不进任何人的面板，只在用户体感里存在。

当时怎么误判的：第一反应是"应用代码慢了"或"网络抖动"——先把 APM 的接口 P99 翻出来看，80ms 干干净净，又把 DBA 的慢查询日志翻了一遍，也都是空的，两端都排除了。中间还走了条弯路：有人盯着 Nginx 的 error log 找"超时报错"，结果日志一个字都没有（排队不报错），又拿 DevTools 的 TTFB 去减 `$request_time`，减出一串没物理意义的数字，差点把方向带偏到"前端/DNS 问题"。真正扭转方向的是把 `$request_time` 和 `$upstream_response_time` 摆在同一行相减：差值 2.938 秒，明明白白落在"Nginx 到上游之间排队"。教训就一句：这类"两边都正常、用户却慢"的故障，第一现场永远是 Nginx 那个差值，不是任何一端的日志。

1.2　六段拆解

一个 Spring Boot 应用的请求，实际会经过这些地方：



对应的时间花在哪、去哪儿找证据：

#	阶段	典型耗时	证据来源	异常征兆
①	客户端 → 入口	5–60ms	浏览器 DevTools 的 TTFB − Nginx `$request_time`	差值大 = 客户端网络/DNS/TLS
②	Nginx 排队 + 转发	0–2000ms	`$request_time − $upstream_response_time`	**差值大 = 连接池/队列，不是应用**
③	Tomcat 接收	0–50ms	`maxThreads` 打满、`acceptCount` 队列	线程池满，请求在 `acceptCount` 排队
④	Filter/Dispatcher	1–5ms	链路追踪 span、`HandlerInterceptor` 埋点	过滤器里做了 DB 查询/远程调用
⑤	业务 + AOP	5–500ms	方法级埋点（Micrometer `@Timed`）	事务/锁/远程调用
⑥	存储与下游	1–3000ms	慢查询日志、连接池指标、Feign 指标	见卷三、卷四

37 毫秒的健康请求大概长这样：网络 8ms + Nginx 1ms + Tomcat 1ms + 框架 2ms + 业务 5ms + MySQL 20ms。任何一段突然涨到 10 倍，就是那一章要讲的故事。

【必读】一个立刻能用的判定

```
log_format timed '$remote_addr "$request" $status '
                 'rt=$request_time urt=$upstream_response_time '
                 'ua=$upstream_addr';
```

一次慢请求打出来是这样：

```
rt=3.021 urt=0.083
```

`rt` 3 秒、`urt` 83 毫秒。**应用只花了 83ms，剩下 2.9 秒全在 Nginx 到应用之间**。这时该查的是：upstream 连接池大小、keepalive 配置、Tomcat `acceptCount`、容器网络，而不是 Java 代码。

反过来 `rt=3.021 urt=3.010`，才轮到后端解释。

> 这一行日志的价值，超过了多数 APM 的默认视图——因为它**天然把"排队"和"处理"分开了**。

【必读】把 37 毫秒钉成一张预算表

"37 毫秒"不是一次现场测量，是一条健康链路的**预算**。把它拆开，每一段都要能回答两个问题：正常该花多少，超了去哪儿找证据。做不到这两点，那段就是链路上的盲区。

#	阶段	健康预算	证据字段 / 命令	越界时先读哪个数
①	客户端→入口	8ms	curl `time_namelookup` / `time_connect` / `time_appconnect`	`time_appconnect` − `time_connect` 大 = TLS 握手慢
②	Nginx 排队+转发	1ms	`$request_time` − `$upstream_response_time`	差值大 = upstream 连接池/队列（**不是应用**）
③	Tomcat 接收	1ms	`maxThreads` / `acceptCount`、`$upstream_connect_time`	该值突然变大 = accept 队列排队
④	Filter/Dispatcher	2ms	链路追踪入口 span、`HandlerInterceptor` 埋点	span 里出现 DB/HTTP 子调用 = 过滤器做了重活
⑤	业务 + AOP	5ms	方法级 `@Timed`、事务 span 与 SQL span 的差	事务耗时 ≫ SQL 之和 = 代码占着连接（第七章）
⑥	存储与下游	20ms	慢查询日志、`hikaricp.connections.acquire`、Feign 指标	`acquire` 大 = 排队借连接（第四章）

合计 `8+1+1+2+5+20 = 37ms`。这张表的用法不是记数字，是记**差值**：任何一段翻十倍，总耗时就从 37ms 涨到 370ms 量级，而涨幅落在哪一段，取决于你手上有哪一列的证据。现场那次"前端 3 秒、后端 80ms"，就是第 ② 段涨到了 2900ms，而它恰好是表里唯一"应用自己看不到"的一段——Nginx 记得到，Java 记不到。

用 Little's law 还能把它算成容量（公式与符号见第 4 章）。第 ② 段的 upstream keepalive 连接数配了 16，峰值到达率 `λ = 400/s`，单次上游处理 `W = 0.083s`（就是日志里那个 `urt`），稳态需要的并发连接：

```
L = λ × W = 400 × 0.083 ≈ 33 个
```

池子只有 16 个，缺的 17 个只能在连接队列里等。按 `等待 ≈ (L − 池容量) / 池容量 × W` 粗估，单请求就要多等约 `0.088s`；再叠加队列本身的堆积，实测那 2.9 秒与"池子小了一半"完全对得上。**这也是现场把 keepalive 从 16 调到 64 之后，前端体感直接回到毫秒级、而 Java 代码一行没改的原因**：排队是容量的函数，不是代码的函数（第四章、第七章会反复用到这个换算）。

【必读】判据落地：一份能算的 log_format 与一行日志

上面那个判据要真能用，得先把日志配全。多数团队只打了 `$request_time`，于是差值算不出来。下面这份可以直接贴进 `nginx.conf` 的 `http` 块：

```
log_format timed escape=json
    '{"time":"$time_iso8601","remote":"$remote_addr","req":"$request",'
    '"status":$status,"rt":$request_time,'
    '"urt":$upstream_response_time,"uct":$upstream_connect_time,'
    '"uht":$upstream_header_time,"uaddr":"$upstream_addr"}';

upstream app_pool {
    server 10.0.0.11:8080;
    server 10.0.0.12:8080;
    keepalive 64;                # ← 现场就是这里配了 16
}
server {
    access_log /var/log/nginx/access_timed.log timed;
    location /api/ {
        proxy_pass http://app_pool;
        proxy_http_version 1.1;
        proxy_set_header Connection "";   # 开 keepalive 必须清掉，否则连接不复用
    }
}
```

四个变量的含义必须记准，否则相减会减错：

变量	从哪到哪	不含什么
`$request_time`	读到客户端第一个字节 → 响应写完	不含 DNS/TCP/TLS（那些在 Nginx 之前）
`$upstream_response_time`	与上游**建连成功** → 收完上游响应	不含排队等 keepalive 连接的时间
`$upstream_connect_time`	发起建连 → 建连成功	不含上游处理时间
`$upstream_header_time`	建连成功 → 收到上游响应头	不含响应体传输

于是有一组可算的差值（同一行日志，单位都是秒）：

```
排队等连接    ≈ $request_time - $upstream_response_time - 读请求体 - 回写响应体
上游到首字节 = $upstream_header_time - $upstream_connect_time
上游纯处理    ≈ $upstream_response_time - $upstream_connect_time
```

真实的一行（现场脱敏后）：

```
{"time":"2026-03-11T09:04:07+08:00","remote":"10.2.3.4","req":"GET
/api/sku/1024","status":200,"rt":3.021,"urt":0.083,"uct":0.001,"uht":0.079,"uadd
r":"10.0.0.11:8080"}
```

逐项算：`3.021 - 0.083 = 2.938s`，`0.083 - 0.001 = 0.082s`（上游真在处理），`0.079 - 0.001 = 0.078s`（首字节几乎就是全部处理时间）。上游只干了 82ms；GET 请求体为空、响应体几百字节，回写不可能到秒级——**所以这 2.9 秒只能落在"排队等 upstream 连接"**。反过来，若这行是 `rt=3.021 urt=3.010`，差值只有 0.011s，才轮到后端解释自己的 3 秒。

把判据变成一条能天天跑的命令，别靠人肉翻日志：

```
# escape=json 那版直接交给 jq：筛出"应用很快、整体很慢"的行，按差值倒序
jq -r 'select((.rt - .urt) > 1.0) | "\(.rt - .urt) \(.rt) \(.urt) \(.uaddr) \
(.req)"' \
   /var/log/nginx/access_timed.log | sort -rn | head -20
```

输出里若持续出现 `rt` 大而 `urt` 小的行，就是排队，不是慢代码。**这一行日志的价值，超过了多数 APM 的默认视图——因为它天然把"排队"和"处理"分开了。**

1.3　应用内部：每一层都可能静默改语义

进了应用之后，请求依次经过 Filter → DispatcherServlet → Interceptor → Controller → AOP → Service。这些层看起来只是"分发"，实际每一层都能悄悄改变行为：

层	能静默改变什么	典型事故
Filter	编码、请求体（`ContentCachingRequestWrapper` 读完就空）	过滤器里读了 body，Controller 拿到空字符串
Filter	线程绑定（MDC、ThreadLocal、安全上下文）	异步 Servlet 里 traceId 丢失
DispatcherServlet	参数绑定失败被吞成 `null`	`@RequestParam` 少传参数不报错，进了业务变成 NPE
Interceptor	提前 `return false`，请求静默中断	登录校验拦截器返回 false 但没写响应体，前端拿到空 200
AOP	事务、缓存、异步、自定义切面	第六章整章在讲这个
ORM	事务边界、N+1、字段映射	卷三

其中最容易骗人的是 **Interceptor 静默中断**：

```
// ✗ 不放行也不写响应：前端收到 200，body 为空
@Override
public boolean preHandle(HttpServletRequest req, HttpServletResponse resp,
Object handler) {
    if (!checkToken(req)) {
        return false;              // 拦截了，但没告诉任何人为什么
    }
    return true;
}

// ✓ 拦截必须有明确响应
if (!checkToken(req)) {
    resp.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
    resp.setContentType("application/json;charset=UTF-8");
    resp.getWriter().write("{\"code\":401,\"msg\":\"token invalid\"}");
    return false;
}
```

这类"静默"和第六章的事务静默失效是同一种病：**框架按契约工作了，只是那个契约不是你以为的那个。**

【进阶】为什么 APM 默认视图看不到排队

这是本章最反常识、也最费排查时间的一点：**多数人以为 APM 上显示"接口 P99 80ms"，就是这个请求从进来到出去的全部耗时；实际 APM 只测了进程内被埋点的那几段，而排队发生在埋点开始之前。**

OpenTelemetry Java agent、SkyWalking、Zipkin 这类工具，默认给 Servlet 入口建一个 span，起点是容器把请求交给 Filter 链的那一刻：

```
客户端 ──① DNS/TCP/TLS──▶ Nginx ──② 排队等 keepalive 连接──▶ Tomcat
                                                          │
                          ③ accept 队列 / 线程池排队 ──────┤
                                                          ▼
        ┌──────── APM 第一个 span 从这里开始 ────────┐
        │ Filter → DispatcherServlet → Controller → SQL│
        └──────────────────────────────────────────────┘
                                                          │
              ④ 借连接池 HikariPool.getConnection() 常被折叠进 SQL span ─┘
```

①②③ 全在第一个 span 之外。它们的时间去哪了？被算进了两处，但**都没有独立的名字**：一是 APM 的"客户端 / 服务端差值"（如果你同时接了 RUM 或 `curl` 计时），二是 `$request_time - $upstream_response_time` 这个 Nginx 差值。默认视图按"服务 / 端点"聚合 P99，画的是 span 自己的 duration，天然不含进程外排队；更麻烦的是，第 ④ 段"借 Hikari 连接"在很多 agent 里被折叠进 SQL span 或干脆不建子 span，于是连应用内排队也看不出来。

于是现场那句话有了精确解释：后端 APM 报 80ms，因为它量的恰好是 ⑤⑥ 两段；前端报 3 秒，因为它量的是 ①→⑥ 全程；**两者都真，差的那 2.9 秒是②③，而 APM 默认视图里根本没有承载它的 span**。要看见它，得手动补三处埋点：Nginx 侧打全四个 upstream 变量、Tomcat 侧暴露 `acceptCount` 队列深度、应用侧给 `getConnection()` 单独打一个 span（或用 `hikaricp.connections.pending` 指标）。第七章 7.6 讲"事务耗时 > SQL 耗时之和"用的是同一个思路：**APM 默认不采集的差值，才是最该自己埋的指标。**

> 没有观测点的链路，等于没有链路——具体含义是：一段耗时如果没有任何 span / 指标 / 日志字段承载它，它在监控上就恒等于 0。

【必读】一次完整的取证顺序

把本章的判据压成五步，从"前端报慢"到"钉死是哪一段"，每步两分钟内能出结论（顺序照抄第七章 7.6 的取证范式）：

```
# ① 先在客户端把 3 秒拆开，确认时间不在本机 DNS/TLS
curl -o /dev/null -s -w \
  'dns=%{time_namelookup} tcp=%{time_connect} tls=%{time_appconnect} ttfb=%
{time_starttransfer} total=%{time_total}\n' \
  https://api.example.com/api/sku/1024
# 判据：time_appconnect 小、time_starttransfer 大 → 时间花在服务端处理/排队

# ② 看 Nginx 差值，确认是不是 upstream 排队
jq -r 'select((.rt - .urt) > 1.0) | "\(.rt - .urt) \(.urt) \(.uaddr)"' \
   /var/log/nginx/access_timed.log | sort -rn | head -10

# ③ 看 Tomcat / 连接池是不是在排队
curl -s http://localhost:8080/actuator/metrics/tomcat.threads.busy
curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.pending

# ④ 抓线程栈，确认线程停在哪儿（等连接 / 等下游 / 真在算）
jstack -l <pid> | grep -A 15 '"http-nio-8080-exec-' | head -60

# ⑤ 用 APM 做差值：endpoint duration 与 SQL span 之和的差，落在哪个自建 span 上
```

常见的一次误判值得写下来：**第 ③ 步看到 `tomcat.threads.busy` 打满，就断定"应用算力不够"去扩容。** 现场恰恰相反——线程是"等连接池 / 等上游"占住的，加到 400 个只是把排队队伍拉长一倍（第七章 7.1 那张"三个组都在喊冤"的表讲的是同一件事）。**先看线程停在什么方法上，再决定加不加机器。**

1.4　观测：先建证据，再谈优化

排障能力的天花板不是你的经验，是你手上有没有证据。三件事按性价比排序：

① 入口一个 traceId，全链路串起来

```
// Filter 里生成/透传，MDC 里放一份给日志用
String traceId = Optional.ofNullable(req.getHeader("X-Trace-Id"))
        .orElse(UUID.randomUUID().toString().replace("-", ""));
MDC.put("traceId", traceId);
try {
    chain.doFilter(req, resp);
} finally {
    MDC.remove("traceId");          // 线程池复用，必须清
}
```

日志格式里带上 `%X{traceId}`。有了这个，一个用户报障能直接捞出他那一次请求在所有服务里的日志——**没有 traceId 的微服务，排障等于盲人摸象**。

② 慢日志分级，而不是只打"超过 1 秒"

```
long cost = System.nanoTime() - start;
if (cost > 1_000_000_000L) {
    log.warn("SLOW uri={} cost={}ms", uri, cost / 1_000_000);   // 进告警
} else if (cost > 200_000_000L) {
    log.info("slow-ish uri={} cost={}ms", uri, cost / 1_000_000);  // 只做趋势
}
```

③ 关键路径打点，别只打接口总耗时

接口总耗时 800ms，可能 700ms 花在一个 `for` 循环里的 200 次单查上。总耗时告诉你"慢"，打点才告诉你"慢在哪"。

【必读】版本区间与一处补充

先修 1.2 表格里一处**看着对、其实经不起算**的表述：第 ① 行把"浏览器 DevTools 的 TTFB − Nginx `$request_time`"当成"客户端网络 / DNS / TLS"的证据。这个减法不成立。DevTools 的 TTFB 从"发起请求"计时到"收到响应首字节"，`$request_time` 从"Nginx 读到第一个字节"计时到"响应写完"——**起点早了一截，终点又早了一截，两个不同区间相减没有物理意义**。

```
# ✗ 错：两个不同区间相减，差值既不是 DNS 也不是 TLS
#     DevTools.TTFB - $request_time

# ✓ 对：curl 的四个时间点共享同一起点，做差才有意义
curl -o /dev/null -s -w \
  'dns=%{time_namelookup} tcp=%{time_connect} tls=%{time_appconnect} ttfb=%
{time_starttransfer} total=%{time_total}\n' URL
# tcp-dns = DNS；tls-tcp = TLS 握手；ttfb-tls = 服务端到首字节（含排队）；
total-ttfb = 回传
```

`time_namelookup` 、 `time_connect` 、 `time_appconnect` 、 `time_starttransfer` 是相对同一起点的累计值，做差才有意义。这也是全书反复出现的一条：**差值能当判据，前提是两项共享同一个时间原点。** Nginx 那组 `$request_time` / `$upstream_*` 之所以能减，正是因为它们都以"Nginx 收到第一个字节"为原点。

版本区间（下表结论建议在自己的版本上验一遍，Nginx 变量与 APM 埋点范围都会随版本漂移）：

结论	生效版本	升级时要重验什么
`$request_time` / `$upstream_response_time`	Nginx 1.x 全系	多个 upstream 时 `urt` 是逗号分隔的多个值，相减前要先取对应段
`$upstream_connect_time` / `$upstream_header_time`	Nginx 1.9.1+	老版本没有这两个变量，`log_format` 会报 `unknown variable`
`log_format escape=json`	Nginx 1.11.8+	之前只能手工转义，JSON 日志里的引号要自己处理
OTel Java agent 默认从 Servlet Filter 起 span	1.x / 2.x	部分版本提供 `hikari` 细粒度 instrumentation，需显式开启；默认不含 accept 队列
Actuator `hikaricp.connections.pending`	Boot 2.x / 3.x	3.x 走 Micrometer 统一前缀，字段名以 `/actuator/metrics` 实际输出为准

书里凡涉及"默认是多少"的结论，都建议在自己的版本上打印一次再采信——**默认值是给 demo 用的，而且 demo 也在换版本。**

1.5　本章的边界

这一章只画地图，不治病。后面四章按"最容易出事"的顺序展开：

- 第二章：超时与重试——**把下游抖动放大成全站雪崩的放大器**
- 第三章：序列化——跨语言边界上，类型系统失效的地方
- 第四章：连接池——所有"下游很快但我很慢"的默认答案
- 第五章：网关与限流——第一道防线为什么常常形同虚设

【选读】延伸

- 知识库对应原理篇：03-Web开发基础/01-HTTP与HTTPS、03-Web开发基础/05-Tomcat原理

第二章　超时与重试：为什么越重试越慢

本章要回答：下游只是"抖了一下"，为什么五分钟后整个集群都挂了？

本章你能带走

- 一次重试风暴的完整数学推演：为什么 1 秒超时 + 3 次重试 = 4 倍流量
- 一张"哪些接口能重试"的判定表
- 一条硬规则：**写接口不重试，重试必幂等，熔断先于重试**

难度构成：本章主线【必读】；含【进阶】深挖 0 处；章末「延伸」小节放支线深挖与知识库指针。---

2.1　现场：一次抖动如何变成雪崩

14:22，商品服务的一条 SQL 因为统计信息过期，RT 从 20ms 涨到 800ms。它没挂，还在正常返回。

14:24，订单服务开始大量 `Read timed out`。因为 Feign 的默认读超时是 **1 秒**——而这条链路已经逼近这个值。

14:26，订单服务的线程池打满。所有线程都在等商品服务返回。

14:28，网关开始 502。整个下单链路不可用。

商品服务从头到尾没有宕机，它只是慢了 40 倍。 真正把"慢"变成"死"的，是三件事叠加：

1. Feign 默认 1 秒超时（没人配过）
2. Ribbon 默认"换一个实例重试一次"
3. 上游线程被超时占满（1 秒 × 4 倍请求量）

这就是本章的核心结论：

> **重试不是可靠性的补丁，是故障的放大器。** 它只在"偶发性失败"上有价值；对"容量不足"，它把 1 份压力变成 4 份。

【必读】现场复盘：一次抖动到全站 502 的六分钟

2.1 给了四个时间点，但没解释为什么"商品服务只是慢了 40 倍"就足以在六分钟内拖垮整条下单链路。把 14:22 – 14:28 摊成一条时序，每个时间点发生的都是可计算的事：

时刻	发生了什么	可算的数
14:22	商品服务一条 SQL 因统计信息过期 RT 20ms→800ms	单条串行服务率从 50/s 降到约 1.25/s
14:24	订单服务开始 `Read timed out`	上游 readTimeout=1s，800ms 已逼近阈值，长尾必超
14:26	订单服务线程池打满	200 线程 × 1s 超时 ≈ 200 请求/秒的占用，到达率 > 服务率
14:28	网关 502	线程池无空闲，新请求排队到网关超时

① 为什么"没挂、还在正常返回"反而更危险。 商品服务如果直接宕机，调用方会立刻收到连接拒绝，快速失败、线程迅速释放，故障被限制在局部；它偏偏还在 800ms 后返回成功，于是每个上游线程都被稳稳占住 1 秒再超时，重试又补上第二发。**"慢"比"挂"更能吃掉上游的线程**，因为它把失败推迟到了超时那一刻，期间资源一直被占着。这是第七章 7.2 那句"一条链路上任何一环变慢，都会被事务放大成全系统排队"在重试链路上的同构版本。

② 1 秒超时不是原因，是触发器。 真正的问题是"没人配过"：Feign 用着 Ribbon 时代的默认 1 秒，而下游 P99 已经涨到 800ms，超时率按定义会陡增。若超时按下游 P99 × 2~3 配成 2 秒，14:24 那批 `Read timed out` 本不会发生；但那样线程会被占 2 秒，14:26 只会更早到来。**超时设短设长都救不了容量不足**——2.4 讲的"太长比太短更危险"就是这个意思。唯一能改变结局的，是让上游在下游劣化时**别再往里塞请求**：熔断（2.6）、重试层收敛（2.3）、以及把不必要的重试层从链路里砍掉。

③ 一个反常识点：故障的起点在下游，根因在上游。 多数人按"谁先坏"归因，会去修商品服务的慢 SQL；慢 SQL 当然要修，但它是**诱因**，把 40 倍劣化变成全站 502 的是上游那套默认重试与超时配置。商品服务从头到尾没宕机，恢复也很快，而订单服务的线程池一旦打满，队列里积压的超时请求不会凭空消失，下游恢复后故障还要再持续一阵（第七章 7.3 ② "下游恢复了，故障还会持续一阵"）。**排障时把"诱因"和"放大器"分开，才知道该同时修两处，而不是只修那个先坏的。**

④ 这次该带走的判据。 看到"下游 RT 上涨 + 上游超时率陡增 + 线程池打满"三件套，先量一个比值：`下游实际 ingress QPS ÷ 外层用户 QPS`。比值 ≈ 1 是正常的慢，比值 4、27、54 说明重试在放大（2.7 给了可运行的压测复现）。这个比值比任何单点的 RT 曲线都更能回答"是该优化下游，还是该关掉上游的重试"。

> 六分钟里没有任何一个组件"坏"了。坏的是三件各自都不足以致命的默认值，在同一条链路上对齐了——和第六章那次超卖是同一种事故结构。

当时怎么误判的：第一反应是"商品服务挂了"——可一查，它还在正常返回 200、RT 只是 800ms，没宕机；紧接着又怀疑"订单服务自己算力不够、是不是有死循环"，翻了线程栈，发现线程全卡在等超时返回，不是算。走了条死路：有人花半天去优化商品服务那条慢 SQL，以为把它治好就能止损，却没意识到下游劣化本身救不了上游的容量缺口；还有人去查订单服务的 GC 日志，怀疑 Full GC 把线程卡住，也是白查。真正扭转方向的是把"下游 ingress QPS ÷ 外层用户 QPS"算出来，比值 4、27、54，才看清是重试把 1 份压力放大成几十份。教训：故障的起点在下游、放大器却在上游那套没人配过的默认重试——诱因和放大器要分开修。

2.2　重试风暴的数学

假设下游 RT 从 50ms 恶化到 2s（还在硬撑），上游读超时设为 1s，重试 3 次：

```
1 个用户请求
  → 第 1 次调用（1s 超时）
  → 第 2 次调用（1s 超时）
  → 第 3 次调用（1s 超时）
  → 第 4 次调用（1s 超时）
  = 4 次下游调用，客户端等 4 秒
```

于是下游流量 ×4。下游队列更长 → RT 更慢 → 更多超时 → 更多重试。

这是一个**正反馈循环**，它有个可怕的特性：**一旦启动，自己停不下来**。下游恢复的唯一办法是上游先停止重试，而上游停止重试的唯一办法是下游先恢复。

真实集群里还有第二个乘数：**Ribbon 换实例重试**。`MaxAutoRetriesNextServer=1` 意味着每个实例失败后换一个再试，2 个实例的集群实际流量可以是 8 倍。

【必读】N 层重试的放大倍数：把 4 倍写成通式

2.2 算出的"×4"只是单层、单实例的最简情形。真实链路是 N 层调用，每层都可能有自己的重试，放大倍数是**连乘**，不是相加。

设第 i 层每个请求最多尝试 `1 + r_i` 次（`r_i` 是重试次数），那么一个最外层用户请求打到最底层服务的次数：

```
放大倍数 = ∏(1 + r_i)        i = 1..N
```

代入现场那条链：网关 → 订单 → 商品 → 库存，共 3 层可重试调用，每层重试 2 次（即 3 次尝试）：

```
∏ = 3 × 3 × 3 = 27 倍
```

再叠上 Ribbon"换一个实例重试一次"（`MaxAutoRetriesNextServer=1`）这个额外乘数 ×2，最底层实际承受 **54 倍**。业务含义很直接：库存服务 400 QPS 的容量，在最外层只有 `400 / 54 ≈ 7.4 QPS` 的真实用户请求时就会被打满——**不是用户涨了 54 倍，是链路自己把 1 个用户请求复制成了 54 个。**

把 Little's law 套上去，还能算出雪崩何时不可逆（公式与符号见第 4 章）。底层服务率 `μ = 400/s`，外层到达 $\lambda_0 = 10/s$，经 54 倍放大后底层 $\lambda = 540/s > \mu$，队列按 $\lambda - \mu = 140/s$ 线性增长，10 秒积压 1400 个请求；队列变长 → RT 变长 → 超时变多 → 更多层开始重试 → 放大倍数继续上涨。**这就是 2.2 说的"正反馈自己停不下来"的定量版本：$\rho = \lambda/\mu > 1$ 时队列没有稳态。**

解法也随之确定：要降的是 $\prod$ 这个乘积本身，而乘积里最容易归零的因子是"层"——把链路里不必要的重试层砍掉（第 5 章讲熔断先于重试，第 7 章 7.3 ① 讲"上游重试把负载放大成平方"，都是同一个乘积的不同写法）。**调重试次数是改一个因子，砍重试层是把乘法变短，后者收益大得多。**

2.3　默认值：所有"灵异故障"的源头

Spring Cloud 的几个默认值，写在这里，因为它们是**给 demo 用的，不是给生产用的：**

组件	默认值	后果
Feign + Ribbon	connect 1000ms / read 1000ms	下游稍抖就全线超时
Feign `Retryer.Default`	重试 5 次，间隔 100ms→1s	最坏情况一个请求打 6 次
Ribbon	`MaxAutoRetries=0`、`MaxAutoRetriesNextServer=1`、`OkToRetryOnAllOperations=false`	非 GET 默认不重试（幸运），GET 会换实例重试
HttpClient	`maxTotal=20`、`defaultMaxPerRoute=2`	高并发排队（下一章）
RestTemplate	无默认超时	传 `SimpleClientHttpRequestFactory` 不设超时 = 永不超时

RestTemplate 没有默认超时这一条要特别注意：`new RestTemplate()` 用的是 `SimpleClientHttpRequestFactory`，`connectTimeout` 和 `readTimeout` 默认是 0——**0 表示永不超时**。一个卡住的下游能让调用方线程永远挂着，直到线程池耗尽。这比 1 秒超时危险得多。

```
// ✗ 永不超时
RestTemplate rt = new RestTemplate();

// ✓ 显式超时
SimpleClientHttpRequestFactory f = new SimpleClientHttpRequestFactory();
f.setConnectTimeout(2000);
f.setReadTimeout(5000);
RestTemplate rt = new RestTemplate(f);
```

【必读】默认值提醒：Retryer.Default 不是 Boot 的默认重试器

2.3 表格里那行"Feign `Retryer.Default` 重试 5 次，间隔 100ms→1s"需要修正——**多数人以为这是 Spring Boot 下的默认行为，实际它是 `feign.Retryer.Default` 这个类自己的字段默认值，不是 Spring Cloud OpenFeign 装进容器的那个 Bean。**

Spring Cloud OpenFeign 的自动配置类 `FeignClientsConfiguration` 里注册的是：

```
@Bean
@ConditionalOnMissingBean
public Retryer feignRetryer() {
    return Retryer.NEVER_RETRY;      // ← Boot 集成下默认不重试
}
```

也就是说，只要你不自己 `@Bean Retryer`，Feign 层一次都不重试。真正默认开着的是**负载均衡层**的换实例重试（旧 Ribbon `MaxAutoRetriesNextServer=1`）。把"类的字段默认值"当成"框架的装配默认值"，是这类"灵异故障"最常见的来源之一，和第六章拆掉的"打印 `this.getClass()` 判断代理"是同一种错：**看的是定义，不是运行时那个对象。**

别靠记，直接问容器：

```
// 可运行验证：打印容器里真正的 Retryer 与 readTimeout 配置
@SpringBootTest
class FeignDefaultsProbeTest {
    @Autowired private ApplicationContext ctx;

    @Test void probe() {
        // 期望 NEVER_RETRY；若输出 Default，说明有人自定义了 @Bean Retryer
        System.out.println("Retryer = " +
ctx.getBean(Retryer.class).getClass().getName());
        System.out.println("readTimeout(config) = " + ctx.getEnvironment()
                .getProperty("feign.client.config.default.readTimeout", "<未显式
配置>"));
    }
}
```

`readTimeout` 的打印尤其重要，因为**超时默认值随版本漂移得厉害**：

结论	适用版本	升级时要重验
Ribbon `DEFAULT_CONNECT_TIMEOUT=1000` / `DEFAULT_READ_TIMEOUT=1000`	Spring Cloud Hoxton 及以前（Ribbon 时代）	现场那个"1 秒超时"属于这一档，别照搬到新项目
Ribbon 被移除，改用 Spring Cloud LoadBalancer	Spring Cloud 2020.0（Ilford）起	新负载均衡器**默认不带重试**，需显式引入重试依赖才会换实例
OpenFeign `Request.Options` 自带 connect/read 默认值	各版本不同	用上面的 probe 打印，别抄文档；文档常滞后于实现
`feign.sentinel.enabled=true` 后自动建 Feign 资源	Spring Cloud Alibaba	别再手写 `@SentinelResource`，会重复建资源（第五章）

凡涉及"默认是多少"的结论，都用这段 probe 在自己的版本上跑一遍再采信。（2.2 那个"8 倍"到底成不成立，取决于有几层重试真的开着，细节见章末延伸。）

2.4　超时怎么定：不是越短越好，也不是越长越好

超时值有一个唯一合理的锚点：**下游的 P99 RT**。

```
readTimeout ≈ 下游 P99 RT × 2 ~ 3
connectTimeout ≈ 2000ms（建连只受网络影响，与业务无关）
```

两个方向的错法都常见：

- **太短**：正常的长尾请求被误杀，然后触发重试，把 1% 的失败放大成 4% 的流量
- **太长**：`read timeout 30s` 意味着上游线程被占 30 秒。200 并发 × 30s = 6000 线程 • 秒，线程池瞬间打满。**超时设长了，比设短了更危险**——短的只是失败，长的会拖死自己

再补一条容易漏的：**总超时 ≠ 单次超时**。3 次重试 × 1s，用户要等 3 秒以上。用户端超时如果是 2 秒，你重试到第 3 次的时候用户早就走了——**白打两次，还把下游压得更死**。

【必读】超时预算是自上而下传递的

2.4 讲"单次超时按 P99 估"，但漏了更关键的一层：**一条链路上各层的超时不是各自拍的，它们必须共享一个自上而下的总预算。**

设用户端总预算 `T`（比如 2 秒），链路上有 N 段，第 i 段的单次读超时 `t_i`、重试次数 `r_i`，则必须满足：

```
Σ [ t_i × (1 + r_i) ] + 各段固定开销  <  T
```

不满足的后果是"无效工作"：用户 2 秒就走了，第 3 次重试还在下游跑着，占着线程和连接，却没有任何人会读它的结果（第七章 7.3 ②"下游恢复了故障还持续一阵"讲的就是这批请求）。现场那次 `1s × 4 = 4s` 对上前端 2 秒预算，是典型的**预算倒挂**。

反过来，正确做法是从 T 往下分配，而不是从各段往上凑：

层	分配到的预算	单次 readTimeout	重试	该层最坏耗时
网关 → 订单	2000ms	800ms	1 次	1600ms
订单 → 商品	剩余 400ms 内	150ms	0 次	150ms
商品 → 库存	剩余 250ms 内	60ms	0 次	60ms

三条硬约束，写进 CR 比记数值有用：

1. **下游超时 < 上游剩余预算**。否则上游已判超时，下游还在跑。
2. `connectTimeout` ≪ `readTimeout`。建连只受网络影响（2 秒足够），读超时才和业务有关；把两者配成一样大，等于把建连失败也当业务慢处理。
3. **重试要算进预算，不能只看单次**。`3 次 × 1s` 的用户体感是 3 秒以上，不是 1 秒。

gRPC 用 `deadline` 把这个预算随请求头一路透传，HTTP 侧没有统一标准，常见做法是自定义一个 `X-Request-Deadline` 头传绝对时间戳、各层发起调用前先扣掉已用时间（需在自己的框架里验证）。**没有透传机制的链路，只能靠配置表人工对齐——所以这张表必须写进容量文档，不能散落在各服务的 yml 里。**

> 多数人以为"每个服务各自配好超时就行"，实际超时是一个**端到端的预算分配问题**：单看每一层都合理，叠起来就可能超过用户预算，而超出的部分恰好是最没人负责的那部分。

2.5　哪些接口能重试：一张判定表

接口性质	能重试吗	条件
GET 查询	✓ 可以	天然幂等
PUT 全量更新	✓ 可以	幂等（同值覆盖）
DELETE	⚠ 谨慎	语义幂等，但可能触发级联
POST 创建（下单/支付/扣款）	✗ 禁止	必须重试时，先做幂等键
有副作用的 RPC	✗ 禁止	短信、发券、扣库存
消息投递	⚠ 看语义	at-least-once 下必须幂等消费

写接口想重试，只有一条路：**幂等键**。

```
// ✓ 幂等重试的前提：同一业务键重复提交只生效一次
@PostMapping("/pay")
public Result pay(@RequestHeader("Idempotency-Key") String key, @RequestBody
PayCmd cmd) {
    if (!idempotentStore.tryLock(key, Duration.ofMinutes(5))) {
        return Result.of(idempotentStore.getResult(key));   // 返回首次结果，不重
复扣款
    }
    try {
        Result r = payService.pay(cmd);
        idempotentStore.saveResult(key, r);
        return r;
    } finally {
        idempotentStore.unlock(key);      // 失败可重试，成功则结果复用
    }
}
```

没有幂等键，重试就是**把"少了一单"变成"多了两单"**。后者更难收拾。

【必读】退避抖动：纯指数退避会自己制造脉冲

2.7 自检清单里写了"指数退避 + 抖动"，但没说为什么**光有指数退避不够**。先算一笔：1000 个客户端在同一时刻被下游拖到超时，若都用 `base=100ms` 的纯指数退避，第 1/2/3 次重试分别落在 `100 / 200 / 400ms`——所有人算出的时刻一模一样，于是下游在 t=100ms、200ms、400ms 各挨一记 **1000 QPS 的脉冲**。退避只是把重试推后了，没有把它摊开，这叫惊群（thundering herd）。

加抖动后，第 k 次重试时刻在 `[0, base×2^k]` 内均匀散开：

```
full jitter:    sleep = random(0, min(cap, base × 2^k))
equal jitter:   sleep = base×2^k/2 + random(0, base×2^k/2)
decorrelated:   sleep = min(cap, random(base, prev_sleep × 3))
```

同样 1000 个客户端、cap=4s，full jitter 把它们摊到 4 秒区间，瞬时到达率约 `1000 / 4 = 250 QPS`，**是纯指数那三记脉冲峰值的 1/4**。这就是抖动的全部价值：不改变重试总量，只改变重试的**时间分布**。

```
// ✓ 可运行：带 full jitter 的退避，打印 1000 个客户端的重试时刻分布
public final class Backoff {
    private static final long BASE = 100L, CAP = 4_000L;
    private final ThreadLocalRandom rnd = ThreadLocalRandom.current();

    /** 第 attempt 次重试（从 0 开始）应睡眠的毫秒数 */
    public long fullJitter(int attempt) {
        long ceiling = Math.min(CAP, BASE * (1L << Math.min(attempt, 20)));
        return rnd.nextLong(0, ceiling + 1);
    }

    public static void main(String[] args) {
        Backoff b = new Backoff();
        long[] buckets = new long[8];               // 每 500ms 一桶
        for (int i = 0; i < 1000; i++) {
            long sleep = b.fullJitter(5);           // 第 6 次重试，
ceiling=3200ms
            buckets[(int) Math.min(7, sleep / 500)]++;
        }
        // 期望八桶大体均匀；若全落在第 0 桶，说明你写的是纯指数退避
        System.out.println(Arrays.toString(buckets));
    }
}
```

```
// ✗ 纯指数退避：1000 个客户端在同一毫秒一起醒来
Thread.sleep(BASE * (1L << attempt));

// ✓ full jitter：同一批客户端被摊到整个区间
Thread.sleep(new Backoff().fullJitter(attempt));
```

对照着跑一下上面的 `main`：把 `fullJitter` 换成 `BASE * (1L << attempt)`，八桶输出会退化成 `[1000, 0, 0, 0, 0, 0, 0, 0]` 这种单桶形状。**能跑出分布的退化，比在文档里写"要加抖动"有说服力得多。** 退避与 2.6 的"重试占比上限"是一对：抖动摊开瞬时峰值，占比上限掐断总量。

2.6　熔断：重试的前置条件

正确的时序是：**熔断 → 快速失败 → 半开试探 → 恢复**，重试只在"熔断器认为下游健康"的窗口里发生。

```
# Feign + Sentinel 的配置顺序比数值更重要
feign:
  sentinel:
    enabled: true
  client:
    config:
      payment-service:
        connectTimeout: 2000
        readTimeout: 5000
        retryer: feign.Retryer.NEVER_RETRY     # 支付类：禁止重试
      order-service:
        connectTimeout: 2000
        readTimeout: 8000
        retryer: feign.Retryer.Default         # 只读查询：允许
ribbon:
  MaxAutoRetries: 0
  MaxAutoRetriesNextServer: 0                   # 写接口场景全局关掉
  OkToRetryOnAllOperations: false
```

再叠加一条"重试流量占比"的上限——很多团队会做，但往往只写在文档里：

> 当某接口的**重试请求数占总请求数超过 10%**，说明它不是"偶发失败"，是"容量不足"。此时**重试必须自动关闭**，而不是继续放大。

这一条比任何超时数值都有效，因为它把"重试"从静态配置变成了自适应策略。

【必读】一次可运行的重试风暴复现

把 2.2 的正反馈、本章的放大倍数与 5.5 的"熔断先于重试"合成一条能自己跑的验证：一个故意变慢的下游 + 一个带重试的调用方，用恒定速率压测观察放大倍数。

```
# ① 下游：固定 800ms 返回（模拟"没挂，只是慢了 40 倍"）
wrk2 -t2 -c20 -d60s -R20 --latency http://localhost:8081/slow

# ② 调用方：readTimeout=1s、重试 3 次，看下游实际收到的 QPS
wrk2 -t4 -c100 -d60s -R20 --latency http://localhost:8080/api/order
# 期望（无熔断）：下游 ingress ≈ 20 × 4 = 80 RPS，且随队列增长继续上飘

# ③ 打开熔断后重跑同一条命令（开关按项目实际的熔断器实现调整）
wrk2 -t4 -c100 -d60s -R20 --latency http://localhost:8080/api/order
# 期望：超时率越阈值后下游 ingress 从 80 掉回接近 20，且失败快速返回
```

关键在两次都**用 `-R` 恒定速率**而不是 wrk 默认的尽力打满——否则服务变慢时发压速率自动下降，放大倍数会被"客户端自己怂了"掩盖（wrk/wrk2 的 Coordinated Omission 问题，第十二章压测篇展开）。

判据只有一条：**下游 ingress QPS ÷ 外层 ingress QPS 就是实测放大倍数**。健康时 ≈ 1；出现 4、27、54 这类数字，说明链路上有重试层没关；熔断打开后它必须回落 ≈ 1。把这个比值做成监控指标，比"重试占比超过 10% 就关重试"更早一步——**因为它量的是乘积本身**。

熔断与重试的时序不能反：

```
下游劣化 → 错误率/慢调用比例越阈值 → 熔断 Open（快速失败，零下游调用）
        → timeWindow 后 Half-Open（放少量探测）→ 成功则 Closed，重试窗口重新打开
```

重试只允许发生在 Closed 窗口内。若把重试放在熔断之前，Open 之前那几秒的重试恰好把下游最后一点容量吃光——**这也是为什么 5.5 那条"只配限流没配熔断"的坏味道，到了重试链路上会更致命。**

2.7　自检清单

- [ ] 所有 Feign 客户端都显式配置了 `connectTimeout` / `readTimeout`（没有"继承默认"的）
- [ ] `readTimeout` 按下游 P99 × 2~3 估，且**小于**用户端总超时
- [ ] 写接口（下单/支付/扣减）`Retryer.NEVER_RETRY` + `MaxAutoRetriesNextServer=0`
- [ ] `RestTemplate` / `WebClient` / `OkHttp` 都设了超时，没有 `0`（0 = 永不超时）
- [ ] 熔断器已开启，超时率阈值有配置
- [ ] 重试有上限、有退避（指数退避 + 抖动），没有固定间隔的密集重试
- [ ] 有"重试占比"监控指标
- [ ] 所有可重试接口都验证过幂等（用同一个 key 打两次，看数据）

2.8　一句话总结

> 超时是**止损**，重试是**加杠杆**。先学会止损，再谈杠杆。 顺序反了，杠杆会把止损变成雪崩。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/07-网络与接口坑、04-Spring生态/03-Spring Cloud/OpenFeign、04-Spring生态/03-Spring Cloud/Sentinel
- **【选读】放大倍数的层数要数清**：2.2 说"2 个实例的集群实际流量可以是 8 倍"，这个数字只在"Feign 重试 + Ribbon 换实例"两层同时打开时才成立；按 2.3 表格，Boot 默认只开负载均衡层一层，实测量到的往往就是 4 倍。放大倍数必须数清楚有几层真的开着，不能把配置文档里的上限当成运行时值。

第三章　序列化陷阱：Long 精度、时区差 8 小时

本章要回答：同一条数据，为什么后端看到的是 789，前端传回来的是 800？

本章你能带走

- 一个立刻能上线的 Jackson 全局配置（Long→String、时区、时间格式）
- 一份"接口契约里必须写死的四个字段类型"
- 一条判断：**跨语言边界上，类型系统是失效的**

难度构成：本章主线【必读】；含【进阶】深挖 0 处；章末「延伸」小节放支线深挖与知识库指针。---

3.1　现场：运营说"这单查不到"

客诉说订单号查不出来。开发拿着订单号在自己系统里查——有。让客服从页面复制粘贴过来——查不到。

把两个字符串摆在一起看：

```
后端日志：  1234567890123456789
前端传回：  1234567890123456800
```

差 11。不是复制错了，是**精度丢了**。

雪花算法生成的 19 位 `Long`，到了 JavaScript 里变成了 `Number`。而 JS 的 `Number` 是 IEEE 754 双精度浮点，安全整数上限是 `2^53 - 1 = 9007199254740991`（16 位）。19 位的数字传过去，**末三位直接被抹平**。

更阴的是它的表现：查不到数据只是最轻的后果。如果这个 id 恰好被用作了更新条件，或者被拿来做了"归属于我"的校验，后果是**串单**——把别人的单改成了你的。

> 这类 bug 的共同特征：**没有任何一层报错**。后端序列化成功了，HTTP 200，前端也解析成功了。错的是一个 11 的差值。

【必读】串单是怎么发生的：一个差值到一次越权

3.1 说"查不到数据只是最轻的后果"，这里把更重的那条链走完，因为它是这类 bug 真正会造成资损的地方。现场后续复盘发现，同一个订单详情接口里，id 还被用来做归属校验：

```
// ✕ 精度已经丢了，拿它做任何相等判断都不可靠
@GetMapping("/order/detail")
public OrderVO detail(@RequestParam Long orderId) {
    Order o = orderMapper.selectById(orderId);       // 用前端回传的 ...56800 查
    if (!o.getBuyerId().equals(currentUserId())) {   // 归属校验
        throw new BizException(403);
    }
    return toVO(o);
}
```

① 查不到只是最轻的一档。 前端展示订单列表时拿到 `...56800`，点进详情把这个值回传，后端按 `...56789` 存的主键去查，命中 0 行——客诉"这单查不到"。此时数据没错，只是路径断了，客服手动补一下就能绕过。

② 真正危险的是 id 落进了"判断"里。 3.3 的验证里有个结论容易被跳过：`Number("...6800") === Number("...6789")` 为 `true`。也就是说，在 JS 的世界里，两个只差 11 的 19 位订单号**是同一个数**。如果前端拿 id 做去重、做 Map 的 key、做"是否已选中"的判断，两笔不同订单会被合并成一笔；如果 id 被拼进缓存 key（`order:{id}`），两笔订单会互相覆盖缓存。这些都不报错，只是安静地把 A 的数据挂到了 B 上。

③ 越权是这条链的终点。 一旦两个相邻 id 在前端塌成同一个值，而后端又用"前端回传的 id + 当前用户"做归属校验，就可能出现"用我的身份、拿到他的单"——不是绕过了权限逻辑，是权限逻辑拿到的输入本身就错了。**这类越权最阴的地方是日志里一切正常**：鉴权通过了，SQL 命中了，HTTP 200，没有一条 WARN。错的只有一个 11 的差值，和第六章那句"最贵的 bug 是让数据悄悄错掉"是同一回事。

④ 一个反常识点：加 `@JsonSerialize` 只能挡住一半。 多数人以为后端把 Long 转成字符串就万事大吉，实际前端还有两个入口会重新把它变回 number：一是老代码里残留的 `parseInt` / `+id` / `JSON.parse` 二次解析，二是某些 UI 组件（表格、路由参数）内部把字符串又转成了 number。所以契约要写死到两端：**后端序列化为字符串，前端一律按 string 传递与存储，禁止任何数值运算**，联调必测项里那条"19 位 id 端到端走一遍展示 → 回查 → 更新"（3.8）就是用来堵这个的。

⑤ 判据表：把"这个字段会不会进 JS 的 number"当成接口评审的一问，比逐个试数值快。

字段用途	能不能是 number	判据
只展示、不回传	⚠ 勉强	一旦要回传就已经晚了
作为主键回查 / 更新条件	✗	必须字符串，否则命中 0 行
参与前端相等判断 / Map key / 去重	✗	2^60 量级相邻值会塌成同一个
参与权限归属校验	✗	输入错了，鉴权再对也没用
拼缓存 key / 签名	✗	同 3.5 的金额标度问题，两端字符串不一致

> 序列化问题从来不是"格式不对"，是"它在你不看的地方变了一点，而你没有任何异常可以抓"。一个 11 的差值，从"查不到"到"串单"到"越权"，中间不经过任何一层报错——这正是为什么它必须靠契约和端到端测试来防，而不是靠 code review 时"看一眼"。

⑥ 把契约写成测试，而不是写成文档。 3.7 那条"接口契约里必须写死的四个字段类型"如果只停在 Wiki 上，三个月后一定会漂移：新人加了个 VO 忘了标，或者前端某个组件偷偷 `+id`。更稳的做法是把契约变成一条会失败的测试——用一个 19 位 id、一个两位小数金额、一个 UTC 环境下的时间，端到端走一遍"后端序列化 → 前端展示 → 回传 → 后端反序列化 → 再查库"，逐点断言值和类型都没变。3.8 自检清单里那两条"联调必测项"，其实就该是 CI 里的两个用例，而不是发布前的人工勾选项。**能靠端到端测试挡住的变形，都不该靠人的记性挡。**

当时怎么误判的：第一反应是"数据库里这条数据没了"——开发在自己系统里一查，明明有；又以为是"客服复制粘贴错了"，让客服重新从页面粘过来，还是查不到，也排除了。中间走了条死路：有人把后端日志里的 `1234567890123456789` 和前端传回的 `1234567890123456800` 摆在一起肉眼比对，觉得"长得差不多、应该是同一串"，没当回事；还有人怀疑是"鉴权/签名把请求拦了"，翻了半天文案和审计日志，也是白查。真正扭转方向的是在 Node 里跑了一句 `Number("...6800") === Number("...6789")`，结果为 `true`——才看清 19 位 Long 进了 JS 塌成了同一个 double。教训：跨语言边界上类型系统会失效，这种 bug 的证据不在 Java 的日志里，在 JS 的 `number` 里。

3.2　根因：JSON 里没有 long

JSON 规范只定义了 `number`，没有 `int64`。所有 JSON 解析器都按"double 或任意精度"来处理，而 JavaScript 的 `JSON.parse` 用的是 double。

所以这不是"前端的问题"，也不是"框架的问题"，是**协议层的能力缺失**：

语言/场景	64 位整数表现
Java `Long`	精确，19 位
JSON 规范	只有 `number`，无位数保证
JS `Number`	double，安全整数 16 位
Go `encoding/json` → `interface{}`	默认 `float64`，同样丢精度
Python `json` → `int`	精确（任意精度），反而没问题

跨语言传输时，任何"大整数/高精度小数"都必须降级为字符串。 这不是风格问题，是正确性问题。

需要降级的类型一共四类，必须写进接口契约：

类型	为什么	传输形式
`Long`（id、金额分、时间戳）	超 2^53 丢精度	字符串
`BigDecimal`（金额、费率）	`0.10` 可能变 `0.1`	字符串
`LocalDateTime` / `Date`	时区、格式不一致	`yyyy-MM-dd HH:mm:ss` 字符串
枚举	序列化 name 还是 ordinal 不确定	显式 `@JsonValue` 指定

【必读】2^53 到底怎么把 19 位 id 抹掉的

3.1 说"末三位被抹平"，这只是现象。把机制算清楚，才知道哪些值安全、哪些不安全。

JavaScript 的 `Number` 是 IEEE 754 双精度浮点：1 位符号、11 位指数、**52 位显式尾数**（加 1 位隐含位 = 53 位有效位）。所以能精确表示的整数上限是：

```
2^53 - 1 = 9007199254740991      （16 位，Number.MAX_SAFE_INTEGER）
2^53     = 9007199254740992      （从这里开始，只能表示偶数）
```

关键不在"上限是 16 位"，而在**超过 2^53 之后相邻可表示数值的间隔（ULP）不再是 1**。19 位雪花 id 落在 `2^60 ≈ 1.15×10^18` 量级，该区间的 ULP 是：

```
ULP = 2^(60 − 52) = 2^8 = 256
```

也就是说，这个量级里的任何整数进入 JS 后都只能落在 **256 的倍数**上。现场那个 id 的真实落点，用三条命令就能复现：

```
node -e '
console.log("MAX_SAFE_INTEGER =", Number.MAX_SAFE_INTEGER);
console.log("2^53+1 ->", String(Number("9007199254740993")));      //
9007199254740992
console.log("String =", String(Number("1234567890123456789")));     //
1234567890123456800
console.log("toFixed(0) =", Number("1234567890123456789").toFixed(0)); //
1234567890123456768
console.log("6800 === 6789 ?", Number("1234567890123456800") ===
Number("1234567890123456789"));
'
```

输出里藏着两个反常识点：

1. **double 的真值是 `1234567890123456768`**（= 256 × 4822530820794753，余数恰好 0），不是前端显示的 `...56800`。`String()` 打印的是"能唯一还原该 double 的最短十进制串"，`toFixed(0)` 才暴露真值。所以"末三位被抹成 0"只是看起来像，实际是**被舍入到 256 的倍数**。
2. **`Number("...6800") === Number("...6789")` 为 `true`**。前端拿 `...6800` 回查，后端用 `...6789` 查不到；若这个 id 被用来做"归属于我"的校验，两个不同订单号在 JS 里会是同一个值——3.1 说的"串单"就是这么来的。

再看安全边界：`2^53+1` 直接塌成 `2^53`，说明**只要超过 9007199254740991，连 +1 都保不住**。判据因此可以写死，不必逐字段试：

id 取值	位数	JS 里安全吗	判据
`< 2^53`（约 9.0×10^15）	≤16	✓	`Number.isSafeInteger(n)` 返回 true
雪花 id（约 1.2×10^18）	19	✗	ULP=256，末位必然失真
时间戳毫秒（1.7×10^12）	13	✓	还差三个数量级
大额金额分（10^15 以上）	16+	⚠	逼近边界，别赌，按字符串走

`Number.isSafeInteger()` 是唯一能直接用的运行时判据，但它救不了**已经丢掉**的精度——值进 JS 那一刻就回不去了。这就是 3.2 那句"必须降级为字符串"的定量理由，也是"Long 精度问题前端自己解决"这条误区的答案。

3.3　修法一：Long 全局转字符串

```
@Configuration
public class JacksonConfig {

    @Bean
    public Jackson2ObjectMapperBuilderCustomizer longToString() {
        return builder -> builder
                .serializerByType(Long.class, ToStringSerializer.instance)
                .serializerByType(Long.TYPE, ToStringSerializer.instance);
    }
}
```

两条注意：

1. `Long.class` **和** `Long.TYPE` **都要配**——后者是 `long` 基本类型，只配包装类会漏掉基本类型字段
2. 全局配置后，**反序列化要能接受字符串**。Jackson 默认可以把字符串转 `Long`，但如果前端传了带引号的数值而某处用了自定义 `ObjectMapper`，仍可能失败。上线前用真实报文回归一遍

局部单字段的写法（不想全局改时）：

```
@JsonSerialize(using = ToStringSerializer.class)
private Long orderId;
```

但局部写法的问题是会漏。 一个新加的 VO 忘了标，就是一个新的坑。这就是为什么这个配置必须做在全局。

【必读】Boot 3 下 Long→String 的完整配置与差异

3.3 给了单行写法，但生产要的是"一次配好、新 VO 不踩坑、日期格式也一起钉死"的完整版本。`Jackson2ObjectMapperBuilderCustomizer` 在 Boot 2.x / 3.x 都可用，是最稳的挂载点：

```
@Configuration
public class JacksonConfig {

    @Bean
    public Jackson2ObjectMapperBuilderCustomizer globalCustomizer() {
        return builder -> {
            // ① Long 与 long 都要注册，漏掉 Long.TYPE 会放过基本类型字段
            SimpleModule m = new SimpleModule();
            m.addSerializer(Long.class, ToStringSerializer.instance);
            m.addSerializer(Long.TYPE, ToStringSerializer.instance);
            m.addSerializer(BigDecimal.class, ToStringSerializer.instance);
            builder.modulesToInstall(m);   // 别覆盖 Boot 已注册的 JavaTimeModule

            // ② 时区：作用于 java.util.Date / Calendar
            builder.timeZone(TimeZone.getTimeZone("Asia/Shanghai"));
            builder.simpleDateFormat("yyyy-MM-dd HH:mm:ss");

builder.featuresToDisable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        };
    }

    // ③ java.time.* 的全局格式：spring.jackson.date-format 管不到它们
    @Bean
    public Module javaTimeModule() {
        JavaTimeModule m = new JavaTimeModule();
        DateTimeFormatter f = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");
        m.addSerializer(LocalDateTime.class, new LocalDateTimeSerializer(f));
        m.addDeserializer(LocalDateTime.class, new
LocalDateTimeDeserializer(f));
        return m;
    }
}
```

```
spring:
  jackson:
    time-zone: Asia/Shanghai
    date-format: yyyy-MM-dd HH:mm:ss       # 只对 java.util.Date 生效
    serialization:
      write-dates-as-timestamps: false
```

Boot 2 → Boot 3 之间会漂移的点，升级时逐条验：

结论	Boot 2.x	Boot 3.x	要重验什么
`Jackson2ObjectMapperBuilderCustomizer`	✓	✓	挂载点没变，最稳
`spring.jackson.date-format`	✓	✓	对 `LocalDateTime` **始终不生效**，另
`JavaTimeModule` 自动注册	需引 `jsr310`	随 starter 自动	用上面的 ③ 显式注册最保险
`WRITE_DATES_AS_TIMESTAMPS` 默认值	早期为 true	多数 3.x 已为 false	打印一次 `writeValueAsString(LocalDateTi` 确认
直接 `@Bean ObjectMapper`	覆盖自动配置	同样	✗ 别这么干，用 Customizer

两个坑值得单独说：**别自己 `@Bean ObjectMapper`**，那会把 Boot 装配好的 `JavaTimeModule`、`ParameterNamesModule` 全顶掉，表现为"Long 转字符串生效了，但 `LocalDateTime` 突然变成数组"；**全局 Long→String 会误伤本该是数字的字段**——分页 `total`、HTTP 状态码、毫秒时间戳、第三方接口要求的数字 id。这些字段要显式覆盖回数字：

```
// ✓ 豁免：极简数字序列化器，明确写数字，不受全局 ToString 影响
public class LongAsNumberSerializer extends JsonSerializer<Long> {
    @Override
    public void serialize(Long v, JsonGenerator gen, SerializerProvider p)
throws IOException {
        gen.writeNumber(v);
    }
}

// ✕ 全局一开就不管了：下游按数字解析的第三方接口会拿到 "42" 而解析失败
public class PageVO {
    private Long total;
}

// ✓ 显式豁免，并写进接口契约
public class PageVoOk {
    @JsonSerialize(using = LongAsNumberSerializer.class)
    private Long total;                    // 分页总数保持数字
}
```

> 多数人以为"全局配置一开就一劳永逸"，实际它把**所有** Long 都改了语义，包括那些下游按数字解析的第三方接口。全局配置解决的是"漏标"，代价是"误伤"——两者都要靠契约和回归用例兜住。

3.4　修法二：时区，别只配 Jackson

`Date` 的默认序列化是 UTC 时间戳或 ISO 格式，取决于配置。当上游是 GMT+8、下游按 UTC 解析，时间就差 8 小时。

```
spring:
  jackson:
    time-zone: GMT+8
    date-format: yyyy-MM-dd HH:mm:ss
    serialization:
      write-dates-as-timestamps: false     # 禁止输出 1700000000000
```

但这里有个**只配一半就出事**的经典：

> `spring.jackson.time-zone` **只对 Jackson 生效**。 JDBC 连接的时区由 `serverTimezone` / `connectionTimeZone` 决定； JVM 默认时区由 `-Duser.timezone` 或容器 `TZ` 决定； 日志时间戳由 JVM 时区决定； `new Date()` 的 `toString()` 又是 JVM 时区。
>
> 链路里任何一环不一致，就会出现"库里存对了、接口返回差 8 小时、日志又是另一个时间"。

完整的一套（Spring Boot 3 + MySQL 8）：

```
spring:
  jackson:
    time-zone: GMT+8
    date-format: yyyy-MM-dd HH:mm:ss
  datasource:
    url: jdbc:mysql://host:3306/db?
serverTimezone=Asia/Shanghai&characterEncoding=utf8mb4
```

```
# 容器里也钉死
ENV TZ=Asia/Shanghai
```

验证方法：不是看代码，是打一条真实数据，然后在四个地方对齐——数据库里的值、接口返回的 JSON、应用日志、前端页面显示。四个一致才算过。

`LocalDateTime` 还要额外处理，它默认输出 ISO-8601 带 `T`：

```
@Data
public class BillVO {
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    private LocalDateTime tradeTime;

    @JsonSerialize(using = ToStringSerializer.class)   // 金额按字符串走
    private BigDecimal amount;
}
```

【必读】时区要三处一起钉死

3.4 反复讲"只配 Jackson 会出事"，这里把三处凑齐并给出能一次验收的验证。差 8 小时的现场，几乎都是三处里有一处用了 UTC：

环节	决定什么	配置项	配错的典型表现
JVM	`new Date()`、`LocalDateTime.now()`、日志时间戳	`-Duser.timezone=Asia/Shanghai` 或容器 `TZ`	日志时间对、写库差 8 小时
JDBC 连接串	驱动如何解释 `TIMESTAMP`、如何写 Java 时间	`connectionTimeZone=Asia/Shanghai`	库里存的值差 8 小时
Jackson	接口 JSON 里 `Date` / `LocalDateTime` 的输出	`spring.jackson.time-zone` + `date-format`	接口返回差 8 小时

完整的一套（Boot 3 + MySQL 8）：

```
spring:
  jackson:
    time-zone: Asia/Shanghai
    date-format: yyyy-MM-dd HH:mm:ss
  datasource:
    url: jdbc:mysql://host:3306/db?
connectionTimeZone=Asia/Shanghai&characterEncoding=utf8mb4
    # Connector/J 8.0.23 之前用 serverTimezone=Asia/Shanghai（旧别名）；
    # 两个属性同时出现时以哪个为准，需在自己的驱动版本上验证
```

```
ENV TZ=Asia/Shanghai
# 或 java -Duser.timezone=Asia/Shanghai -jar app.jar，别依赖基础镜像默认值
```

数据库侧还有一条判据最容易被漏：**`TIMESTAMP` 会按连接时区转换，`DATETIME` 原样存取**。配错时区时 `TIMESTAMP` 字段差 8 小时而 `DATETIME` 正常——"部分字段差、部分正常"不是见鬼，是两种列类型混用。

```
SELECT @@global.time_zone, @@session.time_zone, @@system_time_zone;
SHOW VARIABLES LIKE 'wait_timeout';
```

一次验收，四个点必须同时对齐（照抄 3.4 那句"不是看代码，是打一条真实数据"）：

```
// ✓ 可运行：把四处打印到一条日志里，联调时贴给前端
@Test void timeZoneProbe() {
    System.out.println("JVM  = " + TimeZone.getDefault().getID());
    System.out.println("JSON = " + objectMapper.writeValueAsString(new Date()));
    System.out.println("JDBC = " + jdbcTemplate.queryForObject("SELECT NOW()",
String.class));
    System.out.println("UTC  = " + jdbcTemplate.queryForObject("SELECT
UTC_TIMESTAMP()", String.class));
    // 期望：JVM=Asia/Shanghai；JSON 与 NOW() 一致；UTC_TIMESTAMP 恰好早 8 小时
}
```

> 多数人以为"时区是一个配置项"，实际它是**四个独立进程各有一份默认值**：JVM 一份、驱动一份、MySQL 会话一份、Jackson 一份。任何一份没钉死，链路里就出现两个时间原点，而它们相处得还挺安静——没有异常，只有对账时对不上的 8 小时。

3.5　修法三：BigDecimal 的 0.10 与 0.1

金额差一分钱是怎么来的？一个很隐蔽的路径：

```
// 数据库里是 DECIMAL(10,2) = 0.10
BigDecimal amount = new BigDecimal("0.10");
objectMapper.writeValueAsString(amount);     // "0.1"（stripTrailingZeros 生效）
```

序列化成 `0.1` 本身数值没错，但如果这个字符串被拿去**拼接签名、做 Redis key、和另一端比对**，两端一个 `0.10` 一个 `0.1`，字符串比较直接判不相等。

对策：金额按字符串传，格式在契约里写死两位：

```
@JsonSerialize(using = ToStringSerializer.class)
@JsonFormat(shape = JsonFormat.Shape.STRING)
private BigDecimal amount;      // 契约：固定两位小数字符串，如 "0.10"
```

再补一条：**金额运算永远不要经过 double**。`new BigDecimal(0.1)` 会得到 `0.1000000000000000055511151231257827021181583404541015625`，必须用字符串构造或 `BigDecimal.valueOf`。

【必读】0.10 变 0.1：元凶不是 stripTrailingZeros

3.5 把 `0.10 → 0.1` 归因于 `stripTrailingZeros`，这个结论要修正——**多数人以为金额变形是后端序列化时把尾零去掉了，实际是 JSON `number` 这个类型根本没有"标度"概念。**

先看可运行的证伪：

```
node -e '
console.log("JSON.parse(0.10) =", JSON.parse("0.10"));       // 0.1
console.log("typeof =", typeof JSON.parse("0.10"));           // number
console.log("JSON.stringify(0.10) =", JSON.stringify(0.10));  // 0.1
'
```

`JSON.parse("0.10")` 得到的是 double `0.1`：JSON 的 `number` 只表示数值，`0.10` 与 `0.1` 是同一个数，尾零在解析那一刻就丢了。所以"库里 0.10、接口 0.10、前端 0.1"这条链路里，**后端每一步都没错，错在用了 number 传金额**。拿这个字符串去拼签名、做 Redis key、和另一端比对，两端一个 `0.10` 一个 `0.1`，自然判不相等（3.5 描述的症状是真的，归因错了）。

`stripTrailingZeros` 只在业务代码或某些 ORM 转换显式调用它时才参与，可以用一段 JUnit 直接证伪：

```
@Test void bigDecimalProbe() throws Exception {
    BigDecimal a = new BigDecimal("0.10");
    System.out.println("toString = " + a.toString());                    //
0.10
    System.out.println("Jackson  = " + mapper.writeValueAsString(a));       //
多为 0.10
    System.out.println("strip    = " + a.stripTrailingZeros().toPlainString());
// 0.1
    // 结论：只有主动 strip 才变形；JS 侧是解析即变形
}
```

所以对策不是"小心别 strip"，而是**金额一律走字符串**，把标度放进契约：

```
// ✗ 用 number 传金额：标度在 JSON.parse 那一刻就没了，且 JS 是 double
private BigDecimal amount;

// ✓ 字符串 + 固定两位，标度由契约保证，不由类型保证
@JsonSerialize(using = ToStringSerializer.class)
@JsonFormat(shape = JsonFormat.Shape.STRING)
private BigDecimal amount;      // 契约："0.10"，固定两位小数
```

（"金额运算永远不要经过 double"的量化佐证，及它与 3.3 的 2^53 同构关系，见章末延伸。）

3.6　修法四：Feign 的两个静默丢参

序列化问题在 Feign 上还有一个变体，症状是"参数没了但没报错"：

```
// ✗ GET 请求的 POJO 参数被当成 body
@GetMapping("/orders")
List<Order> list(OrderQuery query);      // Feign 把它序列化成 JSON body
                                          // 而 GET 带 body，多数服务端直接忽略

// ✗ 基本类型没写 @RequestParam，靠编译保留参数名
@GetMapping("/orders")
List<Order> list(Long tenantId);         // 没开 -parameters → 报错；开了 → 也可能对不上

// ✓ 显式标注
@GetMapping("/orders")
List<Order> list(@SpringQueryMap OrderQuery query,
                 @RequestParam("tenantId") Long tenantId);
```

这类问题的排查成本特别高，因为**服务端收到的参数是 `null`，而服务端代码是"参数为空就查出全部"**——于是它返回了所有租户的数据。看起来是"查询不对"，实际是"参数没传过来"。

【必读】Feign 丢参的可运行复现

3.6 说"服务端收到 null 而代码里是『参数为空就查出全部』"，这段给一个不额外引测试桩（WireMock / MockWebServer）的复现，用 JDK 内置 `HttpServer` 记下服务端**实际收到**的请求行，把"静默"变成断言失败：

```
// ✓ 可运行复现：JDK 自带 HttpServer，无需额外测试桩
public class FeignParamProbe {
    static volatile String receivedQuery = null;

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(0), 0);
        server.createContext("/orders", ex -> {
            receivedQuery = ex.getRequestURI().getRawQuery();   // 服务端真正收到
的 query
            byte[] body = "[]".getBytes(StandardCharsets.UTF_8);
            ex.sendResponseHeaders(200, body.length);
            ex.getResponseBody().write(body);
            ex.close();
        });
        server.start();
        String base = "http://127.0.0.1:" + server.getAddress().getPort();

        OrderClient c = Feign.builder().contract(new SpringMvcContract())
                             .target(OrderClient.class, base);
        c.listBad(new OrderQuery(42L));     // ✗ POJO 未标 @SpringQueryMap
        System.out.println("✗ 错误写法收到的 query = " + receivedQuery);
        // 期望 null：GET 带 body，服务端压根没收到 query string
        server.stop(0);
    }

    interface OrderClient {
        @GetMapping("/orders")
        List<Order> listBad(OrderQuery query);                  // ✗ 会被当 body

        @GetMapping("/orders")
        List<Order> listOk(@SpringQueryMap OrderQuery query);  // ✓ 展开成 query
string
    }
}
```

把期望写进断言，它就是一条会失败的测试，而不是一段"据说会丢参"的知识：

```
// ✓ 让契约变成断言：跑不通就说明参数没过去
assertNull(receivedQuery, "未标 @SpringQueryMap 时 GET 参数确实丢了");
// 换成 listOk 再跑一次：assertTrue(receivedQuery.contains("tenantId=42"));
```

两个变体一起验才完整：`@RequestParam("tenantId") Long tenantId` 不写参数名、且编译没开 `-parameters` 时，Feign 抛契约异常（**这个是响的，反而好查**）；真正危险的是 POJO 那条——**它不抛异常，只是把参数吞了**，服务端看到 `null`，正好撞上"参数为空就查出全部"的默认分支，于是返回了所有租户的数据。第 1 章 1.3 那张"静默改语义"表里，Interceptor `return false` 和这一类同源：**框架按契约工作了，只是那个契约不是你以为的那个。**

> 排查这类问题的成本全花在"参数明明传了"上。可运行复现的价值，是把"我以为传了"和"服务端收到了"摆在同一行输出里对比一次。

3.7　接口契约：四个必须写死的类型

把这一节直接抄进团队的接口规范：

> 1. 所有 `id` 类字段（`Long`）—— 传输为字符串
> 2. 所有金额字段（`BigDecimal`）—— 传输为固定两位小数的字符串
> 3. 所有时间字段 —— 传输为 `"yyyy-MM-dd HH:mm:ss"`，且时区固定 `GMT+8`
> 4. 所有枚举字段 —— 传输为 `name`（字符串），不用 `ordinal`

为什么是这四条？因为它们是**唯一会"静默变形"的四种类型**。String、int（小）、boolean 在跨语言传输中不会悄悄变样，而这四种会。

3.8　自检清单

- [ ] 全局配了 `Long → String`，且包含 `Long.TYPE`
- [ ] 所有 VO 的 `LocalDateTime` 都有 `@JsonFormat`（或全局配置了 `JavaTimeModule` 格式）
- [ ] 金额字段按字符串传输，两位小数
- [ ] `spring.jackson.time-zone`、JDBC `serverTimezone`、容器 `TZ` 三处一致
- [ ] Feign 的 GET 接口全部用 `@SpringQueryMap` / `@RequestParam` 显式标注
- [ ] 联调必测项：19 位 id 端到端走一遍"展示 → 回查 → 更新"
- [ ] 联调必测项：时间字段在服务器 UTC 环境下验证一次
- [ ] 枚举用 `@JsonValue` 明确指定序列化形式

3.9　一句话总结

> 序列化的问题从来不是"格式不对"，是**"它在你不看的地方变了一点，而你没有任何异常可以抓"**。 对付这类问题，唯一的办法是把契约写死，然后端到端测一遍。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/07-网络与接口坑、03-Web开发基础/03-RESTful API设计
- 【选读】**后端侧也有一个同款精度坑**：`new BigDecimal(0.1)` 会得到 `0.1000000000000000055511151231257827021181583404541015625`（double 0.1 的精确二进制值），必须用字符串构造或 `BigDecimal.valueOf`。这条与 3.3 的 2^53 是同一套 IEEE 754 机制，只是一个丢精度、一个丢标度。

第四章　连接池耗尽：堵死的不是数据库

本章要回答：为什么下游监控显示 20ms，我的接口却是 3 秒？

本章你能带走

- 一个数字：`HttpClient` 默认单路由连接数是 **2**
- 一个估算公式：`perRoute ≥ QPS × RT(秒) × 1.5`
- 一条认知：**"排队时间"不出现在任何下游的监控里，所以它永远是最先被忽略的那一段**

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

4.1　现场：下游很快，我很慢

现象非常标准，几乎每个团队都遇过一次：

- 订单服务调用库存服务的接口，P99 从 80ms 涨到 2.8s
- 库存服务自己的监控：RT 30ms，CPU 30%，没有慢 SQL
- 订单服务日志：大量 `Timeout waiting for connection from pool`
- 中间件团队说"我们没问题"，业务团队说"我们也没问题"

两边都对。请求根本没发出去。

`Timeout waiting for connection from pool` 这句话的字面意思就是：**我在等一个连接，没等到**。所谓"调用下游花了 2.8 秒"，其中 2.7 秒是在自己进程的池子里排队。

> 记住这个判据：**如果下游的 RT 监控正常，而调用方的 RT 异常，先怀疑连接池，不要怀疑网络。**

【必读】两边监控都正常的那 2.7 秒去哪了

4.1 说"两边都对，请求根本没发出去"，这句值得再拆一层，因为它是全书复用最多的一类误判。把那 2.8 秒按归属切开：

时间段	花在哪	谁的监控里有	谁会为它负责
0–2.7s	在调用方进程里排队等池连接	只有调用方的 `pending` 指标	没人（默认不采集）
2.7–2.8s	网络往返 + 下游处理	下游 RT、调用方 HTTP 指标	下游团队

下游团队看自己的 RT 是 30ms，是对的；调用方看接口 P99 是 2.8s，也是对的。**2.7 秒的排队发生在请求离开调用方进程之前，它既不算"下游耗时"，也不算"网络耗时"，更不算"应用处理"**——所以在下游的监控面板上恒等于 0，在调用方的接口总耗时里却占了大头。第一章 1.2 那个 `$request_time - $upstream_response_time` 判据讲的是同一件事，只是排队发生在 Nginx 而不是 HttpClient 池：

```
排队时间 = 调用方总 RT − 下游 RT − 网络 RTT
         = 2800ms − 30ms − 约 1ms ≈ 2769ms
```

① 一个立刻能用的判据：如果下游 RT 监控正常、而调用方 RT 异常，**先怀疑连接池，不要怀疑网络**。反过来，若调用方与下游 RT 一起涨，才轮到查下游或网络。这条判据不需要任何新工具，只需要把两处的 P99 摆在同一张图上相减——和第七章 7.6 用"事务 span 总耗时 − SQL span 之和"定位事务里的远程调用，是同一个减法。

② 为什么"排队"永远是最先被忽略的一段。 因为它不产生日志、不产生异常、不消耗 CPU，只消耗"等待"。多数监控体系采集的是**事件**（一次 SQL、一次 HTTP、一次 GC），而排队是"什么都没发生"的一段时间；没有事件，就没有 span，没有 span，默认视图里就看不到。要让它显形，只能主动埋两类数据：池子的 `pending` 计数（4.6）和 `acquire` 耗时 P99。**pending > 0 是比接口 P99 早 1~3 分钟的信号**，因为排队总是先发生、后传导到 RT。

③ 一个反常识点：调大池子往往不是最优解，甚至不是正确解。 多数人看到"等连接"就把 `maxTotal` 从 20 调到 200，症状确实缓解了，但 4.3 那三个上限会依次找上门：下游的线程池、数据库的 `max_connections`、以及每个连接占用的内存。更关键的是，**如果池子是被"事务里做远程调用"占满的（4.5、第七章），调大池子只是让更多请求一起挤在下游的慢接口上**，故障从"调用方排队"变成"下游被打挂"。这类场景真正的解法是缩短每个请求占用连接的时间，而不是增加连接数量。判别方法也简单：看 `active` 是随流量起伏（容量不足）还是持续顶满不回落（泄漏 / 长事务占用，4.6 的五步取证）。

④ 现场的最小修复与验收。那次最终改了三处：`defaultMaxPerRoute` 按 `500 × 0.2 × 1.5 = 150` 重算、加 `validateAfterInactivity` 防半开连接、把 `pending` 指标接进告警。压测复现时用恒定速率（4.7 的 wrk2 命令）再打一遍，`pending` 归零、P99 从 2.8s 回到 90ms。**修复只动了配置，但因为有了 `pending` 这个观测点，下一次池子再满时不用再靠两边团队互相举证。**

> 接口慢只有三种可能：算得慢、等得慢、走得慢。连接池属于"等"，而"等"从来不出现在下游的监控面板上——这句话的具体含义，就是上面那张表里"谁会为它负责"那一列写着"没人"。

当时怎么误判的：第一反应是"下游库存服务慢了/挂了"——可库存服务自己的监控 RT 30ms、CPU 30%，活得好好的；又去查网络，ping 和 RTT 都正常，也排除了。走了条死路：有人一头扎进库存服务的慢 SQL 和 GC，想把"下游"治好，方向全反了；还有人抓了订单服务的 jstack，却只盯着"有没有死锁"看，没注意大批线程都停在 `ConcurrentBag.borrow` 等连接，把真线索放过去了。真正扭转方向的是订单服务日志里那行 `Timeout waiting for connection from pool`，再对着池子的 `pending` 指标一看——2.7 秒全在调用方自己进程里排队，根本没发出去。教训：下游监控正常、调用方却异常时，先怀疑连接池，别怀疑网络。

4.2　罪魁祸首：1990 年代的默认值

```
// Apache HttpClient 的默认值是给示例代码用的
CloseableHttpClient client = HttpClients.createDefault();
// 等价于：
// maxTotal = 20
// defaultMaxPerRoute = 2      ← 单下游只能同时 2 个连接
```

`defaultMaxPerRoute = 2` 意味着：对同一个下游，同一时刻最多 2 个请求在跑，其余全部排队。

算一下：下游 RT 200ms，2 个连接 → 理论吞吐 **10 QPS**。而你的接口有 500 QPS。剩下的 490 QPS 全在排队，排 49 秒——实际上早就超时了。

同类默认值还有：

客户端	默认	问题
Apache HttpClient 4	`maxTotal=20`、`perRoute=2`	单下游 2 并发
OkHttp	`maxIdleConnections=5`、`keepAlive=5min`	并发高时频繁建连
RestTemplate（Simple）	无连接池，每请求新建连接	无复用，TIME_WAIT 堆积
HikariCP	`maximumPoolSize=10`	对高并发服务偏小（但默认值本身合理）
Tomcat	`maxThreads=200`、`acceptCount=100`	200 线程打满后请求在 acceptCount 排队

4.3　池子该多大：一个能算的公式

> 单路由连接数 ≥ 目标 QPS × 下游 RT(秒) × 安全系数(1.5)

例：500 QPS × 0.2s × 1.5 = **150**。

这是利特尔法则（Little's Law）的直接应用：`并发数 = 到达率 × 停留时间`，写成标准式就是 `L = λ × W`。本书把 Little's law 的"主场"定在本节，符号统一如下，其它章只引用、不重推：

符号	含义	在本章
`L`	系统内平均在途/并发数	需要的连接数
`λ`（lambda）	平均到达率（如 QPS）	请求到达的速率
`W`	平均停留/服务时间	单次请求占用连接的时长

连接数就是 `L`。同一个 `L = λ × W` 会反复出现：线程池队列长度（第 9 章）、锁等待占用的连接与线程（第 14 章）、雪崩放大（第 2 章）、GC 停顿期间的请求堆积（第 25 章）——公式与符号一律以本节为准。连接数就是"并发数"。连接数就是"并发数"。

```
// ✓ 按公式估算，并暴露监控
PoolingHttpClientConnectionManager cm = new
PoolingHttpClientConnectionManager();
cm.setMaxTotal(300);                          // 所有下游加起来
cm.setDefaultMaxPerRoute(150);                // 单个下游
cm.setValidateAfterInactivity(TimeValue.ofSeconds(2));   // 复用前校验，防半开连接

CloseableHttpClient client = HttpClients.custom()
        .setConnectionManager(cm)
        .evictExpiredConnections()
        .evictIdleConnections(TimeValue.ofSeconds(30))
        .build();
```

`setValidateAfterInactivity` 这一行不能省。下游重启、LB 悄悄掐掉空闲连接之后，池子里会留下"看起来活着、实际已死"的连接。第一次用它就报 `Connection reset`——而这类错误是**偶发**的，最容易被当成"网络抖动"糊过去。

【必读】但池子不是越大越好

三个上限必须一起看：

1. **下游的容量**。给 10 个调用方各配 300 连接，下游就要承受 3000 并发——它可能只有 200 个线程。**你只是把排队从自己进程挪到了下游。**
2. **数据库的连接上限**。MySQL `max_connections` 默认 151。10 个实例各配 50 = 500，直接打满。
3. **内存**。每个连接有缓冲区，几百个连接不算多，但也没必要。

> 池大小是一个**跨团队的约定**，不是单方面的调参。定下来要写进容量文档：这个服务对下游的最大并发是多少。

【进阶】Hikari 池容量公式：从 Little's law 推到默认值 10

4.3 给了 HTTP 侧的 `QPS × RT × 1.5`，这一节补数据库侧。HikariCP 官方 wiki 的"About Pool Sizing"给了一条经验公式：

```
connections = (core_count × 2) + effective_spindle_count
```

它不是拍脑袋，可以从"一次服务里 CPU 干活与等 IO 的比例"推出来。设一次 SQL 的服务时间 `W = C + I`（C 是 CPU 时间，I 是等磁盘/网络的时间），单核要跑满需要同时有 `W / C = 1 + I/C` 个连接在工作；N 核就是：

```
L ≥ N × (1 + I/C)
```

机械盘时代 `I/C` 很大，经验上约等于 `N × 2 + 磁盘可并行数`（每个 spindle 同时只能服务一个 IO）。代进现场那台 4 核、SSD 的机器（`effective_spindle_count ≈ 1`）：

```
L = 4 × 2 + 1 = 9  →  取 10
```

这就是 `maximumPoolSize` 默认值 10 的由来——多数人以为它是"给小项目随便定的保守值"，实际它是按当年一台 4 核机器的磁盘并行度算出来的。 换句话说，默认值不是让你照抄的，是让你按自己机器的 `N` 和 `I/C` 重算的。

另一条更贴合业务的推法用 Little's law：`L = λ × W`。目标 1000 QPS、单事务内 SQL 累计 `W = 8ms`：

```
L = 1000 × 0.008 = 8 个连接（纯数据库时间）
+ 事务里其它开销、峰值余量 ×1.5  →  12~15
```

两条路殊途同归：**池子大小 = 并发数，而并发数由"到达率 × 服务时间"决定，与"我想开多少"无关。** 4.3 那个 HTTP 公式 `QPS × RT × 1.5` 其实是同一个 `L = λ × W` 换了个名字。

实测方法比公式更重要，因为 `I/C` 你测不准：

```
# ① 阶梯调池：maximumPoolSize 依次取 10/20/40/80，每档恒定速率压 3 分钟
for p in 10 20 40 80; do
  echo "=== pool=$p ==="
  wrk2 -t8 -c200 -d180s -R1000 --latency http://localhost:8080/api/order | tee
"pool_$p.log"
  curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.pending >>
"pool_$p.log"
done
```

判据写在曲线上，不写在配置里：

观察	结论
`pending > 0` 且 P99 随池增大而下降	池子确实不够，继续加
加池后 QPS 不再涨、P99 反而升	已过拐点，瓶颈转到下游（下游 `Threads_running` / 锁竞争）
`active` 持续等于 max 且不回落	**不是容量不足，是泄漏**（4.4、下一节取证）
`hikaricp.connections.timeout > 0`	已经有人在报障了

池子不是越大越好，这句能算：10 个实例各配 80 连接 = 800，MySQL `max_connections` 默认 151，直接打满；就算调大上限，InnoDB 并发线程、锁竞争、上下文切换也会让 QPS 在某个点后掉头向下。**池大小是跨团队的约定，不是单方面调参**（4.3 已写过，这里给出它的量化上限）。

4.4　另一条路径：连接泄漏

池子够大了还是耗尽？那是漏了。

```
// ✗ 拿了不还
CloseableHttpResponse resp = client.execute(req);
String body = EntityUtils.toString(resp.getEntity());
// resp 没 close → 连接被这个响应对象一直持有

// ✓ try-with-resources
try (CloseableHttpResponse resp = client.execute(req)) {
    return EntityUtils.toString(resp.getEntity());
}
```

数据库侧同理，但 Spring 帮我挡了大部分（`@Transactional` 会正确释放）。真正容易漏的是**手动 `getConnection()` 和流式查询未关闭**：

```
// ✗ 手动拿连接，异常路径漏 close
Connection c = dataSource.getConnection();
Statement s = c.createStatement();
ResultSet rs = s.executeQuery("select ...");
// 抛异常 → 三个都没关 → 连接永久占用
```

判断是否泄漏，看两个指标：**活跃连接数持续等于 max，且不回落；租借时长 P99 持续上涨**。有这两个就是泄漏，不是容量不足。

【必读】四个 timeout 各管一段

4.4 讲"借连接要等多久"，但 Hikari 里有四个名字都带 timeout 的配置，管的完全不是一回事。调错一个，症状会变得更难懂（第七章 7.4 第四步列的是**跨组件**的四个超时，这一节是**同一个池内部**的四个）：

配置	默认值	谁在等	超时后发生什么	约束
`connectionTimeout`	30000ms	业务线程等池里空出连接	抛 `SQLTransientConnectionException`，**根本没进事务**	最小 250ms
`validationTimeout`	5000ms	借出前探活连接	判定不可用，丢弃再借	必须 < `connectionTime`
`idleTimeout`	600000ms	空闲连接留多久	回收超出 `minimumIdle` 的空闲连接	仅当 `minimumIdl` < `maximumPoolS` 生效
`maxLifetime`	1800000ms	一条连接无论多忙能活多久	退役，关闭重建	必须 < 链路里最小的空闲断连阈值

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 20
      minimum-idle: 10                  # < max，idleTimeout 才生效
      connection-timeout: 3000          # 3s 借不到就失败，别让线程干等 30s
      validation-timeout: 1000          # 必须 < connection-timeout
      idle-timeout: 600000
      max-lifetime: 1740000             # 29min，见下方 wait_timeout 的坑
      keepalive-time: 120000            # 4.x+，定期探活，防中间设备静默掐连接
      leak-detection-threshold: 2000    # 下一节的取证开关，ms
```

几个必须知道的约束与反常识点：

- `connectionTimeout` **超时抛的是** `SQLTransientConnectionException`**，此时事务一次都没开。** 把它和 `@Transactional(timeout=3)` 混为一谈，会去查"事务为什么超时"而永远查不到——第七章 7.4 那张表里，这一行正是"根本没进事务"。
- `idleTimeout` **不是"连接空闲 10 分钟就关"。** 默认 `minimumIdle = maximumPoolSize`，池子从不缩容，这个配置**默认不生效**。只有显式把 `minimumIdle` 调小，它才开始工作。
- `maxLifetime` **要小于链路里最短的那个空闲超时。** MySQL `wait_timeout` 默认 28800 秒（8 小时），但云厂商 LB / 防火墙常把空闲连接 350 秒掐掉，而默认 `maxLifetime` 是 30 分钟——连接会"看起来活着、实际已死"，复用时偶发 `Connection reset`。这正是 4.3 强调 `validateAfterInactivity` 的数据库版。

```
# 取证：把两处超时摆在一起，取最小值定 maxLifetime
mysql -e "SHOW VARIABLES LIKE 'wait_timeout'; SHOW VARIABLES LIKE
'interactive_timeout'"
# 云厂商 SLB / NAT 空闲断连阈值 → 问网络组，常见 350s / 900s
```

> 多数人以为"超时配置就是别让请求等太久"，实际池里这四个 timeout 分属四个完全不同的时机：**借连接、探活、回收、退役**。把它们当同一个旋钮调，最常见的后果是 `validationTimeout > connectionTimeout`，Hikari 启动直接抛 `IllegalArgumentException` 拒绝配置——好在这是少数会明确报错、而不是静默出错的地方。

4.5　第三类：连接被"事务"占着不放

这类最隐蔽，因为池子配置完全正确。

```
@Transactional(rollbackFor = Exception.class)
public void create(Order order) {
    orderMapper.insert(order);                    // ① 拿到连接，事务开启
    inventoryClient.deduct(order.getSku());       // ② HTTP 调用，可能 3 秒
    orderItemMapper.insertBatch(order.getItems()); // ③ 还用同一个连接
}
```

第 ② 步的三秒里，**数据库连接一直被第 ① 步占着，行锁也一直没释放**。

后果是双重的：

- 数据库连接池被"慢业务"占满 → 其他请求拿不到连接
- 行锁持有 3 秒 → 其他事务在这行上等锁 → 连锁排队

这就是第六章讲事务时反复强调的那句：**事务里不要做远程调用**。它不是"规范洁癖"，它直接决定连接池和行锁的吞吐。

【必读】连接泄漏的取证命令序列

池子配置全对、`pending` 却持续大于 0，就是漏了。别先调大池子——那只是把泄漏爆发的时间往后推。按下面五步走，从"现象"到"哪一行代码没还连接"，每步两分钟：

① 先开泄漏检测，让池子自己报出借连接的代码位置

```
spring.datasource.hikari.leak-detection-threshold: 2000   # ms，生产可开，开销很小
```

超过 2 秒未归还，Hikari 打一条 WARN：`Connection leak detection triggered for ... on thread http-nio-8080-exec-7`，并附带**借连接时的堆栈**。这条堆栈直接指向 `getConnection()` 的调用点，是整条取证链里信息量最大的一行。注意边界：最小值 2000ms，且不能大于 `maxLifetime`。

② 看指标，确认是"不回落"而不是"容量不足"

```
curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.active
curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.pending
curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.acquire
curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.timeout
# 判据：active 持续 == maximumPoolSize 且不回落 = 泄漏；
#       随流量起伏 = 容量不足。timeout > 0 说明已经有人在报障了
```

③ jstack 看线程停在哪儿

```
jstack -l <pid> | grep -A 20 'HikariPool\|getConnection' | head -80
# 大量线程停在 ConcurrentBag.borrow / LockSupport.park → 在等连接
# 泄漏线程则停在业务方法（借了没还的地方），对照 ① 的堆栈找同一帧
```

④ 数据库侧交叉验证

```
SHOW PROCESSLIST;                          -- 大量 Sleep 且 Time 很长 = 借了没还/长事务
SELECT trx_id, trx_started, trx_mysql_thread_id, trx_query
  FROM information_schema.innodb_trx ORDER BY trx_started;  -- 老事务 = 4.5 那类
```

⑤ 定位后回到代码：手动 `getConnection()` 的异常路径、流式查询未关 `ResultSet`、`@Transactional` 里做了远程调用（4.5）。前三步是"确认漏了"，第四五步是"漏在哪"。**顺序反过来先抓 dump，会得到 200 个一模一样的 `WAITING`，看不出谁先谁后**（第七章 7.6 的取证顺序是同一条规矩）。

【必读】交叉引用：事务占连接对应第七章

4.5 结尾写"这就是第六章讲事务时反复强调的那句"，这里的章号需要修正：第六章（v2-06）讲的是**事务为什么没生效**（自调用、异常没逃出、传播行为），而"事务里不要做远程调用、行锁与连接被持有到远程返回"是**第七章**（v2-07）整章的主题，7.3 ③ 更是直接点名"先满的往往不是线程池，是连接池"。本书的交叉引用一律按卷二的实际章号走：

现象	该翻哪一章
连接池被"慢业务"占满	**第七章** 7.2–7.4（事务边界 / 远程调用移出事务）
事务压根没开	第六章（代理、五问定位法）
多个事务互相等锁	第十四章（死锁与锁等待）
池子容量与排队	本章 4.3、第一章 1.2（`$request_time` 差值）

一处章号的误差看着很小，但排障时按它去翻第六章，会花半小时看"自调用与代理"，而真正要找的"把 HTTP 调用挪出事务"在第七章。**交叉引用不是装饰，它是排查路径的一部分**——这也是全书坚持用"第 N 章"把 27 章串起来的原因。

4.6　监控：三个必须暴露的指标

池子出事前一定有征兆，前提是你看了：

```
// HttpClient: 暴露到 Micrometer
Metrics.gauge("http.pool.leased", cm, m -> m.getTotalStats().getLeased());
Metrics.gauge("http.pool.pending", cm, m -> m.getTotalStats().getPending());
Metrics.gauge("http.pool.available", cm, m -> m.getTotalStats().getAvailable());
```

指标	含义	告警阈值
`leased`	已借出连接数	持续 > 80% max
`pending`	**在排队等连接的请求数**	**> 0 持续 1 分钟**
`available`	空闲连接	持续 = 0
`acquire 耗时 P99`（HikariCP `connectionAcquireMillis`）	拿连接花了多久	> 50ms

`pending > 0` 是最有价值的信号——它意味着**已经有请求在排队了**，此时接口还没明显变慢，但再涨一点就雪崩。这个指标通常比接口 P99 早 1–3 分钟报警。

HikariCP 的指标名要记一下，因为它是 Spring Boot 默认连接池：

```
hikaricp.connections.active
hikaricp.connections.pending        ← 排队数
hikaricp.connections.acquire        ← 获取耗时
hikaricp.connections.timeout        ← 超时次数，> 0 就是事故
```

`hikaricp.connections.timeout > 0` 基本等同于"已经有人在报障了"。

【必读】用阶梯压测实测池容量（可运行）

公式给的是起点，验收要靠压测。把 4.4 的公式、4.6 的指标和第十二章压测篇的恒定速率模式合成一条能跑的命令序列：

```
# ① 压测前先把指标看板打开（/actuator/prometheus 盯
active/pending/acquire/timeout）
# ② 恒定速率梯度加压：R 固定、只升并发，干净地看出"排队从哪档开始"
for r in 200 400 800 1200; do
  echo "=== R=$r ==="
  wrk2 -t8 -c200 -d180s -R$r --latency http://localhost:8080/api/order | tee
"r_$r.log"
  curl -s http://localhost:8080/actuator/metrics/hikaricp.connections.pending >>
"r_$r.log"
  sleep 30        # 冷却，避免上一档的排队污染下一档
done
```

为什么必须用 `-R` 恒定速率：wrk 默认"发完立刻发下一个"，服务变慢时发压速率自动下降，排队被客户端自己怂掉掩盖（Coordinated Omission），测出来的 P99 偏乐观。恒定 RPS 才能模拟"真实流量不变、服务变慢"的容量语义。

判据（把 4.4 那张表写成验收条件）：

观察	结论	动作
`pending` 从 0 变正的第一档 R	池容量拐点	按 `L = λ × W` 重算 `maximumPoolSize`
加池后 P99 继续降、QPS 继续涨	还没到拐点	继续
加池后 QPS 平台、下游 `Threads_running` 升高	瓶颈转到数据库	停手，别再加池
`active` 不回落、`timeout > 0`	泄漏不是容量	转 4.6 的取证序列

```
// ✓ 把"pending > 0 持续 1 分钟"做成会失败的告警，而不是人工看板
Gauge.builder("hikaricp.connections.pending", pool,
              p -> p.getHikariPoolMXBean().getThreadsAwaitingConnection())
     .register(registry);
// 告警：pending > 0 持续 1min → 早于接口 P99 变慢 1~3 分钟（4.6 已写过这条）
```

> 一次只改一个变量：调 `maximumPoolSize` 时别顺手改 `connectionTimeout`，否则 P99 的变化分不清是池子大了还是等得久了。这也是第八章 8.6 升级迁移清单里"先在旧版本暴露问题，再升级"的同一个方法论。

4.7　自检清单

- [ ] 所有 HTTP 客户端都用了连接池，没有用 `HttpClients.createDefault()` 裸奔
- [ ] `perRoute` 按 `QPS × RT × 1.5` 算过，不是抄的默认值
- [ ] 池大小与下游容量对齐，写进了容量文档
- [ ] 配了 `validateAfterInactivity` / `testOnBorrow`（防半开连接）
- [ ] 所有响应、连接、流都用 `try-with-resources`
- [ ] 事务方法内没有 HTTP/RPC 调用
- [ ] 暴露了 `leased / pending / acquire` 三个指标，`pending > 0` 有告警
- [ ] `hikaricp.connections.timeout` 有告警

4.8　一句话总结

> 接口慢，只有三种可能：**算得慢、等得慢、走得慢**。 连接池属于"等"，而"等"从来不出现在下游的监控面板上。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/07-网络与接口坑、17-常见Bug与解决方案/02-并发与线程池坑、05-数据库/01-MySQL/基础语法

第五章　网关与限流：全局限流为什么没限住

本章要回答：限流规则配了 1000 QPS，为什么峰值打进来 4000，服务还是被打挂？

本章你能带走

- 一个判断框架：限流要回答"限谁"和"在哪限"
- 一张表：单机限流 vs 集群限流的偏差有多大
- 一条最容易忽略的：**规则存在内存里，重启就没了**

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

5.1　现场：限流规则写了，流量一条没少

大促前的压测，运维在网关配了"单接口 1000 QPS"的限流。压测工具打 4000 QPS，预期放过去 1000、拒绝 3000。

实际：全部放过去了，服务被打挂。

三个问题依次暴露：

问题一：规则是单机的，不是全局的。 网关 4 个实例，每个实例独立计数 1000 QPS——**总放行量是 4000**。所谓"全局限流"，实际是"每台机器各限各的，然后再乘一个实例数"。

问题二：规则只在内存里。 配置是通过 Sentinel Dashboard 推的，没做持久化。网关重启过一次（滚动发布），规则丢了，谁也没发现——**Dashboard 上还显示着规则，但那是内存里的副本，重启即空**。

问题三：限流的位置选错了。 有的接口走了网关，有的走内部直连（服务间调用），还有的走了另一条路由前缀。限了主入口，漏了两个旁路。

> 限流失效的三种形态，本质是三个不同的问题：**维度错、位置错、状态错**。

【必读】压测现场：三个问题是怎么逐个显形的

5.1 的三个问题不是一次看出来的，是按"先量总量、再重启、最后核旁路"的顺序逼出来的，这个顺序本身就是可复用的排查路径：

① 先量总放行量，别看单机面板。 压测打 4000 QPS，网关每台实例的 Dashboard 都显示"1000 QPS，未超限"——四个面板各自都对，合起来是 4000。判据是**把 N 台机器的放行数求和，再和规则里那个数字比**，而不是逐台看"有没有超"。这一步直接暴露问题一（维度错）。

② 再滚动重启一台，看规则还在不在。 Dashboard 上规则一直亮着，但那是推送后的内存副本；重启一个实例后重新发压，该实例不再返回 429，总放行量反而上升。**验证限流是否持久化，唯一可靠的动作是重启，不是看控制台**（5.3）。这一步暴露问题二（状态错）。

③ 最后核路由清单，找 bypass。 把网关所有路由前缀、服务间直连的内部域名列一张表，逐条发一个超阈值的请求，看哪些返回了 200。现场漏掉的是一条 `/internal/**` 前缀和一个服务间直连域名。限流是"挡在最外层"，而旁路请求根本没经过最外层——这一步暴露问题三（位置错）。

> 三个问题对应三个不同的判据：求和、重启、清单。它们都不能靠"看一眼配置"完成，这也是为什么 5.7 要把阈值压测变成带期望值的断言——**配置过不等于验证过**。

当时怎么误判的：第一反应是"限流规则根本没配生效"——可 Sentinel Dashboard 上明明显示着 1000 QPS 的规则，又去怀疑"是不是压测工具统计口径错了"，两边看着都像。走了条死路：有人盯了整整一个下午的单台网关面板，每台都写着"1000 QPS、未超限"，四台各自都对，从没把放行数求和；还有人去查 Sentinel 的流控效果是不是被谁改成了"放行"，其实规则早就在一次滚动发布后丢进了内存，Dashboard 上还亮着的是副本。真正扭转方向的是把 N 台实例放行数一加（4×1000=4000），再滚动重启一台复现规则丢失——才看清是"维度错 + 状态错 + 位置错"三件事叠一起。教训：配置过不等于验证过，限流要验的是"流量真的被挡住"，不是"规则真的被显示"。

5.2　维度：单机限流会放大 N 倍

这是最容易犯、也最容易解释不清的错。

```
网关 4 实例，每实例限 1000 QPS
   ↓
集群总放行 = 4 × 1000 = 4000 QPS
   ↓
下游只扛得住 1200 QPS
   ↓
被打挂
```

两种解法，取舍不同：

方案	实现	精度	代价
单机限流（均分）	每实例阈值 = 总数 / 实例数	粗，实例数变化需重配	无额外依赖
集群限流	Redis + Lua 原子计数，或 Sentinel 集群流控	准	多一次 Redis 往返；Redis 本身成为依赖

单机限流的隐藏坑：**流量不均**。K8s 里 4 个 Pod，负载均衡不是绝对均匀的，某台机器可能拿到 40% 的流量，于是它在总流量 40% 的时候就开始拒绝了，而实际集群只用了 40% 的容量。

所以正确做法是：

- **入口按接口/租户做集群限流**（总量要准）
- **应用内按资源（线程池、DB 连接）做单机限流**（保护自己就够，不用准）

【必读】单机限流放大 N 倍的完整算式

5.2 那个 `4 × 1000 = 4000` 是最简情形，完整算式还要算上"流量不均"和"实例数会变"。设集群有 N 个实例、每实例阈值 `q`，单实例限流下：

```
集群实际放行上限 = N × q
```

代入现场：N=4、q=1000 → 4000 QPS，而下游只扛 1200，超了 3.3 倍。这只是"均分且均匀"的理想值，真实 K8s 里负载均衡不是绝对均匀。设第 i 个实例分到的流量比例 `p_i`（`Σp_i = 1`），它会在集群总流量达到 `q / p_i` 时就开始拒绝，于是：

```
集群有效容量 = q / max(p_i)
```

4 个 Pod、最忙那台拿到 35%：`1000 / 0.35 ≈ 2857 QPS`，比理想的 4000 少 28%。更糟的是**误拒**：若按"总数/N"均分改成每实例 250，最忙那台在总流量 `250 / 0.35 ≈ 714 QPS` 就开始返回 429，而此时集群才用了 `714 / 1200 ≈ 60%` 的下游容量——**限流比下游先满，白白丢了 40% 的吞吐**。

还有一层最容易被忽略的漂移：**N 不是常数**。滚动发布时实例从 4 扩到 6，若阈值仍写在每台机器各自的配置里，总放行量从 4000 悄悄变成 6000；缩容到 3 又变成 3000。判据因此可以写死：

> 只要"限流阈值"这个数字出现在**每个实例各自的配置文件**里，它就不是全局限流，而是一个会随发布次数漂移的量。

把三种偏差摆在一起：

偏差来源	算式	现场数字	后果
实例数放大	`N × q`	4 × 1000 = 4000	超下游 3.3 倍
流量不均	`q / max(p_i)`	1000 / 0.35 ≈ 2857	集群有效容量缩水 28%
扩缩容漂移	`N(t) × q`	4→6 实例：4000→6000	阈值随发布变化

正确做法 5.2 已给：**入口按接口/租户做集群限流（总量要准），应用内按资源做单机限流（保护自己就够，不用准）**。集群限流用 Redis + Lua 原子计数（5.4 会给 Lua），把 `N × q` 里的 N 从算式里彻底消掉。

5.3　状态：规则不在配置中心，就是在眼泪里

Sentinel 的规则默认**只存在客户端内存**。Dashboard 推送的是运行时状态，重启即失效。这是最高频的"限流突然失效"原因。

必须做持久化，三种方式：

```
// ① 推模式（推荐）：Nacos / Apollo 作为规则源，客户端监听变更
@Configuration
public class SentinelNacosConfig {

    @PostConstruct
    public void init() throws Exception {
        ReadableDataSource<String, List<FlowRule>> ds = new NacosDataSource<>(
                properties.getServerAddr(), GROUP_ID, DATA_ID,
                source -> JSON.parseObject(source, new
TypeReference<List<FlowRule>>() {}));
        FlowRuleManager.register2Property(ds.getProperty());
    }
}
```

```
② 拉模式：客户端定时轮询配置中心（简单，有延迟）
③ 本地文件：规则写文件，重启加载（适合静态规则，不能动态调）
```

判断是否踩了这个坑，有个简单方法：**重启一个实例，看限流还生不生效。** 如果重启后规则消失而没人察觉，说明你从来没有真正验证过限流——只是"配置过"。

【必读】四种算法的判据对照

5.2 说"滑动窗口 / 令牌桶 / 漏桶"，但没给怎么选。把四种算法横着摆开，每格给判据而不是定义：

算法	临界突刺	允许突发	突发容量	存储/依赖	典型实现	什么时候选它
固定窗口	✗ 有（临界 2 倍）	有限	窗口内全额	1 个计数器	`INCR + EXPIRE`	内部低风险接口，能容忍边界突刺
滑动窗口	✓ 无	有限	窗口内全额	ZSET 每请求一条记录，内存高	Redis ZSET + Lua、Sentinel `LeapArray`	要精确、请求量不大
令牌桶	✓ 无	✓ 明确支持	`burstCapacity`	2 个字段 + Lua	Gateway `RedisRateLimiter`	**网关限流主流**，兼顾平滑与突发
漏桶	✓ 无	✗ 不允许	队列深度	队列	匀速排队、消息消费	"可以慢但不能失败"，削峰填谷

固定窗口的"临界双倍"必须算一次才知道有多危险：限 100 次/秒，在第 59.9 秒打满 100 次、第 00.1 秒又打满 100 次，`0.2` 秒内放进 200 次，**瞬时速率是阈值的 10 倍**（200 / 0.2s = 1000/s）。若下游按 100 QPS 配容量，这两波就能把它打穿。滑动窗口用 ZSET 删掉窗口外记录解决了它，代价是每请求一次写、内存随窗口内请求数线性增长；Sentinel 的 `LeapArray` 用环形数组复用桶（默认 1 秒切 2 个 500ms 桶），是精度与内存的折中。

令牌桶的两个参数必须一起配，配错比不配更隐蔽：

```
# ✓ Gateway 令牌桶：桶容量 ≥ 补充速率，经验值 1.5~2 倍
spring.cloud.gateway.routes[0].filters[0].args:
  redis-rate-limiter.replenishRate: 1000     # 每秒放 1000 个令牌（≈ 压测上限 × 0.7）
  redis-rate-limiter.burstCapacity: 2000     # 桶最多存 2000，允许的突发
  redis-rate-limiter.requestedTokens: 1
```

```
-- ✓ 滑动窗口 Lua：三步必须原子，分开执行并发下两个请求都读到"还剩 1 个"
local key, window, limit, now = KEYS[1], tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)      -- 清窗口外旧记录
if redis.call('ZCARD', key) < limit then
    redis.call('ZADD', key, now, now .. '-' .. math.random())
    redis.call('EXPIRE', key, window)
    return 1                                              -- 放行
end
return 0                                                  -- 拒绝
```

> 为什么必须 Lua：判据是"检查令牌 + 扣减令牌"两步之间不能有其它请求插进来。分开执行时两个并发请求都读到"还剩 1 个"、都放行，限流直接失效——**这是"限流规则配了却没限住"的第四种形态**，前三种在 5.1（维度、位置、状态）。

一个反常识点来自 Sentinel 的 Warm Up：**预热只在流量"从低到高"时生效**，持续高流量下预热早就完成了，起不到保护作用。它是给冷启动（缓存未热、连接池未建）用的，不是给突发流量用的——把 Warm Up 当突发保护配，是选错了流控效果。

5.4　位置：限流要挡在最外层，也要挡在最里层

限流不是一道闸，是三道：

```
① 接入层（Nginx/LB）      —— 挡洪水：按 IP/连接数，防 DDoS 式流量
② 网关（Gateway）          —— 挡业务：按接口/租户/AppId，做配额
③ 应用内（Sentinel 资源） —— 挡自己：保护线程池、DB 连接、下游调用
```

只做 ②、不做 ③ 的后果：网关限了 1000 QPS，但每个请求在下游要放大成 5 次调用，下游承受 5000 QPS——**入口限流不等于下游安全**。

只做 ③、不做 ② 的后果：无效请求（鉴权失败、参数错误）也要先打到应用，白白消耗线程。

```
# Gateway 路由级 + 接口级，两层都要
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/order/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 1000     # 令牌桶填充速率
                redis-rate-limiter.burstCapacity: 1500     # 桶容量（允许的突发）
                key-resolver: "#{@apiKeyResolver}"          # ← 限流维度：按 API
Key
```

`key-resolver` 这一行决定了"限谁"：按 IP、按用户、按 API Key、按接口路径——**维度选错，限流就等于没做**。按 IP 限流在多租户 NAT 场景下会误杀整个出口 IP。

【进阶】Sentinel 与网关层限流的实现差异

5.3/5.4 把算法讲清了，但"在哪限"和"用哪个组件限"是两件事。同一个 1000 QPS 阈值，放在 Gateway 和放在应用内 Sentinel，走的是两条完全不同的链路：

```
Gateway（RedisRateLimiter）                应用内（Sentinel）
请求 → KeyResolver 取维度                  请求 → Entry → Slot Chain
     → ReactiveStringRedisTemplate                ├ NodeSelector / ClusterBuilder
     → Redis 执行 Lua 令牌桶（原子计数）          ├ StatisticSlot（LeapArray 滑动窗口）
     → 放行 / 429                                 ├ SystemSlot → FlowSlot →
DegradeSlot                                       └ 本地判断，零网络往返
     ↑ 每次请求一次 Redis 往返                   ↑ 默认单机计数，集群需 Token Server
     ↑ 天然集群总量（集中计数）
```

维度	Gateway `RequestRateLimiter`	应用内 Sentinel
统计位置	Redis 集中计数	进程内 `LeapArray`
算法	令牌桶（Lua 原子）	滑动窗口（可配 QPS / 线程数）
阈值维度	集群总量，天然准	默认单机，`clusterMode` 才接 Token Server
限谁	`KeyResolver`：IP / API Key / 路由 id	资源名 + `origin`（需 `RequestOriginParser`）
额外能力	只有限流 + 429	熔断降级 / 热点参数 / 系统保护 / Warm Up / 匀速排队
网络开销	每请求一次 Redis 往返，Redis 成为依赖	无
规则持久化	配置即规则（yml / 配置中心）	❗默认内存，重启即丢，必须挂 Nacos（5.3）
失败模式	Redis 挂了要**快速失败**，不能无限等	无外部依赖

选型判据一句话：**"总量要准、维度要细、在入口挡无效请求"→ 网关层；"保护自己的线程池 / DB 连接、要熔断和热点参数"→ 应用内 Sentinel**。5.4 那道三道闸里，② 和 ③ 正好对应这两行，缺一不可：只做 ②，网关限了 1000 QPS 但每个请求在下游放大成 5 次调用；只做 ③，鉴权失败的请求也要先打到应用白耗线程。

两个叠加时的坑必须提前说清：**别同时上两套网关限流**（`sentinel-gateway` 与 `RequestRateLimiter` 二选一，两个都上会双重限流、实际阈值取更小那个）；**Feign 侧别手写 `@SentinelResource`**（`feign.sentinel.enabled=true` 后自动建资源，手写会重复建、统计对不上）；**`blockHandler` 与 `fallback` 不是一回事**（规则拦截走前者，业务异常走后者）。

```
// ✓ 两个都配，语义分开：限流是可预期的，业务异常是意外的
@SentinelResource(value = "queryOrder", blockHandler = "blocked", fallback =
"failed")
public Order query(Long id) { return orderService.get(id); }

// ✗ 只配 fallback：限流时不走它，BlockException 直接冒泡成 500
@SentinelResource(value = "queryOrder", fallback = "failed")
public Order queryBad(Long id) { return orderService.get(id); }

public Order blocked(Long id, BlockException e) {          // 签名 = 原方法 +
BlockException
    throw new BizException(429, "请求过于频繁，请稍后重试");
}
public Order failed(Long id, Throwable t) { return Order.defaultOf(id); }
```

这段 ✗/✓ 对照是本章最该记住的一条：**"规则没限住"和"限住了但没兜住"是两种事故**，后者返回 500 比不限流还糟，因为它把可预期的拒绝伪装成了系统故障。

5.5　限流、熔断、降级：三个词别混着用

这三个经常被当成一件事说，实际解决不同问题：

机制	触发条件	作用对象	效果
限流	流量达到阈值（**主动**）	自己	拒绝多余请求，保护自己
熔断	下游错误率/慢调用率超阈值（**被动**）	下游依赖	快速失败，给下游喘息
降级	熔断打开 / 限流拒绝 / 超时	业务功能	返回兜底数据，保核心链路

三者关系是：**限流保护自己，熔断保护链路，降级保护体验。**

一个常见错误配置：只配了限流、没配熔断。结果下游挂掉之后，请求全部超时（而不是快速失败），线程被占满——限流还没触发，服务已经死了。

【必读】限流 / 熔断 / 降级的判据表

5.5 已经给过一张定义表，这里补一张**判据表**——排障时看的不是"它们是什么"，是"现在该触发哪一个、触发了没有、怎么恢复"：

机制	触发信号（看哪个指标）	主动性	作用对象	恢复方式	不做会怎样	误判点
限流	入口 QPS / 并发线程数 ≥ 阈值	主动	自己（入口）	流量回落即自动恢复，无状态	洪水直接打到应用	把"单机阈值"当全局（5.3）
熔断	下游慢调用比例 / 异常比例 ≥ 阈值	被动	对某个下游的调用	状态机 Closed→Open→Half-Open，靠探测恢复	下游挂了请求仍逐个超时，线程占满	用小样本触发（见下）
降级	限流拒绝 / 熔断打开 / 超时 / 业务异常	结果	业务功能与体验	随触发条件消失而结束	用户看到 500 / 空白页	只配 fallback 不配 blockHandler（5.5）

熔断的"小样本误判"值得算一次。Sentinel 慢调用/异常比例熔断里 `minRequestAmount` 是统计窗口内的最小请求数：若窗口内只有 2 个请求、1 个失败，失败率就是 50%；阈值恰好设 50% 且 `minRequestAmount=1` 时，**一个偶发失败就会打开熔断**。所以这条参数不是可选项：

```
最小样本判据：阈值 50% 时取 minRequestAmount = 20，
则窗口内至少 20 个请求才允许判定，2 个请求的 50% 不再触发
```

三者关系 5.5 写过"限流保护自己、熔断保护链路、降级保护体验"，补一句时序：**限流在请求进来时、熔断在调用下游时、降级在前两者触发之后**。串起来才是完整链路，缺任何一环，故障都会从缺口漏过去。5.5 那个"只配限流没配熔断"的坏味道，用这张表读就是：限流看的是入口 QPS，下游挂掉时入口 QPS 并没超，限流永远不会触发——**它俩看的根本不是同一个指标**。

一个反常识点：多数人以为"熔断打开 = 服务降级了"，实际熔断只是**停止调用下游**，返回什么由降级决定；没配降级时熔断打开的表现是"快速抛异常"，可能比不限流更早把 500 打到用户脸上。**熔断是止损动作，降级才是体验动作，两者必须各自独立配置。**

5.6　网关自身的坑：它不是万能的

网关在链路上是**单点**，它自己也会出问题：

① 网关超时比下游超时长，等于没超时

```
# 网关等 60 秒，下游 5 秒超时 —— 网关这层形同虚设
spring:
  cloud:
    gateway:
      httpclient:
        response-timeout: 3s        # 必须 < 下游总耗时
        connect-timeout: 1000
```

② 网关的连接池也会耗尽（第四章同样适用）。Gateway 用 Reactor Netty，默认连接池有限，高并发下同样排队。

③ 在网关做重活：鉴权时查一次 Redis、查一次用户库、调用一次权限服务——网关从"转发"变成了"业务处理"，它的 RT 直接叠加到所有请求上。**网关里只做 O(1) 的校验**，查库的事交给专门的鉴权服务并加缓存。

④ 跨域与鉴权的顺序：`CorsWebFilter` 必须在鉴权 Filter 之前，否则预检请求 `OPTIONS` 会被鉴权拦掉，前端拿到的错误是"跨域失败"——**而真正原因是 401**。这个错误信息误导过很多人。

【必读】用压测把限流阈值变成会失败的断言

5.3 说"重启一台看规则还在不在"，这补一条更强的：**用恒定速率压测，把阈值算成一个可验证的数字**。工具用 wrk2 / Gatling 的恒定 RPS 模式（第十二章压测篇），不能用默认尽力打满模式——否则服务被限流后客户端自动降速，429 比例会被掩盖。

```
# ① 恒定 2000 RPS 打 30 秒，限流阈值设 1000 QPS
wrk2 -t8 -c200 -d30s -R2000 --latency \
     -H 'X-Client-App: stress' http://gateway/api/orders | tee lim.log

# ② 统计 429 比例：总请求 2000×30 = 60000，期望约 30000 被拒
grep 'Non-2xx' lim.log
# 期望 ≈ 30000：接近 0 → 限流没生效（回到 5.1 三种形态）
#       ≈ 60000 → key-resolver 维度错，把压测流量全挡了

# ③ 同时验"规则持久化"：滚动重启一个网关实例后再跑一遍
wrk2 -t8 -c200 -d10s -R2000 --latency http://gateway/api/orders | grep 'Non-2xx'
# 期望与 ② 同量级：429 掉到 0 → 规则内存态、重启即丢（5.3）
```

把期望写成算式，压测结果才有判据：`拒绝数 ≈ (R − 阈值) × 时长 = (2000 − 1000) × 30 = 30000`，误差超过 10% 就说明维度、状态或位置三者里有一个错了。**这比"配完看一眼 Dashboard"强，因为它验的是流量真的被挡住，不是规则真的被显示出来。**

最后把会漂移的点列出来，升级时逐条重验：

结论	适用版本	升级要重验什么
Gateway `RequestRateLimiter` + `RedisRateLimiter`	Gateway 3.x / 4.x（Boot 2.7 / 3.x）	4.x 起参数名与 `KeyResolver` 的响应式写法
Sentinel 规则默认内存态、需 Nacos 持久化	Sentinel 1.8 / 2.x	各版本 `DataSource` 接口包名可能变
`blockHandler` / `fallback` 语义分工	Sentinel 全系	稳定
Hystrix 已停止维护，新项目用 Resilience4j / Sentinel	2020 起	Boot 3.x 新项目优先 Resilience4j
Resilience4j 配置前缀	Spring Cloud 2023.0.0 前后不同	`resilience4j.circuitbreaker` → `spring.cloud.circuitbreaker.resilience4j`，文档常滞后
Redis 7 单线程执行 Lua 保证原子	Redis 2.6+ / 7.x	稳定，但要给 Redis 配连接超时与快速失败

> 限流最贵的不是配错数字，是**从来没验证过它真的拦住过流量**。一次带期望值的压测，能把 5.1 那三种"限流形同虚设"的形态在上线前全部逼出来。

5.7　自检清单

- [ ] 限流阈值是**集群维度**的，不是"每实例 × N"
- [ ] 规则持久化在配置中心，重启后自动恢复（**验证方法：重启一台，看规则还在不在**）
- [ ] 限流维度明确：按 IP / 用户 / API Key / 接口，且写进了文档
- [ ] 三道闸都有：接入层、网关、应用内资源
- [ ] 熔断独立配置，不依赖限流
- [ ] 网关 `response-timeout` 小于下游总耗时
- [ ] 网关内没有查库/远程调用的重逻辑
- [ ] `CorsWebFilter` 顺序在鉴权之前，`OPTIONS` 放行
- [ ] 被限流的请求有明确响应（429 + 重试建议），不是空 200

5.8　一句话总结

> 限流不是"配一个数字"，是回答三个问题：**限谁、在哪限、规则存在哪。** 三个都答对了才叫限流，答错一个就是心理安慰。

【选读】延伸

- 知识库对应原理篇：04-Spring生态/03-Spring Cloud/Gateway、04-Spring生态/03-Spring Cloud/Sentinel

第六章　事务为什么会静默失效

`@Transactional` 大概是 Java 后端被加得最多、也被误解得最深的一个注解。它的可怕之处不在"不生效"，而在于**不生效的时候，一切看起来都很正常**：方法执行了，SQL 打了，日志没有一条 ERROR，返回值是 success。错的只有数据。

这一章只解决一个问题：**当你怀疑事务没生效时，用最短的路径确认它，并且知道为什么。**

6.1　现场：9:04

预售活动九点开抢，三千件限量商品，九点零四秒售罄。

十点半，运营在群里问了一句："库存怎么是负的？"

查下来是这样：库存表扣了 3,347 件，订单表只有 3,000 条。**多扣了 347 件，但没有对应的订单。** 更奇怪的是，应用日志里没有任何异常堆栈，没有 ERROR，告警一条都没响。

接口的代码大致是这样：

```
@Service
public class SeckillService {

    public Result createOrder(OrderCmd cmd) {
        orderMapper.insert(cmd.toOrder());   // ① 写订单
        this.deductStock(cmd);               // ② 扣库存
        return Result.success();
    }

    @Transactional(rollbackFor = Exception.class)
    public void deductStock(OrderCmd cmd) {
        int affected = stockMapper.deduct(cmd.getSkuId(), cmd.getQty());
        if (affected == 0) {
            throw new BizException("库存不足");   // ③ 扣不到就抛，让事务回滚
        }
    }
}
```

读到这儿你可能已经看出来了。但当时我们没有——因为在监控上，`createOrder` 这个接口的**报错率是 0**，它确实没报错：`BizException` 被全局异常处理器转成了 `Result.fail("库存不足")`，HTTP 200，业务上完全合理。

于是数据变成了这样：

表	期望	实际
`order_info`	3,000 条	3,000 条
`stock`	扣 3,000	扣 3,347

多出来的 347 次，是那些"库存不足"的请求：**库存扣成功了，订单没写进去，异常抛了，但没人回滚。**

> 本章的现场由多个真实故障脱敏合成，代码与数字经过改写。结论对应的是真实排查路径，不是虚构的。

这就是这本书要反复讲的第一个认知：

> **最贵的 bug 不是让服务挂掉的 bug，是让数据悄悄错掉的 bug。** 挂掉的 bug 有告警，错掉的 bug 只有对账。

当时怎么误判的：第一反应是「超卖肯定是并发没控制住」，于是给下单接口加了一把 Redis 分布式锁，压测复现却还是超卖——而且 SQL 明明执行成功、订单却没写进去，锁白加了。接着怀疑「`@Transactional` 没配上」，翻了一遍数据源和事务自动配置，全是正常的；其实当时日志里已经打出 `JDBC Connection ... will not be managed by Spring`，这行本该直接点破「这里没有事务」，却被我们当背景噪音跳过了。真正扭转方向的是在 `deductStock` 第一行打了 `isActualTransactionActive()`，返回 `false`。这一次空转提醒了一句话：这种故障该先问「此刻我到底在不在事务里」，而不是去证明「环境没错」。

6.2　三次误判

我把当时的判断过程完整写出来，因为它比结论有用。

【进阶】现场再拆一层：347 是怎么攒出来的

把当天日志里一个 SKU 的六次请求摊开看，规律就出来了：

请求	① 写订单	② 扣库存	③ 抛异常	结果
A	成功	成功	否	正常
B	成功	成功	否	正常
C	成功	**affected=0**	是	**订单在，库存没扣**
D	成功	**affected=0**	是	同上
E	成功	成功	否	正常
F	成功	**affected=0**	是	同上

等一下——按这张表，应该是"订单比库存多"，可现场是"库存比订单多扣了 347 件"。

原因在 SQL 本身：

```
-- ✗ 无护栏扣减：affected 恒为 1，库存被改成负数
UPDATE stock SET qty = qty - #{qty} WHERE sku_id = #{skuId};

-- ✓ 条件更新 + 影响行数判定，扣不到就是 0 行
UPDATE stock SET qty = qty - #{qty}
 WHERE sku_id = #{skuId} AND qty >= #{qty};
```

当天线上跑的是第一条。库存见底后 `qty` 继续减成负数，`affected` 始终是 1，永远不会抛 `BizException`。于是"抛异常、期望回滚"的分支根本没被触发，而**订单写入和库存扣减分属两个自动提交的连接**：订单写进去、库存扣成负数，两边各自提交，谁也不欠谁。

这一点很关键，也很容易被"事务失效"这个结论盖过去：**事务没生效，是这次数据不一致的必要条件，不是充分条件。** 如果扣减 SQL 自带 `qty >= #{qty}` 的护栏，就算事务没开，最坏结果也只是"订单多、库存没扣"，对账一眼能看出来；正是"事务没开 + 更新无护栏"两个漏洞叠在一起，才把错误变成了负数库存这种要花一个半小时才敢确认的形态。

> 排查时别只找那个"最像根因"的错。多数线上事故是两三个各自都不足以致命的缺口，在同一条路径上对齐了。

【必读】误判一：以为是并发问题（花掉 40 分钟）

第一反应很自然：超卖嘛，肯定是并发没控制好。于是加了一把 Redis 分布式锁，锁 key 是 `sku:{skuId}`。

压测复现——**还是超卖**，而且这次日志更清楚了：扣库存的 SQL 明明执行成功了，订单表却没记录。

加锁解决的是"同一时刻多个请求同时扣同一行"，而这里的问题是"一个请求内部的两次写不同生共死"。**两个问题长得很像，根因完全不同。** 分布式锁加得再对，也救不了本地事务的缺失。

【必读】误判二：以为注解没配上（花掉 30 分钟）

接下来检查配置，全部正常：

- `@EnableTransactionManagement` 在（其实 Boot 的 `DataSourceTransactionManagerAutoConfiguration` 已经自动开了）
- `spring.datasource` 配的是同一个库，没有多数据源
- 日志能打出 `JDBC Connection [HikariProxyConnection] will not be managed by Spring`——**这一行是当天的关键线索，但被我们跳过了**

> 这条日志的意思是"这条连接不归 Spring 管"。它在**没有事务**的 SQL 上出现是正常的；当它出现在你以为是事务的方法里，就是在明说"这里没有事务"。

【必读】误判三：怀疑数据库（花掉 20 分钟）

```
SHOW TABLE STATUS WHERE Name = 'stock';   -- Engine: InnoDB    ✓
SELECT @@transaction_isolation;           -- REPEATABLE-READ  ✓
```

一切正常。此时已经一个半小时过去。

【必读】转折：一行日志

有人在 `deductStock` 的第一行加了这么一句：

```
log.info("inTx={}",
TransactionSynchronizationManager.isActualTransactionActive());
```

输出是 `inTx=false`。

注意这里有个陷阱，也是本章第一个要拆掉的东西：

网上大量文章教你打印 `this.getClass()` 看是不是 `XXX$$SpringCGLIB$$0`，以此判断代理有没有生效。**这个方法永远是错的。** 因为在目标方法的代码里，`this` 恒等于目标对象本身，永远不可能打印出代理类名——不管代理有没有生效。用这个方法"验证"过的人，得到的结论要么是误判，要么是恰好蒙对。

真正能回答"我在不在事务里"的，只有一句 `isActualTransactionActive()`。它查的是当前线程绑定的 `ConnectionHolder` 上有没有真实事务在跑。**这是全章最值得记住的一行代码。**

【必读】复盘：这一小时四十分钟花在哪

时段	动作	拿到的事实	为什么没用
10:30–11:10	加分布式锁、压测复现	仍超卖，且 SQL 成功但无订单	并发控制与原子性是两件事
11:10–11:40	翻事务配置与数据源	全部正常；跳过了一条 `will not be managed by Spring`	只验证了"配置存在"，没验证"运行时是否在事务里"
11:40–12:00	查表引擎与隔离级别	InnoDB / RR，全正常	前提没错，但前提不是本题
12:00	一行 `isActualTransactionActive()`	`false`	直接命中

前三步有一个共同毛病：**都在验证"环境对不对"，没有一步在问"程序此刻到底处于什么状态"。** 这类排查特别耗时间，因为环境永远能查出一堆"正常"，而"正常"会给人一种正在推进的错觉。

真正该在第一步做的就是那个成本最低、信息量最大的动作——在可疑方法的第一行问一句"我现在在不在事务里"。它是布尔值，不需要推理，不需要看配置，也不需要等压测。**排查顺序的价值，往往比排查技巧大。**

6.3　一个事实决定所有失效：你调用的是代理

Spring 的事务不是"给方法开个开关"，而是一层 AOP。容器启动时，它给 `SeckillService` 生成了一个代理对象（Boot 默认 CGLIB，即子类），**容器里放的是代理，不是你写的那个类**。

调用链是这样的：

```
外部调用 seckillService.createOrder()
        │
        ▼
  SeckillService$$SpringCGLIB$$0        ← 容器里真正被注入的东西
        │  createOrder() 无事务注解，直接透传
        ▼
  SeckillService（目标对象）
        │
        ├─ orderMapper.insert()          正常执行
        │
        └─ this.deductStock(cmd)
                │
                └─ this 就是目标对象本身，① 不经过代理
                   ② 拦截器没被触发
                   ③ 事务从未开启
                   ④ 注解等于一行注释
```

`this` 是目标对象，不是代理。**所有"事务失效"的问题，根因都收敛到这一件事上：只要调用路径绕过了代理，注解就是废的。**`@Cacheable`、`@Async`、`@Scheduled`、自定义切面，全部同理。

绕过代理的路径只有那么几条，穷举得出来：

路径	是否走代理	说明
跨 Bean 调用（注入进来的）	✓ 走	注入的是代理
`this.method()` 同类自调用	✗ 不走	本章的现场
`new Service().method()`	✗ 不走	根本不在容器里
父类方法里调 `this.xxx()` 且子类覆写了	✗ 不走	真实类型是目标对象
方法被声明为 `private`	✗ 不走	子类无法覆写，静默跳过
方法被声明为 `final`	✗ 不走	子类无法覆写，**静默失效**
类被声明为 `final`	✗ 全类失效	无法生成子类，CGLIB 整体不可用
`protected` / 包级私有（Spring 6+）	✓ 走	6.0 起 CGLIB 支持，但团队统一 public 理解成本最低

`private` 和 `final` 这两条要特别小心，因为它们**不报错**。

这里要纠正一个流传很广的说法：有资料称 Spring Boot 3 检测到 `private` 方法上的 `@Transactional` 会在启动时直接报错。实际不会。它只是静默跳过这个注解（源码里 `allowPublicMethodsOnly()` 判定不通过就直接返回 `null`），最好的情况是打一行默认级别很低的日志，生产环境你看不见。**加了注解、没生效、没提示——这是最需要靠规约来防的一类。**

【必读】启动期：代理是什么时候生成的

理解"代理"不能只停留在"容器里放的是代理"这句话。它在两个阶段各做了一件事：

```
启动期  @EnableTransactionManagement
          ├ 注册 InfrastructureAdvisorAutoProxyCreator（一个 BeanPostProcessor）
          └ 注册 TransactionInterceptor +
BeanFactoryTransactionAttributeSourceAdvisor
        Bean 初始化后置阶段
          └ 扫到 @Transactional → 为该 Bean 生成代理，替换容器里的实例

运行期  proxy.createOrder()
          └ TransactionInterceptor#invokeWithinTransaction
              ① createTransactionIfNecessary()      getTransaction：拿连接、关自动
提交、绑 ThreadLocal
              ② invocation.proceed()                 执行业务方法
              ③ commitTransactionAfterReturning()  正常返回 → commit
              ④ completeTransactionAfterThrowing() 异常且匹配规则 → rollback
              ⑤ cleanupTransactionInfo()            解绑 ThreadLocal、恢复被挂起的资
源
```

两个推论，比背流程有用：

1. **代理是在 Bean 初始化完成后才生成的。** 所以 `@PostConstruct` 里调用本类的另一个事务方法，不仅绕过代理，连代理都还不存在（第八章会讲这类初始化顺序问题）。
2. **事务的"决策点"在方法进入时**（`getTransaction` 那一刻），不是提交时。传播行为、隔离级别、超时、只读，全在进门那一瞬间定死；方法体里再怎么改注解、换数据源，都来不及了。

【必读】三个常被当成"事务失效"的邻居

它们表现相似，根因完全不同，按五问排查会白费力气：

现象	真实原因	一句判据
`readOnly = true` 的方法照样写进去了	`readOnly` 只是给驱动和 ORM 的**提示**，不是写保护	去掉注解，行为不变
注解加在接口方法上，实现类没生效	Boot 默认 CGLIB 代理，读的是**实现类**上的注解	把注解挪到实现类方法上
回滚了，但自增 ID 还是跳号	自增序列不参与事务，InnoDB 亦然	别拿 ID 连续性做任何业务假设

第三条尤其容易在复盘里被当成"回滚没生效"：数据确实回滚了，只有 `AUTO_INCREMENT` 计数器没回退。如果你们用 ID 是否连续来判断事务有没有回滚，会得到完全错误的结论。

6.4　五问定位法

下次怀疑事务，按这个顺序问，五问能覆盖九成场景，而且每一步都有成本极低的验证手段。

【必读】Q1　我现在到底在不在事务里？

不要推理，直接问程序：

```
// 临时断言，排障和单测时用；上生产前删掉或改成 DEBUG 级
log.info("inTx={}",
TransactionSynchronizationManager.isActualTransactionActive());
```

或者更狠一点，直接把假设变成契约：

```
Assert.state(TransactionSynchronizationManager.isActualTransactionActive(),
        "deductStock must run in transaction");
```

`false` → 往下看 Q2、Q5。`true` → 事务开着，问题一定在 Q3、Q4（回滚没触发）。

这一步决定了后面所有的排查方向，别跳。

【必读】Q2　调用穿过代理了吗？

看调用点，不看注解。三类高危写法：

```
// ✗ 同类自调用
this.deductStock(cmd);

// ✗ 在 lambda / 内部类里调外部方法，看似正常，实际是目标对象引用
items.forEach(i -> this.deductStock(i));

// ✗ 从父类模板方法里回调
protected void execute() { this.doBiz(); }   // 子类覆写的 doBiz 上是目标对象
```

【必读】Q3　异常逃出去了吗?

回滚的触发点在 `TransactionInterceptor` 的 catch 块里——**只有异常穿过代理边界，拦截器才看得见**。

```
// ✕ 吞掉：拦截器看到的是正常返回 → 提交
@Transactional(rollbackFor = Exception.class)
public void transfer(Cmd cmd) {
    accountMapper.debit(cmd);
    try {
        accountMapper.credit(cmd);
    } catch (Exception e) {
        log.error("credit failed", e);      // 异常到此为止
    }
}
```

同一类问题还有几种变体，都很难看出来：

- `finally` 里 `return`，把异常吃掉了
- 自定义 `@Around` 切面 catch 后没重抛（**切面顺序问题，下一章展开**）
- 用 `CompletableFuture` 包了一层，异常被吞进了 `future`
- 全局异常处理器 `@RestControllerAdvice`——这个**不影响**，它在事务边界之外，别冤枉它

要"记日志但仍回滚"，只有两种写法：

```
catch (Exception e) {
    log.error("credit failed", e);
    TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();  // ① 标记回滚
    // 或 throw e;                                                          // ② 原样抛出
}
```

【必读】Q4　异常类型对得上吗?

`DefaultTransactionAttribute#rollbackOn` 的实现只有一句：

```
return ex instanceof RuntimeException || ex instanceof Error;
```

也就是说：`IOException`、`SQLException`、你的自定义业务受检异常，**抛了也不回滚**。

这条不该靠人记，该靠规约：

> 项目里所有 `@Transactional` 一律写 `rollbackFor = Exception.class`。新人不理解不要紧，照抄就行——不理解但写对，比理解但漏写强。

【必读】Q5　是同一个线程、同一个数据源吗？

事务的物理载体是**绑定在线程上的那个连接**（`TransactionSynchronizationManager` 里的 `ThreadLocal<Map<DataSource, ConnectionHolder>>`）。所以：

- **换线程 = 换事务**。`@Async`、线程池、`CompletableFuture.supplyAsync`、`new Thread()`，一律不传播
- **换数据源 = 换事务**。多数据源不显式指定 `transactionManager`，可能挂到错误的管理器上

```
// ✗ 事务内用线程池并发写库
@Transactional(rollbackFor = Exception.class)
public void batchSave(List<Item> items) {
    Lists.partition(items, 500).forEach(batch ->
            pool.execute(() -> itemMapper.batchInsert(batch)));   // 子线程：裸连接，自动提交
    throw new BizException("触发回滚");                            // 主事务回滚，子线程数据还在
}
```

`@Transactional` 和 `@Async` 叠在同一个方法上，是最容易骗过 CR 的写法：注解都在，语义全丢。异步方法里的"事务"和调用方期望的"同一个事务"没有任何关系。

Q0（数据库层）：如果五问都排除了还回滚不了，去查表引擎。MySQL 的 MyISAM 不支持事务，回滚命令发下去毫无反应、**也不报错**。老库 InnoDB/MyISAM 混用时表现为"部分回滚"——最容易被误判成"事务时灵时不灵"。

```
SHOW TABLE STATUS WHERE Name = 'order_info';   -- Engine 必须是 InnoDB
```

【进阶】五问之外：事务明明开着，为什么还是"全回滚了"

有一类问题五问答完都是"是"，事务确实在跑，但结果仍然和预期相反。它出在**传播行为**上，值得单独拎出来。

```
// ✕ 以为 catch 住内层异常，外层就能保住订单
@Transactional(rollbackFor = Exception.class)
public void place(Order order) {
    orderMapper.insert(order);
    try {
        stockService.deduct(order.getSku(), order.getCount());   // 内层
REQUIRED
    } catch (Exception e) {
        log.warn("deduct failed, ignore", e);
    }
}
// 结果：提交时抛 UnexpectedRollbackException，订单也没了
```

内层 `REQUIRED` 和外层**共用同一个物理事务对象**。内层抛异常时，拦截器已经在这个共享对象上打了 `rollback-only` 标记；外层 catch 掉的只是异常对象，标记还在。等到外层想 commit，事务管理器发现标记，直接回滚并抛 `UnexpectedRollbackException`。

> **catch 住的是异常，不是事务状态。** 这是"我明明处理了，怎么还是全没了"最常见的答案。

三种传播行为在这个场景里的差别，记住这张表就够：

传播行为	物理连接	内层提交	内层回滚	外层回滚	主要代价
`REQUIRED`（默认）	共用 1 个	不提交，等外层	标记 rollback-only，**外层必回滚**	全部回滚	catch 也救不回来
`REQUIRES_NEW`	挂起外层，**再借 1 个**	独立提交	只回滚内层	不影响已提交的内层	同线程瞬时占 2 个连接；改同一行会自死锁
`NESTED`	共用 1 个，建保存点	只释放保存点	回滚到保存点	**照样带走内层**	依赖 JDBC Savepoint，JTA 下可能不支持

`REQUIRES_NEW` 有两个坑必须在写之前想清楚。一是**挂起不是免费的**：`doSuspend` 把外层的 `ConnectionHolder` 从 ThreadLocal 解绑，内层再从池里借一个新连接，这期间同一个线程占着两个连接，批量场景会把事务问题直接变成连接池耗尽（第四章）。二是**自死锁**：外层已经更新过某行、持着行锁，内层 `REQUIRES_NEW` 又去更新同一行，内层等外层释放，外层等内层返回，双方一起耗到 `innodb_lock_wait_timeout`。

```
// ✕ 外层已锁同一行，内层 REQUIRES_NEW 再更新它 = 自死锁
@Transactional(rollbackFor = Exception.class)
public void settle(Long orderId) {
    orderMapper.lockById(orderId);
    auditTxService.markSettled(orderId);        // REQUIRES_NEW，更新同一行
}

// ✓ 要么同事务内更新，要么把加锁推迟到内层之后
```

最后一条经验：**能不用传播行为就不用。** 上面那个"内层失败不影响外层"的需求，最稳的解法往往不是挑一个传播行为，而是把可失败的动作移出事务、改走补偿（第七章、第 18 章）。传播行为是在同一个调用栈里做精细控制，越嵌套越难推理；能用事件或补偿把调用栈拆开，就别用挂起。

6.5　修复的四个层次

同一个故障，有四种修法，成本与收益完全不同。**当天止血和长期正确，往往是两个不同的动作**，分清楚。

【进阶】L0　先把结论钉死：一个能跑的最小验证

改代码之前，先把"我们到底搞清楚了没有"变成一段可重复执行的测试。下面这段 JUnit 同时验证三件事：跨 Bean 调用有事务、同类自调用没有事务、`private` 方法上的注解被静默跳过而不报错。

```
@SpringBootTest
class TransactionProbeTest {

    @Autowired private OrderService orderService;

    @Test
    void probe() {
        log.info("跨 Bean 调用: {}", orderService.probeExternal());   // true
        log.info("同类自调用: {}", orderService.probeSelfInvoke());   // false
        log.info("private 方法: {}", orderService.probePrivate());     //
false，且启动不报错
    }
}

@Service
class OrderService {
    @Autowired private StockService stockService;

    public boolean probeExternal() { return stockService.inTx(); }
    public boolean probeSelfInvoke() { return inTx(); }
    public boolean probePrivate() { return privateInTx(); }

    @Transactional(rollbackFor = Exception.class)
    public boolean inTx() {
        return TransactionSynchronizationManager.isActualTransactionActive();
    }

    @Transactional(rollbackFor = Exception.class)   // private: Spring 6 静默跳过
    private boolean privateInTx() {
        return TransactionSynchronizationManager.isActualTransactionActive();
    }
}
```

期望输出只有一行是 `true`。**特别留意第三行**：如果它输出 `false` 而应用正常启动，就说明知识库里那句"Spring 6 / Boot 3 会对 private 上的 `@Transactional` 启动 fail-fast"是错的——真实行为是 `allowPublicMethodsOnly()` 判定不通过直接返回 `null`，不报错、不警告。

这类"一句话就能证伪"的断言，比在群里争论半小时有用。建议把它留在 `book-demo` 仓库里，跟着每次 Spring 版本升级一起跑：**框架行为会变，测试不会替你圆谎。**

【必读】L1　止血：让调用走代理

改动最小的三种方式，按推荐度排：

```
// ① 拆 Bean（推荐）：结构最清晰，天然走代理
@Service
@RequiredArgsConstructor
public class SeckillService {
    private final StockService stockService;

    public Result createOrder(OrderCmd cmd) {
        orderMapper.insert(cmd.toOrder());
        stockService.deduct(cmd);          // 跨 Bean → 代理生效
        return Result.success();
    }
}

// ② TransactionTemplate（推荐）：不依赖代理，粒度最精确
public Result createOrder(OrderCmd cmd) {
    orderMapper.insert(cmd.toOrder());
    txTemplate.executeWithoutResult(status -> {
        int affected = stockMapper.deduct(cmd.getSkuId(), cmd.getQty());
        if (affected == 0) {
            throw new BizException("库存不足");   // 逃出 → 回滚
        }
    });
    return Result.success();
}

// ③ 自注入 / AopContext（只应急，别进主干代码）
@Lazy @Resource private SeckillService self;        // self 是代理
self.deduct(cmd);
```

第 ③ 种在紧急 hotfix 里能用，但别留着：它把"依赖 AOP 机制"这件事藏进了业务代码，单测里 `self` 是 null，换代理方式（JDK → CGLIB）会静默坏掉。**应急可以，留坑不行。**

【必读】L2　修正边界：事务只包数据库操作

止血之后要问一个更重要的问题：**这个事务是不是本来就不该这么大？**

```
// ✗ 事务里做远程调用：连接和行锁被持有到远程返回
@Transactional(rollbackFor = Exception.class)
public void create(Order order) {
    orderMapper.insert(order);
    inventoryClient.deduct(order.getSku());   // HTTP，可能 3 秒
    orderItemMapper.insertBatch(order.getItems());
}

// ✓ 远程调用移出事务，事务只包本地写
public void create(Order order) {
    inventoryClient.deduct(order.getSku());
    txService.saveOrder(order);               // @Transactional(timeout = 5)
}
```

这条在下一章展开讲，先记住一句话：**事务里做 HTTP 调用、发 MQ、传大文件，等于把行锁的持有时间交给别人决定。**

【必读】L3　结构性预防：让错误写不出来

前两层修的是这一次，这一层修的是下一次。

1. **规约**：所有 `@Transactional` 必须带 `rollbackFor = Exception.class`；禁止在事务方法内做远程调用（CR checklist 项）
2. **静态检查**：自定义注解处理器扫 `final` / `private` 方法上的 `@Transactional`，编译期报错
3. **单测契约**：关键写路径的测试里加 `isActualTransactionActive()` 断言，把"必须有事务"变成可执行断言
4. **日志**：把 `will not be managed by Spring` 这类连接日志纳入日志平台监控，出现即告警

【必读】L4　兜底：承认代码会错

> **能靠对账发现的问题，都不算灾难；只能靠客诉发现的问题，才是。**

这一层不是技术方案，是工程态度：库存与订单的日终对账、支付与流水的 T+0 校验、关键表的变更流水。事务写得再对，也挡不住下一个没意识到自调用的人。

6.6　自检清单

贴进 CR 模板，或者做成 IDEA Live Template。十项，三十秒过一遍。

- [ ] `@Transactional` 方法都是 `public`，都不是 `final`，所在类不是 `final`
- [ ] 调用方是**另一个 Bean**，不是 `this.xxx()`
- [ ] 注解带 `rollbackFor = Exception.class`
- [ ] 方法内的 `catch` 要么重抛，要么 `setRollbackOnly()`
- [ ] 事务内没有 HTTP / RPC / MQ 发送 / 大文件 IO
- [ ] 事务内没有线程池、`@Async`、`CompletableFuture`
- [ ] 多数据源场景显式指定了 `transactionManager`
- [ ] `REQUIRES_NEW` 的使用有明确理由（独立成败），不是随手加的
- [ ] 涉及的表引擎是 InnoDB
- [ ] 这个写路径有对账或补偿手段

6.7　版本区间与复现环境

本章结论对应 **Spring Framework 6.0–6.2 / Spring Boot 3.x、JDK 17–21、MySQL 8.0（InnoDB）**。几个会随版本漂移的点，升级时重新验一遍：

结论	生效版本	可能的变化
默认 CGLIB 代理	Boot 2.x 起	`spring.aop.proxy-target-class=false` 会切回 JDK 代理，接口注解的判定随之改变
`protected` / 包级私有方法可被代理	Spring 6.0 起（CGLIB）	6.0 之前一律只看 public
`private` 上的注解静默跳过	至今	团队规约仍应统一 public，别赌框架会不会改
`TransactionExecutionListener`	Spring 6.1 新增	6.1 之前没有，做事务级埋点得自己写同步回调
`@TransactionalEventListener` 固定回原线程执行	Spring 6.1 起	之前异步多播器会影响它

书里所有"不报错"的判断，都建议你用上面的 L0 测试在自己的版本上跑一遍再采信——**这类结论最怕的就是跨版本照抄。**

6.8　本章的边界

这一章讲的是「本地事务为什么没开」。还有两类问题它没覆盖，在后续章节：

- **事务开了但锁住了不该锁的东西** → 第七章（事务与远程调用、行锁持有时间）
- **事务之间互相等待** → 第十四章（死锁与锁等待）
- **事务跨不了服务** → 卷四（消息表、Saga、最终一致）

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/03-Spring事务与AOP失效、04-Spring生态/01-Spring Framework/事务管理、04-Spring生态/06-Spring源码专题/事务传播机制

第七章　事务里做远程调用：行锁被持有 3 秒之后

本章要回答：一个和事务完全无关的下游接口变慢，为什么会把本地数据库拖死？

本章你能带走

- 一条锁等待的完整链条：从"下游慢了 3 秒"到"本地连接池打满"
- 一个判断：事务的边界应该由**数据一致性**决定，不由**代码层次**决定
- 一条可以直接写进 CR 的规矩：**事务方法内禁止 HTTP / RPC / MQ**

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

7.1　现场：库存服务"变慢"了三天

一个电商系统，下单失败率从 0.3% 涨到 6%，持续三天，每天早上九点到十一点最严重。

排查过程很绕，因为每个组件单独看都正常：

- 订单服务：CPU 40%，GC 正常，但线程池 `active` 打满
- 库存服务：RT 从 80ms 涨到 900ms（**下游**，不是数据库）
- MySQL：CPU 30%，慢查询日志里有大量 `Lock wait timeout exceeded`

第一反应是"库存服务的数据库有锁冲突"，于是 DBA 去看锁：

```
SELECT * FROM performance_schema.data_lock_waits;
```

看到的是：**同一个 `sku_id` 的行锁，被一个事务持有 3 秒以上，后面排了七八个事务在等。**

但那个持有锁的事务，其 SQL 只花了 8ms。

【必读】三次误判：每个组件都在喊冤

这三天里，三个组各查了一遍，结论都是"不是我的问题"：

谁查的	看到的	得出的结论	错在哪
订单组	线程池 active 200 打满、CPU 才 40%	"下游太慢，加线程"	线程不是不够，是被等锁占住了；加到 400 只是把排队时间翻倍
库存组	自己接口 P99 900ms，SQL 单条 8ms	"我们慢一点但不至于拖垮别人"	900ms 是**持锁时长**，不是它的响应时间
DBA	有 `Lock wait timeout`，但库 CPU 30%	"锁冲突，让业务改 SQL"	SQL 没问题，是事务边界把锁框大了

三个结论单看都自洽，合起来正好绕过了真相：**没有任何一个组件"坏"了，坏的是它们之间被事务绑在一起的时间。** 这类故障最难查，因为每个团队的监控都显示"我这边正常"，而成本全部落在了跨组件的等待上。

第二个要记的教训是"加线程"。等锁型故障加线程，等价于给排队的队伍扩容——**队伍变长了，服务速率没变**。真正能改变服务速率的，只有缩短每个请求的持锁时间。

当时怎么误判的：第一个反应是「下游变慢，那就加线程扛」，订单组把 Tomcat 线程从 200 调到 400，失败率反而更高——加线程只是把等锁的队伍排得更长，是一条走弯路的死线索。库存组和 DBA 也各查了一遍，DBA 看到 `Lock wait timeout` 第一反应是「业务 SQL 写得不好、有锁冲突」，盯着慢查询优化，方向也偏了。真正扭回来的，是有人把持锁事务的 `ACTIVE` 时长拉出来看：那条 SQL 只花了 8ms，事务却开着 3 秒以上——锁根本不是被 SQL 占着，是被事务里那次 HTTP 调用占着。这一下把方向从「调 SQL / 加线程」扭回「缩短持锁时间」。

7.2　根因：事务里那次 HTTP 调用

代码是这样的：

```
@Transactional(rollbackFor = Exception.class)
public void deductStock(Long skuId, int qty) {
    int affected = stockMapper.deduct(skuId, qty);      // ① 更新，持有行锁
    if (affected == 0) {
        throw new BizException("库存不足");
    }
    marketingClient.recordSale(skuId, qty);             // ② 调营销服务，HTTP
    logMapper.insertSaleLog(skuId, qty);                // ③ 再写一条日志
}
```

第 ② 步是压垮系统的关键。它的语义只是"通知营销系统记一笔销量"，失败了大不了补，但它被放在了事务里，于是：

```
① 更新 stock 行 → 行锁 + 数据库连接，被这个事务独占
② HTTP 调营销服务 → 营销服务 P99 从 20ms 涨到 900ms
   → 这 900ms 里，行锁一直被占着，连接也一直被占着
③ 后面的请求全部在等这把锁
   → 等锁的线程占着 Tomcat 线程
   → Tomcat 线程池 200 打满
   → 新请求排队 → 接口超时
```

一条链路上任何一环变慢，都会被事务放大成全系统的排队。

这就是"事务边界的成本"：

事务里做的事	占用的资源	时间尺度
一条 UPDATE	行锁 + 1 个连接	毫秒
一次 HTTP 调用	行锁 + 1 个连接 + 1 个线程	百毫秒~秒
一次 RPC 广播（10 个实例）	行锁 + 1 个连接	秒级
上传/下载文件	行锁 + 1 个连接 + 大内存	秒~分钟

事务内每多一个外部调用，行锁的持有时间就被拉长一个数量级。

【进阶】把 900ms 算成一条公式

"下游慢了 900ms"听起来只是慢了一点。把它换成吞吐，才知道有多严重。

同一个热点 SKU 的扣减必须串行（行锁），每次持锁 0.9 秒，于是：

```
该 SKU 的最大吞吐 = 1 / 0.9s ≈ 1.1 次/秒
```

活动期间这个 SKU 的到达率是 40 次/秒。**到达率 40，服务率 1.1，队列必然无限增长**——这不是"有点慢"，是一个数学上不收敛的系统。

再看线程。按 Little's law，队列长度 `L = λ × W`（定义与符号见第 4 章）。取 `λ = 40/s`、等待时间 `W = 50s`（`innodb_lock_wait_timeout` 默认值），稳态需要 **2000 个线程**才排得下，而 Tomcat 只有 200 个。也就是说：

```
200 个线程 / 40 次每秒 ≈ 5 秒    → 线程池被打满
之后每个新请求都在排队，RT 从 900ms 涨到超时
```

这解释了现场最反直觉的一点：**下游 P99 只从 20ms 涨到 900ms，下单失败率却从 0.3% 涨到 6%。** 因为失败率不是由"慢了多少"决定的，是由"到达率 × 等待时间是否超过线程池容量"决定的。跨过那条线，系统从"变慢"直接跳到"不可用"。

把远程调用移出事务后，持锁时间从 900ms 回到 8ms，同一 SKU 的吞吐上限变成约 125 次/秒，**是原来的 110 倍**。这也是为什么这类修复往往不需要加机器、不需要改 SQL，只需挪一行代码。

> 看到"某个接口变慢"，先别问它慢了多少毫秒，先问：**这段时间里，它占着什么别人也要用的东西？** 占着行锁，就是全局问题；只占着自己的 CPU，才是局部问题。

7.3　为什么很多团队都这么写

因为这种写法"看起来是对的"：

- 业务上确实要"扣了库存才记销量"
- 代码顺序符合业务顺序
- 本地测试全过（营销服务很快）
- CR 时没人反对——它没有语法错误，没有坏味道，只有一个看不见的时间耦合

真正的问题在于：**它把本地事务的持有时间，交给了另一个团队的发布窗口。** 营销服务哪天上了个慢功能，你们的库存就开始超时。

【必读】三个反直觉的后果

事务里放远程调用，麻烦不止"变慢"这一种：

① 上游重试会把负载放大成平方。 调用方超时后重试一次，旧请求还占着锁没释放，新请求又来排队。若有 N 层调用链都配了重试，一次下游抖动会产生 `2^N` 个持锁事务（第二章）。**这类雪崩的解药是缩短持锁时间，不是调重试次数。**

② 下游恢复了，故障还会持续一阵。 队列里积压的等锁事务不会凭空消失，它们仍要逐个抢锁、逐个超时。现场常见"下游 P99 已经回到 20ms，但订单失败率又花了十几分钟才降下来"。如果这时候有人根据"下游已恢复但系统仍异常"判断"根因不在下游"，就会第二次走错方向。

③ 先满的往往不是线程池，是连接池。 每个持锁事务都占着一个数据库连接；Hikari 默认 `maximumPoolSize=10`，比 Tomcat 的 200 小一个数量级。所以实际顺序常常是：连接池先耗尽 → 后续请求在 `getConnection()` 上等 `connectionTimeout`（默认 30 秒）→ 这些等待又占住 Tomcat 线程 → 线程池最后才满。**只盯着 Tomcat 线程池看，会晚一步才发现起点**（第四章）。

7.4　修法：把事务收窄到只包数据库

【必读】第一步：远程调用移出事务

```
// ✓ 事务只包本地写；远程调用放外面
public void deductStock(Long skuId, int qty) {
    stockTxService.deduct(skuId, qty);        // ① 短事务：只有 UPDATE + 日志
    try {
        marketingClient.recordSale(skuId, qty);   // ② 事务外，慢也不占锁
    } catch (Exception e) {
        log.warn("recordSale failed, will retry later sku={}", skuId, e);
        saleRetryQueue.offer(new SaleRecord(skuId, qty));   // ③ 失败进补偿
    }
}

@Service
public class StockTxService {
    @Transactional(rollbackFor = Exception.class, timeout = 3)
    public void deduct(Long skuId, int qty) {
        int affected = stockMapper.deduct(skuId, qty);
        if (affected == 0) {
            throw new BizException("库存不足");
        }
        logMapper.insertSaleLog(skuId, qty);
    }
}
```

三个改动各自的作用：

- `stockTxService` 独立 Bean：跨 Bean 调用才走代理（第六章）
- `timeout = 3`：给事务内的**数据库操作**一个累计上限（注意它管不到阻塞中的 HTTP 调用，见 7.6）
- 远程调用失败进队列：**从"强一致"降级为"最终一致"**，这是必须做的取舍

【必读】第二步：明确一致性等级

这一步是设计决策，不是代码技巧。要做的事：**在文档里写清楚"扣库存"和"记销量"的一致性是哪个级别**。

方案	一致性	代价	适用
同一事务（现状）	强一致	锁持有时间 = 最慢的外部调用	只适合纯本地操作
事务 + 事务后调用	扣库存强一致，销量最终一致	需补偿	**推荐**，绝大多数业务够用
事务 + 本地消息表	最终一致，可靠	需消费者	不能丢的场景
TCC / Saga	跨服务强一致	复杂度高	资金类核心链路

大多数业务"扣了库存但销量少记一笔"是可以接受的（第二天对账补上），而"库存没扣但订单创建了"是不可接受的。**分清这两种，才知道哪些能移出事务。**

【必读】第三步：事务里发 MQ 的正确姿势

很多人把"移出事务"理解成"事务提交后发消息"，但顺序写错了：

```
// ✗ 消息先发，事务后提交：事务回滚了，消息已经出去了
mqProducer.send("order-created", order);
txService.create(order);

// ✓ 用事务同步回调，提交后才发
TransactionSynchronizationManager.registerSynchronization(
    new TransactionSynchronization() {
        @Override
        public void afterCommit() {
            mqProducer.send("order-created", order);
        }
    });

// ✓ 或者更简洁：@TransactionalEventListener
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onCreated(OrderCreatedEvent event) {
    mqProducer.send("order-created", event.orderId());
}
```

注意：`AFTER_COMMIT` 只保证"事务已提交"，不保证"消息一定发出"。 进程在 `afterCommit` 和 `send` 之间崩掉，消息就丢了。要真正不丢，得用本地消息表（卷四第 18 章）。

【必读】第四步：四种 timeout 别混着调

修复里同时出现了四个"超时"，它们管的完全不是一回事。调错一个，症状会变得更难懂：

配置	单位	管的是什么	超时后发生什么
`@Transactional(timeout = 3)`	秒	该事务**内数据库操作**的累计时间	抛 `TransactionTimedOutException`，事务回滚
HTTP client `readTimeout`	毫秒	一次网络等待	抛 `SocketTimeoutException`，由你的 catch 决定
`innodb_lock_wait_timeout`	秒	一个事务**等行锁**的最长时间（默认 50）	当前语句报错回滚，事务本身可能还在
Hikari `connectionTimeout`	毫秒	从池里**借连接**的等待（默认 30000）	抛 `SQLTransientConnectionException`，根本没进事务

一个实用的配置关系：**HTTP 超时 < 事务 timeout < 锁等待 timeout < 调用方超时**。反过来的典型坏味道是：HTTP 允许 10 秒，事务 timeout 设 3 秒——事务早就被判超时了，HTTP 还在傻等，最后抛出来的异常和真正的失败点对不上，日志里全是误导信息。

【必读】传播行为不是解法

有人会想："把远程调用包进 `REQUIRES_NEW`，不就不占外层事务了吗？"两件事都不会改善：

- **连接还是占着。** `REQUIRES_NEW` 挂起外层连接、再借一个新连接，同一线程瞬时占 2 个（第六章）。远程调用的 900ms 里，新旧两个连接都被占着。
- **行锁还是被外层持有。** 外层在调用前已经 `UPDATE stock`，锁一直挂到外层提交；内层开多少个新事务都改变不了这一点。如果内层也更新同一行，还会等到 `innodb_lock_wait_timeout` 触发自死锁。

`NESTED` 更不用说——它和外层共用同一个物理事务，保存点只解决"局部回滚"，一毫秒也不会减少持锁时间。**事务边界问题的解只有一类：把不该在边界里的代码挪出去。** 换传播行为只是把边界挪了个位置。

7.5　行锁的等待时间：一个必须知道的数字

MySQL 的 `innodb_lock_wait_timeout` 默认是 **50 秒**。

这意味着：一个被阻塞的事务，会**等 50 秒**才放弃。50 秒里它一直占着一个数据库连接和一条 Tomcat 线程。

200 个 Tomcat 线程 / 50 秒 → 只要 4 个热点行被卡住，线程池就会被打满。

所以有两个可调项，但优先级不同：

```
-- ① 缩短等待：让失败来得快一些（推荐，但要考虑业务能忍多久）
SET GLOBAL innodb_lock_wait_timeout = 5;

-- ② 缩短持有：从根上减少锁时间（真正的解法）
-- 事务里不加远程调用、不 sleep、不做大计算
```

调小 timeout 是止痛药，收窄事务是治病。 止痛药有副作用：原本"等 50 秒能成功"的请求，现在 5 秒就失败了，失败率会上升。所以调之前先确认：等待是因为锁竞争，还是因为某个事务持有太久。前者要优化 SQL/加索引，后者要改代码。

【必读】一个必须说清的边界：`timeout = 3` 管不到 HTTP

前面给短事务加了 `@Transactional(timeout = 3)`，容易让人以为"事务最多 3 秒，锁最多被占 3 秒"。**这是错的，而且错得很危险。**

Spring 的事务超时不是墙上时钟的定时器，没有后台线程在 3 秒时来打断你。它的实现是把 deadline 记在 `ConnectionHolder` 上，在**每次向事务资源申请连接 / 执行语句时**检查是否超期（`ResourceHolderSupport#getTimeToLiveInMillis`）。也就是说：

```
@Transactional(timeout = 3)
public void deduct(Long skuId, int qty) {
    stockMapper.deduct(skuId, qty);          // 8ms，此处检查：未超时
    httpClient.post(url, body);              // 阻塞 900ms……甚至 30 秒
                                             // 期间没有任何 SQL，也就没有任何检查点
    logMapper.insertSaleLog(skuId, qty);     // 回来才检查：已超期 → 回滚
}
```

HTTP 卡住的时候，事务安静地占着行锁和连接，**没有人打断它**。等它终于返回、下一条 SQL 触发检查，才会抛 `TransactionTimedOutException` 并回滚——但那时锁已经多占了 30 秒，队列也已经堆满了。

所以 `timeout` 的真实定位是：**给"事务内的数据库工作"设上限，用来兜住意外的慢 SQL，不是给整个方法设墙钟上限。** 要兜住外部调用，只能靠 HTTP client 自己的超时；要兜住持锁时长，只能靠把外部调用移出事务。这一点很多资料写反了，值得在 CR 里反复讲。

7.6　怎么发现这类问题

三个信号，从弱到强：

① 慢查询日志里有 `Lock wait timeout exceeded`

```
SHOW ENGINE INNODB STATUS\G
-- 看 TRANSACTIONS 段：
-- TRANSACTION 12345, ACTIVE 3 sec
--   MySQL thread id 88, query id 999 ... update stock set ...
--   ------- TRX HAS BEEN WAITING 3 SEC FOR THIS LOCK TO BE GRANTED
```

`ACTIVE 3 sec` 就是持有时间。**如果这个数字远大于 SQL 本身的执行时间，说明锁被业务代码占着，不是在等 IO。**

② 接口 RT 上涨，但数据库 CPU 不动

数据库不忙、连接数很高、`Threads_running` 不高——典型的"等锁"特征。真忙的话 CPU 会上去。

③ APM 里"事务耗时 > SQL 耗时之和"

一个事务里有 3 条 SQL 共 15ms，但事务总耗时 900ms。中间那 885ms 去哪了？**不在 SQL 里，就在事务里的其他代码里。** 这是最快能定位的指标，如果你有事务级别的埋点。

【必读】一次完整的取证顺序

把现场那三天的排查压缩成可复用的五步，从"现象"到"证据"，每步都能在两分钟内出结果：

① 确认是等锁，不是慢 SQL

```
SELECT trx_id, trx_started, trx_mysql_thread_id, trx_query
  FROM information_schema.innodb_trx
 ORDER BY trx_started;
```

`trx_started` 距今几秒、而 `trx_query` 只是一条点查 UPDATE，就是"事务开着但没在干活"。

② 找出谁在等谁

```
SELECT * FROM performance_schema.data_lock_waits;
SELECT * FROM performance_schema.data_locks
 WHERE lock_type = 'RECORD';
```

MySQL 8.0 用 `performance_schema`，5.7 用 `information_schema.innodb_locks` / `innodb_lock_waits`，别把两套视图搞混。

③ 看事务实际活跃时长

```
SHOW ENGINE INNODB STATUS\G
-- TRANSACTIONS 段：TRANSACTION 12345, ACTIVE 3 sec
-- ------- TRX HAS BEEN WAITING 3 SEC FOR THIS LOCK TO BE GRANTED
```

`ACTIVE 3 sec` 远大于该事务所有 SQL 的耗时之和，就是业务代码占着锁的铁证。

④ 确认线程在等什么

```
jstack -l <pid> | grep -A 12 '"http-nio-8080-exec-'
```

如果大量线程停在 `SocketInputStream.read`（等 HTTP）或 `HikariPool.getConnection`（等连接），而不是停在 JDBC，方向就明确了。

⑤ 用 APM 做差值

事务 span 总耗时 900ms，其中 SQL span 加起来 15ms，**中间 885ms 就是答案所在**。这可能是全书性价比最高的一个监控指标，可惜多数 APM 默认不采集事务级 span，需要自己埋一个。

> 取证要按"锁 → 事务 → 线程 → 差值"的顺序走。反过来先抓线程 dump，会得到 200 个一模一样的 `WAITING`，看不出谁先谁后。

7.7　自检清单

- [ ] 所有 `@Transactional` 方法里没有 HTTP / RPC / MQ 发送 / 文件 IO
- [ ] 所有 `@Transactional` 都配了 `timeout`
- [ ] 需要"事务后执行"的逻辑用 `@TransactionalEventListener(AFTER_COMMIT)` 或同步回调
- [ ] 需要跨服务一致的场景，写清了用最终一致还是 TCC，文档里有
- [ ] `innodb_lock_wait_timeout` 按业务容忍度调过（不是默认 50 秒）
- [ ] 热点行的更新语句走索引（否则会锁全表扫描到的行）
- [ ] 有"事务耗时 vs SQL 耗时"的对比监控

【必读】把规矩交给机器，别交给记性

"事务里禁止远程调用"写进 CR checklist，能挡住一部分；但代码是流动的，三个月后总有人为了省事又加回去。更稳的做法是让它变成一条会失败的测试：

```
// ArchUnit：事务方法不允许调用 HTTP / MQ 客户端
@ArchTest
static final ArchRule no_remote_calls_in_transaction =
    methods().that().areAnnotatedWith(Transactional.class)
        .should().notCallMethodWhere(
            targetOwner(assignableTo(RestTemplate.class))
              .or(targetOwner(assignableTo(KafkaTemplate.class))))
        .because("事务内远程调用会把行锁持有时间交给下游（第 7 章）");
```

扫不到运行时反射和 lambda，也不该指望它全能——**它的价值是让"这是团队约定"变成"这是构建失败"**。配合每次代码改动自动跑，比每季度的规范宣讲有效得多。

只留两条人工判断给 CR：一是**一致性等级有没有写进设计文档**（7.4 第二步），二是**补偿路径有没有对账**（7.4 第一步）。这两条机器查不了，但它们才是真正决定数据会不会错的地方。

7.8　一句话总结

> 事务边界不是"这个方法做完"，是"这些数据要一起对"。 你把一个 HTTP 调用放进事务，等于把行锁的钥匙交给了别人的发布计划。

【选读】延伸

- 知识库对应原理篇：04-Spring生态/01-Spring Framework/事务管理、17-常见Bug与解决方案/03-Spring事务与AOP失效、05-数据库/01-MySQL/事务与锁

第八章　循环依赖与 Bean 初始化顺序

本章要回答：为什么老项目升级到 Spring Boot 3 之后，启动直接失败？为什么 `@PostConstruct` 里拿到的值是 null？

本章你能带走

- 三级缓存的真实作用（不是"为了性能"）
- 一张表：哪四种循环依赖**根本无解**
- 一条判断：**循环依赖是设计信号，不是配置问题**

难度构成：本章主线【必读】；含【进阶】深挖 2 处；章末「延伸」小节放支线深挖与知识库指针。---

8.1　现场：升级当天起不来

一个跑了五年的项目升 Spring Boot 3，启动报错：

```
The dependencies of some of the beans in the application context form a cycle:

   orderController
┌──────┐
|  orderService defined in file [.../OrderService.class]
↑      ↓
|  payService defined in file [.../PayService.class]
└──────┘
```

原因很简单：**Spring Boot 2.6 起，`spring.main.allow-circular-references` 默认改为 `false`。**

官方在 2.6 的发布说明里写得很直白：循环依赖是设计问题，以前能启动是"宽容"，不是"支持"。他们把这个宽容关掉了。

于是有两个选择：

```
# ✗ 打开开关（把问题延后，不是解决）
spring:
  main:
    allow-circular-references: true
```

```
// ✓ 消除环（需要改代码，但一次改对）
```

绝大多数团队选了第一个，因为"先上线再说"。这不是不能理解，但要清楚代价：**环还在，而且从"启动就报"变成"运行期才炸"**——比如加了 `@Async` 之后，注入进对方的是代理还是原始对象，行为会变。

【必读】升级当天的三个错误决策

报错摆在屏幕上，解决方案看起来有三条，我们按顺序试了两条半：

顺序	决策	当时理由	后来的代价
①	打开 `allow-circular-references: true`	"先把版本升上去，环以后再拆"	环从此不再报错，变成年终技术债清单里永远排不到的一行
②	把构造器注入改成 `@Autowired` 字段注入	"Spring 官方不就是这么解决的吗"	环隐形了；半年后给 `OrderService` 加 `@Async`，启动又炸
③	重启三次，怀疑是 Bean 加载顺序偶发	"昨天还能起"	白白花掉 40 分钟，顺序是确定的，只是没看懂报错

第 ③ 条最值得写下来：**容器创建单例的顺序是确定的，`BeanCurrentlyInCreationException` 不会偶发。** 说"偶发"，通常只是没意识到测试环境和生产环境的 Bean 数量不同，导致环被触发的路径不同。把确定性故障当偶发故障查，是排查里最贵的一类误判（第 6 章讲的是同一件事的另一面）。

第 ② 条则是本章后半段的主线：**把构造器注入改成字段注入，等于把编译期能看见的设计问题，换成了运行期才可能暴露的代理问题。** 它俩不是"两种写法"，是"两种错误暴露时机"。

当时怎么误判的：报错摆在屏幕上，第一反应是「重启试试，可能是 Bean 加载顺序偶发」——连着重启三次，当然还是失败，这条「偶发」最耽误事，因为容器创建单例的顺序是确定的，根本不会偶发。接着又试了打开 `allow-circular-references: true` 和把构造器注入改成字段注入，两者都让启动「暂时」过了，却把环藏了起来。真正扭回来的，是有人把 `BeanCurrentlyInCreationException` 的 message 认真读了一遍——那张环图已经把 `orderService → payService → orderService` 画得清清楚楚，根本不需要猜。教训是：把「确定性故障」当「偶发故障」查，是这类问题里最贵的一类误判。

8.2　三级缓存到底在解决什么

被问过一百遍的问题：″为什么是三级不是两级？″

先说结论：**不是为了性能，是为了"延迟决定要不要生成代理"+"保证代理唯一"。**

三个 Map 各管一段：

缓存	内容	作用
一级 `singletonObjects`	完整成品 Bean	正常 `getBean` 命中这一层
二级 `earlySingletonObjects`	提前暴露的引用（可能是代理）	保证多次提前引用拿到**同一个**对象
三级 `singletonFactories`	`ObjectFactory` 工厂	只在"被提前引用时"才执行，此刻才决定要不要代理

A ↔ B 的时序，用一句话概括：

> A 实例化完（还是空壳），把自己的**工厂**登记进三级缓存 → 去注入 B → B 需要 A → 从三级缓存取出工厂执行 → 得到 A 的早期引用（需要代理就此刻生成）→ 放进二级 → B 完成 → A 拿到 B，继续初始化 → A 完成，进一级。

【进阶】源码里到底发生了什么

三个缓存是 `DefaultSingletonBeanRegistry` 里的三个 Map，查找顺序固定：

```
// DefaultSingletonBeanRegistry#getSingleton（简化）
protected Object getSingleton(String beanName, boolean allowEarlyReference) {
    Object singletonObject = this.singletonObjects.get(beanName);        // ① 成品
    if (singletonObject == null && isSingletonCurrentlyInCreation(beanName)) {
        synchronized (this.singletonObjects) {
            singletonObject = this.earlySingletonObjects.get(beanName);  // ② 早期引用
            if (singletonObject == null && allowEarlyReference) {
                ObjectFactory<?> factory =
this.singletonFactories.get(beanName);  // ③ 工厂
                if (factory != null) {
                    singletonObject = factory.getObject();               // ★ 此刻才决定是否代理
                    this.earlySingletonObjects.put(beanName, singletonObject);
                    this.singletonFactories.remove(beanName);
                }
            }
        }
    }
    return singletonObject;
}
```

A ↔ B 的完整时序：

```
getBean(A)
 ├ createBeanInstance(A)                    → A 裸对象
 ├ addSingletonFactory(A, () -> getEarlyBeanReference(A))   ← 只登记工厂，不执行
 ├ populateBean(A) 发现需要 B
 │   └ getBean(B)
 │      ├ createBeanInstance(B)             → B 裸对象
 │      ├ addSingletonFactory(B, ...)
 │      ├ populateBean(B) 发现需要 A
 │      │   └ getSingleton(A, true) → 三级工厂执行
 │      │        ├ A 无需代理 → 返回原始 A
 │      │        └ A 需代理   → 当场 wrapIfNecessary，生成 early proxy
 │      │      结果放二级、删三级（保证后面再引用 A 拿到同一个对象）
 │      ├ initializeBean(B)                 → B 完成
 │      └ addSingleton(B)                   → B 入一级
 ├ A 拿到 B，继续 populateBean
 ├ initializeBean(A)                        → 若已提前代理，此处靠
earlyProxyReferences 去重
 └ addSingleton(A)                          → 清二级/三级
```

两个细节别跳过：

- `addSingletonFactory` **登记的是工厂，不是对象。** 这一层"延迟"就是三级缓存的全部意义：只有真的被提前引用了，才需要决定要不要生成代理。
- `earlyProxyReferences` **保证同一个 Bean 只有一个代理。** A 在早期引用时已经生成过代理，初始化后置阶段不会再造一个；否则容器里会出现两个代理实例、其中一个没有事务，单例语义直接破掉。

【进阶】推演一下"两级为什么不够"

方案	结果
只有一级	空壳不可见 → 无限递归
两级：实例化后立即放**原始对象**	B 注入的是原始 A，A 后续生成的代理对 B 不可见 → **B 里的事务/日志全部失效**（第六章的坑换个姿势又来了）
两级：实例化后立即**生成代理**	所有 Bean 无论有没有循环依赖都要提前代理，还可能在未就绪时触发 Advisor 解析
三级工厂（现状）	按需代理，只在真的被提前引用时才执行；二级保证唯一性

"B 注入原始 A 导致 AOP 失效"这一条尤其值得记住——它是一个"修了循环依赖、坏了事务"的经典连锁事故。如果你见过"加了 `@Transactional` 但某些调用路径没事务"，除了自调用，也要怀疑这条链路。

【必读】报错里的环怎么读

`BeanCurrentlyInCreationException` 的 message 已经把环画出来了，不需要猜：

```
The dependencies of some of the beans in the application context form a cycle:

   orderController
┌──────┐
|  orderService defined in file [.../OrderService.class]
↑     ↓
|  payService defined in file [.../PayService.class]
└──────┘
```

读法：`orderController → orderService → payService → orderService`。**箭头回到谁，谁就是要被拆的那个点**；`Currently in creation:` 后面那条链则告诉你创建到哪一步卡住了。

还有一个更容易被忽略的变体，通常出现在"打开开关跑了一段时间，后来给某个 Bean 加了 `@Async` 或新切面"之后：

```
Bean with name 'orderService' has been injected into other beans [payService]
in its raw version as part of a circular reference, but has eventually been
wrapped.
This means that said other beans do not use the final version of the bean.
```

翻成白话：**别人先拿走了原始对象，容器后来又把它包装成了代理，于是别人手里的那个"不是最终版"。** Spring 拒绝在这种状态下静默启动。这正是第 ① 个决策（打开开关）埋的雷——环还在，AOP 一加就炸，而且报错信息比原来的环更难懂。

> 这条报错也是第六章那条代理线索的另一种形态：不是"调用没走代理"，而是"注入进来的根本不是代理"。

8.3　四种根本无解的循环依赖

Spring 的三级缓存只能解决"**单例 + setter/字段注入**"这一类。其他四种，直接报错，没有配置能救：

场景	为什么无解	对策
构造器互相依赖	对象还没造出来，没东西可提前暴露	改注入方式，或 `@Lazy` 放构造参数上
prototype 互相依赖	prototype 不缓存，没有"提前引用"这回事	重构，或 `ObjectProvider`
`@Async`/`@Transactional` **+ 配置类** `@Bean` **互调 +** `proxyBeanMethods=true`	配置类增强代理形成调用环	拆配置类，或 `proxyBeanMethods=false`
`@DependsOn` **成环**	显式声明的强顺序依赖成环	去掉不必要的 `@DependsOn`，改用事件

第一种最常见：

```
// ✗ 构造器互依赖：启动即失败，无解
@Service
public class OrderService {
    private final PayService payService;
    public OrderService(PayService payService) { this.payService = payService; }
}

@Service
public class PayService {
    private final OrderService orderService;
    public PayService(OrderService orderService) { this.orderService =
orderService; }
}
```

很多人这里会犯一个错：**改成字段注入让它能启动**。

```
// ✕ 能启动了，但问题被藏起来了
@Service
public class OrderService {
    @Autowired private PayService payService;      // 编译期不再强制，环隐形了
}
```

这是最糟的处理方式。它不是"解决"，是把一个编译期可见的问题变成了运行期才暴露的问题，而且顺带引入了第六章讲的代理陷阱。

8.4　四种正确的破坏方式

按推荐度排序：

【必读】⓪ 破坏之前：先把依赖图画出来

先别急着挑方案。打开报错里的那条链，或者用 `Actuator /beans`、IDEA 的 Diagrams 把相关 Bean 的依赖画出来，然后只问一个问题：**这两个类，谁在业务上更"上游"？**

- 订单与支付互相调用，通常是**领域方向画反了**：支付不该反向调用订单，它只该接收"订单已创建"这个事实。
- 服务 A 与 B 互相需要对方的能力，通常是**有一个能力放错了类**：把共用的只读查询下沉成第三个 Bean，环自己就断了（方案 ②）。
- 实在分不出上下游，多半是这两个类**本来就该是一个类**，或者边界切得太碎。

如果跳过这一步直接上 `@Lazy`，等于承认"我不打算判断方向，只让容器别报错"。这也是为什么前面把 `@Lazy` 排在最后：**它绕开的不是环，是那次本该做的设计判断。**

【必读】① 用事件解耦（首选）

```
@Service
@RequiredArgsConstructor
public class OrderService {
    private final ApplicationEventPublisher publisher;
    private final OrderRepository repository;

    @Transactional(rollbackFor = Exception.class)
    public void create(Order order) {
        repository.save(order);
        publisher.publishEvent(new OrderCreatedEvent(order.getId()));   // 单向
    }
}

@Component
public class PayListener {
    @EventListener
    public void onCreated(OrderCreatedEvent event) {
        // 支付侧逻辑，不反向依赖 OrderService
    }
}
```

事件把"双向"变成"单向"。**而且它顺手解决了另一个问题：支付逻辑不再阻塞下单的主流程。**

注意一点：`@EventListener` 默认是**同步**的，异常会传播回发布方，也就在同一个事务里。要想异步或事务后执行，用 `@Async` + `@TransactionalEventListener(AFTER_COMMIT)`，并接受"异步无事务"的事实（第六章 Q5）。

【必读】② 拆出第三个 Bean（最干净）

```
@Service
public class OrderQueryService {          // 只读能力下沉
    private final OrderRepository repository;
    public Order find(Long id) { ... }
}

@Service
public class OrderService {
    private final PayService payService;          // 依赖支付
    private final OrderQueryService queryService; // 依赖查询
}
```

大多数循环依赖的本质是"这个类既做写又做读，另一边既要用它的写又要用它的读"。**按读写拆开，环通常自己就断了。**

【必读】③ `ObjectProvider`：显式表达"用时才取"

```
@Service
public class PayService {
    private final ObjectProvider<OrderService> orderService;

    public void reconcile(Long orderId) {
        orderService.getObject().markReconciled(orderId);   // 首次调用才解析
    }
}
```

比 `@Lazy` 好在语义：**代码里能看出"这是延迟依赖"，而不是靠一个注解暗示。**

【必读】④ `@Lazy`：最短的改法，最模糊的语义

```
public OrderService(@Lazy PayService payService) {
    this.payService = payService;   // 注入的是延迟解析代理
}
```

能用，但两个代价：一是语义藏在注解里，读代码的人不知道这里的调用会在什么时候解析；二是它只是**解耦了实例化顺序**，两个类依然互相认识——**设计问题没变**。

> 团队约定：`@Lazy` 允许出现在遗留代码的 hotfix 里，新代码禁止使用。

【必读】破坏之后必须做的一件事

改完代码、能启动，不等于环没了。至少补两条，否则半年后它会以另一种形态回来：

1. **把开关关回去。** 删掉 `spring.main.allow-circular-references: true`，让容器继续当那个"会报错的设计检查器"。开着开关做重构，等于蒙着眼睛验收。
2. **加一条会失败的测试。** 用 ArchUnit 或 Spring 的 `ApplicationContextRunner` 断言关键包之间无环；更省事的版本是直接断言启动时没有 `BeanCurrentlyInCreationException`——但那只能挡住"启动即失败"，挡不住 raw version 那类。

```
// ArchUnit: service 包内不允许成环（切片按包，粒度按团队约定）
@ArchTest
static final ArchRule no_cycles_between_slices =
    slices().matching("com.example..service.(*)..")
            .should().beFreeOfCycles();
```

破坏是改设计，不是改配置；而设计要能被机器守住，才不会随人员流动退化。

8.5　初始化顺序：另一类"静默错"

循环依赖会让启动失败，看得见。更麻烦的是**没有循环依赖、但初始化顺序错了**——启动成功，值是 null。

```
// ✕ 在 @PostConstruct 里依赖另一个 Bean 的初始化结果
@Component
public class CacheWarmer {
    @Autowired private DictService dictService;

    @PostConstruct
    public void warm() {
        dictService.loadAll();     // DictService 可能还没初始化完
    }
}
```

`@PostConstruct` 的时机是"当前 Bean 属性注入完成之后"，**不保证依赖的 Bean 已经完全初始化**。如果 `DictService` 的 `@PostConstruct` 还没跑，`loadAll()` 可能读到空数据。

更隐蔽的一种：

```
// ✕ 静态字段持有 Bean：类加载时机早于容器
@Component
public class AppContext {
    public static DictService dictService;      // 有人在 @PostConstruct 里赋值

    @Autowired
    public void setDictService(DictService s) { dictService = s; }
}
```

静态字段被谁、在什么时候赋值，完全取决于 Bean 创建顺序。测试环境 Bean 少，顺序碰巧对了；生产环境加了几个 Bean，顺序变了，值就变 null。

正确做法：把"初始化完成后再做"的逻辑，交给 `ApplicationReadyEvent`。

```
@Component
@RequiredArgsConstructor
public class CacheWarmer {
    private final DictService dictService;

    @EventListener(ApplicationReadyEvent.class)
    public void warm() {
        dictService.loadAll();      // 此时全部单例 Bean 已就绪
    }
}
```

三个事件的时机，记一下：

事件	时机	适合做什么
`@PostConstruct`	当前 Bean 注入完成	**只操作自己**
`ContextRefreshedEvent`	容器 refresh 完成	缓存预热（可能有多次 refresh）
`ApplicationReadyEvent`	应用可对外服务	**所有需要"全部就绪"的逻辑**

【必读】另外三种"启动成功但值是错的"

`@PostConstruct` 只是这一类问题的入口，还有三种长得不一样：

① 静态字段持有容器 Bean。 前面写过，赋值时机取决于创建顺序。补一句更隐蔽的：测试环境 Bean 少，顺序碰巧对了；生产环境多几个 `@Configuration`，顺序一变就是 `NullPointerException`，而且**只在生产复现**。凡是 `static` 引用了 Spring Bean 的地方，都该直接列成代码红线。

② `@PostConstruct` 里调远程接口。 启动阶段下游可能还没就绪、网络策略还没放行，一次超时能把应用启动拖到健康检查失败，容器反复重启。它和"值是 null"不同，但同属"把运行期逻辑放进了初始化期"。

③ `@DependsOn` 被当成排序工具。 `@DependsOn` 只保证"先创建"，不保证"先完成业务初始化"；两个 Bean 互相 `@DependsOn` 还直接成环。要表达"等全部就绪"，用事件，不要用依赖声明。

把这几种和 8.5 的事件时机表合起来看，其实只有一条规矩：**初始化阶段只操作自己；任何"需要别人已经好了"的逻辑，都挂到 `ApplicationReadyEvent`。** 静态字段、远程调用、`@DependsOn` 之所以容易出事，都是因为它们在偷偷违反这一条。

8.6　自检清单

- [ ] 没有 `spring.main.allow-circular-references: true`（或已列入技术债清单并标注原因）
- [ ] 所有循环依赖都消除了，不是用 `@Lazy` 绕过去的
- [ ] 没有"构造器互依赖 + 改成字段注入"这种伪修复
- [ ] `@PostConstruct` 里没有调用其他 Bean 的业务方法
- [ ] 需要"全部就绪"的初始化逻辑挂在 `ApplicationReadyEvent`
- [ ] 没有静态字段持有容器 Bean
- [ ] 配置类里没有跨 `@Bean` 方法互相调用（或已设 `proxyBeanMethods = false`）

【必读】升级迁移清单（Boot 2.5 → 3.x）

如果你们正准备做那次升级，按这个顺序走，比"报错了再查"省一天：

步骤	动作	验收
1	升级前先在当前版本设 `allow-circular-references: false`，启动一次	提前暴露所有环，不与框架升级混在一起排查
2	按 8.4 的 ⓪ 画出每个环的方向，逐个选破环方式	每处改动有"谁依赖谁"的结论，不是只加注解
3	全局搜索 `@Autowired` 字段注入、静态字段持有 Bean、`@PostConstruct` 调其他 Bean	清单清零或有豁免说明
4	升级 Spring Boot，保持开关关闭	能启动，且日志无 raw version 警告
5	给关键包加无环测试	后续 CR 由构建守住

第 1 步是关键：**先在旧版本把环暴露出来，再升级。** 两件事一起做，报错里既有框架 API 变更、又有循环依赖，很难分清哪个是根因——这也是所有大版本升级都该遵守的"一次只改一个变量"。

8.7　一句话总结

> 循环依赖不是"Spring 的一个限制"，是**你的两个类互相认识**这个事实，被容器替你指了出来。 关掉检查不会让关系消失。

【选读】延伸

- 知识库对应原理篇：04-Spring生态/06-Spring源码专题/循环依赖、04-Spring生态/06-Spring源码专题/Bean创建流程、04-Spring生态/01-Spring Framework/Bean生命周期

第九章　线程池：任务去哪了

本章要回答：异步任务"提交成功了"，为什么什么都没发生？服务为什么 CPU 只有 10% 却完全无响应？

本章你能带走

- 三个默认值炸弹（无界队列 / 无限线程 / 默认拒绝静默）
- 一条几乎没人注意的真相：`max` **参数在队列很大时永远是摆设**
- 一个高频事故模型：`submit()` + 不 `get()` = 异常人间蒸发

难度构成：本章主线【必读】；含【进阶】深挖 2 处；章末「延伸」小节放支线深挖与知识库指针。---

9.1　现场：两种"假死"

同一个服务，两周内出了两次事故，症状完全相反。

第一次：OOM。

```
java.lang.OutOfMemoryError: Java heap space
```

dump 下来，80% 的堆是 `LinkedBlockingQueue$Node` 链。原因：一个导出功能在促销日被点了 3000 次，任务全堆在队列里，每个任务持有 Excel 上下文。

第二次：CPU 10%，线程 200 个全 BLOCKED，接口全超时。 服务像死了一样，但 CPU 几乎不动。

第一次是"任务太多"，第二次是"线程全卡住"。看起来是两个问题，**其实是同一个问题的两面：你从来没有真正控制过这个池子。**

当时怎么误判的：第一次 OOM，第一反应是「导出功能写得太重、Excel 上下文没释放」，顺着堆 dump 去查大对象，方向沾边却没摸到根——根是无界队列把任务无限堆积。第二次更迷惑：服务像死了一样，可 CPU 只有 10%，于是先去查 GC、查下游 RT，都「正常」，这条「CPU 不高就是没忙」差点把人带偏。真正扭回来的，是 `jstack` 一拉，四十多个线程齐刷刷停在 `FutureTask.get` 上——有人在无超时地 `get()` 一个再也不会返回的下游。这一下把方向从「查 CPU / 查 GC」扭回「查线程在等什么」：`CPU 低 + 线程全 BLOCKED` 几乎可以判定为「有人在无超时地等」。

9.2　三个默认值炸弹

【必读】炸弹一：`Executors` 的工厂方法

```
// ✗ 队列容量 = Integer.MAX_VALUE（等价无界）
ExecutorService pool1 = Executors.newFixedThreadPool(8);
ExecutorService pool2 = Executors.newSingleThreadExecutor();

// ✗ 最大线程数 = Integer.MAX_VALUE（来多少建多少）
ExecutorService pool3 = Executors.newCachedThreadPool();
```

`newFixedThreadPool` 用的是 `new LinkedBlockingQueue<Runnable>()`——**不传容量时容量是 `Integer.MAX_VALUE`**。生产者比消费者快时，任务无限堆积：

- 每个任务对象持有上下文（请求参数、闭包、大数据）
- 堆慢慢涨，直到 `OOM: Java heap space`
- MAT 打开 dump，看到的是 `LinkedBlockingQueue$Node` 占掉七八成堆

`newCachedThreadPool` 是另一个方向的炸法：`maximumPoolSize = Integer.MAX_VALUE` + `SynchronousQueue`（不存任务），来一个建一个线程。每线程默认栈 1MB，几千个线程先触发：

```
java.lang.OutOfMemoryError: unable to create new native thread
```

这个 OOM 和堆大小无关，加 `-Xmx` 一点用没有。它受 OS 线程数（`ulimit -u`）和容器 pids limit 约束。

> 阿里开发手册把"禁用 Executors 创建线程池"列为【强制】，原因就是这两个。

【必读】炸弹二：队列太大，max 成了摆设

```
// ✗ 看起来配了 max=16，实际线程数永远停在 8
new ThreadPoolExecutor(8, 16, 60L, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(100_000));
```

线程池的提交流程是固定顺序，**不是"先开到 max"**：

```
1. 运行线程 < core          → 建核心线程执行
2. 核心满了                 → 进队列排队          ← 卡在这里
3. 队列也满了               → 才建非核心线程到 max
4. 线程到 max 且队列满      → 拒绝策略
```

队列 10 万，意味着第 3 步几乎永远到不了。**你配的 max=16 是一个永远不会被使用的数字。**

想让 max 生效，队列必须有界且**相对小**——小到"队列满了以后应该扩线程"这件事真的会发生。典型值：core=8, max=16, queue=200。

【必读】炸弹三：默认拒绝策略会"静默丢"

```
// ✗ DiscardPolicy：丢任务，不报错，不告警，无痕迹
new ThreadPoolExecutor(8, 8, 0, SECONDS, new ArrayBlockingQueue<>(100),
        new ThreadPoolExecutor.DiscardPolicy());
```

四个策略的取舍：

策略	行为	什么时候用	风险
`AbortPolicy`（默认）	抛 `RejectedExecutionException`	大多数场景，**快速失败**	调用方没 catch 会抛到用户
`CallerRunsPolicy`	提交者线程自己执行	需要背压、任务可降级为同步	**提交者是 Tomcat/Netty IO 线程时会拖死容器**
`DiscardPolicy`	静默丢弃	几乎不适用	丢数据且无痕迹，最危险
`DiscardOldestPolicy`	丢队首再试	允许丢旧数据的日志类	同样静默

`CallerRunsPolicy` 的坑值得单独说：它在"线程池是自己业务线程"的场景下是优雅的背压；但如果提交动作发生在 **Tomcat worker 线程**里，被拒绝的任务会被这个 worker 同步执行——**容器的 IO 线程被拖去做业务，吞吐直接塌。** 用之前先问：谁在提交？

【进阶】execute() 的三分支：把"max 是摆设"钉到源码上

9.2 用四句话概括了提交顺序，但"队列满了才扩到 max"到底由哪几行代码决定，值得摊开看一次。`ThreadPoolExecutor#execute` 的主体不过二十来行，全部压在一个 `ctl` 变量上：

```
public void execute(Runnable command) {
    if (command == null) throw new NullPointerException();
    int c = ctl.get();
    if (workerCountOf(c) < corePoolSize) {            // 分支① 建核心线程
        if (addWorker(command, true)) return;
        c = ctl.get();
    }
    if (isRunning(c) && workQueue.offer(command)) {    // 分支② 入队
        int recheck = ctl.get();
        if (!isRunning(recheck) && remove(command)) reject(command);
        else if (workerCountOf(recheck) == 0) addWorker(null, false);
    }
    else if (!addWorker(command, false))               // 分支③ 扩到 max
        reject(command);                               // 分支④ 拒绝
}
```

`ctl` 是一个 `AtomicInteger`：高 3 位存运行状态，低 29 位存工作线程数，`workerCountOf(c)` 就是 `c & CAPACITY`。把这条判定画成流程，三分支的边界一目了然：

```
                    execute(command)
                          |
          workerCountOf(c) < corePoolSize ?
              | 是                    | 否
              ▼                       ▼
   addWorker(command, true)     isRunning && workQueue.offer()
   建核心线程，任务直接跑          | 入队成功          | offer 返回 false
                                  ▼                   ▼
                          recheck 一次状态      addWorker(command, false)
                          必要时补个空 worker   用 maximumPoolSize 判定
                                                  | 仍失败
                                                  ▼
                                            handler.reject()
```

关键在分支②：`offer` 一旦成功就返回了，**根本走不到分支③**。所以"max 生效"的源码级条件不是"线程数到了 core"，而是"`offer` 返回 false，也就是队列满了"。队列传 `new LinkedBlockingQueue<>(100_000)`，意味着要往里堆满十万个任务才会触发扩容；而在那之前，8 个核心线程早已停在那里不动了。`addWorker` 的第二个布尔参数才是两个阈值的分水岭：`true` 拿 `corePoolSize` 比，`false` 拿 `maximumPoolSize` 比——同一个方法，两把尺子。

反常识点：多数人以为线程池"先扩线程、再排队"，标准的 `ThreadPoolExecutor` 恰恰相反，是"先入队、后扩线程"。设计意图是优先复用已有线程、避免频繁创建销毁。真正"先扩到 max 再入队"的是 Tomcat 的 `TaskQueue`：它改写了 `offer()`，在线程数还小于 max 时故意让入队失败，把 `execute` 逼进分支③，好让 Web 请求优先多开线程而不是排队。**同一个 `execute`，换一个队列实现，扩容行为就翻了过来**——所以背下"四步流程"不如记住"`offer` 卡在哪一步"。

顺手算一笔：core=8、单任务 20ms，8 个线程满负荷吞吐是 `8 / 0.02 = 400 任务/秒`。要让十万容量的队列堆满、触发分支③，生产者得长期以远高于 400/s 的速率灌入且十分钟不回落；现实里队列还没堆到那个量级，堆内存先被任务对象撑爆了。**"max 生效"和"OOM"在赛跑，超大队列让 OOM 稳赢**（9.2 炸弹一的源码解释）。

补一句容易被跳过的：`ctl` 的高 3 位还压着线程池的五个状态（RUNNING、SHUTDOWN、STOP、TIDYING、TERMINATED），低 29 位才是 workerCount。分支②入队成功后那段 `recheck` 不是冗余——从 `offer` 到 `recheck` 之间，另一个线程可能已经调了 `shutdown()`；此时要把刚入队的任务 `remove` 掉并走拒绝，否则任务会永远躺在队列里没人执行。这也解释了为什么"入队"和"确认池还活着"必须分成两步写。一个判据：**队列里的任务数，只有在池处于 RUNNING 时才等于"待执行数"**；一旦开始关闭，`getQueue().size()` 就不再代表任何承诺——9.5 要求把 `queued` 接进监控，前提正是池还活着。

顺带把 `execute` 与 `submit` 在源码上的分岔点说清：`submit` 最终也调 `execute(futureTask)`，三分支一步没少。所以"submit 的任务排队 / 被拒绝"和 execute 完全同构，差别只在异常去向（9.3）。**池参数、队列、拒绝策略对两种提交一视同仁，别以为换了 submit 就能绕开容量模型。**

9.3　事故模型：`submit()` 吞掉了异常

这是"任务执行了但什么都没发生"的第一大根因，也是第二次事故的直接原因。

```
// ✗ 异常人间蒸发
Future<?> f = pool.submit(() -> riskyCall());
// 没有 f.get() → riskyCall 抛的异常被封装进 Future，永远没人看
```

两种提交方式的异常行为**完全不同**：

方法	异常去向	结果
`execute(Runnable)`	抛给线程的 `UncaughtExceptionHandler`	至少进日志
`submit(...)`	**封装进返回的 Future**	不 `get()` 就永远消失

```
// ✓ 方案一：任务内兜底（最省事，推荐）
pool.execute(() -> {
    try {
        riskyCall();
    } catch (Throwable t) {
        log.error("async task failed", t);
        throw t;
    }
});

// ✓ 方案二：get 带超时，集中处理
try {
    Result r = future.get(3, TimeUnit.SECONDS);
} catch (TimeoutException e) {
    future.cancel(true);
    return fallback();
} catch (ExecutionException e) {
    log.error("task failed", e.getCause());     // 真正的异常在这里
}
```

顺带一个高频致命写法：

```
// ✗ get() 不带超时 = 永久阻塞许可
Result r = future.get();     // 下游假死 → 当前线程永远 BLOCKED
```

第二次事故就是这么来的：40 个业务线程全卡在 `get()` 上，等一个再也不会返回的下游。**CPU 低、线程 BLOCKED、接口全超时**——这个组合几乎可以判定为"有人在无超时地等"。`jstack` 一看，40 个线程停在 `FutureTask.get`。

【必读】submit 的异常存进了哪里：FutureTask#setException 与 get() 的交接

9.3 那张表说异常"封装进返回的 Future"，但"封装"这个词太笼统。`AbstractExecutorService#submit` 先把任务包成 `FutureTask`（`newTaskFor` → `new FutureTask<>(callable)`），再交给 `execute`。**池子里跑的从来不是你写的 Runnable，是 `FutureTask`**。异常就是在这一层被截住的：

```
// FutureTask#run（简化）
public void run() {
    try {
        Callable<V> c = callable;
        if (c != null && state == NEW) {
            V result; boolean ran;
            try { result = c.call(); ran = true; }
            catch (Throwable ex) { result = null; ran = false; setException(ex);
}  // ★
            if (ran) set(result);
        }
    } finally { /* runner = null; 处理中断 */ }
}

protected void setException(Throwable t) {
    if (UNSAFE.compareAndSwapInt(this, stateOffset, NEW, COMPLETING)) {
        outcome = t;                                    // 异常存进 outcome 字段
        UNSAFE.putOrderedInt(this, stateOffset, EXCEPTIONAL);
        finishCompletion();                             // 唤醒阻塞在 get() 的线
程
    }
}
```

`setException` 只做三件事：把 `Throwable` 放进 `outcome` 字段、把状态从 `NEW` 推到 `EXCEPTIONAL`、唤醒等待者。**它不打印、不抛出、不通知任何日志框架**——异常此刻变成了一个躺在对象字段里的普通引用。真正把它变回异常的是 `get()`：

```
// FutureTask#get → awaitDone → report
private V report(int s) throws ExecutionException {
    Object x = outcome;
    if (s == NORMAL) return (V) x;
    if (s >= CANCELLED) throw new CancellationException();
    throw new ExecutionException((Throwable) x);        // ★ 包一层才抛出
}
```

```
submit(task) → FutureTask(task) → execute(futureTask) → worker 调
futureTask.run()
      c.call() 抛异常 → setException(t): outcome = t, state = EXCEPTIONAL
      ├─ 无人 get()        → outcome 无人读，随 Future 被 GC，异常彻底消失
      └─ 有人 get(timeout) → report() → throw new ExecutionException(t)
```

一句话：**异常不是"被打了日志"，是被存进了一个字段；`get()` 不是"触发计算"，是把 `outcome` 里的 `Throwable` 包成 `ExecutionException` 重新抛出来**。这也正是 9.3 里 `e.getCause()` 才是真异常的原因——`ExecutionException` 只是那层包装纸。这个"包一层就没人看得见根因"的形态，第 11 章讲自定义异常丢 `cause` 时还会原样再见一次。

【必读】afterExecute 的正确写法：用 submit 时 t 恒为 null

顺着上面的链路，能拆掉一个流传很广的误判。**多数人以为给线程池重写 `afterExecute(Runnable r, Throwable t)` 就能兜住所有异常，实际上只要任务是 `submit` 提交的，`t` 永远等于 null。** 原因就在 `runWorker()`：只有 `task.run()` 直接抛出的 `Throwable` 才会作为第二个参数传进 `afterExecute`；而 `FutureTask.run()` 早把异常 `catch` 掉、塞进 `outcome` 了，压根没往 `runWorker` 外面抛。于是钩子拿到的 `r` 是那个 `FutureTask`，`t` 却是空的。

```
// ✗ 以为能拿到异常：submit 的任务，t 恒为 null，什么都不会打
@Override protected void afterExecute(Runnable r, Throwable t) {
    super.afterExecute(r, t);
    if (t != null) log.error("task failed", t);
}

// ✓ 官方 Javadoc 范例：识别 Future，主动 get() 把异常"钓"出来
@Override protected void afterExecute(Runnable r, Throwable t) {
    super.afterExecute(r, t);
    if (t == null && r instanceof Future<?>) {
        try {
            ((Future<?>) r).get();
        } catch (CancellationException ce) {
            t = ce;
        } catch (ExecutionException ee) {
            t = ee.getCause();                  // 取回真正的异常
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt(); // 恢复中断标志
        }
    }
    if (t != null) log.error("uncaught in async task", t);
}
```

这里的 `get()` 不带超时是安全的：任务既然走到了 `afterExecute`，说明 `run()` 已经结束、状态已是终态，`get()` 会立刻返回，不会阻塞。它和 9.3 那句"`get` 必须带超时"并不矛盾——那条约束的是**还没跑完**的 Future，这条读的是**已经跑完**的 Future。

顺带修正 9.3 表格里一处偏乐观的说法：`execute` 的异常写的是"抛给 `UncaughtExceptionHandler`，至少进日志"。更准确的是——默认 `ThreadFactory` 只把它打到 `System.err`，是否落进应用日志取决于容器怎么重定向标准错误；**而 `submit` 的异常连 `UncaughtExceptionHandler` 都到不了**，因为它被 `FutureTask.run()` 吞在了内部。所以"配了 handler 就高枕无忧"是假象：handler 能兜住的只有 `execute` 提交的任务。

结论是兜底要成对做：任务内 `try-catch` 记日志（9.3 方案一）管 `execute`，`afterExecute` 里主动 `get()` 管 `submit`，需要返回值的场景再加带超时的 `get`。三条各管一段，缺一条就有一段静默。

9.4　ThreadLocal：串台比泄漏更可怕

线程池复用的副作用，是两个：

① 内存泄漏

`ThreadLocalMap` 的 `Entry` 继承 `WeakReference<ThreadLocal>`：**key 是弱引用，value 是强引用**。ThreadLocal 对象被回收后 key 变 null，但 `线程 → ThreadLocalMap → Entry → value` 这条强引用链还在。线程是池化的长驻对象，**线程不死，value 不释放**。

② 数据串台（更危险）

```
// ✗ 不 remove：下一个请求可能读到上一个用户
private static final ThreadLocal<LoginUser> CTX = new ThreadLocal<>();

public void handle(Request req) {
    CTX.set(req.getUser());
    service.process();        // 抛异常时 remove 被跳过
    CTX.remove();             // 而且不在 finally 里 = 等于没写
}
```

worker 处理完请求 A 归还池中，下个请求 B 复用同一线程，`CTX.get()` 读到的是 A 的用户。**表现为"偶发拿到别人的登录态/租户 ID"——这是数据事故，不是性能问题。**

```
// ✓ 唯一正确写法
public void handle(Request req) {
    try {
        CTX.set(req.getUser());
        service.process();
    } finally {
        CTX.remove();          // 必须 finally
    }
}
```

`InheritableThreadLocal` 也救不了跨线程池传值：它只在**线程创建那一刻**从父线程拷贝一次，而 worker 早就创建好了，之后父线程改值它看不到。跨池传递用阿里开源的 `TransmittableThreadLocal`（TTL），它通过装饰 `Runnable`/线程池在**提交时快照、执行时回放**。

```
ExecutorService ttlPool = TtlExecutors.getTtlExecutorService(pool);
```

【进阶】一段能跑出串台的复现：一个线程的池就够了

9.4 说"下一个请求可能读到上一个用户"，这句话最好用一段能跑的测试钉死，别停在口头。单线程池把"复用"变成必然，不需要压测、不需要并发：

```
@Test
void threadLocalLeaksAcrossRequests() throws Exception {
    ExecutorService pool = Executors.newSingleThreadExecutor();  // 只有 1 个
worker，必复用
    ThreadLocal<String> CTX = new ThreadLocal<>();

    pool.submit(() -> CTX.set("用户A")).get();          // 请求 A：set 后故意不
remove
    String seenByB = pool.submit(() -> CTX.get()).get(); // 请求 B：什么都没
set，直接读

    assertEquals("用户A", seenByB);   // ✓ 断言成立：B 读到了 A 的上下文
    pool.shutdown();
}
```

这个测试**故意让断言通过**——它证明的不是"代码对"，而是"漏洞真实存在"。把 `CTX.set` 换成登录态、租户 ID 或 traceId，就是 9.4 说的那类数据事故。再加一段对照，证明 `finally remove` 确实补上了缺口：

```
// ✓ 对照：加上 finally，第二条任务读到 null
pool.submit(() -> { try { CTX.set("用户A"); } finally { CTX.remove(); }
}).get();
assertNull(pool.submit(() -> CTX.get()).get());
```

线上怀疑串台时，取证顺序是：`jstack -l <pid> | grep "order-async-"` 先看池子里 worker 的数量和名字（9.5 坚持命名前缀就是为了这一刻），再用 `jmap -dump:format=b,file=heap.hprof <pid>` 配 MAT 看 `ThreadLocalMap$Entry` 里 value 的类型与数量。**能稳定复现的串台反而好查，线上几乎都是偶发的**——因为它取决于"上一次请求有没有走到 finally"，而这跟异常路径有关：恰恰是出错的那次请求漏了 `remove`，下一位用户才中招。

【必读】TransmittableThreadLocal 的边界：传引用、要装饰、不包治百病

先补正 9.4 一处只说了一半的话：原文讲 `InheritableThreadLocal` "之后父线程改值它看不到"。更危险的另一半是——**worker 复用时，它会一直攥着上一次任务创建那一刻拷贝的旧值**，也就是"串台"同样适用于它。线程池里的 worker 只在首次创建时从父线程拷一次，之后父线程怎么改都与它无关，它停在了一个陈旧帧上。所以那句话该读成：`InheritableThreadLocal` 在线程池下不是"看不到新值"，是"永远停在一个旧值上"。

TTL（`TransmittableThreadLocal`）靠"提交时快照、执行时回放、执行后恢复"三拍解决，但有三个边界，用之前必须清楚：

边界	具体是什么	判据
必须装饰	要么 `TtlExecutors.getTtlExecutorService(pool)` 装饰池，要么 `TtlRunnable.get(task)` 装饰任务	两者都不做，TTL 退化成普通 `InheritableThreadLocal`
装饰要在最外层	快照发生在被装饰的 `Runnable` 被提交那一刻	中间若又被别的包装类套了一层原始 `Runnable`，快照链就断了
传的是引用	快照保存的是对象引用，不是深拷贝	value 若可变（`Map`、`User`），子线程改字段，父子看到的是同一个对象

第三条最容易被当成 bug：想隔离，就得自己存不可变快照。另外 TTL 只管"任务提交"这一条路径，`parallelStream` 用的 `ForkJoinPool`、`CompletableFuture` 默认池、JDK 21 虚拟线程、第三方库自起的线程，各有各的装饰方式，别假定一个 `getTtlExecutorService` 全覆盖。

这条线索和另外两章是同一根线：第 6 章 Q5 讲"换线程 = 换事务"，根因正是 `TransactionSynchronizationManager` 用 `ThreadLocal<Map<DataSource, ConnectionHolder>>` 把连接绑在线程上——**事务上下文和这里的登录态，是同一个 ThreadLocal 机制的两副面孔**，线程池一复用就一起失效。第 7 章那次"事务里做远程调用"的池子若要传上下文，踩的也是它。至于"多开线程能否救场"，第 4 章连接池与第 7 章第③点已经给了答案：先满的往往不是线程池，是 Hikari 默认只有 10 个连接的连接池。

泄漏与串台该怎么区分，给一条取证判据：串台是"值不对"，通常能靠日志里的用户 ID 对上；泄漏是"老年代缓慢上涨"，MAT 里看 `ThreadLocalMap$Entry` 的数量只增不减。前者加 `finally remove` 就好，后者还要查是不是把大对象（一次查询的全量结果、整个请求体）塞进了 ThreadLocal——`remove` 只解绑引用，塞进去的对象若还被别处持有，一样回收不掉。**两者都靠 `remove` 收口，但一个是数据事故、一个是容量事故，别混着查。** 另外 `InheritableThreadLocal` 的泄漏更隐蔽：值会随子线程创建一路拷贝，父线程早已结束，那批拷贝还挂在各条 worker 上。

TTL 还有一条部署边界值得写进 CR：`TtlExecutors.getTtlExecutorService(pool)` 返回的是**包装后的 ExecutorService**，如果框架（某些 RPC / 任务调度库）内部又拿原始 `pool` 去提交，装饰就绕过去了。稳妥做法是"对外只暴露装饰后的那个实例"，并把断言加进单测：提交一个任务，断言子线程 `TTL.get()` 等于主线程设置的值。**跨线程传值这类问题，靠约定守不住，得靠一条会失败的断言。**

9.5　一个可用的池子长什么样

```
ThreadPoolExecutor bizPool = new ThreadPoolExecutor(
        8,                                          // core：CPU 密集 ≈ 核数，IO
密集按阻塞系数放大
        16,                                         // max：队列有界时才有意义
        60L, TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(200),               // 有界：堆积上限可见
        new ThreadFactoryBuilder()
                .setNameFormat("order-async-%d")     // jstack 里一眼认出
                .setUncaughtExceptionHandler((t, e) -> log.error("uncaught {}",
t.getName(), e))
                .build(),
        (r, executor) -> {                           // 自定义拒绝：告警 + 落补偿
            alert.send("order-async rejected");
            rejectMapper.insert(r.toString());
            throw new RejectedExecutionException("order-async saturated");
        });

// 提交处兜底：绝不让任务无声消失
try {
    bizPool.execute(() -> doAsync(task));
} catch (RejectedExecutionException e) {
    log.warn("pool saturated, degrade to sync", e);
    doAsync(task);                                   // 或丢进 MQ 延时重试
}
```

三个必须暴露的指标：

```
Metrics.gauge("pool.active",  bizPool, ThreadPoolExecutor::getActiveCount);
Metrics.gauge("pool.queued",  bizPool, p -> p.getQueue().size());
Metrics.gauge("pool.completed", bizPool,
ThreadPoolExecutor::getCompletedTaskCount);
```

`pool.queued` **是唯一能提前预警的指标。** 队列水位持续上涨，比 OOM 早几分钟到几十分钟；而 `active` 打满只是"现在很忙"，未必出事。

【必读】线程数不是拍脑袋：CPU 密集 / IO 密集的估算式

9.5 的样例池写了 `core=8`，这个 8 从哪来？《Java Concurrency in Practice》给的估算式（Goetz）是：

```
N_threads = N_cpu × U_cpu × (1 + W/C)

N_cpu = Runtime.getRuntime().availableProcessors()    CPU 核数
U_cpu = 目标 CPU 利用率（0~1）
W/C   = 等待时间 / 计算时间
```

任务类型	W/C	取 U_cpu	结果	判据
CPU 密集（加解密、序列化、计算）	≈ 0	1	≈ N_cpu，工程上常写 N_cpu + 1	几乎没有阻塞，多开只会增加上下文切换
IO 密集（查库、调 RPC、写文件）	远大于 0	1	N_cpu × (1 + W/C)	大部分时间挂在等，要靠多线程把 CPU 填满

举一个能算的：一次任务计算 10ms、等下游 90ms，`W/C = 9`，8 核、目标打满 CPU，则 `N_threads = 8 × 1 × (1 + 9) = 80`。业务含义是——**要吃满 8 个核，得有 80 个线程同时在"等"**，因为每个线程 90% 的时间是挂起的。把 core 设成 8，CPU 只会用到约 10%，这正是 9.1 第二次事故"CPU 只有 10% 却无响应"的另一种成因：不一定是卡死，也可能是线程数压根不够把等待时间填满。

队列容量同样能算，别拍 `200`。按 Little's law `L = λ × W`（定义与符号见第 4 章）：峰值 `λ = 1000 任务/秒`、允许排队 `W = 0.2 秒`，稳态队列长度 `L = 200`。设 200 刚好容纳这一波突发；设 10 万等于放弃背压（9.2 炸弹一），设 10 又会让拒绝策略频繁触发。**队列是"能容忍排多久"的量化结果，不是随手填的整数。**

三条边界比公式本身重要。① `W/C` 要**实测**：用 APM 拉一次任务里 IO 等待与 CPU 时间的占比，公式给起点、压测定终点。② 线程池再大也突破不了下游容量：80 个线程抢 Hikari 默认 10 个连接，多出的 70 个只是在 `getConnection()` 上排队（第 4 章、第 7 章第③点），**线程数要和下限最紧的那层对齐**。③ 版本区间：JDK 21 起 IO 密集可用 `Executors.newVirtualThreadPerTaskExecutor()`，虚拟线程不池化、不设 core/max，改用 `Semaphore` 限下游并发；但上面的估算式对 JDK 8/11/17 的平台线程池仍然成立。Spring Boot 3.x 默认跑在 JDK 17+，别把虚拟线程的结论照搬回平台池。

把估算和拒绝策略连起来看，才是完整的容量模型。仍用上面的 `λ=1000/s`、服务率 `μ=8/0.02=400/s`：一旦 `λ>μ`，队列按 `λ-μ=600/s` 的速度线性增长，200 的队列 `200/600≈0.33 秒` 就满，之后全靠拒绝策略决定命运。`AbortPolicy` 快速失败把压力还给上游，`CallerRunsPolicy` 让提交线程自己跑、相当于把 `λ` 压回 `μ` 附近（这正是它叫"背压"的原因），而 `DiscardPolicy` 让 `λ` 表面依旧 1000/s、只是悄悄丢掉 600/s。**三者里只有背压真正改变了到达率，另两个只是改变了"谁承担损失"**（第 7 章 Little's law 的同一个式子）。

最后给一张把三者对齐的表，配池时横着过一遍：

配置项	由什么决定	配错的典型症状
core / max	`N_cpu × U_cpu × (1+W/C)` + 下游最紧的容量	CPU 低但 RT 高（线程不够），或大量超时（线程多于连接池）
队列容量	`L = λ × W`，W 是能容忍的排队时长	无界 → OOM；过小 → 拒绝策略频繁触发
拒绝策略	提交方是谁、能不能背压	`CallerRunsPolicy` 用在 IO 线程 → 容器吞吐塌

9.6 自检清单

- [ ] 没有用 `Executors.*` 建池，全部手动 `new ThreadPoolExecutor`
- [ ] 队列有界，且大小与 core/max 匹配（队列太大 = max 失效）
- [ ] 线程有命名前缀（出事时 `jstack` 能认出来）
- [ ] 拒绝策略明确，不是 `DiscardPolicy`；用了 `CallerRunsPolicy` 的确认过提交方不是 IO 线程
- [ ] 所有 `submit` 要么 `get`，要么任务内 try-catch
- [ ] 所有 `get` 都带超时
- [ ] 所有 `ThreadLocal` 都在 `finally` 里 `remove`
- [ ] 跨线程池传上下文用 TTL，不用 `InheritableThreadLocal`
- [ ] 暴露了 `active / queued / completed` 三个指标

9.7　一句话总结

> 线程池不是"把任务扔出去就不管"的管道，它是一个**有容量上限、有拒绝语义、有异常去向**的系统。 你不定义这三件事，它就替你定义——而它的默认值，是按 demo 的规模写的。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/02-并发与线程池坑、02-Java进阶/02-并发编程/线程池、02-Java进阶/02-并发编程/ThreadLocal

第十章　并发下的集合与可见性

本章要回答： `ConcurrentModificationException` 为什么单线程也会抛？为什么加了 `synchronized` 还是读到旧值？

本章你能带走

- 一个反直觉的事实：**CME 和并发无关**
- 一个最阴险的 bug 类型：`equals` / `hashCode` 不成对，不报错、只丢数据
- 一条判断：`volatile` 管可见性，不管原子性

难度构成： 本章主线【必读】；含【进阶】深挖 2 处；章末「延伸」小节放支线深挖与知识库指针。---

10.1　现场：发布后用户数据"串了"

一次上线后，客服陆续报"我明明看的是张三的订单，点进详情变成李四的"。

日志里没有异常，数据库是对的，接口返回也是对的——**只有在特定路径下才串**。

最后定位到一段缓存代码：

```
// ✕ 用 HashMap 做本地缓存，多线程读写
private final Map<Long, User> cache = new HashMap<>();

public User get(Long id) {
    User u = cache.get(id);
    if (u == null) {
        u = userMapper.selectById(id);
        cache.put(id, u);          // 并发 put：可能触发扩容
    }
    return u;
}
```

`HashMap` 在并发 `put` 时，JDK 7 会形成环形链表导致 CPU 100%（经典的"并发扩容死循环"）；**JDK 8 改成了尾插，不再死循环，但依然会丢数据、丢 entry、甚至 `get` 到 null 后返回错误的引用。**

JDK 8 修复了"死循环"，很多人以为"那就安全了"——**没有，它只是从"卡死"变成了"悄悄错"**，而后者更难查，因为不报错。

这类 bug 的共性：**它不崩，它只是让数据变得不可信。**

当时怎么误判的：客诉「张三的订单点开变李四」，第一反应是「缓存读串了或者序列化有问题」，于是在缓存层加日志、换 JSON 工具，都是白查；又怀疑「是不是数据库里就存错了」，拉 SQL、对账流水，数据又都是对的——这条「查库」是完全走弯的死线索，因为问题根本不在存储。真正扭回来的，是有人注意到这段代码用的是 `HashMap` 做本地缓存，且读写都在多线程下，换成 `ConcurrentHashMap` 后串数据消失。教训很硬：这类「不报错、只错数据」的并发 bug，测试环境几乎复现不出来，只能靠规约（共享 Map 一律用并发容器）防，不能靠「我先查查」。

10.2　CME 和并发没有关系

这是最常被误解的一条。

```
// ✗ 单线程，照样抛 ConcurrentModificationException
// 注意：列表里要留一个「被删元素之后」的元素（这里 "d"）。
// 否则删完直接到末尾、没机会再调 next()，foreach 会静默结束、不抛 CME
List<String> list = new ArrayList<>(List.of("a", "b", "c", "d"));
for (String s : list) {
    if (s.equals("b")) {
        list.remove(s);
    }
}
// Exception in thread "main" java.util.ConcurrentModificationException
```

原因不是"另一个线程改了"，而是：

- `foreach` 编译成 `Iterator`
- `Iterator` 每次 `next()` 前检查 `modCount == expectedModCount`
- `list.remove()` 让 `modCount++`，`expectedModCount` 没变
- 检查失败 → 抛 CME

注意：`next()` 只在"还有下一个元素"时才会被调用。若被删元素之后已经没有剩余元素（例如刚好删的是迭代末尾那个），循环会直接结束，连 `next()` 都没机会再调，自然不抛 CME——这正是很多人以为"我试过，没抛"的原因：把列表多留一个元素（如上面示例的 "d"），它就会稳定抛出。

这是 fail-fast 设计，不是并发错误。 名字里的 "Concurrent" 误导了一代人。

```
// ✓ 三种正确写法
// ① Iterator.remove
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) it.remove();
}

// ② removeIf（JDK 8+，最简洁）
list.removeIf(s -> s.equals("b"));

// ③ 收集后批量删
List<String> toRemove = list.stream().filter(s -> s.equals("b")).toList();
list.removeAll(toRemove);
```

反过来说：**真正的并发修改，`ArrayList` 不一定抛 CME，而是给你一个错的结果。** CME 是"运气好的情况"——它至少让你知道了。

【进阶】modCount 与 expectedModCount：把 CME 钉到源码上

10.2 给了一段单线程复现，但"`modCount` 对不上就抛"这句话值得落到源码。它不是并发检测，是一次普通的整数比较：

```
// AbstractList：结构性修改计数器
protected transient int modCount = 0;

// ArrayList.Itr（迭代器）
private class Itr implements Iterator<E> {
    int expectedModCount = modCount;          // 创建迭代器时快照一次

    public E next() {
        checkForComodification();
        ...
    }
    final void checkForComodification() {
        if (modCount != expectedModCount)
            throw new ConcurrentModificationException();
    }
    public void remove() {
        ...
        expectedModCount = modCount;          // 只有迭代器自己删，才同步回去
    }
}
```

`foreach` 编译后就是 `Iterator` 遍历。`list.remove(s)` 走的是 `ArrayList` 的 `fastRemove`，它只做 `modCount++`，**不会去改任何一个已存在迭代器的 `expectedModCount`**；等下一次 `next()` 调 `checkForComodification()`，两个整数不等，异常抛出。这就是"单线程也 CME"的全部机制，和"几个线程"没有任何关系。反过来，`Iterator.remove()` 在删完后把 `expectedModCount = modCount` 同步了一遍，所以 10.2 的三种正确写法里它安全，`removeIf`（JDK 8+，内部用迭代器删）也安全。

```
创建 Itr：expectedModCount = modCount = 3
list.remove(x)：modCount → 4，expectedModCount 仍是 3
next() → checkForComodification()：4 != 3 → ConcurrentModificationException
```

顺带把"视图"这条线接上：`subList` 返回的 `SubList` 也沿用了同一套 `modCount` 快照，所以对原列表做一次结构性修改，视图再 `get(0)` 就抛 CME（10.3）。**CME、`subList` 失效、`Arrays.asList` 定长，看着是三个坑，底层是同一套"结构变更计数 + 快照比对"**。

再补两个常被问到的点。`modCount` 是 `int`，极端频繁的结构修改会溢出回绕，理论上能让 `modCount` 恰好又等于 `expectedModCount`，把一次本该抛的 CME 吞掉——这也是 Javadoc 强调"不要依赖 fail-fast 保证"的原因之一，虽然工程上几乎遇不到。更实际的是 `CopyOnWriteArrayList`：写操作时复制整个底层数组，迭代器永远遍历那个旧快照，因此它的迭代器**从不抛 CME**，代价是写放大与读到旧数据。判据是读写比：读远多于写、且能容忍短暂旧值（监听器列表、白名单）才用，写密集场景它的 `O(n)` 复制会把 CPU 打满。它和 `ConcurrentHashMap` 一样，解决的是"不崩"，不是"强一致"。

10.3　视图陷阱：`Arrays.asList` 与 `subList`

```
// ✗ UnsupportedOperationException
List<String> list = Arrays.asList("a", "b");
list.add("c");
```

`Arrays.asList` 返回的是 `java.util.Arrays$ArrayList`——一个**定长视图**，不是 `java.util.ArrayList`。只支持 `set`/`get`，不支持 `add`/`remove`。而且它**和原数组共享底层存储**：

```
String[] arr = {"a", "b"};
List<String> list = Arrays.asList(arr);
list.set(0, "x");
System.out.println(arr[0]);      // "x" — 原数组被改了
```

```
// ✓ 要可变就包一层
List<String> list = new ArrayList<>(Arrays.asList("a", "b"));
```

`subList` 同理，它是**视图不是拷贝**：

```
// ✗ 对原列表结构修改后，视图立刻失效
List<String> sub = list.subList(0, 10);
list.add("x");
sub.get(0);        // ConcurrentModificationException
```

```
// ✓ subList 后立即拷贝
List<String> sub = new ArrayList<>(list.subList(0, 10));
```

这两个坑的共同点：**你在用一个"看起来是集合、实际是窗口"的东西。** 它不会在你写代码时提醒你，只在运行时用异常（或更糟，用错误数据）告诉你。

【必读】fail-fast 只是"尽力而为"，不是并发保证

10.2 结尾说"CME 是运气好的情况"，这里要再收紧一次。**反常识点：多数人以为 `ConcurrentModificationException` 是并发修改的保护网，实际它是单线程迭代器给自己的一个 best-effort 自检，从来不是并发控制手段。** `ArrayList` 的 Javadoc 写得很直白：fail-fast 行为"无法给出任何硬性保证"，只能"尽最大努力"抛，不能拿来写正确性依赖。

为什么并发下靠不住？三条：① `modCount` 是普通 `int`、不是 `volatile`，一个线程改了，另一个线程不保证立刻看到；② 读写之间没有 happens-before 边（见 10.5、JMM 篇），迭代器可能压根没观察到那次修改；③ 就算抛了，也是在**已经读到脏数据之后**才抛。于是真实并发修改 `ArrayList` 的典型结果不是 CME，而是 `size` 不对、元素丢失、或 `get` 到半写入的引用——**不抛异常，只给错数据**，和 10.1 的 `HashMap` 是同一副面孔。

场景	会不会抛 CME	判据
单线程 foreach 里 `list.remove()`	几乎必抛	`modCount` 与快照不等，且同一线程一定看得到
单线程用 `Iterator.remove()`	不抛	删完同步了 `expectedModCount`
多线程并发改 `ArrayList`	不保证，常常不抛	`modCount` 非 volatile、无 HB 边，检测本身不可靠
想靠 CME 保护并发正确性	✗ 错误用法	它只是调试期提示，不是同步原语

一句话：**能在测试环境复现的 CME 是"运气好"，真正上线的并发 bug 往往一声不吭。** 要并发安全就换 `ConcurrentHashMap` / `CopyOnWriteArrayList`，别指望一个异常来兜底（10.7 自检清单第一条）。

【必读】三种"看起来是 List"的东西：asList / subList / List.of 对照表

10.3 讲了 `Arrays.asList` 和 `subList`，但还差一个常一起踩的 `List.of`（JDK 9+）。把三者横着摆开，每格给"能不能"而不是定义，用时照着判：

写法	底层是什么	`set` 改元素	`add` / `remove`	能在
`Arrays.asList(arr)`	`java.util.Arrays$ArrayList` 定长视图	✓	✗ `UnsupportedOperationException`	✓
`list.subList(a,b)`	`ArrayList$SubList` 视图	✓	✓ 会写回原列表	✓
`List.of(...)`	不可变 `ImmutableCollections.ListN`	✗	✗	✗ 本
`new ArrayList<>(...)`	真正的 `java.util.ArrayList`	✓	✓	✓

```
// ✗ 三个一行代码的翻车
List<String> a = Arrays.asList("a", "b"); a.add("c");       //
UnsupportedOperationException
List<String> b = new ArrayList<>(List.of("a", "b", "c")).subList(0, 2);
b.get(0);                                                    // 若原列表后来被动过
→ CME
List<String> c = List.of("a", null);                         //
NullPointerException

// ✓ 一律先拷贝成真正可变的 ArrayList，再谈增删改
List<String> safe = new ArrayList<>(Arrays.asList("a", "b"));
List<String> slice = new ArrayList<>(source.subList(0, 10));
```

共同点：它们都是"看起来是集合、实际是窗口或只读快照"。 `List.of` 尤其容易骗人——它连 `set` 都不让改，也不收 `null`，放进老代码里"只是想把几个值凑成 List"，结果在 `add` 处炸。记住一条就够：**只要后面要动，就 `new ArrayList<>(...)` 包一层**，这是四个陷阱里唯一通用的解法。

视图还有两个不显眼、但线上真出过事的坑。一是 **subList 会强引用原列表**：一个 100 万元素的大列表取 `subList(0,10)` 后只留着这 10 个元素的视图，视图内部仍持有整个大数组的引用，于是那 100 万个对象一个都回收不掉——这是"明明只用了几个元素却 OOM"的典型，`new ArrayList<>(sub)` 拷贝同时也是在切断这条引用链。二是 `Collections.unmodifiableList` 包装的仍是**原列表的视图**：它只挡住通过包装对象的写，原引用照样能改，改完包装对象读到的也跟着变，和 `List.of` 那种真正的不可变副本不是一回事。判据只有一句：**只要还想"独立使用"，就拷贝，不要包装。**

10.4　最阴险的一种：`equals` / `hashCode` 不成对

```
// ✗ 只重写 equals
public class User {
    private Long id;
    private String name;

    @Override
    public boolean equals(Object o) { ... }   // 没重写 hashCode
}
```

后果：

```
Map<User, String> map = new HashMap<>();
map.put(new User(1L, "张三"), "A");
map.get(new User(1L, "张三"));      // null —— 取不出来
```

因为 `HashMap` 先用 `hashCode` 定位桶，再在桶内用 `equals` 比较。`hashCode` 没重写，两个"逻辑相等"的对象 hash 不同，落进**不同的桶**，`equals` 根本没机会被调用。

这个 bug 不抛异常、不报错、不打日志，只是"存进去取不出来"。 排查时人通常先怀疑 SQL、怀疑缓存、怀疑事务，最后才想到对象相等性。

规则很简单：**`equals` 和 `hashCode` 必须成对重写**。用 IDE 生成，或者用 Lombok 的 `@EqualsAndHashCode`（注意 `@Data` 已经包含它，但继承场景要显式配 `callSuper`）。

顺带一个 BigDecimal 的变体：

```
new BigDecimal("1.0").equals(new BigDecimal("1.00"));      // false（比 scale）
new BigDecimal("1.0").compareTo(new BigDecimal("1.00"));   // 0（只比值）
```

放进 `HashSet` 会出现"两个相等的金额都在里面"。**金额比较大小时永远用 `compareTo`。**

【必读】HashMap 是怎么把数据"静默丢掉"的

10.4 说"`hashCode` 没重写，落进不同的桶"，把这句落到源码，才知道"静默"二字从哪来。`HashMap` 存、取都先算一次桶下标：

```
// HashMap#hash：高 16 位混入低 16 位，减少碰撞
static final int hash(Object key) {
    int h; return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
// putVal / getNode 里定位桶：
//   (n - 1) & hash        n 是 table 长度，必为 2 的幂
```

`getNode` 的比较顺序是：先比 `hash`，`hash` 不等直接跳过；只有 `hash` 相等才轮到 `equals`。两个"逻辑相等"但没重写 `hashCode` 的对象，继承的是 `Object.hashCode()`（通常和对象地址相关），`hash` 大概率不同，于是——

```
put(new User(1,"张三"))  → hash = H1 → 桶 5      存进 table[5]
get(new User(1,"张三"))  → hash = H2 → 桶 9      去 table[9] 找，空的 → 返回 null
```

`equals` 从头到尾**没有被调用的机会**，因为它在"同一个桶内"才生效。这也解释了为什么这个 bug 不抛异常、不打日志：`get` 返回 `null` 是完全合法的 API 行为，调用方只当"缓存没命中"。

```
// 可运行验证：打印出来，比讲十遍管用
User u1 = new User(1L, "张三"), u2 = new User(1L, "张三");   // 只重写了 equals
System.out.println(u1.equals(u2));                    // true
System.out.println(u1.hashCode() == u2.hashCode());   // false ← 病根
Map<User, String> m = new HashMap<>();
m.put(u1, "A");
System.out.println(m.size() + " / " + m.get(u2));     // 1 / null ← 存进去取不出
```

判据只有一条：**`equals` 相等的两个对象，`hashCode` 必须相等**，且两者用同一组字段（IDE 生成，或 Lombok `@EqualsAndHashCode`，继承场景显式 `callSuper = true`）。把这段验证留在单测里，比在 CR 里靠人眼盯注解靠谱得多。

把 10.1 那个现场也接到源码上：JDK 8 的 `HashMap` 把扩容从头插改成尾插，确实消掉了 JDK 7 的环形链表死循环，但并发 `put` 依旧不安全——两个线程同时触发 `resize`、各自建新 table，后写回的覆盖前一个，已迁移的 entry 直接丢失；`++size` 也不是原子的。所以"JDK 8 修好了"的准确说法是**从"卡死"降级成"丢数据"**，而不是"并发安全"。正解只有一个：共享 Map 一律 `ConcurrentHashMap`，且记住它的 `get` / `put` 各自线程安全，但"先 `get` 判空再 `put`"这种复合检查仍不是原子的，要用 `computeIfAbsent`。**复合操作要原子，靠的是把检查与写入合成一次调用，不是把每一步都换成线程安全的类**——这条和 10.1 现场那段 `if (u == null) { ...; cache.put(...) }` 正好对上。

10.5　可见性：`volatile` 管什么

```
// ✗ 这个循环可能永远不退出
private boolean running = true;

public void run() {
    while (running) {          // 线程可能一直读到缓存的 true
        doWork();
    }
}
public void stop() { running = false; }
```

两个线程，一个读一个写，`running` 没有同步措施。**读线程可能永远看不到写线程的修改**——因为 JMM 允许线程把变量缓存在工作内存（寄存器/CPU cache）里。

```
// ✓ volatile 保证可见性
private volatile boolean running = true;
```

但 `volatile` **只保证可见性和有序性，不保证原子性**：

```
// ✗ volatile 不能解决竞态
private volatile int count = 0;
public void incr() { count++; }        // 读-改-写三步，中间可被打断
```

要原子性，用 `AtomicInteger` 或加锁。

一个必须知道的关联：**双重检查锁里的 `volatile` 不能省**。

```
// ✓ 单例的正确写法
private static volatile Singleton instance;      // volatile 不可省

public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance;
}
```

`instance = new Singleton()` 不是原子的，分三步：分配内存 → 初始化 → 赋值引用。没有 `volatile` 时，编译器/CPU 可能重排成"分配 → 赋值 → 初始化"，于是另一个线程可能拿到**一个引用不为 null、但字段还没初始化的对象**。这是 JMM 里最经典的一个例子，也是面试题里唯一值得背的一个。

【必读】volatile 的两层语义与 happens-before：它凭什么能发布别的变量

10.5 说"`volatile` 管可见性、不管原子性"，这句对，但只说了一半。`volatile` 其实给了两层语义，第二层才是它真正值钱的地方：

语义	保证什么	对应机制	判据
可见性	写立即刷回主内存、使其他线程副本失效；读强制从主内存取	读写插入内存屏障	一个写线程、多个读线程的状态标志
有序性（禁重排）	volatile 写之前的普通写不会被重排到写之后；读之后的普通读写不会上浮	`StoreStore`/`StoreLoad`/`LoadLoad`/`LoadStore` 屏障	双重检查锁、安全发布
原子性	✗ 只保证单次读/写，`count++` 这种读-改-写不管	无互斥	见下一节反例

第二层的正式名字叫 happens-before（HB）。JMM 里最常用的一条是：**对 volatile 字段的写，happens-before 后续对该字段的读**；再加上程序顺序与传递性，就得到"volatile 写之前的所有普通写，对 volatile 读之后的所有普通读可见"。八条常用 HB 规则里，日常推理用得最多的是这五条：

HB 规则	内容
程序顺序	同一线程内，前面的操作 HB 后面的操作
监视器锁	对同一把锁的 unlock HB 后续 lock
volatile	volatile 写 HB 后续对该字段的读
线程启动	`Thread.start()` HB 新线程中的任何操作
传递性	A HB B 且 B HB C ⇒ A HB C

这也补正 10.5 里一处不够严谨的比喻：把"工作内存"直接说成"寄存器 / CPU cache"容易误导，它是 JMM 的**抽象**，涵盖缓存、寄存器、写缓冲以及编译器优化，不是某个具体硬件部件。真正决定可见性的从来不是"变量放在哪"，而是**两个线程之间有没有一条 HB 边**。`SafePublication` 里 `ready` 加 volatile 之所以能让 `data = 42` 被看到，靠的正是这条"写 HB 读 + 传递性"，而不是 volatile 有什么广播魔法。

【进阶】一段 JMM 反例：volatile 为什么不能替代锁

反常识点：多数人以为 `volatile` 是"轻量级的 synchronized"，能顺手替代锁；实际它一点互斥都不提供，凡是需要"读旧值、算新值、写回"的复合操作，加了 volatile 照样丢数据。

```
// ✗ volatile 保不住 i++：读-改-写三步，中间可被别的线程插进来
private volatile int count = 0;
public void incr() { count++; }

// ✓ 要原子性，用原子类（或加锁）
private final AtomicInteger count = new AtomicInteger();
public void incr() { count.incrementAndGet(); }
```

`count++` 在字节码里是 `getfield → iconst_1/iadd → putfield` 三步。volatile 只保证每次 `getfield` 读到的是最新值，却拦不住两个线程**同时读到同一个最新值**、各自加一、再各自写回——两次自增只落了一次。下面这段能跑的验证，把"看着没问题的代码"直接证伪：

```
@Test
void volatileIsNotAtomic() throws Exception {
    for (int round = 0; round < 5; round++) {
        VolatileCounter c = new VolatileCounter();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int i = 0; i < 8; i++)
            pool.submit(() -> { for (int j = 0; j < 10_000; j++) c.incr(); });
        pool.shutdown(); pool.awaitTermination(1, TimeUnit.MINUTES);
        System.out.println(c.count);   // 期望 80000，实际常见 5xxxx~7xxxx，且每轮不同
    }
}
```

换成 `AtomicInteger` 后稳定输出 80000。想看更硬的理论结论，可以用 `jcstress` 跑无锁用例——普通单元测试几乎测不出可见性 bug，这类工具才是为 JMM 准备的。

更要命的是"跨多个变量的不变式"。就算每个字段都是 volatile，`if (a + b == 100)` 这类约束也守不住，因为两个字段之间没有任何一起变的保证：**volatile 保护的是单次访问，锁保护的是一段临界区**。要"一起变"，就得用同一把 `synchronized`、或把状态封进不可变对象整体替换。这条边界和 10.1 的 `HashMap`、10.4 的 `equals/hashCode` 是同一种教训——不报错、只错数据的 bug，靠规约和单测防，别靠"我加了 volatile"的自我安慰。

把 volatile 和锁摆在一起，选型就不必背了：

需求	用什么	判据
一个状态标志，一写多读	`volatile boolean`	只读不依赖旧值
单个变量的原子累加	`AtomicInteger` / `LongAdder`	读-改-写要一气呵成
多个变量要"一起变"	同一把 `synchronized` / `ReentrantLock`	存在跨字段不变式
安全发布一个构造好的对象	`final` 字段或静态初始化器	构造期 `this` 不逃逸

最后一行接 10.5 的双重检查锁：`volatile` 在那里防的是"引用已赋值、对象没初始化"的重排，属于**有序性**那一层，跟可见性、原子性都不同。同一个关键字在三个场景里干三件事，正是它最容易被误用的根源。再给一条版本与工具的边界：`volatile` 的语义由 JSR-133（Java 5 起）定死，JDK 8 / 17 / 21 一致，不会随版本漂移；**会漂移的是你自己的代码有没有建立 HB 边**，怀疑可见性问题时用 `jcstress` 压，别用 `Thread.sleep` 试——sleep 不建立任何 HB 关系，加一行打印"就好了"只是 synchronized 的副作用掩盖了问题。

10.6　`SimpleDateFormat`：共享即出错

```
// ✗ 静态共享：多线程下解析出错、抛 NumberFormatException
private static final SimpleDateFormat SDF = new SimpleDateFormat("yyyy-MM-dd");
```

`SimpleDateFormat` 内部持有 `Calendar`，`parse`/`format` 会改它的状态。并发调用会互相污染：

- 解析出**错误的日期**（不抛异常，最可怕）
- 抛 `NumberFormatException` / `ArrayIndexOutOfBoundsException`

```
// ✓ JDK 8+ 一律用 DateTimeFormatter（不可变、线程安全）
private static final DateTimeFormatter FMT = DateTimeFormatter.ofPattern("yyyy-
MM-dd");
```

如果必须用 `SimpleDateFormat`（老代码），两条路：`ThreadLocal` 包一层，或者每次 `new`（开销可接受，别为省一个对象引入并发 bug）。

（换成 `DateTimeFormatter` 之后仍有两个边界——严格模式 `ResolverStyle.STRICT` 与年份用 `uuuu` 而非 `yyyy`——见章末「延伸」。）

10.7　自检清单

- [ ] 多线程共享的 `HashMap` / `ArrayList` 全部换成了 `ConcurrentHashMap` / `CopyOnWriteArrayList`
- [ ] 没有在 `foreach` 里 `add/remove`，用 `removeIf` 或 `Iterator.remove`
- [ ] `Arrays.asList` 的结果没有直接 `add/remove`；`subList` 后没有再动原列表
- [ ] 所有放进 `HashMap/HashSet` 的类都成对重写了 `equals` + `hashCode`
- [ ] 金额比较用 `compareTo`，不用 `equals`
- [ ] 跨线程共享的标志位是 `volatile`
- [ ] 计数器用 `Atomic*`，不用 `volatile int` 自增
- [ ] 双重检查锁的实例字段带 `volatile`
- [ ] 没有静态共享的 `SimpleDateFormat`

10.8　一句话总结

> 并发 bug 分两种：**报错的**（CME、NPE）和**不报错的**（丢数据、串数据、读到旧值）。 前者靠经验，后者只能靠规约——因为它在测试环境几乎不会出现。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/01-Java语言与集合坑、02-Java进阶/02-并发编程/Java内存模型JMM、02-Java进阶/02-并发编程/volatile
- **【选读】DTF 迁移的两个边界**：`DateTimeFormatter` 解决的是"线程安全"，不自动解决"校验严格"——默认 `ResolverStyle.SMART` 仍会把 `2026-02-30` 修成 `2026-02-28`，要严格就得显式 `withResolverStyle(ResolverStyle.STRICT)`；且严格模式下年份用 `uuuu` 而非 `yyyy`（`yyyy` 是纪元年，解析无纪元日期会抛异常）。这是从 `SimpleDateFormat` 迁到 DTF 最常踩的一行。

第十一章　全局异常处理吃掉了堆栈

本章要回答：为什么监控面板上"错误率 0%"，用户却在报障？

本章你能带走

- 一个致命的组合：`@ExceptionHandler` 返回 200 + 监控按状态码统计 = **错误全部隐形**
- 一张表：`@RestControllerAdvice` 管不到的五个地方
- 一条原则：**异常处理的目标是"让人看见"，不是"让用户看不见"**

难度构成：本章主线【必读】；含【进阶】深挖 2 处；章末「延伸」小节放支线深挖与知识库指针。---

11.1　现场：错误率 0，客诉 40 单

第六章那个超卖事故，事后复盘时发现了一个更让人后背发凉的事实：

> 事故持续了 47 分钟，监控上**错误率一直是 0.00%**。

为什么？因为异常确实被抛了，但被全局异常处理器接住了，转成了：

```
HTTP 200
{"code": 500, "msg": "库存不足", "data": null}
```

对监控来说，**HTTP 200 就是成功**。APM 按状态码统计错误率，网关按 5xx 触发告警，日志平台按 `ERROR` 级别聚合——而这里连一行 ERROR 都没有，只有 `log.warn`。

于是形成了一条完美的盲区：

```
业务异常 → Advice 捕获 → 转成 200 + code=500 → 监控看到"成功"
                                                    ↓
                                              用户看到失败，但系统认为一切正常
```

这就是本章的核心：

> **统一响应体是好设计，"统一成 200"是坏设计。** 前者统一了格式，后者统一掉了可观测性。

当时怎么误判的：监控面板上错误率一直是 0.00%，客诉却堆了 40 单，第一反应是「APM 采集坏了，或者网关统计口径错了」，运维查了一遍埋点和 Prometheus 配置，全正常——这条「监控坏了」是当时最耽误事的死线索。又有人怀疑「是前端显示 bug，数据其实是好的」，可用户端明明在报错。真正扭回来的，是有人把一次失败请求的响应体抓出来看：HTTP 200、`{"code":500,...}`，异常早被全局异常处理器接住转成了「成功」。这一下把方向从「查监控 / 查前端」扭回「查异常是怎么被消音的」：可观测性不是某一层加个注解，是「异常 → 日志级别 → 状态码 → 采集字段」这条链上每一环都别把信息抹掉。

11.2　根因：三件事被混在一起了

`@RestControllerAdvice` 一次做了三个决定，而这三个决定本该分开：

决定	应该怎么做	常见做法
响应体格式	统一 `{code,msg,data}`	✓ 统一了
HTTP 状态码	保留语义（400/404/500）	✗ 一律 200
日志级别	4xx 用 warn，5xx 用 error + 堆栈	✗ 一律 warn 或不打

第一个是接口设计，第二个是**可观测性基础设施**，第三个是**排障能力**。混在一起的代价就是：格式统一了，事故看不见了。

【必读】一个必踩的坑：`void` handler

```
// ✗ 响应体为空，状态码 200，堆栈没打
@ExceptionHandler(Exception.class)
public void handle(Exception e) {
    System.out.println(e.getMessage());     // 只取 message，没有堆栈
}
```

这个写法在大多数团队的代码库里都能找到至少一处。它的三个问题：

1. 用户收到空响应，不知道发生了什么
2. `getMessage()` 不带堆栈，线上定位等于没有线索
3. `System.out` 不进日志文件，重启即失

【必读】ExceptionHandlerExceptionResolver：一条异常怎样被选中处理器

11.2 说"Advice 把异常接住了"，但"接住"这两个字背后是一整条解析链。请求在 `DispatcherServlet#doDispatch` 里抛异常后，会依次交给容器里的一组 `HandlerExceptionResolver`，负责 `@ExceptionHandler` 的是 `ExceptionHandlerExceptionResolver`：

```
DispatcherServlet#processDispatchResult
        └ processHandlerException(request, response, handler, ex)
              └ 遍历 HandlerExceptionResolver 链
                    ├ ExceptionHandlerExceptionResolver   ← @ExceptionHandler 在
这里
                    ├ ResponseStatusExceptionResolver     ← @ResponseStatus
                    └ DefaultHandlerExceptionResolver     ← Spring 内置标准异常
```

`ExceptionHandlerExceptionResolver#doResolveHandlerMethodException` 只做一件事：调 `getExceptionHandlerMethod(handlerMethod, exception)` 找到该用哪个方法，再 `invokeAndHandle` 执行它。找方法的顺序是优先级的核心：

```
① 先查 Controller 本类：ExceptionHandlerMethodResolver(handlerType)
      → 本类里 @ExceptionHandler 方法，按异常类型匹配
② 本类没有，再查 @ControllerAdvice Bean
      → ControllerAdviceBean.findAnnotatedBeans(applicationContext)
      → 按 @Order / Ordered / @Priority 排序，值越小越先
③ 在单个 Resolver 内，用"异常深度"选最具体的方法
      → ExceptionHandlerMethodResolver#getMappedMethod：
        对 exception 的每一级父类找映射，深度最浅（最具体）的胜出
```

于是优先级可以横着记成一张表：

维度	谁优先	判据
类型匹配	越具体越优先	`@ExceptionHandler(OrderNotFoundException.class)` 先于 `Exception.class`
位置	Controller 本类先于 Advice	本类有匹配方法，就不再找 Advice
多个 Advice	`@Order` 值小的先	兜底 `Exception` handler 放**最低**优先级，否则会盖住别人的具体 handler
同类型冲突	启动期报 `IllegalStateException`	两个 Advice 在同优先级声明同一异常，含糊不得

反常识点：多数人以为"多个 `@RestControllerAdvice` 谁生效看扫描顺序"，实际未指定 `@Order` 时顺序不确定、也不该依赖；把兜底 handler 放在 `@Order(Ordered.LOWEST_PRECEDENCE)` 的类里，是让具体异常不被吞掉的关键。这条和第 6 章"代理只认调用路径"是同一种思路：**框架不会替你猜意图，优先级得自己显式写死。**

【进阶】void handler 为什么"吃掉"堆栈：三个层次

11.2 列了 `void` handler 的三个症状，这里补上"为什么"。**它不是"少打了一行日志"这么简单，而是异常在框架语义上被判定为"已处理"了。**

```
// ✗ 响应体空、状态码 200、堆栈彻底不传播
@ExceptionHandler(Exception.class)
public void handle(Exception e) {
    System.out.println(e.getMessage());
}

// ✓ 保留状态码 + 完整堆栈 +（包装时）保留 cause
@ExceptionHandler(Exception.class)
public ResponseEntity<ApiResult<Void>> handle(Exception e) {
    log.error("unhandled exception", e);              // 最后一参传 e，Slf4j 才打堆栈
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ApiResult.fail(500, "系统繁忙"));
}
```

三层原因叠在一起：① 返回 `void` 且类上有 `@ResponseBody`（`@RestControllerAdvice` 自带），Spring 认为"你已给出响应"，于是响应以 200 提交、异常**不再向容器传播**，`BasicErrorController` 那套兜底也不会触发；② `e.getMessage()` 只是个字符串，`StackTraceElement[]` 一次都没被读，唯一能定位到行号的信息被丢掉；③ `System.out` 走标准输出，不进日志框架，重启即失，也不参与日志平台的 `ERROR` 聚合（11.1 那条盲区正是由此而来）。

正确写法里最容易漏的是 `cause`。业务代码包装异常时若不把原始异常传进去，即便 handler 打了堆栈，也只剩一句"创建订单失败"：

```
// ✗ 断链：唯一索引冲突、字段超长、连接断开，全没了
catch (SQLException e) { throw new BizException("创建订单失败"); }
// ✓ 保留 cause：日志里能看到完整的 Caused by 链
catch (SQLException e) { throw new BizException("创建订单失败", e); }
```

`log.error("...", e)` 与 `new BizException("...", e)` 是同一条规矩的两处落点：**最后一个参数永远把异常对象本身传下去，别只传 `getMessage()`**。第 23 章讲取证时，线上唯一能还原现场的，就是这条没被剪断的 `Caused by` 链。

【必读】一条错误的完整旅程：从抛到监控看不见

把 11.1 那 47 分钟拆成时序，才明白"错误率 0.00%"不是监控坏了，是每一环都按设计工作、合起来却成了一堵墙：

```
Controller 抛 BizException("库存不足")
   → ExceptionHandlerExceptionResolver 选中 handleBiz
   → log.warn(...)                    ① 日志级别 WARN，平台按 ERROR 聚合 → 不计入
   → ResponseEntity.status(200)       ② 网关/APM 按状态码统计 → 计为成功
   → {"code":500,...}                 ③ 前端按 code 判失败，但监控没采 code 字段
   → 监控面板：错误率 0.00%，告警 0 条
```

三个"各自正确"的决定叠在一起：统一响应体统一掉了状态码，业务异常用 warn 统一掉了 ERROR 日志，监控只认状态码统一掉了业务码。**可观测性不是某一层加个注解，是"异常 → 日志级别 → 状态码 → 采集字段"这条链上每一环都别把信息抹掉。** 把代价量化一下：47 分钟里约 40 单客诉、按每单关联 3 个订单估算，业务侧比技术侧早约 40 分钟发现故障——第 23 章讲取证时，"谁先发现"往往比"怎么修"更决定损失大小。

补一张口径表，监控接入时照着对齐，别让"错误率"这个词各说各话：

统计口径	看哪个字段	本章现场里的值	问题
HTTP 状态码	status	200	业务失败被记成成功
日志级别	level	WARN	进不了 ERROR 告警
业务码	code	500	多数监控默认不采集
端到端	前端埋点 / 客诉	失败	唯一真实的信号，却最晚到

11.3　正确写法：分层 + 保留语义 + 保留堆栈

```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    /** 业务异常：可预期，4xx，不打堆栈（避免日志噪音） */
    @ExceptionHandler(BizException.class)
    public ResponseEntity<ApiResult<Void>> handleBiz(BizException e) {
        log.warn("biz error code={} msg={}", e.getCode(), e.getMessage());
        return ResponseEntity.status(e.getHttpStatus())
                .body(ApiResult.fail(e.getCode(), e.getMessage()));
    }

    /** 参数校验：@RequestBody 失败 */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResult<Void>>
handleValid(MethodArgumentNotValidException e) {
        String msg = e.getBindingResult().getFieldErrors().stream()
                .map(f -> f.getField() + ": " + f.getDefaultMessage())
                .collect(Collectors.joining("; "));
        return ResponseEntity.badRequest().body(ApiResult.fail(400, msg));
    }

    /** 参数校验：@RequestParam / @PathVariable 失败 */
    @ExceptionHandler(ConstraintViolationException.class)
    public ResponseEntity<ApiResult<Void>>
handleConstraint(ConstraintViolationException e) {
        return ResponseEntity.badRequest().body(ApiResult.fail(400,
e.getMessage()));
    }

    /** 兜底：5xx，必须 error 级别 + 完整堆栈 */
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResult<Void>> handleOther(Exception e) {
        log.error("unhandled exception", e);          // ← 堆栈在这里
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(ApiResult.fail(500, "系统繁忙，请稍后重试"));
    }
}
```

四个关键点，缺一个就留一个盲区：

1. **`BizException` 单列**：可预期的业务失败用 `warn`，不打堆栈——否则日志里全是噪音，真正的 error 被淹没
2. **兜底 `Exception` 打 `error` + 堆栈**：`log.error("...", e)` 最后一个参数传异常对象，Slf4j 才会打印堆栈。写成 `log.error("unhandled " + e.getMessage())` 就等于没有
3. **HTTP 状态码保留语义**：`ResponseEntity.status(...)`。前端和监控都靠这个
4. **别用 `e.printStackTrace()`**：进 stderr，不进日志框架，收集不到

11.4　它管不到的五个地方

这是最容易被忽略的一段：`@RestControllerAdvice` 只能接住 **Controller 层方法体内抛出的异常**。以下五个地方，它一次都接不住：

位置	异常会去哪	现象
Filter	容器 → 默认错误页 / 空响应	前端收到 500 但响应体是 HTML
拦截器 `preHandle`	同上	登录校验抛异常，前端拿到空白页
`@Async` 方法	线程池的 UncaughtExceptionHandler	静默失败，无任何记录（第九章）
定时任务	调度框架的日志	任务失败无人知，直到业务方发现数据没生成
`@PostConstruct` / 启动期	容器启动失败	这个还好，至少起不来

Filter 里的异常最典型：

```
// ✕ Filter 抛异常，Advice 接不住
public void doFilter(...) {
    String token = req.getHeader("Authorization");
    if (parse(token) == null) {
        throw new UnauthorizedException();       // → 容器错误页，用户看到 HTML
    }
    chain.doFilter(req, resp);
}

// ✓ Filter 内自己处理
public void doFilter(...) throws IOException {
    try {
        if (parse(req.getHeader("Authorization")) == null) {
            writeJson(resp, 401, "unauthorized");
            return;
        }
        chain.doFilter(req, resp);
    } catch (Exception e) {
        log.error("filter error", e);
        writeJson(resp, 500, "internal error");
    }
}
```

一条实用规则：凡是"在 Controller 之外"的代码，异常必须自己处理完，不能指望 Advice。

【进阶】五类异常的兜底层：别都指望 Advice

11.4 那张表列了 Advice 管不到的五个地方，但"异常会去哪"只写了现象，排障时更需要的是"该去哪层兜"。补一列，把责任钉死：

位置	异常最终落到哪	该去哪层兜	判据
Filter	容器默认错误页 / 空响应	**Filter 内**自己 `try-catch` 写 JSON（11.4 代码）	请求还没进 `Dispatcher`
拦截器 `preHandle`	同上	拦截器内处理，或下放到 Filter	比 Controll
`@Async` 方法	线程池的 `UncaughtExceptionHandler`	配 `AsyncUncaughtExceptionHandler`，实现 `AsyncConfigurer`	第 9 章：换 丢了上下文
定时任务	调度框架日志	给 `TaskScheduler` 配 `ErrorHandler`，任务体内 try-catch	`@Scheduled` 求线程
`@PostConstruct` / 启动期	容器启动失败	起不来就是最好的兜底，别 catch 掉	第 8 章：fai 反而省事

```
请求 ── Filter ── preHandle ── DispatcherServlet ── Controller ── @ExceptionHandler
        ↑            ↑                                  ↑
      自己兜       自己兜                          Advice 只管这一段
    （Advice 根本还没被创建/调用）
```

一条实用规则再强调一次：**凡是在 Controller 之外的代码，异常必须自己处理完**。第 9 章的 `@Async` 静默失败、11.4 的 Filter 拿到 HTML 错误页，都是同一条边界的两种表现——异常发生在 `DispatcherServlet` 的管辖范围之外，`ExceptionHandlerExceptionResolver` 连被调用的机会都没有。第 2 章讲的接口超时异常若抛在 Filter 或异步线程里，同样不会进这套体系，别在监控里等那条 ERROR。

安全框架是这条边界上最容易漏的一块：Spring Security 的 401 / 403 大多产生在 **Filter 链**里，压根没到 `DispatcherServlet`，所以 `@RestControllerAdvice` 接不住，用户拿到的是容器默认错误页或空响应。正解是在安全配置里给 `http.exceptionHandling(...)` 指定 `authenticationEntryPoint` 与 `accessDeniedHandler`，自己写 JSON 和状态码；唯一的例外是 `@PreAuthorize` 在 Controller 方法上抛的 `AccessDeniedException`，它确实能进 Advice，别顺手被兜底 `Exception` handler 转成 500。**判据：看异常发生在 Filter 还是 Controller 方法，前者归安全配置，后者才归 Advice。**

再补一个和 11.3 呼应的细节：`ResponseEntityExceptionHandler` 里那些 `handleXXX` 方法之所以能"内置处理"标准异常，是因为它们本身就用 `@ExceptionHandler` 声明了一长串类型（含 `MethodArgumentNotValidException`、`NoHandlerFoundException` 等）。**继承它等于一次性继承了几十个 handler**，代价是它返回的 `ProblemDetail` 与你自定义的 `{code,msg,data}` 未必是同一套格式——用之前先决定全项目统一到哪一套。

11.5　404 为什么也不进 Advice

Spring Boot 把"没有匹配的 handler"交给 `BasicErrorController` 处理，走的是错误页机制，不经过 `@ExceptionHandler`。

要让 404 也统一格式，两个配置缺一不可：

```
spring:
  mvc:
    throw-exception-if-no-handler-found: true      # ① 无 handler 时抛异常
  web:
    resources:
      add-mappings: false                          # ② 关掉静态资源兜底，否则永远匹配得上
```

第 ② 条特别容易漏。不关静态资源映射，任何路径都会被 `ResourceHttpRequestHandler` 接住，`NoHandlerFoundException` 永远不抛——**配了第 ① 条也没用**。

【必读】404 / NoHandlerFound 与 Boot 3：那条配置在 3.2 起已经变了

11.5 给了"两个配置缺一不可"，这在 Spring Boot 2.7 / 3.0 / 3.1 上是对的，但**照搬到 Boot 3.2+ 就会踩空**——这是本章最该改掉的一处"看着对、其实已经错"的结论。

变化来自 Spring Framework 6.1（随 Boot 3.2 发布）：

版本	无 handler / 缺静态资源	要不要配 `throw-exception-if-no-handler-found`
Boot 2.7 及更早	交给 `BasicErrorController` 出 Whitelabel 页	要，配 `true` 才抛 `NoHandlerFoundException`
Boot 3.0 / 3.1	同上，旧机制	要，且必须关 `add-mappings`
Boot 3.2（Framework 6.1）起	拆成两个异常：`NoHandlerFoundException`（无任何 handler）、`NoResourceFoundException`（静态资源找不到）	**该配置项已标记 deprecated**（3.4 计划移除），默认就会抛异常

`WebMvcProperties#setThrowExceptionIfNoHandlerFound` 出现在 Boot 3.2.0 的官方 Deprecated List 里，理由正是"默认已改为抛异常"。但 11.5 那句"不关静态资源映射，配了第①条也没用"**依然成立，而且更重要了**：只要 `spring.web.resources.add-mappings` 保持默认 `true`，`ResourceHttpRequestHandler` 会接住 `/**`，任何路径都"匹配得上"，于是走到的是"静态资源没找到"这条分支——在 6.1 之后抛的是 `NoResourceFoundException`，而不是 `NoHandlerFoundException`。**只写了针对 `NoHandlerFoundException` 的 handler，就会漏掉它**，这才是 Boot 3 下 404 统一格式最常见的新坑。

```
// ✓ Boot 3.2+：一个 handler 同时接住两种"找不到"
@RestControllerAdvice
class ApiErrors extends ResponseEntityExceptionHandler {   // 父类已内置处理这两类，返回 ProblemDetail 404
    // 只在这里加自己的业务异常 handler 即可
}

// 或者显式声明，别只写一个：
@ExceptionHandler({NoHandlerFoundException.class,
NoResourceFoundException.class})
public ProblemDetail notFound(Exception e) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, "资源不存在");
}
```

若项目还依赖静态资源直链、不能关 `add-mappings`，就照 11.5 结尾那条走自定义 `ErrorController` 覆盖 `/error`。**这几个具体类名与默认行为建议在自己的 Boot 版本上验证一次再定稿**——框架在 3.2 前后对 404 的处理换过实现，跨版本照抄配置是这类问题的高发来源（第 8 章讲升级时是同一个教训）。

两条配置与两个 handler 别叠着用，会互相打架：既开了 `throw-exception-if-no-handler-found`（旧版本）又自定义了 `ErrorController` 覆盖 `/error`，请求可能被处理两次、响应体写两遍。Boot 3 还多一个 `spring.mvc.problemdetails.enabled=true`：开启后 `ResponseEntityExceptionHandler` 会以 `application/problem+json` 返回 `ProblemDetail`，和项目里自定义的 `{code,msg,data}` 是**两套格式**，要么统一到 ProblemDetail，要么别开这个开关、自己写 handler。一句话：**404 的兜底方式只能选一种**——配置项、`ErrorController`、`@ExceptionHandler` 三选一，混用比不选更难查。

版本区间再收一句：本章关于状态码、日志级别、`cause` 链的结论对 Boot 2.7 / 3.x 全系列成立，不随版本漂移；**会漂移的只有 404 这一处实现**（3.2 前后换了异常类型与默认行为）。所以升级 Boot 时，别的断言可以照抄，唯独 11.5 那两个配置和 `NoResourceFoundException` 要在目标版本上重跑一次 11.7 的四点验证——第 8 章讲大版本升级"一次只改一个变量"，此处同理。

11.6　异常链：不要断掉堆栈

```
// ✕ 断链：原始异常丢了
try {
    orderMapper.insert(order);
} catch (SQLException e) {
    throw new BizException("创建订单失败");          // 根因（SQL 错误）彻底消失
}

// ✓ 保留 cause
} catch (SQLException e) {
    throw new BizException("创建订单失败", e);       // 堆栈链完整，排障能看到 SQL 报
错
}
```

断链的后果：日志里只有一句"创建订单失败"，看不到唯一索引冲突、看不到字段超长、看不到连接断开。**排查从"看日志"退化成"猜"。**

排查时看日志的第一件事就是找 `Caused by`。如果一次报错里没有 `Caused by`，而你知道底层是 SQL/网络调用，那大概率是有人把 cause 丢了。

11.7　一个自检用的"事故模拟"

想验证你的异常体系有没有盲区，做一件事就够了：**在代码里人为抛一个异常，然后看四个地方**。

```
1. 用户看到的响应          → 有没有 code/msg，状态码对不对
2. 监控面板的错误率        → 有没有涨
3. 日志平台                → 有没有 ERROR + 完整堆栈
4. 告警                    → 有没有触发
```

四个都对，才算这套体系是通的。四个里有一个不对，那个地方就是一个未来的盲区。

建议每个团队在联调环境做一次，五分钟的事，能提前发现 80% 的可观测性缺陷。

【必读】一段会失败的事故模拟测试：断言堆栈没被吞

11.7 给的是"人为抛异常后看四个地方"的人工清单，它能发现盲区，但会随人员流动退化。把它变成一条构建里会失败的测试，才守得住。下面用 `MockMvc` + Logback 的 `ListAppender` 直接断言"堆栈没被吞"：

```
@SpringBootTest
@AutoConfigureMockMvc
class ExceptionObservabilityTest {

    @Autowired private MockMvc mvc;

    @Test
    void stackMustNotBeSwallowed() throws Exception {
        Logger log = (Logger)
LoggerFactory.getLogger(GlobalExceptionHandler.class);
        ListAppender<ILoggingEvent> appender = new ListAppender<>();
        appender.start(); log.addAppender(appender);

        mvc.perform(get("/probe/boom"))
           .andExpect(status().isInternalServerError())          // ① 不是 200
           .andExpect(jsonPath("$.code").value(500));            // ② 响应体带业
务码

        ILoggingEvent evt = appender.list.stream()
                .filter(e -> e.getLevel() ==
Level.ERROR).findFirst().orElseThrow();
        assertThat(evt.getThrowableProxy()).isNotNull();         // ③ 有
Throwable，不是只打了 message
        assertThat(evt.getThrowableProxy().getClassName())
                .isEqualTo(IllegalStateException.class.getName());
        log.detachAppender(appender);
    }
}
```

三条断言各自守一个盲区：① 守 11.1 的"一律 200"，② 守统一响应体，③ 守 11.2 的"`void` handler 只打 message"——`getThrowableProxy()` 非空，等价于"`log.error("...", e)` 真的把异常对象传进去了"。若有人把 handler 改成 `log.error("unhandled " + e.getMessage())`，第三条立刻红。

再补一条对异常链的断言，防 11.6 那种断链：在 `/probe/boom` 里故意 `new BizException("外层", new SQLException("唯一索引冲突"))`，然后断言 `evt.getThrowableProxy().getCause().getClassName()` 含 `SQLException`。**"有 ERROR"不等于"能排障"，`Caused by` 链完整才是。**

把这条测试接进 CI，配合第 2 章超时演练（人为让下游超时，看异常有没有被这套体系如实记录）一起跑，就形成了第 23 章取证闭环的最后一环：**线上出事时要看的 ERROR 日志、状态码、`Caused by` 链，在联调环境先被断言过一遍**。第 6 章那次"错误率 0%、客诉 40 单"的超卖事故，若有这条测试，本可在发布前就被拦下。

如果不想引 Logback 的 `ListAppender`，Spring Boot 自带 `OutputCaptureExtension` 更省事：`@ExtendWith(OutputCaptureExtension.class)` 加 `CapturedOutput output`，直接 `assertThat(output).contains("IllegalStateException")` 就能守堆栈。把这类测试挂到 CI 的必经路径上，再加一条门禁指标：**联调环境人为抛异常后，错误率面板必须在一分钟内上升**。这条比任何单测都直接，因为它验的是"日志 → 采集 → 聚合 → 告警"整条链，而不是某一行 Java 代码。第 2 章的超时演练、第 23 章的取证清单，最终都要落到这个"链路通不通"的验证上。

最后把这套体系的责任划清，别再把它当成"前端显示友好"的事：`@RestControllerAdvice` 管的是 **Controller 之内**的异常如何变成响应，`AsyncUncaughtExceptionHandler` 管 **@Async**，`ErrorController` / Filter 管**进入 Controller 之前与 404**，日志与监控平台管**异常有没有被人看见**。四层各管一段，任何一层缺位，11.1 那种"错误率 0、客诉 40 单"就会重演。**异常处理的目标从来不是让用户看不见，是让该看见的人第一时间看见。**

还有一类"进了 Advice 却仍然看不见"的情况：在 `@ExceptionHandler` 方法里**再次抛异常**（比如拼装响应体时序列化失败、或 handler 自己调了另一个会抛的方法），此时框架认为处理失败，请求会回落到 `/error`，统一格式整个失效，日志里只留一条容器级的 500。判据：如果前端偶尔收到 Whitelabel 页或 HTML，先查兜底 handler 里有没有 `throw`、有没有调用可能失败的序列化 / 远程方法。同理，handler 里别做"再查一次库补全信息"这种事——**异常处理器必须是纯函数：只读入参、只拼响应、只打日志，任何可能失败的副作用都会把最后一层兜底也拖下水。**

把门禁再量化一层：假设联调环境每千次请求注入 1 次人为异常，若错误率面板纹丝不动，说明采集链至少断了一环；按 11.1 的口径反推，生产环境同样会"错误率 0、客诉不断"。**这条断言的价值不在覆盖率，在于它验的是链路而不是代码**——单测全绿但监控瞎眼的系统，和没有异常体系的系统，在事故当晚是等价的。

11.8　自检清单

- [ ] 兜底 handler 用 `log.error("...", e)`，带堆栈
- [ ] 业务异常单列，用 warn，不打堆栈
- [ ] 响应状态码保留语义（400/401/404/500），不是一律 200
- [ ] 没有 `void` 类型的 `@ExceptionHandler`
- [ ] 没有 `e.printStackTrace()` / `System.out`
- [ ] Filter、拦截器、`@Async`、定时任务内部都有 try-catch
- [ ] 404 走统一格式（两个配置都配了）
- [ ] 所有自定义异常的包装都传了 `cause`
- [ ] 监控的错误率不是只按 200/500 统计（要看业务 `code` 字段）
- [ ] 做过"人为抛异常"的四点验证

11.9　一句话总结

> 异常处理的第一目标是**让人看见**，第二目标才是**让用户看着舒服**。 顺序反了，你就有了一套对用户友好、对运维瞎眼的系统。

【选读】延伸

- 知识库对应原理篇：04-Spring生态/02-Spring Boot/全局异常处理、17-常见Bug与解决方案/03-Spring事务与AOP失效

第十二章　索引失效的七种写法

本章要回答：索引明明建了，为什么 `EXPLAIN` 还是 `type=ALL`？

本章你能带走

- 七种失效写法，每一种都附"为什么"，不只是"不能这么写"
- 一个最隐蔽的：**隐式类型转换的方向搞反，结论就反**
- 一条判读标准：`type` 至少要到 `range`/`ref`，`Using filesort` 也算不及格

难度构成：本章主线【必读】；含【进阶】深挖 3 处；章末「延伸」小节放支线深挖与知识库指针。---

12.1　现场：索引建了三个月，一次没生效

一个查询接口上线后一直慢，DBA 看了说"你这表没索引"，开发说"我建了"。

```
SHOW INDEX FROM order_info;
-- idx_phone (phone), idx_create_time (create_time), idx_status_user (status,
user_id)
```

索引都在。但慢查询日志里长这样：

```
SELECT * FROM order_info WHERE phone = 13800138000;
```

`phone` 是 `varchar(20)`。查询里传的是**数字**。

`EXPLAIN` 的结果：`type=ALL, rows=2847391`——全表扫描。

索引存在，但优化器用不上。 这不是"索引坏了"，是"这条 SQL 把索引变成了一个无法定位的表达式"。

当时怎么误判的：第一反应是"索引被删了 / 重建后没生效"——查 `SHOW INDEX FROM order_info`，索引都在；又怀疑"统计信息过期，优化器选错了计划"，`ANALYZE TABLE` 之后 `EXPLAIN` 依旧 `type=ALL`，像是在白忙。两条都排除后，才有人把慢日志里那条 SQL 的 `phone` 入参（数字）跟表里 `varchar(20)` 的列类型对一下，发现是隐式类型转换让列被函数包裹、索引用不上。这段空转了将近半天，也印证了一句话：这种"有索引却全表扫"的第一现场，永远是 **SQL 参数类型 vs 列类型**，不是重建索引。

12.2　失效的七种写法

【必读】① 对索引列用函数

```
-- ✗ 索引存的是 create_time 原值，DATE(...) 的结果没有索引
WHERE DATE(create_time) = '2026-11-08'

-- ✓ 改范围查询，让列以原值参与比较
WHERE create_time >= '2026-11-08 00:00:00'
  AND create_time <  '2026-11-09 00:00:00'
```

为什么：B+ 树按 `create_time` 的原始值排序。你要求"按 `DATE(create_time)` 的结果查找"，索引里没有这个值，只能逐行算一遍。

MySQL 8.0 有函数索引（`CREATE INDEX idx ON t ((DATE(create_time)))`），但**能改写 SQL 就别加函数索引**——函数索引占空间，且对使用者不透明。

同类还有：`substring(...)`、`UPPER(...)`、`amount + 0`、`id * 2`。**只要列在等号左边被运算过，索引就退化成"列上的一个普通表达式"。**

【必读】② 隐式类型转换（最隐蔽）

```
-- phone 是 varchar(20)，这里传的是数字
WHERE phone = 13800138000        -- ✗ 索引失效
WHERE phone = '13800138000'      -- ✓ 走索引
```

为什么：MySQL 比较 `varchar` 和数字时，会把**字符串转成数字**再比。也就是等价于：

```
WHERE CAST(phone AS SIGNED) = 13800138000    -- 列被函数包裹 → 失效
```

但方向反过来，结论就反了：

```
-- user_id 是 int，这里传的是字符串
WHERE user_id = '123'            -- ✓ 索引仍然有效
```

因为这次是把**常量** `'123'` 转成数字，列保持原样，索引照常可用。

> **记这条只需要记一句话：转换发生在"列"上就失效，发生在"常量"上就没问题。**
> 判断方法：看列类型。varchar 列传数字 → 失效；int 列传字符串 → 不失效。很多人把这两个方向记反，导致调优调错地方。

顺带说一句：字符集 / 排序规则不一致也会让 JOIN 的索引失效，这一侧的深挖见章末延伸。

【必读】③ 前导通配符

```
-- ✗ B+ 树按前缀有序，'%手机' 无法定位起点
WHERE product_name LIKE '%手机'

-- ✓ 业务允许就去掉前导 %
WHERE product_name LIKE '手机%'
```

为什么：索引是一棵按字符串排序的树。"以 X 开头"能定位到起点，"包含X"不能——你必须从第一个叶子扫到最后一个。

真要"包含查询"，三条路：上 ES、建全文索引（`FULLTEXT`）、或者用倒排表自己维护关键词。**别指望 MySQL 的普通索引。**

【必读】④ `OR` 连接了无索引的列

```
-- ✗ remark 没索引 → 整个 OR 退化为全表扫描
WHERE user_id = 1 OR remark = 'x'

-- ✓ 方案一：给 remark 加索引
-- ✓ 方案二：拆成 UNION（如果两个条件都走索引）
SELECT * FROM t WHERE user_id = 1
UNION
SELECT * FROM t WHERE remark = 'x'
```

为什么： `OR` 要求"两个条件都能用索引"才有可能走 index merge。只要一侧走不了，优化器通常直接放弃，因为分开查再合并的成本可能更高。

注意 `UNION` 会去重排序，如果确认无重叠且不介意重复，用 `UNION ALL` 更快。

【必读】⑤ 违反最左前缀

```
-- 联合索引 (status, user_id)
WHERE user_id = 100              -- ✗ 跳过了 status
WHERE status = 1 AND user_id = 100   -- ✓
WHERE status = 1                 -- ✓ 用了 status
```

为什么： `(status, user_id)` 的排序是"先按 status，再按 user_id"。跳过第一列，剩下的部分在整张表里是**无序**的。

一个常被问的细节：

```
WHERE status = 1 AND amount > 100 AND user_id = 100   -- 索引 (status, user_id)
```

这里 `user_id` 在 `amount` 之后，但**仍然能用上**——因为 MySQL 会重排等值条件，`user_id = 100` 会被提到 `status = 1` 旁边。范围条件会"截断"后面的列，等值条件不会。

口诀：**等值列在前，范围列在后；一旦某列走了范围，它后面的列只能用于过滤（ICP），不能用于定位。**

【必读】⑥ 深分页：优化器主动放弃索引

```
-- ✕ 扫描 100 万行、回表 100 万次、丢掉前 100 万行
SELECT * FROM order_info ORDER BY id LIMIT 1000000, 10
```

这一条特殊：**索引没失效，是优化器算完成本后主动不用**。详见下一章。

【必读】⑦ `!=` / `NOT IN` / `IS NOT NULL` 在某些数据分布下失效

```
WHERE status != 1        -- 如果 99% 的行 status 都是 1 → 全表扫更快
WHERE name NOT IN (...)  -- 同上
```

为什么：索引的价值在于"快速定位少量行"。如果要返回 99% 的行，走索引 = 随机 IO 一百万次，全表扫 = 顺序 IO 一次，后者更快。

这一条不是绝对的"写法错误"，是"数据分布决定"。判断依据：`EXPLAIN` 的 `rows` 和实际返回行数的比例。**低于 20% 选择性才值得走索引。**

【必读】先把账算清楚：三层 B+ 树能放多少行、回表要几次 IO

"索引失效"这四个字之所以吓人，是因为没人算过它到底贵在哪。把 B+ 树的账摊开，后面七种写法为什么致命就不需要背了。

InnoDB 默认页大小 16KB。非叶子节点只存"索引键 + 页指针"，用 `BIGINT` 主键（8 字节）加 6 字节页指针，一项约 14 字节，一页能放 `16384 / 14 ≈ 1170` 个指针，也就是扇出 1170。叶子节点存整行，按每行 1KB 估算，一页放约 16 行。于是三层的容量是：

```
两层：1170 个叶子页 × 16 行 ≈ 1.9 万行
三层：1170 × 1170 个叶子页 × 16 行 ≈ 2190 万行
```

这就是"三层 B+ 树撑两千万行"的来历。点查时根节点常驻 Buffer Pool，实际磁盘 IO 约 2~3 次，工程上常说"3~4 次"。**这个"矮胖"结构是全章的定价基准**：一次索引点查的价值，就是拿 3 次左右的 IO 换掉全表扫描。

回表要把这个数翻倍。二级索引的叶子只存"索引列值 + 主键值"，`SELECT *` 拿到主键后，还得再走一遍聚簇索引的 B+ 树取整行，于是：

> 二级索引点查 = 3 次（二级索引 B+ 树）+ 3 次（聚簇索引回表）≈ 6 次 IO

更要命的是**回表是随机 IO**：二级索引上相邻的键，主键未必相邻，聚簇索引的页可能散落在各处。顺序扫描一页能顺带读十几行，随机回表一次往往只为一行。按一次随机读 0.1ms（SSD）到 10ms（HDD）的常见量级估，100 万次回表就是 100 秒到上万秒的差距——具体数字要在自己的硬件上实测，但量级关系不会变：**顺序 IO 与随机 IO 不在一个计价单位上**。

所以七种失效写法里，前三种（函数、隐式转换、前导通配）真正的损失不是"没走索引"，是把"3 次定位"变成了"逐行计算 + 可能的百万次随机回表"。第十三章的深分页，就是把这个放大到了 100 万倍。

【必读】七种失效的 EXPLAIN 判据逐条对照

七种写法不该靠"记住不能这么写"，要靠一条能当场判的命令。下面把每种写法对应的 `EXPLAIN` 关键列摆齐，判据只看三处：`type`、`key`、`Extra`。

写法	type	key	Extra 关键信号	一句判据
① 列上套函数 `DATE(create_time)=...`	ALL	NULL	Using where	列没有以原值参与比较
② varchar 列传数字	ALL	NULL	Using where	看列类型，字符串列被转数值
③ `LIKE '%手机'`	ALL	NULL	Using where	前缀不确定，定位不到起点
④ `OR` 一侧无索引	ALL	NULL	Using where	任一分支走不了索引
⑤ 跳过最左列	ALL 或 index	可能非 NULL	Using index / Using where	key_len 没覆盖首列
⑥ 深分页回表太多	ALL（主动）	NULL	可能无 Using where	possible_keys 有值但 key 为 NULL
⑦ `!=` / `NOT IN` 命中率高	ALL（主动）	NULL	Using where	rows 接近表总行数

第 ⑥、⑦ 两种要单独标出来：它们的 `possible_keys` 往往**有值**，只是 `key` 为 `NULL`。这不是失效，是优化器算过账后的主动放弃（见本章后面"区分度算式"）。把这两类当成"索引坏了"去重建索引，是白忙一场。

给两段能直接对照的输出片段，取自一张 280 万行的 `order_info`：

```
EXPLAIN SELECT * FROM order_info WHERE phone = 13800138000;
-- type=ALL  key=NULL  rows=2847391  Extra=Using where          ×

EXPLAIN SELECT * FROM order_info WHERE phone = '13800138000';
-- type=ref  key=idx_phone  rows=1                              ✓
```

再看第 ⑤ 种，判据藏在最容易被忽略的 `key_len` 里：

```
-- 联合索引 (status, user_id)
EXPLAIN SELECT * FROM order_info WHERE user_id = 100;
-- type=index  key=idx_status_user  Extra=Using index          × 全索引扫，不是定位
-- key_len 只覆盖首列长度，user_id 未参与定位
```

`type=index` 看起来"用了索引"，其实是顺着叶子链表扫完整棵索引树，和全表扫一个量级。**看到 `type=index` 不要高兴**，12.3 的判读表把它单列成陷阱，原因就在这里。

线上按这个顺序取证，两分钟能定性：先 `EXPLAIN` 看 `type/key/rows/Extra`，再 `SHOW INDEX FROM t` 确认索引真存在，最后 `SHOW CREATE TABLE t` 看列类型与排序规则。别一上来就 `ANALYZE TABLE` 或重建索引——那是第 ⑥、⑦ 类才用得上的动作。

【进阶】隐式转换的方向：为什么反着就没事，以及字符集那一刀

"转换发生在列上就失效，发生在常量上就没问题"这句话好记，但有两处容易被记成错的，值得拆开。

先把方向钉死。 官方比较规则里，一个字符串与一个数值比较，走的是"其余情况按双精度浮点比较"，被转的永远是**字符串那一侧**：

列类型	传入参数	被转的是	索引	结论
VARCHAR	数字 13800138000	**列**（逐行转 double）	✗ 失效	varchar 列传数字失效
INT	字符串 '123'	**常量**（一次转换）	✓ 有效	int 列传字符串不失效

所以 `WHERE user_id = '123'` 能用索引，不是因为"字符串也能走索引"，而是因为转换只发生在常量上、列保持原样。`CAST(phone AS SIGNED)` 这种写法在讲方向时够用，但**转换目标写窄了**：真实规则是转成 double，不是有符号整数。这个差别不只是措辞——double 只有约 15~16 位有效数字，19 位的订单号转过去会丢精度，两个本不相等的单号可能被判成相等：

```
SELECT ('1234567890123456789' + 0) = ('1234567890123456799' + 0) AS same;
-- 若输出 same=1，说明两个不同单号被判成相等：这已不是"慢一点"，是查错行
```

亲手跑一遍比记结论可靠，**这个结果需在自己的版本上验证**。这也是本章最想纠正的一点：多数人以为隐式转换只是性能问题，实际它同时是**正确性问题**。同理，`'123abc'`、`'0123'` 这类常量侧转换虽然不动索引，却会截断或改变数值语义——"索引有效"不等于"可以这么传"。

再说字符集那一刀。 JOIN 两侧的列如果字符集或排序规则不一致，MySQL 得按其中一方的 collation 对列值做转换才能比较，等价于在列上套了一层函数，被驱动表的索引同样用不上；严重时直接报 `Illegal mix of collations`。这里的反常识点是：多数人以为"都是 utf8mb4 就没事"，其实 **collation 不同照样失效**，而且 `utf8` 与 `utf8mb4` 在 MySQL 里是两个字符集。

```
SHOW FULL COLUMNS FROM order_info;      -- 看 Collation 列
SHOW CREATE TABLE order_info;           -- 对比两边 DEFAULT CHARSET / COLLATE
```

版本上也要留意：5.7 建表常见默认 `utf8mb4_general_ci`，8.0 起默认 `utf8mb4_0900_ai_ci`，老库与新表混用、或分库分表后跨库 JOIN（第十七章）时最容易踩。建表时显式写死 `DEFAULT CHARSET=utf8mb4 COLLATE=...`，比事后排查便宜得多。

12.3　怎么判读：`EXPLAIN` 的四个字段

```
EXPLAIN SELECT ...;
```

字段	看什么	及格线
`type`	访问方式	至少 `range`；`ALL`（全表）和 `index`（全索引扫）都不及格
`key`	实际用的索引	为 `NULL` = 没用索引；`possible_keys` 有但 `key` 为 NULL = 优化器主动放弃
`rows`	预估扫描行数	接近表总行数就是没走索引
`Extra`	额外动作	`Using filesort`、`Using temporary` 要优化；`Using index` 是好事（覆盖索引）

`type` 的优劣顺序（从好到差）：

```
system > const > eq_ref > ref > range > index > ALL
```

`index` 是个陷阱：它看起来"用了索引"，实际是"扫了整棵索引树"，和全表扫一个量级。看到 `type=index` 不要高兴。

另外两个值得认识的 `Extra`：

- `Using index condition` —— 索引下推（ICP）生效，把 Server 层过滤下推到引擎层，是好事
- `Using where` —— 拿到行之后 Server 层还要再过滤，说明索引的过滤能力不够

【必读】Extra 字段对照：覆盖索引、ICP、回表三者差在哪

`Extra` 是 12.3 判读表里信息量最大的一列，但它给的是"引擎额外做了什么"，不是"好/坏"的标签。把最容易混的四个摆在一起，每格给判据：

Extra	引擎实际做了什么	回表次数	判据
`Using index`	覆盖索引：所需列全在索引里，直接返回	**0 次**	把 `SELECT *` 改成只取索引列后出现
`Using index condition`	ICP：把非前缀列的过滤下推到引擎层，在索引上先筛再回表	减少（只回表命中的行）	联合索引有"跳过的列"被过滤时出现
`Using where`	Server 层拿到行后再按 WHERE 过滤	已回表	单独出现是中性，`ALL + Using where` 才是灾难
`Using filesort` / `Using temporary`	额外排序 / 临时表	不定	需优化，通常靠复合索引消掉

ICP（索引下推，5.6 起）值得单独画一下，因为它解释了一个常见困惑：为什么联合索引里"跳过的那一列"有时还有用。

```
没有 ICP（5.6 之前）：
  Server 层 ── WHERE a=1 AND c=3 ──▶ 引擎按 a=1 取出所有索引项
  引擎回表取每一行 ──▶ Server 再判断 c=3 ──▶ 不符合的丢掉
  代价：为所有 a=1 的行回表，哪怕 c=3 只剩 1 行

有 ICP：
  Server 层 ── WHERE a=1 AND c=3 ──▶ 引擎按 a=1 定位
  引擎在索引上先判断 c=3（索引项里就有 c）──▶ 只对命中的行回表
  代价：回表次数从"a=1 的行数"降到"a=1 且 c=3 的行数"
```

所以 12.2 第 ⑤ 条那句"范围条件后面的列只能用于过滤（ICP），不能用于定位"，说的就是这件事：**它不能帮你少走索引，但能帮你少回表**。两者都减少 IO，减少的却是不同环节。

覆盖索引则是把回表直接归零。二级索引叶子本来就带着主键值，若 `SELECT` 的列全在索引里（含主键），引擎无需再走聚簇索引，`Extra` 出现 `Using index`。这就是第十三章延迟关联能成立的前提——子查询只 `SELECT id`，`id` 就在索引上，翻 100 万行也不回表。

一个反常识点收尾：多数人以为 `Using where` 是坏信号，其实 `ref + Using where` 是健康组合（索引定位后做最终校验）；只有它跟在 `ALL` 后面，才意味着"全表捞回每一行再逐行过滤"。**看 `Extra` 必须连着 `type` 一起看。**

【进阶】区分度算式：优化器为什么"主动"放弃索引

12.2 第 ⑦ 条写"低于 20% 选择性才值得走索引"，这里要把两个被混用的词分开，否则会调错方向。

区分度（selectivity） 是列本身的性质，算式是：

```
selectivity = COUNT(DISTINCT col) / COUNT(*)
```

越接近 1 越好。性别、只有三五个值的状态列，区分度接近 0，单列索引基本没用——这是"该不该建索引"的判断。

过滤后返回比例 是这条 SQL 的性质，优化器真正算的是它：

```
hit_ratio ≈ 预估返回行数 / 表总行数 ≈ rows / TABLE_ROWS
```

`hit_ratio` 越大，走"二级索引 + N 次回表"就越不划算。成本模型大致是两边比大小：

```
走索引成本 ≈ 二级索引顺序扫描 + hit_ratio × 总行数 × 单次随机回表成本
全表扫成本 ≈ 聚簇索引顺序扫描（一次顺序 IO 读完全表）

当 hit_ratio × 总行数 × 随机IO单价 > 全表顺序扫描成本
    → 优化器选 ALL，key=NULL
```

代一组数：280 万行的表，`status != 1` 命中 90%，走索引要回表约 252 万次；即便每次回表只要 0.1ms（SSD、且假设全不命中缓存），也是 252 秒量级，而全表顺序扫一遍是秒级。**优化器放弃索引不是算错了，是算对了。** 那个"20%~30%"只是经验上的分水岭，不是硬阈值：它随行宽、Buffer Pool 命中率、磁盘类型、MySQL 版本的成本常数一起漂移，必须"需在自己的版本上验证"。

这解释了 12.2 第 ⑥、⑦ 两类"失效"：`possible_keys` 有值、`key` 为 `NULL`，索引完好无损，是优化器算过账主动不选。硬上 `FORCE INDEX` 往往更慢，而且数据分布一变就成了新的故障源。

有三件事会让这本账算歪：统计信息过期（`ANALYZE TABLE` 刷新）、数据分布倾斜（8.0 可用 `ANALYZE TABLE t UPDATE HISTOGRAM ON col WITH 64 BUCKETS` 建直方图）、以及参数化查询绑定变量导致计划与实测不符。判据是 `EXPLAIN` 的 `rows` 与真实返回行数差 100 倍以上——先 `ANALYZE TABLE`，再谈改写。8.0.18 起 `EXPLAIN ANALYZE` 能给出 `actual time / rows / loops`，但它**会真实执行 SQL**，生产上别对 `UPDATE`/`DELETE` 直接跑。

一个反常识点：多数人调索引时在"怎么让优化器用上索引"上使劲，而对第⑥、⑦ 类问题，正确动作恰恰相反——**要么减少返回行数（改写 SQL、加更精准的条件），要么接受全表扫**。第十三章的深分页，就是 hit_ratio 被 `LIMIT offset` 撑大之后优化器主动弃索引的同一个故事。

12.4　索引不是越多越好

每加一个二级索引，代价是：

- **写放大**：每次 INSERT/UPDATE 都要维护这棵 B+ 树
- **空间**：二级索引要复制主键值，主键越宽，所有索引越大
- **Buffer Pool 占用**：索引页也要进内存，缓存命中率下降

经验值：**单表索引控制在 5–8 个以内**。定期清理没被用过的：

```
SELECT * FROM sys.schema_unused_indexes;          -- 从未被使用的索引
SELECT * FROM sys.schema_redundant_indexes;       -- 冗余索引（A 和 A,B 同时存在）
```

MySQL 8.0 还有个安全删索引的机制：

```
ALTER TABLE order_info ALTER INDEX idx_phone INVISIBLE;   -- 先隐藏，观察一周
-- 没出问题再真正 DROP
```

`INVISIBLE` 让优化器看不见这个索引，但索引还在、还能瞬间恢复。**这是线上删索引唯一安全的方式**，直接 `DROP` 是不可逆的。

【进阶】一个能跑的验证脚本：把七种写法亲手证伪一遍

上面所有判据都可以在自己库里复现。下面这段脚本建一张小表、灌入数据、逐条 `EXPLAIN`，读者复制就能把七种写法证伪一遍；它同时是 12.4 ″索引不是越多越好″ 的验收工具——先证明索引用得上，再谈删。

```
-- MySQL 8.0；5.7 把 COLLATE 换成 utf8mb4_general_ci
DROP TABLE IF EXISTS idx_lab;
CREATE TABLE idx_lab (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  phone       VARCHAR(20)  NOT NULL,
  user_id     INT          NOT NULL,
  status      TINYINT      NOT NULL,
  city        VARCHAR(20)  NOT NULL,
  create_time DATETIME(3)  NOT NULL,
  remark      VARCHAR(64),
  PRIMARY KEY (id),
  KEY idx_phone (phone),
  KEY idx_status_user (status, user_id),
  KEY idx_suc (status, user_id, city),
  KEY idx_create_time (create_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- 交叉连接造 1000 行，不依赖存储过程
INSERT INTO idx_lab (phone, user_id, status, city, create_time, remark)
SELECT CONCAT('138', LPAD(n, 8, '0')), n, n % 3,
       CONCAT('c', n % 7), NOW(3) - INTERVAL n SECOND, CONCAT('r', n)
FROM (
  SELECT d1.i + d2.i * 10 + d3.i * 100 AS n
  FROM (SELECT 0 i UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3
UNION ALL SELECT 4
        UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL
SELECT 8 UNION ALL SELECT 9) d1
  CROSS JOIN (SELECT 0 i UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT
3 UNION ALL SELECT 4
        UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL
SELECT 8 UNION ALL SELECT 9) d2
  CROSS JOIN (SELECT 0 i UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT
3 UNION ALL SELECT 4
        UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL
SELECT 8 UNION ALL SELECT 9) d3
) x;
ANALYZE TABLE idx_lab;

-- 看 possible_keys：✕ 写法里索引名不出现，✓ 写法里出现 idx_phone
EXPLAIN SELECT * FROM idx_lab WHERE phone = 13800138000;              -- ✕ 隐式
转换
EXPLAIN SELECT * FROM idx_lab WHERE phone = '13800138000';            -- ✓
EXPLAIN SELECT * FROM idx_lab WHERE DATE(create_time) = '2026-11-08'; -- ✕ 函数
EXPLAIN SELECT * FROM idx_lab WHERE remark LIKE '%手机';               -- ✕ 前导
通配
```

```
EXPLAIN SELECT * FROM idx_lab WHERE user_id = 100;                    -- ✗ 跳过
最左列

-- Extra 三连：Using index / Using index condition / ALL + Using where
EXPLAIN SELECT id FROM idx_lab WHERE status = 1;                      -- 覆盖索
引
EXPLAIN SELECT * FROM idx_lab WHERE status = 1 AND city = 'c1';       -- ICP
（city 被下推）
EXPLAIN SELECT * FROM idx_lab WHERE remark = 'x';                     -- 全表 +
Using where

-- 造倾斜分布，观察优化器主动放弃索引（需按自己版本验证）
UPDATE idx_lab SET status = 1 WHERE id > 10;
ANALYZE TABLE idx_lab;
EXPLAIN SELECT * FROM idx_lab WHERE status != 1;                       -- 看
key 是否为 NULL
```

小表上优化器可能因为"全表更便宜"而不选索引，所以判据以 `possible_keys` 是否出现索引名为主，`key` 与 `rows` 为辅；**`EXPLAIN` 结果会随数据量、统计信息、MySQL 版本变化**，放大到十万行以上再观察更稳。字符集排序规则是脚本里唯一需要按环境调整的地方。

线上取证把这套动作压缩成三步：`EXPLAIN` 定性 → `SHOW INDEX` / `SHOW CREATE TABLE` 查列类型与排序规则 → 只有怀疑统计过期才 `ANALYZE TABLE`。顺序反了，就会在第 ⑥、⑦ 类问题上白白重建一次索引。

12.5　自检清单

- [ ] 所有 `WHERE` 里的索引列都没有被函数/运算包裹
- [ ] 传参类型与列类型一致（varchar 列传字符串，int 列传数字）
- [ ] 没有前导通配符 `LIKE '%x'`（或已上 ES）
- [ ] `OR` 两侧的列都有索引（或已拆 `UNION`）
- [ ] 联合索引满足最左前缀，等值列在前、范围列在后
- [ ] JOIN 的两张表字符集与排序规则一致
- [ ] 关键 SQL 的 `EXPLAIN` 有留存，`type` 不低于 `range`
- [ ] 单表索引数量 ≤ 8，且用 `schema_unused_indexes` 清理过
- [ ] 删索引走 `INVISIBLE` 观察期，不直接 `DROP`

12.6　一句话总结

> 索引失效从来没有"玄学"，只有四种原因：**列被包住了、类型对不上、前缀定位不了、优化器算过了不划算**。 前三种改 SQL，第四种改数据模型。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/04-MySQL与ORM坑、05-数据库/01-MySQL/索引原理、05-数据库/01-MySQL/执行计划
- **【选读】字符集 / 排序规则不一致也会废掉 JOIN 索引**：两张表一边 `utf8mb4`、一边 `utf8`（或 collation 不同），JOIN 时 MySQL 要按一方的 collation 对列值做转换，等价于在列上套了一层函数，被驱动表的索引同样用不上，严重时报 `Illegal mix of collations`。用 `SHOW CREATE TABLE` 对比两边 `DEFAULT CHARSET` / `COLLATE` 即可定位——很多"JOIN 特别慢"的锅在这里。（`utf8` 与 `utf8mb4` 是两个字符集、collation 不同照样失效的进阶展开见 12.2 的进阶小节。）

第十三章　深分页与慢 SQL：从 8 秒到 200 毫秒

本章要回答： `LIMIT 1000000, 10` 只返回 10 条，为什么这么慢？

本章你能带走

- 深分页的真实成本：**不是"少了 100 万行"，是"扫描 100 万 + 回表 100 万"**
- 三种改法：延迟关联、游标翻页、覆盖索引，各自的适用边界
- 一条判断：**优化器不用索引，常常是因为它算过账，觉得不值得**

难度构成： 本章主线【必读】；含【进阶】深挖 2 处；章末「延伸」小节放支线深挖与知识库指针。---

13.1　现场：导出功能拖垮了主库

一个后台导出页面，默认每页 10 条。运营往下翻到第 8000 页，浏览器转圈 8 秒，然后主库 CPU 冲到 90%，**其他接口跟着一起慢**。

SQL 很简单：

```
SELECT * FROM order_info ORDER BY id LIMIT 80000, 10;
```

`id` 是主键，`ORDER BY id` 走主键索引，看起来没毛病。但 `LIMIT 80000, 10` 的真实执行过程是：

```
① 沿主键索引顺序扫描
② 每扫到一行，回表取整行数据
③ 计数，前 80000 行全部丢弃
④ 只返回第 80001~80010 行
```

扫描 8 万行、回表 8 万次，只为返回 10 行。 成本随页码线性增长——第 1 页 1ms，第 8000 页 8 秒。

更糟的是它占着连接，而且 `SELECT *` 让回表的每一行都要读完整行（可能带 TEXT 字段）。

当时怎么误判的：第一反应是"深分页慢，那就加索引 / 加机器"——`id` 本就是主键索引、加了两台从库后主库 CPU 照样冲到 90%，都不对症；又怀疑"`SELECT *` 把 TEXT 字段拉太多拖慢了回表"，于是只 `SELECT id` 试了一遍，单条快了，但翻到第 8000 页依旧 8 秒。两条都排除后，才有人把 `LIMIT 80000, 10` 的执行过程一层层拆开：扫 8 万行、回表 8 万次、丢掉前 8 万行，瓶颈在"回表次数"而不是"扫描行数"。这一下把方向从"扩容"扭回"减少回表"，后面的延迟关联才对路。

13.2　三种改法

【必读】① 延迟关联（Deferred Join）

思路：**先在索引上拿到主键，再回表拿数据。** 让"丢弃 8 万行"这件事不产生回表。

```
-- ✗ 回表 8 万次
SELECT * FROM order_info ORDER BY id LIMIT 80000, 10;

-- ✓ 子查询只走索引（覆盖索引，不回表），外层按 id 精确回表 10 次
SELECT o.* FROM order_info o
INNER JOIN (
    SELECT id FROM order_info ORDER BY id LIMIT 80000, 10
) t ON o.id = t.id;
```

关键在子查询里只有 `id` 一列——`id` **就在索引上，不需要回表**。所以子查询是"纯索引扫描 8 万行"，外层只回表 10 次。

收益：从"8 万次回表"降到"10 次回表"。这是性价比最高的一种改法，因为它**不需要改前端、不需要改接口协议**。

【必读】② 游标翻页（Keyset Pagination）

思路：**不要"第 N 页"，要"比上一页最后一条更靠后的 10 条"。**

```
-- ✗ 偏移量翻页
SELECT * FROM order_info ORDER BY id LIMIT 80000, 10;

-- ✓ 游标翻页：把上一页最后一条的 id 带过来
SELECT * FROM order_info WHERE id > 80000 ORDER BY id LIMIT 10;
```

这条 SQL 的 `type` 是 `range`，扫描 10 行，**耗时恒定**，不管翻到第几页。

代价非常明确，必须先说清楚：

代价	说明
不能跳页	用户不能"直接去第 500 页"，只能上一页/下一页
排序字段必须唯一且有序	用 `create_time` 排序要加 `id` 作为第二排序键，否则会漏/重
前端要改	传 `lastId` 而不是 `pageNo`

```
-- 复合游标：按 create_time 排序时，要用 (create_time, id) 做游标
SELECT * FROM order_info
WHERE (create_time, id) > ('2026-11-08 10:00:00', 80000)
ORDER BY create_time, id
LIMIT 10;
```

只要业务允许（列表页、消息流、日志查询基本都是这种场景），游标翻页是最优解。 它把"越翻越慢"直接变成"永远一样快"。

【必读】③ 覆盖索引：让查询根本不需要回表

如果返回的字段都在索引里，MySQL 直接从索引返回：

```
-- 联合索引 (status, create_time, id)
SELECT id, status, create_time FROM order_info
WHERE status = 1 ORDER BY create_time DESC LIMIT 10;
-- Extra: Using index  ← 全程不回表
```

适用条件很苛刻：**返回的列必须全部在索引里**。做列表页（只返回摘要字段）时很有效，做详情页时没用。

【必读】把 100 万偏移量算成一笔账

13.1 现场里"第 1 页 1ms、第 8000 页 8 秒"不是两个孤立的数字，把它们相除就能验出成本模型：

```
8s / 1ms = 8000 = 页码数  →  耗时 ∝ 页码
```

`LIMIT offset, N` 的执行成本是线性的：

```
扫描索引项数 = offset + N
回表次数（SELECT *）= offset + N
丢弃行数     = offset
返回行数     = N
有效产出比   = N / (offset + N)
```

代入 offset = 1,000,000、N = 10：

```
扫描并回表 1,000,010 次，丢弃 1,000,000 行，返回 10 行
有效产出比 = 10 / 1,000,010 ≈ 0.001%
```

也就是说 **99.999% 的工作没有任何产出**。现场那个 8 秒，买到的就是 10 行数据。

再把回表换成 IO 账（第十二章算过：一次二级索引点查约 3 次 IO，回表再约 3 次，且回表是随机 IO）。取单次随机读 0.1ms（SSD）到 10ms（HDD）的常见量级：

```
1,000,010 × 0.1ms = 100 秒（SSD，且假设全不命中缓存）
1,000,010 × 10ms  = 约 2.8 小时（HDD）
```

这两个数字要在自己的硬件上实测，Buffer Pool 命中会把它们压下来，但**量级关系不变**：深分页的瓶颈从来不是"扫描 100 万行索引"，是"100 万次随机回表"。顺序扫索引 100 万项可能只要几十毫秒，随机回表 100 万次是另一个数量级。这也解释了为什么 `SELECT id`（覆盖索引）能救命——它把第二项整个删掉了。

反常识点：多数人以为 `LIMIT 1000000, 10` 慢在"数据库翻了 100 万行"，于是想靠加机器、加缓存解决。实际慢在"每一行都回表读了整行又扔掉"，**这是 SQL 写法制造的无用功，加机器只是把无用功复制 N 份**。13.5 把"加机器"排在最后一步，就是这个道理。

【必读】延迟关联前后：把耗时算式写出来

13.2 给了 SQL，这里把收益算成可核对的式子。改法的核心是让"丢弃 100 万行"这件事**不产生回表**：

```
-- ✗ 每扫一行就回表一次
SELECT * FROM order_info ORDER BY id LIMIT 1000000, 10;

-- ✓ 子查询只取 id（覆盖索引，不回表），外层按 10 个 id 精确回表
SELECT o.* FROM order_info o
INNER JOIN (
    SELECT id FROM order_info ORDER BY id LIMIT 1000000, 10
) t ON o.id = t.id;
```

设单次回表成本 `R`（随机 IO），单次索引项顺序扫描成本 `S`，经验上 `R` 比 `S` 大一到两个数量级。两种写法的成本：

```
改前 = (offset + N) × R                      ≈ 1,000,010 × R
改后 = (offset + N) × S + N × R              ≈ 1,000,010 × S + 10 × R
收益 ≈ 1 - S/R                               → 接近 100%
```

代一组便于心算的数：`R = 0.1ms`、`S = 0.001ms`：

```
改前 ≈ 100 秒
改后 ≈ 1.0 秒 + 0.001 秒 ≈ 1 秒
```

约两个数量级。**具体倍数取决于行宽、Buffer Pool 命中率与是否有 TEXT 列，需在自己的数据上实测**；标题里"8 秒到 200 毫秒"是 40 倍，属于这一类改法的常见区间，不是承诺值。

一个必须现场核对的前提：子查询的 `EXPLAIN` 里 `Extra` 要出现 `Using index`。如果子查询写成了 `SELECT id, status`，而 `(id, status)` 不在同一棵索引上，就会退化成"边翻边回表"，延迟关联白改。取证命令就一条：

```
EXPLAIN SELECT id FROM order_info ORDER BY id LIMIT 1000000, 10;
-- key=PRIMARY, Extra=Using index  ✓ 才成立
```

还有一处容易被忽略：延迟关联**仍然要顺序翻过 100 万个索引项**，它只是把回表从 100 万次降到 10 次。所以它是"必须跳页"场景下的最优解，不是"任意页都快"——真正恒定耗时的是游标翻页（13.2 ②）。两者的差别是 O(offset) 与 O(1)，别混。

【进阶】覆盖索引为什么能免回表：一次点查的两次 B+ 树

延迟关联之所以成立，全靠"覆盖索引"这一个机制。把它画开：

```
聚簇索引（PRIMARY）：叶子 = 整行数据
二级索引 idx_x(x)  ：叶子 = (x 的值, 对应主键 id)

SELECT * FROM t WHERE x = ? 的执行链路：
  ① 走 idx_x 的 B+ 树，定位到叶子，拿到 (x, id=100)      ← 第 1 次 B+ 树
  ② 拿 id=100 再走 PRIMARY 的 B+ 树，取出整行            ← 第 2 次 B+ 树（回表）
  ③ 返回整行

SELECT id FROM t WHERE x = ? 的执行链路：
  ① 走 idx_x 的 B+ 树，叶子上就已经有 id                 ← 只有第 1 次
  ② 直接返回，Extra: Using index，回表 0 次
```

关键在二级索引叶子**本来就带着主键值**：`id` 是索引的一部分，不需要去聚簇索引取。所以"覆盖"的判据很死——`SELECT` 的列、以及 `WHERE` / `ORDER BY` 用到的列，必须全部落在同一棵索引里，多一列都不行。

写法（索引 (status, create_time, id)）	回表次数	Extra	判据
SELECT id, status, create_time	0	Using index	全在索引内，覆盖
SELECT id, status, amount	命中行数次	Using where	amount 不在索引，逐行回表
SELECT *	命中行数次	通常无 Using index	破坏覆盖

这就把 13.2 ③ 那句"适用条件很苛刻"说清楚了：**多取一列就前功尽弃**，所以覆盖索引只适合列表页、下拉框、摘要接口这类字段固定的查询，详情页永远用不上。

一个反常识点：多数人以为 `SELECT *` 的代价只是"多传了几个字段"。更贵的是它悄悄把"索引扫描"变成了"大量随机回表"——第十二章的 `Extra` 对照表里，`Using index` 消失、`Using where` 出现，就是这件事的现场证据。表里有 `TEXT` / `JSON` 时更严重，每次回表都可能触发额外的离页读取。**改 `SELECT *` 往往是列表接口性价比最高的一次改动，一行代码、零索引变更。**

13.3　什么时候该用哪种

场景	推荐	原因
后台管理列表，允许改协议	游标翻页	恒定耗时，一劳永逸
前端已定死 pageNo，不能改	延迟关联	只改 SQL，不动接口
只返回少数字段	覆盖索引	彻底不回表
需要总数 COUNT(*)	单独处理（见下）	深分页的 COUNT 往往比数据本身更慢

`COUNT(*)` 是深分页的隐藏杀手。 分页组件要显示"共 128 万条"，这个数字每次翻页都算一遍：

```
SELECT COUNT(*) FROM order_info WHERE status = 1;   -- 走索引但也要扫全部匹配行
```

三种处理：

1. **缓存总数**：按条件缓存 5 分钟，翻页时不重算
2. **不显示精确总数**：显示"约 128 万条"，或只显示"是否有下一页"
3. **维护计数表**：写入时维护，牺牲实时性换性能

第 2 种最常被忽略，但它是最有效的。 用户真的需要知道"精确 1,284,731 条"吗？

【必读】游标翻页的三个边界：跳页、删除、乱序

13.2 ② 列了三条代价，这里把最容易被低估的两条展开——尤其是"删除"，它比"不能跳页"隐蔽得多。

边界一：不能跳页。 `WHERE id > lastId` 只认"上一页最后一条"，前端拿不到"第 500 页"的起点。Google 式"最多翻 100 页"或直接改成筛选条件，本质都是回避这个限制，不是解决它。

边界二：偏移量翻页本身会漏行 / 重行。 这条常被算到游标头上，其实正好相反。翻页之间若有写入，偏移量会整体漂移：

```
t0：SELECT ... LIMIT 10, 10   → 拿到第 11~20 行，lastId=20
t1：另一事务插入了 id=5 的新行（或删除了 id=3）
t2：SELECT ... LIMIT 20, 10   → 原第 21 行现在排到第 22 位，第 20 行被重复看到
```

游标翻页用 `id > 20` 定位，插入/删除都不影响结果——**它比偏移量翻页更健壮**，这也是 Feed 流、消息流天然该用它的原因。

边界三：游标键选错，删除与更新会静默漏数据。 真正的坑在这里。如果游标取的不是不可变主键，而是会变的排序列（排行榜分数、`update_time`、状态），那么上一页最后那一行被删除或改了分数之后，下一页的起点就对不上了：

```
-- ✗ 游标取了可变列，且排序键不唯一：漏行 / 重行
SELECT * FROM t WHERE score > 90 ORDER BY score LIMIT 10;

-- ✓ 复合游标：完整元组比较，第二排序键用唯一且不可变的 id
SELECT * FROM t
WHERE (create_time, id) > ('2026-11-08 10:00:00', 80000)
ORDER BY create_time, id
LIMIT 10;
```

注意元组比较必须写全 `(create_time, id) > (?, ?)`，只写 `create_time > ?` 时，同一秒内的多行会被整批跳过。这就是为什么 13.2 ② 要求"排序字段唯一且有序"——不是洁癖，是漏行与否的分界。

把"删除致漏"亲手跑一遍，比记结论可靠：

```
-- 复现：游标取了可变排序列，删除后漏行
-- 会话 A：第一页最后一条 score=90、id=7
SELECT * FROM t_rank ORDER BY score DESC, id DESC LIMIT 10;   -- 记下 last=(90,
7)
-- 会话 B：删掉 id=7 这一行，或把它的 score 改成 95
DELETE FROM t_rank WHERE id = 7;
-- 会话 A：第二页
SELECT * FROM t_rank WHERE score < 90 ORDER BY score DESC, id DESC LIMIT 10;
-- ✗ 只用 score 做游标，score 恰好等于 90 的其他行被整批跳过
-- ✓ 用完整元组 (score, id) < (90, 7) 才不会漏
```

判据只有一条：**游标值必须来自上一页那一行的"不可变列"**。删除之所以危险，不是因为 MySQL 记不住位置，而是你交给它的锚点本身可能已经不存在、或已经变了。用自增主键做锚点时，删除一行只是让那个 id 消失，后续 `id > lastId` 依然连续，不受影响——这也是"主键游标"比"业务字段游标"更稳的根本原因。

一句话收口：**游标翻页的性能是免费的，正确性要自己挣**。游标键必须满足唯一、有序、不可变三条，自增主键是唯一几乎不用想的选项；一旦用了业务字段，就得接受它在删除与更新下的边界。分库分表之后（第十七章）跨分片排序还得把分片键拼进游标，是同一问题的放大版；而排序能不能走索引，回到第十二章的最左前缀。

13.4　其他几类慢 SQL

【必读】隐式转换与索引失效

见上一章。这里只补一个排查动作：**慢查询日志里 `Rows_examined` 远大于 `Rows_sent`，就是"扫了很多只返回一点"**，典型是索引没走对。

```
-- 慢查询日志里看这三个数
-- Query_time: 8.02  Rows_examined: 80010  Rows_sent: 10
```

`Rows_examined / Rows_sent = 8001` —— 这个比值超过 100 就该查索引了。

【必读】N+1 查询

ORM 的经典病：

```
// ✗ 1 次查订单 + N 次查用户
List<Order> orders = orderMapper.selectByStatus(1);      // 1 次
for (Order o : orders) {
    o.setUser(userMapper.selectById(o.getUserId()));     // N 次
}
```

100 条订单 = 101 次 SQL。每次 2ms，总共 202ms——**单看每条都很快，加起来就是慢**。

```
// ✓ 批量查再内存拼装
List<Long> userIds = orders.stream().map(Order::getUserId).distinct().toList();
Map<Long, User> userMap = userMapper.selectByIds(userIds).stream()
        .collect(Collectors.toMap(User::getId, u -> u));
orders.forEach(o -> o.setUser(userMap.get(o.getUserId())));
```

MyBatis-Plus 的 `@TableField(exist = false)` 关联字段、JPA 的 `@ManyToOne(fetch = LAZY)` 都是 N+1 的高发地。**开启 SQL 日志，数一下一次请求打了几条 SQL**，是最快的发现方式。

【必读】批量写：伪批量

```
// ✗ 循环 insert：1000 次网络往返
for (Order o : list) orderMapper.insert(o);

// ⚠ saveBatch 但不加参数：仍是逐条 insert
// jdbc url 必须加 rewriteBatchedStatements=true，否则 MySQL 驱动不会真正合并
```

MyBatis-Plus 的 `saveBatch` 底层是 JDBC batch，但 **MySQL 驱动默认不重写 SQL**，所以每次都发一条完整 INSERT。加上参数才真正合并成 `INSERT INTO ... VALUES (...),(...),(...)`：

```
jdbc:mysql://host/db?rewriteBatchedStatements=true
```

这一行参数能把 1000 条插入从 3 秒降到 80ms。**它是最容易被忽略的一个"性能开关"**。

【必读】N+1 与伪批量：两段能直接跑的复现

13.4 给了改法，这里给"怎么证明它真的发生了"——这两类问题都不会报错，只能靠数 SQL 条数抓。

N+1 的可运行取证。 不靠肉眼数日志，把条数变成断言：

```
// ✗ 1 次查订单 + N 次查用户：100 条订单 = 101 次 SQL
List<Order> orders = orderMapper.selectByStatus(1);
for (Order o : orders) {
    o.setUser(userMapper.selectById(o.getUserId()));
}

// ✓ 批量查 + 内存拼装：2 次 SQL
List<Long> ids = orders.stream().map(Order::getUserId).distinct().toList();
Map<Long, User> map = userMapper.selectByIds(ids).stream()
        .collect(Collectors.toMap(User::getId, u -> u));
orders.forEach(o -> o.setUser(map.get(o.getUserId())));
```

```
// 可运行验证：用 p6spy / Druid 统计一次请求内的 SQL 条数
@Test
void noNPlusOne() {
    SqlCounter.reset();                 // 自定义拦截器累计 execute 次数
    List<Order> orders = orderService.listWithUser(1);
    assertThat(SqlCounter.count()).isLessThanOrEqualTo(2);   // ✗ 原实现会是 101
}
```

算式比感觉可靠：单次 SQL 2ms、单次网络 RTT 0.5ms，100 条订单：

```
N+1   = 101 × (2ms + 0.5ms) ≈ 253ms
批量  = 2   × (2ms + 0.5ms) ≈ 5ms
```

单看每一条都很快，加起来就是慢——这类问题在测试库（订单只有 3 条）里永远不会暴露，必须造满一页数据再跑。

伪批量的可运行取证。 `saveBatch` 不等于批量，别只看 URL，去数服务端到底收到几条 `INSERT`：

```
# 打开 general log，执行批量插入后数语句条数
mysql -e "SET GLOBAL general_log = ON;"
grep -c "INSERT INTO" /var/lib/mysql/*.log        # ✗ 未开 rewrite：1000 条
                                                  # ✓ 开了 rewrite：2 条（每批
500）
```

```
# JDBC URL：唯一真正让驱动合并多值 INSERT 的开关
jdbc:mysql://host:3306/db?rewriteBatchedStatements=true
```

反常识点：多数人以为调了 `saveBatch`、URL 里加了参数就万事大吉。**改了配置不等于生效**——驱动版本、是否启用服务端预编译、批内 SQL 模板是否完全一致，都会让重写失效；上面那条 `grep -c` 才是验收标准。另外合并后的单条 SQL 别超过 `max_allowed_packet`，按 500~1000 行一批是稳妥起点。

这两条和 13.1 的现场是同一类病：**慢在往返次数，不在单条耗时**。深分页放大的是回表次数，N+1 放大的是网络往返次数，伪批量放大的是事务提交次数——三者都是"把一个 O(1) 的动作写成了 O(N)"。

13.5　优化顺序：不要一上来就加索引

正确的顺序，成本从低到高：

```
1. 看 SQL 本身：有没有 SELECT *、有没有 N+1、有没有循环查库
2. 看索引：EXPLAIN，type 和 rows
3. 改 SQL：延迟关联、游标翻页、去掉函数
4. 加/调索引（注意写放大）
5. 改数据模型：冗余字段、读写分离、分表
6. 上缓存 / 异构（ES）
```

大多数"慢 SQL"在第 1、3 步就解决了。**直接跳到第 4 步加索引，往往加了也用不上**（因为问题是写法，不是索引）。

【进阶】优化顺序四步：为什么加机器总是最后一步

13.5 给了六步清单，把它压成四个决策点更好记，也更容易在 CR 里执行：

步骤	动作	取证命令 / 判据	跳过它的代价
① 看执行计划	先定位到具体哪条 SQL	慢日志按 `Rows_examined/Rows_sent` 排序；`EXPLAIN` 看 `type/key/rows/Extra`	凭感觉改 SQL，改错地方
② 改写 SQL	去掉 `SELECT *`、拆 N+1、延迟关联、游标翻页	改前改后 `rows` 与耗时留痕对比	加了索引也用不上
③ 调索引	补复合索引、覆盖索引、刷统计信息	`ANALYZE TABLE`；8.0 可建直方图	单表索引超 8 个，写放大
④ 加机器 / 分表	读写分离、缓存、ES、分库分表	只在 ①②③ 都做完仍不达标时	把无用功复制 N 份

顺序反了会怎样，用现场的数字算一遍。那条 8 秒的深分页 SQL 占着 1 个数据库连接跑 8 秒：

```
单实例：QPS=1、耗时 8s → 并发占用连接 ≈ 1 × 8 = 8 个
Hikari maximumPoolSize 默认 10 → 两个这样的请求就把池占满
加 10 台应用 → 10 × 10 = 100 个连接同时压向同一主库
→ 主库连接池先崩，其他接口一起慢（现场正是这样）
```

按 Little's law `L = λ × W`，加机器提高的是到达率 `λ`，不改服务时间 `W`。**只要 `W` 里的 8 秒还是 8 秒，扩容只是让队列更长、让主库更早打满。** 这和第 4 章连接池耗尽、第 7 章"加线程等于给排队队伍扩容"是同一条公式的三个现场。

取证顺序也按这四步走，两分钟内不要跳步：

```
-- ① 定位：慢日志三件套 Query_time / Rows_examined / Rows_sent
--    比值 > 100 就查索引
EXPLAIN SELECT ...;                          -- 看 type、key、rows、Extra
-- ② ③ 之间的分水岭：统计信息是否过期
ANALYZE TABLE order_info;                    -- 刷新后再 EXPLAIN 对比
-- 8.0.18+ 想看真实耗时（会真实执行，生产别对 UPDATE/DELETE 跑）
EXPLAIN ANALYZE SELECT ...;
```

版本上有三个会漂移的点：`EXPLAIN ANALYZE` 需 8.0.18+；8.0 起已移除查询缓存，别再按 `query_cache_size` 做规划；直方图（`UPDATE HISTOGRAM`）也是 8.0 才有的。分库分表之后（第十七章），深分页要改成"每片各取 TopN 再归并"，是步骤 ④ 里的另一套题；缓存（第十五、十六章）同理，属于最后一步，不是开场白。

一句话：**先把"每个请求做了多少无用功"降下来，再谈用多少台机器去扛。** 九成慢查询死在步骤 ②③，架构级手段是最后的大招。

13.6　自检清单

- [ ] 列表接口没有超过 100 页的偏移量翻页（或有游标翻页方案）
- [ ] 深分页的 `COUNT(*)` 有缓存或不精确返回
- [ ] 没有 `SELECT *`（尤其表里有 TEXT/JSON 时）
- [ ] 没有循环里查库（N+1），已批量查询
- [ ] 批量插入的 JDBC URL 带 `rewriteBatchedStatements=true`
- [ ] 慢查询日志有 `Rows_examined / Rows_sent` 比值监控（> 100 告警）
- [ ] 关键 SQL 的 `EXPLAIN` 有版本化管理
- [ ] 优化先改 SQL，再考虑加索引

13.7　一句话总结

> 深分页的慢，不是"数据库不行"，是**你在让它做了 8 万次没有产出的工作**。优化 SQL 的本质，是让数据库只做"有产出"的那部分工作。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/04-MySQL与ORM坑、05-数据库/01-MySQL/SQL优化、05-数据库/01-MySQL/执行计划

第十四章　锁与死锁：两个事务互等的那 50 秒

本章要回答：为什么一条 UPDATE 会让整个表卡住？为什么死锁之后应用只报"未知错误"？

本章你能带走

- 死锁的三个真实根因（不是"运气不好"）
- 一个立刻见效的招：**能降 RC 就降 RC，间隙锁直接消失**
- 一条判据：`UPDATE` 没走索引 = 锁住全表扫到的所有行

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

14.1　现场：一条 UPDATE 锁住了整张表

促销期间，一个"更新商品库存"的操作让商品列表页全部超时。

```
UPDATE product SET stock = stock - 1 WHERE sku_code = 'A1001';
```

`sku_code` 是业务字段，**没有索引**。

于是 InnoDB 做的事情是：全表扫描找 `sku_code = 'A1001'` 的行，**扫过的每一行都加上行锁**。

这张表 80 万行。这一次 UPDATE 锁住了 80 万行。其他任何涉及这些行的写操作全部阻塞，读操作在 RR 隔离级别下如果走了当前读（`SELECT ... FOR UPDATE`、`UPDATE`、`DELETE`）也一起阻塞。

> `UPDATE` **没走索引 = 锁表**（准确说是锁住所有被扫描到的行）。这一条比任何锁优化技巧都重要。

当时怎么误判的：第一反应是"商品列表页超时，是查询太多 / 慢 SQL 堆积把连接占满了"——查慢查询日志，没有异常长 SQL；又怀疑"是不是死锁，两个事务互等对方手里的行"，拉 `SHOW ENGINE INNODB STATUS` 也没看到明显的等待环。两条都排除后，才有人对那条 `UPDATE` 单独跑一次 `EXPLAIN`，发现 `type=ALL`——`sku_code` 根本没索引，全表扫描把扫到的每一行都加上了行锁。这段绕路印证了一句话：锁问题的第一现场永远是 `EXPLAIN` **确认 UPDATE 走没走索引**，而不是先猜死锁。

14.2　死锁的三个真实根因

死锁不神秘，只有三种成因：

【必读】① 加锁顺序不一致

```
// 线程 1：账户 A → B        线程 2：账户 B → A
synchronized (accountA) {           synchronized (accountB) {
    synchronized (accountB) {           synchronized (accountA) {
        transfer(A, B);                     transfer(B, A);
```

两个线程各持一把锁、各等另一把，标准死锁。

数据库里同样：

```
-- 事务 1
UPDATE account SET balance = balance - 100 WHERE id = 1;
UPDATE account SET balance = balance + 100 WHERE id = 2;

-- 事务 2（反向）
UPDATE account SET balance = balance - 100 WHERE id = 2;
UPDATE account SET balance = balance + 100 WHERE id = 1;
```

解法：全局统一加锁顺序。转账按账户 ID 排序，永远先锁小的：

```
Account first  = from.getId() < to.getId() ? from : to;
Account second = first == from ? to : from;
synchronized (first) { synchronized (second) { transfer(from, to); } }
```

这是最有效、最便宜的解法——**它把"环"的可能性直接消除**，而不是靠重试去化解。

【必读】② 无索引 UPDATE 全表加锁（上一节）

两个事务各扫全表、各锁一半、再互相等对方手里的行。这类死锁在高并发写同一张表时特别容易触发。

【必读】③ 间隙锁（Gap Lock）

这是最反直觉的一个。

RR（可重复读）隔离级别下，InnoDB 为了防止幻读，会对**索引记录之间的间隙**加锁。于是：

```
-- 事务 1
SELECT * FROM t WHERE id BETWEEN 10 AND 20 FOR UPDATE;   -- 锁住 (10,20) 这个间隙

-- 事务 2
INSERT INTO t (id, ...) VALUES (15, ...);                -- 要插入 15，落在间隙里 → 阻塞
```

更隐蔽的是：**两个事务各插一个不存在的 id，也可能死锁**——因为插入意向锁和间隙锁互相等待。

【必读】InnoDB 四种锁：把判据摆进一张表

14.2 讲了三个根因，但它们背后是同一套锁粒度。先把四种锁摆齐，每格给判据而不是定义——排查时你手里只有 `data_locks` 的一行输出，需要反推它属于哪种锁。

锁类型	锁的对象	典型触发	RR	RC	8.0 `data_locks` 的 `LOCK_MODE`（需按自己版本核对）
记录锁 Record Lock	单条索引记录	唯一索引等值命中	✓	✓	`X,REC_NOT_GAP` / `S,REC_NOT_GAP`
间隙锁 Gap Lock	索引记录**之间**的区间	等值未命中、范围当前读	✓	✗（基本无）	`X,GAP` / `S,GAP`
临键锁 Next-Key Lock	记录 + 前面的间隙	RR 下范围当前读的默认单位	✓	✗	`X` / `S`
意向锁 Intention Lock	**表级**标记	加任何行锁之前先加	✓	✓	`IX` / `IS`（`LOCK_TYPE=TABLE`）

还有第五种常被漏掉的：**插入意向锁**（`INSERT_INTENTION`），它是一种特殊的间隙锁，多个事务往同一间隙的不同位置插入互不冲突，但**与已存在的间隙锁冲突**。14.2 ③ 那句"两个事务各插一个不存在的 id 也会死锁"，机制就在这里：两边先各自持有间隙锁，再各自请求插入意向锁，环就成了。

意向锁最容易被误解。多数人以为 InnoDB 只有行锁，其实加行锁前必须先在同一张表上拿到意向锁（`IX` 对应写、`IS` 对应读）。它的作用不是锁数据，而是让"表里有没有行被锁住"变成 O(1) 判断：另一个事务想加表锁时，看一眼表上的意向锁即可，不必逐行扫描。**意向锁之间互相兼容，也不与行锁冲突**，它只和表级锁打交道。

把加锁范围推一遍，用一张只有 10、20、30 三行、`id` 为主键的表：

```
WHERE id = 20 FOR UPDATE                → 命中，仅记录锁 20
WHERE id = 25 FOR UPDATE                → 未命中，间隙锁 (20, 30)   ← 最易忽略
WHERE id BETWEEN 10 AND 20 FOR UPDATE   → 临键锁 (-∞,10]、(10,20] 及间隙 (20,30)
WHERE id > 15 FOR UPDATE                → (10,20] 起，一路锁到 supremum
```

第二条是全章最反常识的一句：**"没查到"不等于"没加锁"**，恰恰相反，等值未命中会锁住整段间隙，阻塞别人插入。14.1 现场那条无索引 UPDATE 之所以危险，也是这个机制的极端版——扫过的每一行、每段间隙都加锁，等于把整张表框住（第十二章的 `EXPLAIN` 判据在这里同样适用：先确认 UPDATE 走没走索引）。

两阶段锁协议收口：锁在语句执行时加上，**直到事务提交/回滚才释放**，不是语句结束就放。所以第 7 章"事务里做远程调用"真正拉长的，是这四种锁的持有时间，而不只是连接时间。

14.3　降 RC：最有效的一招

把隔离级别从 RR 降到 RC（读已提交），**间隙锁基本消失**（除了外键和唯一性检查场景），死锁概率大幅下降。

```
# my.cnf
transaction_isolation = READ-COMMITTED
```

或者只针对当前会话：

```
SET SESSION transaction_isolation = 'READ-COMMITTED';
```

代价要说清楚：

隔离级别	幻读	间隙锁	适用
RR（默认）	基本避免（当前读）	有	需要可重复读语义
RC	可能出现	基本没有	**大多数互联网业务**

为什么大多数业务可以降 RC：因为业务代码很少真的依赖"同一事务内两次读结果一致"。而 RC 带来的好处是实打实的——死锁少了、锁范围小了、并发高了。很多公司（包括多数互联网大厂）的默认就是 RC。

但有一条要注意：**RC 下 binlog 必须是 ROW 格式**，否则主从可能不一致。MySQL 5.7+ 默认就是 ROW，基本不用管，但升级老库时要确认。

【必读】RR 与 RC 到底差在哪：一张加锁差异表

14.3 给了结论"能降 RC 就降 RC"，这里补齐机制——它不是"少了一种锁"这么简单。

维度	RR（MySQL 默认）	RC	对死锁的影响
ReadView	事务首次快照读生成并复用	**每条 SELECT 新建**	不直接相关（MVCC，不锁）
间隙锁 / 临键锁	当前读默认 Next-Key	**基本关闭**（外键、唯一性检查除外）	消除"间隙互撞"这一类死锁
不匹配行的行锁	持有到事务结束	**语句执行完即释放**	锁范围与持有时间同时缩小
半一致读	无	`UPDATE` 遇已锁行可先读最新已提交版本再判断	减少无谓等待
幻读	快照读靠 MVCC、当前读靠 Next-Key 防	可能出现	业务代价

真正让死锁率"断崖式下降"的是中间两行：**RC 不只是关掉间隙锁，还会在语句结束后立刻释放那些没匹配上的行锁**。RR 下这些锁要一直挂到 `COMMIT`，于是两个事务各自扫过的行越多、重叠越多、持锁越久，成环的概率就越大。

给个可量化的对照。RR 下 `WHERE id BETWEEN 10 AND 20 FOR UPDATE`，锁的是"命中的记录 + 夹在中间的间隙 + 右端间隙"，锁对象数约等于"命中行数 + 区间数"；任何落进这些间隙的 `INSERT` 都要排队。RC 下只锁命中的记录：

```
RR：锁对象 ≈ 3 段区间 + 2 条记录 = 5 个可冲突点
RC：锁对象 ≈ 2 条记录          = 2 个可冲突点
```

并发写同一张表时，可冲突点从 O(区间) 降到 O(命中行)，这才是"最有效的一招"的物理原因。

版本上有两处漂移要记：一是变量名，5.7 查隔离级别用 `tx_isolation`，8.0 改名 `transaction_isolation`，写运维脚本时以 `SHOW VARIABLES LIKE '%isolation%'` 实测为准，别照抄；二是 `binlog_format` 必须为 `ROW`，否则 RC 下主从可能不一致（5.7.7 起默认已是 ROW，但升级老库要确认）。降级别前还有一句必须写进设计文档：**业务是否真的依赖"同一事务内两次读结果一致"**——依赖就别降，用别的方式拆事务。

一个反常识点：多数人以为"隔离级别越高越安全，所以别动它"。在高并发写入下，RR 的间隙锁冲突反而比 RC 更重；`SERIALIZABLE` 更极端，连普通 `SELECT` 都隐式加共享锁，并发度断崖式下跌。**隔离级别是并发度与一致性的取舍，不是安全等级。**

14.4　死锁之后：应用看到的什么

死锁发生时，InnoDB 会**自动回滚其中一个事务**（代价小的那个），并抛错：

```
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting
transaction
```

对应 Java 异常是 `DeadlockLoserDataAccessException`（Spring 转译后）。

关键问题：**很多应用把它当成了"未知系统错误"，然后给用户返回"系统繁忙"。**

```
// ✗ 死锁被兜底成 500，业务方无从判断
catch (Exception e) {
    return Result.fail(500, "系统繁忙");
}
```

正确做法是**识别并重试**——死锁是"可恢复"的，重试一次大概率成功：

```
// ✓ 死锁重试：最多 3 次，带随机退避
@Retryable(retryFor = DeadlockLoserDataAccessException.class,
           maxAttempts = 3,
           backoff = @Backoff(delay = 50, multiplier = 2, random = true))
@Transactional(rollbackFor = Exception.class)
public void transfer(Long from, Long to, BigDecimal amount) { ... }
```

三个要点：

1. **只重试死锁/锁超时**，不要重试所有异常
2. **退避要带随机**，否则重试的线程又会撞在一起
3. **重试必须幂等**——重试前事务已回滚，业务上要能接受"重新来一次"

【必读】死锁日志怎么读：WAITING FOR THIS LOCK 与 HOLDS THE LOCK

14.4 讲了应用侧看到的异常，这里讲数据库侧留下的证据。`SHOW ENGINE INNODB STATUS\G` 的 `LATEST DETECTED DEADLOCK` 段，把两个事务各自"在等什么""已持有什么"都写好了，不需要猜。下面是一段典型输出（字段已标注）：

```
------------------------
LATEST DETECTED DEADLOCK
------------------------
*** (1) TRANSACTION:
TRANSACTION 42123, ACTIVE 3 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 2 lock struct(s), heap size 1136, 1 row lock(s)
*** (1) WAITING FOR THIS LOCK TO BE GRANTED:    ← ① 事务1 在等什么
RECORD LOCKS ... index `PRIMARY` of table `db`.`account`
trx id 42123 lock_mode X locks rec but not gap waiting
Record lock, heap no 3 PHYSICAL RECORD: ...

*** (2) TRANSACTION:
TRANSACTION 42124, ACTIVE 2 sec starting index read
*** (2) HOLDS THE LOCK(S):                      ← ② 事务2 已持有什么
RECORD LOCKS ... index `PRIMARY` ... trx id 42124 lock_mode X locks rec but not
gap
*** (2) WAITING FOR THIS LOCK TO BE GRANTED:    ← ③ 事务2 在等什么
RECORD LOCKS ... index `PRIMARY` ... lock_mode X locks rec but not gap waiting
*** WE ROLL BACK TRANSACTION (1)                ← ④ 谁被回滚
```

读法只有一条主线：**把 ① 和 ② 对上，再看 ③ 落在谁的记录上**。事务 1 等的锁正是事务 2 持有的、事务 2 等的锁正是事务 1 已持有的，环就闭合了，再回代码里找两条 SQL 的加锁顺序（14.2 ①）。最后看 ④，被回滚的是 InnoDB 判定"代价较小"的那个事务，应用侧重试它即可。事务 1 持有什么，日志里不一定列全，用 `performance_schema.data_locks` 按 `trx_id` 补齐（见下一节）。

三个措辞的判据值得单独记：

日志里的措辞	含义	判据
`locks rec but not gap`	记录锁，只锁这一行	精确命中，冲突面最小
`locks gap before rec`	间隙锁	等值未命中 / 范围，RR 特有
无修饰的 `lock_mode X`	临键锁（记录 + 间隙）	范围当前读的默认单位
`ACTIVE 3 sec`	事务已活跃时长	远大于 SQL 耗时 = 锁被业务代码占着

最后一行是和第 7 章对上的接口：日志里 `ACTIVE 3 sec` 而该事务所有 SQL 加起来只有 8ms，中间那 3 秒就锁在事务里的远程调用上。

一个反常识点，也是最容易吃亏的：`SHOW ENGINE INNODB STATUS` **只保留最近一次死锁**，再发生一次就被覆盖，历史查不到。线上要常态化开 `innodb_print_all_deadlocks = ON`，把所有死锁打进 error log，否则只能守株待兔。另外 14.4 的 `@Retryable(retryFor = DeadlockLoserDataAccessException.class)` 只覆盖死锁（`ERROR 1213`），锁等待超时是另一个错误码 `ERROR 1205`、Spring 里是 `CannotAcquireLockException`——重试策略要把两者分开写，否则 14.5 那 50 秒的等待会以"未知错误"的面目出现。

14.5　锁等待：`innodb_lock_wait_timeout`

死锁会被自动检测并立即回滚（毫秒级）。但**锁等待不会**——它会一直等到超时，默认 **50 秒**。

```
ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction
```

50 秒意味着：一个被阻塞的请求，占着一个数据库连接和一条 Tomcat 线程 50 秒。**200 个线程 ÷ 50 秒 = 只要 4 个热点行就能打满线程池**（第七章讲过这个链条）。

```
-- 看当前谁在等、谁在持锁
SELECT * FROM performance_schema.data_lock_waits;
SELECT * FROM performance_schema.data_locks;

-- 看事务持有了多久
SELECT trx_id, trx_state, trx_started,
       TIMESTAMPDIFF(SECOND, trx_started, NOW()) AS held_sec,
       trx_query
FROM information_schema.innodb_trx
ORDER BY held_sec DESC;
```

`held_sec` 是关键：**如果这个数字远大于 SQL 执行时间，说明锁是被业务代码占着的**（事务里有远程调用、有 sleep、有大计算），不是 SQL 慢。这就是第七章那个故事的数据库视角。

【必读】5.7 与 8.0 的两套锁视图：别把表名搞混

14.5 的取证 SQL 里同时出现了 `information_schema.innodb_trx` 和 `performance_schema.data_locks`，这两套东西不是同一个版本层的，照抄容易在 5.7 上报"表不存在"。

想看什么	MySQL 5.7	MySQL 8.0	关键字

事务列表/活跃时长	`information_schema.innodb_trx`	`information_schema.innodb_trx`（仍在）	`trx_i`
			`trx_s`
			`trx_s`
			`trx_q`

持有/等待的锁	`information_schema.innodb_locks`	`performance_schema.data_locks`	`LOCK_` `LOCK_` `LOCK_`
谁阻塞谁	`information_schema.innodb_lock_waits`	`performance_schema.data_lock_waits`、`sys.innodb_lock_waits`	`block` / `wait`

8.0 把 `innodb_locks` / `innodb_lock_waits` 移除了，改由 `performance_schema` 提供；`innodb_trx` 因为好用被留了下来，两个版本列名基本一致。所以**"看事务"是通用的，"看锁"要按版本换表名**。

真正的增量是 8.0 `data_locks` 里的 `LOCK_DATA`：它直接给出被锁住的具体值（例如 `20`、或 `supremum` 伪记录），配合 `LOCK_MODE` 的 `X,GAP` / `X,REC_NOT_GAP` 就能一眼看出"这是记录锁还是间隙锁、锁在哪个区间"——5.7 的 `innodb_locks` 只给出锁的汇总信息，判读要靠猜。这也是为什么上一节的死锁日志在 8.0 上更好读。

按版本选一套执行，别混用：

```
-- ① 通用：按活跃时长找长事务（5.7 / 8.0 都能跑）
SELECT trx_id, trx_state, trx_started,
       TIMESTAMPDIFF(SECOND, trx_started, NOW()) AS held_sec,
       trx_mysql_thread_id, trx_query
FROM information_schema.innodb_trx
ORDER BY trx_started;

-- ② MySQL 5.7：谁在等谁
SELECT * FROM information_schema.innodb_lock_waits;

-- ③ MySQL 8.0：阻塞关系 + 锁明细（列名以 DESC 实测为准）
SELECT waiting_pid, waiting_query, blocking_pid, blocking_query
FROM sys.innodb_lock_waits;
SELECT ENGINE_TRANSACTION_ID, LOCK_TYPE, LOCK_MODE, LOCK_DATA, INDEX_NAME
FROM performance_schema.data_locks
WHERE LOCK_TYPE = 'RECORD';
```

`trx_query` 有个坑要记：事务当前空闲时它是 `NULL`，别以为"没 SQL 就没事务"——一个 `BEGIN` 之后迟迟不提交、又没在跑语句的空事务，照样持着锁。这正是第 7 章"事务耗时 > SQL 耗时之和"的数据库视角：`held_sec` 远大于 SQL 执行时间，锁就是被业务代码占着的。

一个反常识点：多数人排查锁等待第一反应是 `SHOW PROCESSLIST`。它只能看到"有个连接在 Sleep"，看不到谁阻塞谁、锁在哪一行。**先看 `innodb_trx` 定长事务，再看锁视图定阻塞关系**，顺序反了会在一堆 `Sleep` 里迷路。

【必读】50 秒的连锁反应：从一条锁等待到连接池打满

14.5 已经给了 "200 线程 ÷ 50 秒 = 4 个热点行打满线程池"。把这条链补完整，会发现**先崩的往往不是线程池**。

`innodb_lock_wait_timeout` 默认 50 秒。一个被阻塞的事务，在这 50 秒里同时占着 1 个数据库连接和 1 条 Tomcat 线程。按 Little's law `L = λ × W`（定义与符号见第 4 章），设热点行的写请求到达率 `λ = 20/s`、等待时间 `W = 50s`：

```
稳态需要的并发等待数 L = 20 × 50 = 1000 个
Hikari maximumPoolSize 默认 10  → 10 个连接瞬间占满
剩余请求卡在 getConnection()，再等 connectionTimeout（默认 30 秒）
这些等待又占住 Tomcat 线程 → 200 个线程池打满
上游超时重试（第 2 章）→ λ 翻倍 → 正反馈
```

链条是：**连接池先满 → 借连接等待 → Tomcat 线程池满 → 上游重试放大**。Hikari 默认 10 个连接比 Tomcat 200 小一个数量级，所以只盯线程池看会晚一步才发现起点（第 4 章）。

更值得警惕的是**默认值之间的倒挂**：

配置	默认值	管的是什么
`innodb_lock_wait_timeout`	50 秒	等行锁
Hikari `connectionTimeout`	30 秒	借连接
HTTP client `readTimeout`	应用自定	等网络
`@Transactional(timeout = 3)`	应用自定	事务内数据库操作累计时间

锁等待 50 秒比借连接 30 秒还长，意味着**很多请求还没等到锁的结果，就先因为借不到连接超时了**。于是日志里 `Lock wait timeout exceeded`（`ERROR 1205`）和 `Connection is not available` 混在一起，排查时容易把"连接池不够"当成根因去调大池子——池子调大只是让更多线程一起等那 50 秒。第 7 章给的"HTTP 超时 < 事务 timeout < 锁等待 timeout < 调用方超时"就是用来消除这种倒挂的。

还有一条前提容易被漏掉：死锁能被"立即回滚（毫秒级）"是因为 `innodb_deadlock_detect` 默认 `ON`。**关掉它**（高并发下有人为省检测开销这么干）之后，死锁不再被立即拆解，会退化成锁等待，一路耗到 50 秒超时——14.4 那套"识别 1213 并重试"的策略就失效了，必须同时覆盖 `ERROR 1205` / `CannotAcquireLockException`。是否关闭、以及各小版本的行为，需在自己的版本上验证。

所以调这个参数的正确顺序是：先缩短**持有**（事务里不加远程调用、UPDATE 走索引、大事务拆小），再缩短**等待**（按业务容忍度把 50 秒降到 3~10 秒让失败来得快一些）。反过来只调小 timeout，原本"等 50 秒能成功"的请求会变成 5 秒就失败，失败率反而上升——**止痛药不能当治病用**，这是 14.5 那句结论的完整理由。

14.6　乐观锁：另一种选择

不是所有并发都要悲观锁。库存扣减这类"读多写少冲突"的场景，乐观锁往往更合适：

```
UPDATE stock
SET quantity = quantity - 1, version = version + 1
WHERE sku_id = 1001 AND version = 5 AND quantity >= 1;

-- affected rows = 0 → 说明被别的请求改过，重试或失败
```

三个注意：

- `AND quantity >= 1` **不能省**，否则会扣成负数（这是一个比锁更常见的 bug）
- 重试次数要有限（3 次），高冲突场景乐观锁会退化成"一直重试"
- `affected = 0` 有两种原因：版本冲突、库存不足——**这两者的业务处理完全不同，需要先查一次区分**

```
// ✓ 区分两种 0
int affected = stockMapper.deductWithVersion(skuId, version);
if (affected == 0) {
    Stock s = stockMapper.selectBySku(skuId);
    if (s == null || s.getQuantity() < 1) {
        throw new BizException("库存不足");
    }
    throw new RetryableException("版本冲突，请重试");   // 交给上层重试
}
```

【进阶】乐观锁的 ABA：版本号为什么必须单调递增

14.6 给了 `version = version + 1` 的写法，这里补一个"为什么不能用别的字段代替 version"。

ABA 说的是：一个值从 A 改成 B、又改回 A，用"值相等"做判断会误以为"没变过"，从而覆盖掉中间发生过的事。数据库里最典型的踩法，是拿业务字段当比较基准：

```
-- ✗ 用 status / amount 当 CAS 基准：中间改过又改回，判断会失效
UPDATE t_order SET amount = 100 WHERE id = 1 AND amount = 100;

-- ✓ 用单调递增的 version：任何一次修改都会让它前进，不会回退
UPDATE t_order SET amount = 100, version = version + 1
WHERE id = 1 AND version = 7;
-- affectedRows = 0 → 已被他人修改，重新读取后重试
```

一段能跑的复现（两个会话）：

```
会话 A 读到 status=1、version=7
会话 B 把 status 改 1→2→1（ABA），version 被推到 9
会话 A 用 status=1 做条件更新：成功，中间那次变更被无声覆盖    ✗
会话 A 用 version=7 做条件更新：affectedRows=0，安全失败        ✓
```

反常识点：多数人以为 ABA 只是面试里的 CAS 八股，跟数据库无关。只要用**会回退的字段**（状态、分数、余额）做"读出来再比回去"的判断，就会踩；`version` 之所以安全，唯一原因是它**只增不减**，中间无论怎么改都留了痕。同理，别拿时间戳当版本：同一毫秒内两次更新、或时钟回拨，都会让"比较"失去意义；`version` 也别用会溢出回绕的小类型。

重试成本要算，否则高冲突场景下乐观锁会退化。设单次更新冲突概率为 `p`，重试次数服从几何分布，期望执行次数：

```
E = 1 / (1 - p)
p = 0.5  → E = 2 次
p = 0.9  → E = 10 次
p = 0.99 → E = 100 次
```

冲突率越高，乐观锁越是在做无用功：`p` 超过 0.5 就该换悲观锁，或者干脆像第 22 章秒杀扣减那样用一条 `UPDATE ... WHERE qty >= ?` 的条件更新，把判断与扣减交给行锁一次完成（连版本号都省了）。这也呼应 14.6 的第 ③ 条：`affected = 0` 除了"版本冲突"和"库存不足"，还有第三种——记录已被物理删除，三者业务处理完全不同，重试前必须先查一次区分。

最后把边界写清楚：重试的方法必须**幂等**（第 19 章），且每次重试是一个独立事务——14.4 的重试若写在同一个 `@Transactional` 方法里，第一次失败已经把事务标成 rollback-only，提交时会抛 `UnexpectedRollbackException`（第 6 章）。乐观锁不是"不加锁就万事大吉"，它只是把锁的成本从"持有"挪到了"失败重来"。

14.7　自检清单

- [ ] 所有 `UPDATE` / `DELETE` 的 `WHERE` 条件都走了索引（**这一条能消掉一半锁问题**）
- [ ] 事务内没有远程调用（第七章），没有 `sleep`，没有大循环
- [ ] 多资源操作有全局统一的加锁顺序
- [ ] 隔离级别评估过：能降 RC 就降（确认 binlog 是 ROW 格式）
- [ ] 死锁异常被识别并重试（`DeadlockLoserDataAccessException`），不是兜底成 500
- [ ] 重试是幂等的，带随机退避，有次数上限
- [ ] `innodb_lock_wait_timeout` 按业务容忍度调过
- [ ] 库存类扣减有 `AND quantity >= 1` 防负

14.8　一句话总结

> 死锁不是"数据库的 bug"，是**你的代码给资源排了一个环**。 排环的方式只有两种：统一顺序，或者不持有（一次拿全、或超时放弃）。

【选读】延伸

- 知识库对应原理篇：05-数据库/01-MySQL/事务与锁、17-常见Bug与解决方案/04-MySQL与ORM坑、05-数据库/01-MySQL/MVCC

第十五章　缓存一致性：先删缓存还是先更新库

本章要回答：为什么"先删缓存再更新库"是错的？延迟双删的 sleep 到底该睡多久？

本章你能带走

- 一张五种方案的取舍表（含各自的致命缺陷）
- 一个结论：**没有强一致，只有"不一致窗口多长"**
- 一条底线：**删缓存失败必须有兜底链路**

难度构成：本章主线【必读】；含【进阶】深挖 3 处；章末「延伸」小节放支线深挖与知识库指针。---

15.1　现场：用户改了昵称，三小时后才生效

一个很常见的客诉：用户改了昵称，页面上还是旧的，刷新、换浏览器、重新登录都没用，过了三个小时突然好了。

排查：数据库里是新的，Redis 里是旧的，而且这个 key 的 TTL 是 3 小时。**说明缓存没被删掉。**

代码：

```
// ✕ 先删缓存，再更新数据库
public void updateNickname(Long userId, String name) {
    redis.delete("user:" + userId);        // ① 删缓存
    userMapper.updateNickname(userId, name);  // ② 更新 DB
}
```

看起来"删了缓存就干净了"，实际有一个致命窗口：

```
时刻  线程 A（写）              线程 B（读）
 t1    删除缓存
 t2                             缓存未命中 → 读 DB（旧值）
 t3    更新 DB（新值）
 t4                             把旧值写回缓存          ← 脏数据回种
 t5                             缓存里是旧值，TTL 3 小时
```

线程 B 在 t2 到 t4 之间读了旧值，并在 t4 写回缓存。之后缓存就是脏的，直到 TTL 过期。

这个窗口的长度 = "读请求从查 DB 到写缓存"的耗时。并发越高，撞上的概率越大。**它不是小概率事件，是必然事件**，只是早晚。

当时怎么误判的：第一反应是"客户端脏了"——清缓存、换浏览器、重新登录，页面还是旧的，说明不是客户端；又怀疑"是不是 Canal / 同步链路把旧值又写回了缓存"，去查 binlog 消费记录，根本没有这条昵称的变更事件。两条都排除后，才有人去翻写接口的代码顺序：先 `redis.delete`、再 `userMapper.updateNickname`，并发下一个读线程在"删缓存之后、更库之前"把旧值填回了缓存。方向从"缓存服务"扭回"代码里删缓存的时机"，问题立刻就清楚了。

15.2　正确姿势：Cache Aside（先更库，再删缓存）

```
// ✓ 标准姿势
@Transactional(rollbackFor = Exception.class)
public void updateNickname(Long userId, String name) {
    userMapper.updateNickname(userId, name);      // ① 在事务内更新 DB（此时尚未提交）
    cacheDeleteAfterCommit(userId);               // ② 提交后再删缓存
}
```

为什么这样能解决大部分问题？因为"先更库再删缓存"的窗口是：

```
t1  线程 A：更新 DB（新值）
t2  线程 B：读缓存命中旧值 ← 有可能，但紧接着 A 会删掉
t3  线程 A：删缓存
```

最坏情况：B 在 t2 读到旧值，返回给用户一次。**但缓存已经被删了**，下一次请求会重新加载新值。**脏数据不会留在缓存里。**

对比一下两个方案的本质区别：

方案	脏数据在哪	影响时长
先删缓存再更库	**留在缓存里**	直到 TTL 过期（可能几小时）
先更库再删缓存	只在一次响应里	一次请求（毫秒级）

这就是"删缓存"必须放在"更库"之后的唯一原因：让脏数据没有机会停留在缓存里。

【必读】两个仍然存在的极端情况

即使是 Cache Aside，也有两个裂缝，必须知道：

① 删缓存失败——DB 更新成功，Redis 删除报错（网络抖动、Redis 主从切换）。此时缓存里是旧值，会一直脏到 TTL。

② 读请求回种与删除的竞态——极端时序下，读线程在"删缓存"之后、把旧值写回缓存，仍然可能脏。

① 是必须处理的；② 概率极低，靠 TTL 兜底即可。 很多文章把 ② 说成"所以要延迟双删"，这是过度设计。

【进阶】把两个极端情况画成时序图

先把 15.2 代码里那行注释纠正一下：`userMapper.updateNickname(...)` 执行时事务**还没有提交**，方法体里的 SQL 只是执行、没落定，真正的提交在方法返回之后。所以"先更新 DB（事务提交）"这个标注是错的；`cacheDeleteAfterCommit` 之所以要注册 `afterCommit`，正因为删除必须发生在提交之后。

15.2 里那两条"裂缝"值得摊开看，因为触发条件和后果完全不同。先把时间轴画出来，再谈该不该处理。

① 读写并发：读回填晚于写删除

前提：缓存此时是空的。读线程 B 要经历"查 DB → 拿到旧值 → 写回缓存"三步，写线程 A 经历"更新 DB → 提交 → 删缓存"三步。脏数据出现的唯一条件是 **B 的"写回"落在 A 的"删"之后**：

时刻	写线程 A	读线程 B	缓存状态
t1	更新 DB（新值，未提交）	—	empty
t2	—	miss → 查 DB	empty
t3	—	读到旧值（A 未提交）	empty
t4	提交 + 删缓存	—	empty
t5	—	把旧值 SET 回缓存	旧值（脏）
t6	—	—	旧值活到 TTL

注意 t3：A 的事务还没提交，其他连接读到的仍是旧值，这是 InnoDB 的 MVCC 决定的，不是 bug。真正致命的是 t5 落在 t4 之后——**删除已经发生过了，没有第二次删除来收拾这次回种**。窗口长度 = B 从"查 DB"到"SET"的耗时，通常 1~5ms；A 从"更新"到"删"要跨一次事务提交，通常 5~20ms。于是 B 比 A 快是常态，撞上 t5 晚于 t4 需要 B 被 GC、网络或慢查询拖慢。**概率低，但不是零**；一旦发生，脏数据和 15.1 那个"先删再更"的事故一样，要活到 TTL（现场里就是 3 小时）。

② 删缓存失败：DB 新、缓存旧

时刻	写线程 A	Redis	缓存状态
t1	更新 DB 并提交	—	旧值
t2	发起 DEL user:1001	网络超时 / 主从切换	旧值（DEL 没生效）
t3	catch 住异常，记日志	—	旧值
t4	—	—	旧值活到 TTL

这里没有并发、没有竞态，就是一次普通的失败。**它和 ① 的区别在于：① 是概率问题，② 是必然问题**——只要删缓存有失败率（Redis 抖动、连接池耗尽、网络分区、主从切换窗口），每失败一次就留一份长期脏数据。按 0.1% 的删除失败率、每天 100 万次更新算，一天就是 1000 个脏 key。**这才是必须在架构上兜底的那一个。**

> 多数人以为 Cache Aside 的两个裂缝"同等危险、都要靠延迟双删补"，实际 ① 靠短 TTL 就能自愈，② 才是会持续留脏的那个；把预算花在 ① 上是典型的用力错地方。

【必读】四种组合对照表：会不会脏、脏多久

"先删缓存还是先更库"其实只是问题的一半，另一半是"缓存那一步做的是删除还是更新"。两个维度叉开是四种组合，把"会不会脏、脏多久、判据"摆在同一张表里，比背结论有用：

#	组合	写顺序	会不会脏	脏多久	判据（一眼认出）
1	更新缓存 + 先缓存	SET 新值 → 更 DB	会	并发写乱序时长期脏；更 DB 失败则库旧缓存新	两个写线程的 SET 顺序与 UPDATE 相反，缓存留旧值
2	更新缓存 + 先库	更 DB → SET 新值	会	同上；SET 失败后脏到 TTL	缓存值是多表聚合/计算得来，重算比删除贵
3	删除缓存 + 先缓存	DEL → 更 DB	**必现**	直到 TTL（现场 3 小时）	15.1 的时序：读写并发，旧值被回种
4	删除缓存 + 先库（Cache Aside）	更 DB → DEL	极窄窗口会	一次请求（毫秒）；**删失败则到 TTL**	读回填晚于写删除（①），或 DEL 抛异常（②）

三条读表的规矩：

- **"更新缓存"两格都不该选。** 一是计算新值可能很贵（缓存里往往是几张表拼出来的 VO），二是并发写没有顺序保证：A 先算完、B 后算完，B 的 `SET` 先到、A 的后到，缓存里就是 A 的旧值。**删除是幂等的，天然没有覆盖问题。**
- **第 3 格是唯一"必现"的。** 它的窗口等于整个 DB 写耗时，并发一高几乎每次都撞；其余三格的脏要么是一次响应，要么靠更新本身兜住。
- **第 4 格的"脏多久"分两段：**正常路径毫秒级（下次读自愈），异常路径（删失败）直接跳到 TTL。这也是为什么 15.3 说"兜底比顺序更重要"——**选对第 4 格只解决了九成，剩下一成全在删除失败那一刻。**

> 反常识点：多数人以为"先更库 + 更新缓存"（第 2 格）比"先更库 + 删缓存"（第 4 格）更及时，实际恰好相反——更新缓存把"算新值"放进了写路径，既引入并发覆盖，又白存了可能没人读的值；删缓存把成本推迟到下次读，还顺带做了懒加载。

15.3　删缓存失败怎么办：必须有兜底

这是生产环境的底线，比选哪种顺序更重要。

```
// ✓ 删除失败进重试队列
private void cacheDeleteAfterCommit(Long userId) {
    TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization() {
        @Override
        public void afterCommit() {
            try {
                redis.delete(key(userId));
            } catch (Exception e) {
                log.error("cache delete failed, enqueue retry userId={}",
userId, e);
                retryQueue.offer(new CacheEvictTask(key(userId)));   // ① 本地重
试队列
                // 或 mqProducer.sendDelay("cache-evict", key, 5s);   // ② MQ 延
迟重试
            }
        }
    });
}
```

三种兜底方式，从轻到重：

方式	实现	可靠性	复杂度
本地重试队列	内存队列 + 定时重试	中（进程挂了就丢）	低
MQ 延迟重试	投递消息，消费端删除	**高**	中
Canal 订阅 binlog	监听 DB 变更，异步删缓存	最高（与业务零耦合）	高

Canal 方案的价值在于**它是唯一不依赖业务代码正确性的方案**——不管谁改的库（包括手工 SQL、其他服务），缓存都会被删。多业务线共用一个缓存时，这个优势非常明显。

【必读】订阅 binlog：把删除交给数据变更事件

Canal / Debezium 这类 CDC 方案的价值，不在于"比应用内删除更快"，而在于**把"删缓存"从业务代码里挪了出去**。链路是：

```
业务写 MySQL（只写库，不碰 Redis）
   │ 事务提交 → binlog（ROW 格式，含主键与前后镜像）
   ▼
Canal 伪装成 replica，向主库发 dump 请求拉 binlog
   │ 解析 RowChange：库名 / 表名 / 事件类型 / 主键值
   ▼
投递 MQ（或直连消费者），至少一次投递 → 消费端幂等
   ▼
删除 user:{id}；多级缓存时同时广播失效 L1（第 16 章）
```

三个必须说清的点：

1. **它不依赖谁改的库。** 应用忘了删、别的服务改的、DBA 手工 `UPDATE`、批量脚本，只要产生 binlog 就会触发删除。多业务线共用一个缓存时，这是唯一能收敛的方案。
2. **它是最终一致，不是强一致，延迟有量级。** 正常链路：binlog 落盘 → Canal 拉取（长轮询，通常几十毫秒内）→ 解析投递 → 消费者删 Redis，端到端**几十毫秒到几百毫秒**；MQ 积压或主从延迟大时会到**秒级**，大事务回放慢时更久（第 17 章讲复制延迟的成因）。所以"用户改完昵称立刻刷新"这类**写后立刻读**的请求不能指望 CDC，得让读路径直接读主库或跳过缓存。
3. **至少一次投递要求消费端幂等。** 删除天然幂等，这点是它比"更新缓存"省事的地方；Canal 重启、位点回放会重复投递，消费者不能假设"一条事件只来一次"。

把这个 `now - ts` 差值做成 P99 监控，比争论"CDC 是不是强一致"有用得多。

维度	应用内 afterCommit 删除	Canal / Debezium 订阅
触发源	业务代码	数据库 binlog
覆盖手工改库	✗ 不覆盖	✓ 覆盖
与业务耦合	高（每个写点都要写对）	低（业务零侵入）
端到端延迟	毫秒级（提交即删）	几十毫秒~秒级
失效来源	删失败、进程崩	复制延迟、MQ 积压、位点回放
运维成本	低	高（Canal 集群 + 位点 + 高可用）

版本区间：MySQL 8.0 默认 `binlog_format = ROW`（`STATEMENT` 下解析不出可靠的行镜像）、Canal 1.x、Debezium 2.x、Spring Boot 3.x / Redis 7。**具体延迟务必在自己的链路压测**——书里给的是量级，不是承诺。

【必读】删缓存失败的兜底：本地消息表与重试队列

15.3 给了"失败进队列"的雏形，这里把它补成一条**不会丢**的链路，并说清为什么 `afterCommit` 里塞内存队列还不够。

内存队列的漏洞很直白：`retryQueue.offer(task)` 成功了，但任务还在 JVM 堆里；进程重启、Pod 被驱逐、OOM，任务全丢。**删除失败本来就是低频事件，可一旦叠加发布/重启，丢的就是一批永久脏 key。** 要可靠，失败记录必须落在数据库或 MQ 里。

方案 A：本地消息表（最稳，和第 18 章是同一套机制）

把"要删的 key"和"业务更新"放进**同一个本地事务**：

```
-- 与业务 UPDATE 同事务，要么都成功要么都回滚
CREATE TABLE cache_evict_task (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  cache_key   VARCHAR(200) NOT NULL,
  status      TINYINT NOT NULL DEFAULT 0,   -- 0 待删 1 已删 2 死信
  retry_cnt   TINYINT NOT NULL DEFAULT 0,
  next_retry  DATETIME NOT NULL,
  create_time DATETIME NOT NULL
);
```

```
// ✓ 业务更新 + 失败登记同事务，不丢
@Transactional(rollbackFor = Exception.class)
public void updateNickname(Long userId, String name) {
    userMapper.updateNickname(userId, name);
    evictTaskMapper.insert(new EvictTask(key(userId)));
    TransactionSynchronizationManager.registerSynchronization(
        new TransactionSynchronization() {
            @Override public void afterCommit() {
evictExecutor.submit(key(userId)); }
        });
}

// ✗ 只在 afterCommit 里 catch 后塞内存队列：进程一挂，任务蒸发
@Override public void afterCommit() {
    try { redis.delete(key); }
    catch (Exception e) { retryQueue.offer(key); }   // 只在 JVM 内存里
}
```

后台任务扫 `status = 0 AND next_retry <= NOW()`，删除成功置 1；失败按指数退避累加 `retry_cnt`，超阈值置 2 并告警。**关键在"登记和业务更新同事务"**——即使 `afterCommit` 没跑、进程崩了，表里也留着待删记录，重启后扫描任务会捡起来。这正是第 18 章"可靠消息最终一致"的本地消息表模式，缓存删除只是它的一个特例；那一章展开事务与消息的完整边界，这里只借"落库不丢"这一点。

方案 B：MQ 延迟重试：删除失败投递延迟消息、超限进死信告警；比 A 轻，但投递本身也可能失败，生产上常两者叠用。

退避次数和窗口直接决定脏数据活多久，15.7 的算式会用到这组数字：

```
第 1~5 次重试间隔：1s, 2s, 4s, 8s, 16s
累计到最终成功/死信：1+2+4+8+16 = 31s
```

两个容易漏的细节：一是**删除必须幂等**（`DEL` 天生幂等，别换成"更新"）；二是**重试要限速**——Redis 抖动恢复瞬间，几千个重试任务一起砸过去会把刚恢复的实例再打趴（第 16 章雪崩讲的就是这个放大）。

15.4　延迟双删：一个被高估的方案

```
// ⚠ 延迟双删
redis.delete(key);
userMapper.update(user);
Thread.sleep(500);          // ← 睡多久？
redis.delete(key);
```

问题就在那个 500ms：

- 睡太短：读请求还没回种完，第二次删也白删
- 睡太长：接口 RT 直接 +500ms，而且占着 Tomcat 线程
- **该睡多久取决于"读请求从查 DB 到写缓存"的耗时分布，而这个分布随流量、GC、网络变化，永远测不准**

所以在工程上它是一个"看起来严谨、实际靠运气"的方案。

它的合理定位：**只作为过渡手段，用在不能改架构、又有偶发脏读的老系统上。**新系统不要用，直接用 Cache Aside + 删除重试。

【进阶】延迟双删为什么被高估：四个失效场景

15.4 说 sleep 时长是玄学，这里把"玄学"拆成四个能点名的失效场景——它们都不是"概率低"，而是设计上就补不住。

① 第二次删除早于读回种。 双删的 sleep 要大于"读请求从查 DB 到写回缓存"的 P99，但这个耗时随 GC、慢查询、网络抖动变化，没有上界。只要某次回种比 sleep 晚，第二次 `DEL` 就打在空缓存上，旧值随后照样写回。**用固定值覆盖一个长尾分布，本身就是错的。**

② 读写分离下回种的是从库旧值。 这是最隐蔽的一条：读请求走从库，主库提交后到从库追平有复制延迟（第 17 章），读线程在 sleep 期间从**从库**读到旧值、写回缓存；等第二次删除发生时，它删掉的可能是别的线程刚重建的新值，而旧值回种发生得更晚。sleep 要同时覆盖"主从延迟 + 回种耗时"，而主从延迟平常几毫秒、大事务时能到几分钟。**两段不可预测的长尾相加，还是不可预测。**

③ 第二次删除也会失败，而且没有兜底。 双删的两次 `DEL` 通常写在同一个方法里，第二次失败照样没人重试。它把"一次删除失败"变成"两次删除失败"，概率上只是 p^2，但**一旦发生，脏数据同样活到 TTL**——兜底链路（15.3）一次都没建立。

④ 异步双删会被进程重启吃掉。 把第二次删除放 `scheduledExecutor.schedule(...)` 看似不阻塞接口，但延迟任务在堆里：发版、Pod 重建、OOM 后任务消失，只剩第一次删除生效，退化成"先删再更库"的坏组合。

还有一笔性能账：`Thread.sleep(500)` 占着 Tomcat 线程不放。按第 7 章的算法，200 个线程、每次写请求白占 0.5 秒，这个接口的写吞吐上限就是 `200 / 0.5 = 400 次/秒`；促销时写昵称请求到 600 次/秒，线程池直接打满——**为补一个毫秒级竞态，付出一个数量级的吞吐。**

失效场景	第二次删除为何没用	调大 sleep 能否解决
回种晚于 sleep	打在空缓存上	✗ 长尾无上界
主从延迟下的旧值回种	删的时机与数据来源错位	✗ 延迟可到分钟级
第二次 DEL 自身失败	无重试兜底	✗ 与时长无关
异步任务随进程重启丢失	任务没执行	✗ 与时长无关

> 反常识点：多数人以为延迟双删"多删一次总归更安全"，实际它引入的两个新风险（sleep 期间的吞吐损失、异步任务丢失）比它想补的毫秒级竞态更贵。**它降低概率，不提供保证；一致性问题要的是保证，不是概率。**

它的定位仍是 15.4 那句：只作老系统的过渡，新系统用 Cache Aside + 删除重试 + 短 TTL。

15.5　五种方案对比

方案	顺序	一致性	复杂度	致命缺陷
先删缓存再更库	删 → 更	差	低	**脏读窗口，旧值回种缓存**
Cache Aside	更 → 删	较高（最终一致）	低	删缓存失败需兜底；极窄竞态窗口
延迟双删	删 → 更 → sleep → 删	中	中	**sleep 时长是玄学**
MQ 重试删除	更 → 删，失败投 MQ	高	中	需 MQ 与重试告警
Canal 订阅 binlog	更 → binlog → 删	高	高	链路长，运维成本

默认选 Cache Aside。 只有在"缓存被多方共用"或"一致性要求高"时，才考虑 Canal。

15.6　三个常见误区

① "更新缓存"而不是"删除缓存"

```
// × 更新缓存：并发下会写入旧值
redis.set(key, newUser);
```

两个线程并发更新，A 先算好新值、B 后算好新值，但 B 的写入先到达，A 的写入后到达——**缓存里是 A 的旧值**。删缓存不会出现这个问题，因为删除是幂等的。

② "缓存和 DB 双写，用事务保证"

Redis 不参与数据库事务。`@Transactional` 回滚了，Redis 的写入不会回滚。**跨存储的"事务"不存在**。

③ "用分布式锁保证一致性"

给每次读都加锁，等于把缓存退化成"串行访问数据库"——**为了 1% 的不一致，牺牲 99% 的性能**。这不是解决问题，是把问题换成了另一个。

15.7　TTL：最后一道防线

不管一致性做得多好，**TTL 必须设**。它是所有方案的最后兜底：

```
// ✓ 基础 TTL + 随机抖动（同时防雪崩，见下一章）
long ttl = 3600 + ThreadLocalRandom.current().nextLong(300);
redis.setex(key, ttl, value);
```

一个必须接受的现实：

> **缓存一致性不存在"做对"这回事，只存在"不一致窗口有多长"。** 你的目标不是消除它，是把它控制在业务能接受的范围（比如 1 秒），并且有对账兜底。

【进阶】TTL 兜底能把窗口压到多久：一个算式

15.7 说"TTL 是最后一道防线"，但"最后一道"能兜多久是可以算的。把不一致窗口记成 `T`，它由三条路径里**最快**的那条决定：

```
T = min( TTL , T_retry , T_cdc )

TTL     = 缓存 key 的存活时间
T_retry = 删除重试链路把脏 key 清掉所需时间
T_cdc   = binlog 订阅链路的端到端延迟（没有这条就当 +∞）
```

代现场的 numbers：`TTL = 3h = 10800s`，重试退避 5 次累计 31s（15.3），CDC 正常 P99 约 0.5s。

```
无兜底：       T = 10800 s              （3 小时，就是客诉里"三小时后才生效"）
+ 删除重试：   T = min(10800, 31) = 31 s
+ CDC 订阅：   T = min(10800, 31, 0.5) = 0.5 s
```

缩小倍数：`10800 / 31 ≈ 348 倍`、`10800 / 0.5 ≈ 21600 倍`。**业务含义：把窗口从 3 小时压到 30 秒内，靠的不是把 TTL 调小，而是给"删除失败"建一条会重试的链路。** 单靠调 TTL 会把命中率一起压掉——TTL 从 3 小时改成 30 秒，回源 QPS 大约放大 `10800 / 30 = 360 倍`，DB 先扛不住。**兜底链路和 TTL 是互补的，不是互相替代的。**

抖动同理：`3600 + random(300)` 把 10 万个 key 的过期摊到 300 秒窗口，每秒过期数从 10 万降到 ≈ 333 个（第 16 章雪崩的量化版本）。

最后留一段可运行验证，读者可以直接证伪"删失败后靠 TTL 自愈要多久"：

```
# 1) 写入一个带 3 小时 TTL 的旧值，模拟删缓存失败
redis-cli SET user:1001 "old_name" EX 10800
# 2) 看它还要活多久
redis-cli TTL user:1001          # → 10799 左右，单位秒
# 3) 触发重试链路后再查；正确的兜底应把它变成 -2（已不存在）
redis-cli TTL user:1001          # → -2；若仍是 10700，说明兜底根本没跑
```

把第 3 步接进 15.3 的重试消费者，这条命令就从演示变成对账：抽样比对 `DB 值 vs 缓存值`，不一致的 key 打印 `TTL`——接近满值是刚发生的竞态，掉了一大截是删失败的长期脏数据，优先级完全不同。

15.8　自检清单

- [ ] 缓存更新用的是**删除**，不是更新
- [ ] 顺序是"先更库，再删缓存"
- [ ] 删缓存放在事务提交之后（`afterCommit` 或 `@TransactionalEventListener`）
- [ ] 删缓存失败有重试链路（本地队列 / MQ / Canal）
- [ ] 没有用 `Thread.sleep` 做延迟双删（或已标记为技术债）
- [ ] 所有缓存 key 都有 TTL，且带随机抖动
- [ ] 关键数据有对账（缓存 vs DB 的定期比对）
- [ ] 没有给读路径加分布式锁来"保证一致"

15.9　一句话总结

> 缓存一致性的关键不是"删几次"，是**"脏数据有没有机会留在缓存里"**。 先更库再删缓存，就是让脏数据最多活一次请求；这已经够了。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/05-Redis与缓存坑、06-缓存/缓存一致性、06-缓存/Redis基础

第十六章　缓存三大问题与热 Key

本章要回答：为什么加了 32 个 Redis 节点，还是有一个 key 把 CPU 打满？

本章你能带走

- 穿透/击穿/雪崩三者的准确区别（多数人说不清）
- 一个真相：**热 key 是 cluster 的盲区，加节点救不了**
- 一条红线：`KEYS *`、`FLUSHALL`、大 key `DEL` 都是线上事故

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

16.1　现场：32 个节点，一个 key 打满 CPU

促销预热时，所有请求打同一个商品：`item:hot:1001`。

Redis Cluster 32 个节点，其中**一个节点 CPU 100%**，其余 31 个几乎空闲。这个节点的所有读写全部超时，于是所有依赖它的接口一起超时。

运维的第一反应是"加节点"——但集群已经是 32 个了。

问题在于：**Redis Cluster 按 key 的 CRC16 取模分 16384 个 slot，同一个 key 永远落在同一个 slot、同一个节点上。** 加再多节点，`item:hot:1001` 还是在那一个节点上。

> **热 key 的问题不是"集群容量不够"，是"单个 key 的吞吐有物理上限"。** 一个 Redis 单节点大约能扛 8–10 万 QPS（简单命令），超过就是单点瓶颈，和集群规模无关。

当时怎么误判的：第一反应是"集群容量不够，加节点"——运维已经把 Redis Cluster 从更少扩到 32 个，加完还是那一个节点 CPU 100%，纯属白加；又怀疑"是不是某个节点硬件 / 配置不均，成了拖后腿的慢节点"，查其余 31 个节点的 CPU 几乎全空，没有普遍瓶颈。两条都排除后，才有人去算 CRC16 分 slot 的规则：同一个 key 永远落同一个 slot、同一个节点，加再多节点也救不了单 key 的吞吐上限。这一下把方向从"扩容"扭回"单 key 拆分 / 本地缓存"，后面的热 key 解法才对路。

16.2　穿通雪：三个概念，别混着说

问题	现象	根因	关键词
穿透	大量请求查**不存在**的数据	缓存和 DB 都没有，每次都打到 DB	数据不存在
击穿	某个**热点 key 突然失效**	一万个请求同时重建同一个 key	单个 key 过期
雪崩	**大量 key 同时失效** 或 Redis 整体不可用	TTL 设成同一个值 / 宕机	面

一句话区分：**穿透是"查不存在的"，击穿是"一个热的没了"，雪崩是"一片同时没了"。**

【必读】穿透：空值缓存 + 布隆过滤器

```
// ✓ 空值也缓存，短 TTL
User u = userMapper.selectById(id);
if (u == null) {
    redis.setex("user:" + id, 60, "NULL");    // 30~60 秒，别太长
    return null;
}
```

```
// ✓ 布隆过滤器：先挡一层
if (!bloomFilter.mightContain(id)) {
    return null;              // 一定不存在，直接返回
}
```

布隆过滤器的两个特性必须知道：

- **只会误判"存在"，不会误判"不存在"** → 所以它能挡住不存在的请求，不会误杀真实数据
- **不支持删除** → 数据删了，布隆里还在，会短时间放行。要删除就得用 counting bloom 或定期重建

另一个坑：布隆过滤器**必须预热**。服务启动时没加载数据就上线，等于没开。

【必读】击穿：互斥重建 或 逻辑过期

```
// ✓ 方案一：互斥锁重建（只有一个线程回源）
String val = redis.get(key);
if (val == null) {
    if (redis.set(lockKey, "1", "NX", "EX", 10)) {   // SET NX EX 10：原子加锁带过期（SETNX 无此签名）
        try {
            val = loadFromDb();
            redis.setex(key, ttl, val);
        } finally {
            redis.delete(lockKey);
        }
    } else {
        Thread.sleep(50);                            // 没抢到，等一会儿重试
        return get(key);
    }
}
```

```
// ✓ 方案二：逻辑过期（不删 key，后台异步刷新）
class CacheVal {
    Object data;
    long expireAt;        // 逻辑过期时间
}
// 读到逻辑过期 → 返回旧值 + 异步刷新，用户永远不等
```

两种方案的取舍很明确：

方案	一致性	用户等待	复杂度
互斥重建	强（一定拿到新值）	**要等重建完成**（可能几百 ms）	中
逻辑过期	弱（短时间返回旧值）	**零等待**	中

热点商品详情页选逻辑过期，库存/价格这类选互斥重建。

【必读】雪崩：TTL 加抖动 + 高可用

```
// ✓ 基础 TTL + 随机抖动，消除"同时过期"
long ttl = 3600 + ThreadLocalRandom.current().nextLong(600);   // 3600~4200 秒
```

这是最便宜、最有效的一招——**一行代码消灭"同时失效"**。

Redis 整体不可用的雪崩，靠三件事：

1. **高可用**：哨兵 / Cluster，主从自动切换
2. **多级缓存**：本地缓存（Caffeine）挡一层，Redis 挂了还有本地
3. **熔断降级**：Redis 超时直接返回兜底值，不要死等

【必读】穿透、击穿、雪崩的判据表

16.2 那张表给的是定义，排障时要的是**判据**——看到监控上一条曲线，第一分钟就能把它归到三类中哪一类。三者的根因都落在"回源 QPS 突增"，但突增的形状完全不同：

判据	穿透	击穿	雪崩
查的数据存不存在	**不存在**（缓存和 DB 都没有）	存在，只是缓存那一刻没了	存在，但一大批 key 同时没了
受影响的 key 数量	大量**不同的**非法 key	**恰好 1 个**热 key	成片 / 全实例
DB QPS 曲线	随攻击流量平稳抬高	单 key 过期瞬间一根尖针	批量过期时台阶式跃升
缓存命中率	掉，且 `keyspace_misses` 全是新 key	短暂掉，随即恢复	大面积掉
第一分钟敲什么	`SLOWLOG GET` + 入参分布，看是不是随机 ID	`TTL hotkey` + `OBJECT FREQ hotkey`	`INFO stats` 看 `expired_keys` 突增、`evicted_keys`
最便宜的一招	空值缓存 + 布隆	互斥重建 / 逻辑过期	TTL 抖动

取证顺序建议固定下来，别一上来就抓线程 dump（第 7 章"锁→事务→线程→差值"的思路在缓存同样适用）：

```
# ① 先看命中率与过期曲线，区分"面"还是"点"
redis-cli INFO stats | grep -E
'keyspace_hits|keyspace_misses|expired_keys|evicted_keys'
# ② 看慢命令，确认是不是 KEYS / 大集合把主线程堵了
redis-cli SLOWLOG GET 10
# ③ 看单 key 热度（需 maxmemory-policy 为 *-lfu，见本节末尾）
redis-cli OBJECT FREQ item:hot:1001
# ④ 看客户端都在等谁
redis-cli CLIENT LIST | head -20
```

> 反常识点：多数人以为"命中率掉"就等于雪崩，实际穿透和击穿也会让命中率掉，三者区别在"掉完之后能不能自己回来"。**击穿的针尖在重建完成后立刻恢复，穿透不挡就一直低，雪崩要等一批 key 全部重建完；看恢复速度比看掉多少更有判据价值。**

【必读】布隆过滤器误判率：m、n、k 的算式

16.2 说布隆"只会误判存在"，但没说误判率怎么定。它完全由三个量决定：

```
n = 预计装入的元素个数
m = 位数组长度（bit）
k = 哈希函数个数
p = 误判率（false positive）

m = - n · ln p / (ln 2)²        k = (m / n) · ln 2
p = ( 1 - e^(-k·n/m) ) ^ k
```

代一组具体数字：`n = 1,000,000`（100 万商品 ID）、目标 `p = 0.01`（1%）。

```
m = -1e6 × ln(0.01) / (0.6931)²
  = 1e6 × 4.6052 / 0.4805 ≈ 9,585,058 bit ≈ 1.14 MB
k = (9.585e6 / 1e6) × 0.6931 ≈ 6.64 → 取 7

反代验证：p = (1 - e^(-7×1e6/9.585e6))^7
           = (1 - e^(-0.730))^7 = 0.518^7 ≈ 0.0100
```

业务含义：100 万 ID、1% 误判，只要 1.14 MB 内存和 7 次哈希，就换来了"不存在"的请求 99% 被挡在 DB 之前。 若把 `p` 压到 0.001，`m` 约 1.71 MB（1.5 倍内存换 10 倍精度），`k` 约 10——注意 `k` 变大意味着每次查询的 CPU 也涨，**误判率不是越低越好，要连哈希次数一起算**。

RedisBloom 把这些计算封装了，可以直接核对：

```
# 需在自己的版本上验证 BF.* 是否内置；独立模块时代要先装 RedisBloom
redis-cli BF.RESERVE product:bf 0.01 1000000
redis-cli BF.ADD product:bf 1001
redis-cli BF.EXISTS product:bf 99999        # 0 = 一定不存在
redis-cli BF.INFO product:bf                 # 看 Capacity / 实际占用
```

用 Python 复算同一组数字，复制就能证伪"1.14MB"：

```
import math
n, p = 1_000_000, 0.01
m = -n * math.log(p) / (math.log(2) ** 2)
k = (m / n) * math.log(2)
p_real = (1 - math.exp(-k * n / m)) ** k
print(f"m={m/8/1024/1024:.2f}MB  k={k:.2f}  p={p_real:.4f}")
# m=1.14MB  k=6.64  p=0.0100
```

两个坑配着公式记：一是**漏灌比误判严重**——真实 ID 没灌进过滤器会被判"一定不存在"，直接误杀真实请求，所以上线必须先全量预热；二是**元素超容量后误判率飙升**，`BF.INFO` 容量使用率超过 80% 就该规划重建（用"新 key + 双读灰度 + 切换"，别原地清空）。标准布隆不支持删除，根因是多个元素共享 bit 位，清零会误杀别人；要删就换布谷鸟过滤器或定期重建。

【必读】互斥重建与逻辑过期：两段代码对照

16.2 给了两个方案的伪代码，这里补上能直接抄的写法，顺带修掉一个流传很广的假 API。

先纠一个错：16.2 击穿代码里的 `setnx(lockKey, "1", 10)` 看着合理，其实**标准 Redis 没有这个签名**——`SETNX key value` 只有两个参数，不带过期；想要"不存在才写 + 自动过期"得用 `SET key value NX EX ttl`。混写的结果要么锁永不过期（持锁者崩了别人永远等），要么 TTL 根本没设上。这一点在 16.6 的锁坑里还会再出现。

```
// ✕ 伪 API：SETNX 没有第三个参数；value 用固定 "1"、finally 直接 DEL
if (redis.setnx(lockKey, "1", 10)) {
    try { val = loadFromDb(); redis.setex(key, ttl, val); }
    finally { redis.delete(lockKey); }     // 可能删掉别人刚拿到的锁
}

// ✓ 互斥重建：原子加锁 + 唯一 token + double-check + Lua 比对释放
private static final String UNLOCK_LUA =
    "if redis.call('get', KEYS[1]) == ARGV[1] then "
    + "return redis.call('del', KEYS[1]) else return 0 end";

public Product queryWithMutex(long id) {
    String key = "product:" + id;
    Product v = getFromCache(key);
    if (v != null) return v;

    String lockKey = "lock:product:" + id;
    String token = UUID.randomUUID().toString();
    Boolean ok = redisTemplate.opsForValue()
            .setIfAbsent(lockKey, token, Duration.ofSeconds(3));   // SET NX EX
3
    if (Boolean.TRUE.equals(ok)) {
        try {
            Product again = getFromCache(key);          // double-check：别人可能
已重建
            if (again != null) return again;
            Product p = productMapper.selectById(id);   // 全局只有一个线程回源
            setToCache(key, p, jitterTtl());
            return p;
        } finally {
            redisTemplate.execute(new DefaultRedisScript<>(UNLOCK_LUA,
Long.class),
                    List.of(lockKey), token);            // 只删自己的锁
        }
    }
    for (int i = 0; i < 3; i++) {                        // 等待者只重读缓存，不查
库
        sleepQuietly(50);
        Product p = getFromCache(key);
        if (p != null) return p;
    }
    return fallbackOf(id);                               // 有限次 + 兜底，绝不无限
自旋
}
```

```
// ✓ 逻辑过期：Redis 不设 TTL，value 自带 expireAt，返回旧值 + 异步刷新
record ExpiringValue<T>(T data, long expireAtMillis) {}

public Product queryLogicalExpire(long id) {
    ExpiringValue<Product> v = getExpiring(key(id));
    if (v == null) return loadWithMutex(key(id), id);      // 冷启动仍需一次互斥
    if (System.currentTimeMillis() < v.expireAtMillis()) {
        return v.data();                                    // 未过期，直接返回
    }
    refreshExecutor.submit(() -> {                          // 过期：先返回旧值
        String lock = "lock:refresh:" + key(id);
        if (Boolean.TRUE.equals(redisTemplate.opsForValue()
                .setIfAbsent(lock, "1", Duration.ofSeconds(30)))) {
            try {
                Product fresh = productMapper.selectById(id);
                setExpiring(key(id), new ExpiringValue<>(fresh,
                        System.currentTimeMillis() + 1800_000));
            } finally { redisTemplate.delete(lock); }
        }
    });
    return v.data();                                        // 用户零等待
}
```

两处细节比代码本身值钱：**等待者醒来必须读缓存，不能查库**（持锁者很可能已把值写回去；醒来还查库，锁就白加了）；**锁 TTL 按 DB 查询 P99 的 2~3 倍定**，不是拍 30 秒，太短会并发重建，太长则持锁者崩溃后其他人干等。

维度	互斥重建	逻辑过期
一致性	强（一定拿到新值）	弱（返回短时旧值）
用户等待	要等重建（几百 ms）	零等待
实现要点	token + Lua + double-check + 限次	`expireAt` 字段 + 异步线程池 + 刷新去重
风险	锁等待堆积	刷新失败 → 长期旧值；物理不过期 → 内存膨胀
适用	库存、价格	商品详情、榜单

④ TTL 抖动的随机区间算法（顺带把 16.2 雪崩那行补全）：`ttl = base + U(0, jitter)`，`U` 取均匀分布，`jitter` 取 `base` 的 10%~20%。它消除"同时过期"的原理不是"让每个 key 不一样"，而是**把 M 个 key 的过期时刻摊到宽度 jitter 的窗口**：均匀假设下同一秒过期数 ≈ `M / jitter`。取 `base=1800s, jitter=300s, M=10万`，则 `≈ 333 个/秒`；若全写成固定 1800s，就是 `10万/秒` 的瞬时回源，差 300 倍（第 15 章 TTL 算式用的是同一套推导）。**空值缓存的 TTL 也要抖动**，否则一批非法 ID 的空值会约好一起过期，制造一次穿透尖针。

16.3　热 key：三个解法，按成本排

【必读】① 本地缓存（最有效）

```
// Caffeine：热点数据放进程内，直接绕开 Redis
Cache<Long, Item> local = Caffeine.newBuilder()
        .maximumSize(10_000)
        .expireAfterWrite(Duration.ofSeconds(3))     // 短 TTL 保证一致性
        .build();
```

一个 8 核机器能扛几十万 QPS 的本地读。**热点 key 的流量被拦在进程内，Redis 完全不受影响。**

代价是**一致性变弱**：本地缓存 3 秒过期，意味着最坏 3 秒不一致。对商品详情、配置、字典这类数据完全够用；对库存、价格不行。

【必读】② key 拆分（读扩散）

把一个热 key 拆成 N 份，请求随机打散：

```
// 写：写 N 份
for (int i = 0; i < 10; i++) redis.set("item:1001:" + i, val);

// 读：随机选一份
int idx = ThreadLocalRandom.current().nextInt(10);
String val = redis.get("item:1001:" + idx);
```

代价：写放大 10 倍、数据有短暂不一致（写 N 份不是原子的）、删除要删 N 个。**只适合"读极多、写极少"的场景**（如秒杀商品快照）。

【必读】③ 读写分离（治标）

从节点分担读流量。但注意：**主从复制是异步的，热 key 的写入峰值会打满主节点**，读写分离只解决读。

【进阶】热 key 探测：三种手段的代价

16.1 说"加节点救不了热 key"，前提是**先知道哪个 key 热**。三种手段的代价差了一个数量级，别一上来就开 `MONITOR`。

① `redis-cli --hotkeys`（最省，但有前提）

```
# 必须先把淘汰策略切到 LFU，否则拿不到热度数据
redis-cli CONFIG SET maxmemory-policy allkeys-lfu
redis-cli --hotkeys
redis-cli OBJECT FREQ item:hot:1001     # 单点确认，返回 0~255 的对数频率
```

它靠 LFU 计数器采样统计，**不阻塞主线程**，但要满足两个前提：策略是 `*-lfu`（默认 `noeviction` 或 `*-lru` 时拿不到），且实例开了 `maxmemory`。**这意味着为了探测去改生产淘汰策略本身就有风险**——切到 `allkeys-lfu` 后内存打满会开始淘汰，得先确认容量余量。

② `MONITOR`（能看全，但可能自己制造故障）

```
redis-cli MONITOR | head -100000
```

`MONITOR` 会把**每一条命令**复制一份写进客户端输出缓冲区。按 16.1 那个节点 8 万 QPS 算，一条简单命令文本约 60 字节，就是 `8万 × 60B ≈ 4.8 MB/s` 的额外输出；更糟的是它同步推送，消费稍慢就顶到 `client-output-buffer-limit`，Redis 主动断开监控连接还算好的，极端情况下缓冲区膨胀拖慢主线程。**线上开 MONITOR 等于给已经 CPU 100% 的节点再加一份日志负载**——这正是 16.5 把它列进红线、却又让人忍不住去开的原因。真要用，限定几百条、低峰、单节点。

③ 客户端埋点（最可控，推荐做成长效机制）

在缓存访问层按 key 计数，本地聚合后上报，把统计压力留在应用侧：

```
// LongAdder 按 key 分桶，每 10 秒上报一次 TopN，避免每次访问都发指标
private final ConcurrentHashMap<String, LongAdder> counter = new
ConcurrentHashMap<>();

public <T> T get(String key, Class<T> type) {
    counter.computeIfAbsent(key, k -> new LongAdder()).increment();
    return redisTemplate.opsForValue().get(key);   // 序列化按项目封装
}
// 定时任务：取 TopN 上报，超过 1w QPS 告警（性能篇的经验阈值）
```

代价是**只能看到自己这个应用**，多应用共用一个 Redis 时要把各实例 TopN 汇总；好处是零 Redis 侵入、能带业务维度（哪个接口、哪个用户段打热了 key），还能顺手统计本地缓存命中。

手段	是否侵入 Redis	覆盖多应用	主要代价	适用
`--hotkeys` / `OBJECT FREQ`	中（要改淘汰策略）	全实例	需 LFU + maxmemory	临时巡检
`MONITOR`	**高**（同步复制每条命令）	全实例	输出缓冲区膨胀，可能压垮节点	低峰、短时、单节点
客户端埋点	零	单应用，需汇总	要写聚合与上报代码	**长效机制，推荐**

> 反常识点：多数人以为"用 Redis 自带命令查热 key 最准也最安全"，实际 `--hotkeys` 要先改淘汰策略、`MONITOR` 会放大故障；**最安全的探测恰恰不在 Redis 里，而在你自己的客户端。**

16.4　大 key：比热 key 更常见的杀手

判定标准（经验值）：

类型	大 key 阈值
String	> 10 KB（有人说几百 KB，但 10KB 的 key 几万个就是几百 MB）
Hash / List / Set / ZSet	元素数 > 5000，或总大小 > 1 MB

大 key 的三个危害：

1. **网络阻塞**：一次 `GET` 拉 1MB，千兆网卡跑满
2. **删除阻塞**：`DEL` 大 key 是**同步**操作，删一个百万元素的 Hash 会卡住 Redis 几百毫秒
3. **集群倾斜**：一个大 key 可以把一个节点的内存打满

```
# 定位
redis-cli --bigkeys                    # 采样扫描，找最大的 key
redis-cli MEMORY USAGE key_name        # 精确看单个 key 大小

# 删除（必须用 UNLINK，异步释放）
redis-cli UNLINK big_key
```

`UNLINK` 把"释放内存"这一步放到后台线程，主线程只做"从字典里摘掉"，不会阻塞。**线上删大 key 只能用 `UNLINK`，用 `DEL` 就是一次事故。**

【必读】大 key 的渐进式删除与拆分

16.4 给了 `UNLINK`，这里补两件它没说的事：**UNLINK 之后内存不会立刻降**，以及**老版本 / 集合类大 key 怎么拆**。

```
# ✓ 4.0+：异步释放，主线程只从字典摘掉 key
redis-cli UNLINK big_hash_key
# 确认后台真的在回收，别把"内存没降"当成 UNLINK 失效
redis-cli INFO stats | grep lazyfreed_objects

# ✓ 让过期 / 淘汰 / DEL 也走异步释放
redis-cli CONFIG SET lazyfree-lazy-expire yes
redis-cli CONFIG SET lazyfree-lazy-eviction yes
# 各 lazyfree 开关的默认值需在自己的版本上验证

# ✓ 无 UNLINK 的老版本：HSCAN 分批 HDEL，每次只删 500 个 field
redis-cli HSCAN big_hash_key 0 COUNT 500
# 循环取 field 后逐批 HDEL，最后再 DEL 空壳
```

`UNLINK` 把释放交给后台线程（`lazyfree`），所以对**百万元素的 Hash**，`DEL` 是主线程同步 `O(N)` 释放、卡几百毫秒，`UNLINK` 是主线程 `O(1)` 摘 key、后台慢慢还内存。**`UNLINK` 后 `used_memory` 不立刻下降是正常的**，看 `lazyfreed_objects` 增长即可——16.4 那句"必须用 UNLINK"没错，但别拿"内存马上下了"当验收标准。

拆分比删除更根本。**大 Hash 按业务键分桶**：`user:profile:{id}` 装不下，就按 `id % 64` 拆成 64 个小 Hash；**大 List/ZSet 按时间分片**：`feed:global` 按月拆成 `feed:2026-11`。拆完单 key 元素数落到 5000 以内，`HGETALL` / `ZRANGE 0 -1` 这类 O(N) 命令自然不再堵主线程。写侧加一道阈值告警（value > 10KB 或集合 > 5000 元素直接拒绝），比事后 `--bigkeys` 巡检便宜得多。

一条取证顺序，照着敲就能判断"卡顿是不是大 key 释放造成的"：

```
redis-cli SLOWLOG GET 10                          # DEL / HGETALL 是否上榜（单位微秒）
redis-cli --bigkeys                               # 采样找各类型最大 key，SCAN 安全
redis-cli MEMORY USAGE big_hash_key SAMPLES 0     # 精确看字节数
redis-cli INFO memory | grep mem_fragmentation_ratio
```

`mem_fragmentation_ratio > 1.5` 才值得处理，`> 2` 考虑 `activedefrag yes` 或低峰滚动重启；`< 1` 说明部分内存被换出到 swap，是更严重的信号，别当成"内存省了"。**版本区间**：`UNLINK` / `lazyfree` 自 Redis 4.0 起，`--memkeys`、字段级 TTL（`HEXPIRE`）是 7.4+，`CLUSTER SLOT-STATS` 是 8.4+；具体命令是否可用，在自己的实例上用 `COMMAND INFO UNLINK` 核一下再写进运维脚本。

16.5　线上禁用命令

这几条不是"优化建议"，是红线：

命令	危害	替代
`KEYS *`	遍历全部 key，**阻塞主线程**，百万 key 卡几秒	`SCAN` 渐进遍历
`FLUSHALL` / `FLUSHDB`	清库，不可逆	无（禁用）
`HGETALL` 大 hash	一次拉几百 MB	`HSCAN` 分批
`DEL` 大 key	同步释放，阻塞	`UNLINK`
`SMEMBERS` 大 set	同上	`SSCAN`
`MONITOR`	生产开它等于给 Redis 加了个同步日志	用 slowlog

配置层面也加一道保险：

```
rename-command KEYS ""
rename-command FLUSHALL ""
rename-command FLUSHDB ""
```

16.6　Redisson 分布式锁的三个坑

缓存章节里带一句锁，因为库存扣减常常用：

① 误删别人的锁

```
// ✗ 自己用 SETNX + DEL 实现：value 是固定的 "1"、finally 直接删
//    A 超时释放后 B 拿到锁，A 的 DEL 删掉的其实是 B 的锁
redisTemplate.opsForValue().setIfAbsent(lockKey, "1", Duration.ofSeconds(30));
try { doDeduct(skuId); }
finally { redisTemplate.delete(lockKey); }
```

自己用 `SETNX` + `DEL` 实现就会踩上面那个坑；Redisson 的 `unlock()` 内部用 Lua 脚本校验 value，**不会误删别人的锁**，但锁已过期时调用会抛 `IllegalMonitorStateException`（详见本节末尾的小节）。

② watchdog 只在没指定 leaseTime 时生效

```
// ✓ 不指定 leaseTime → watchdog 每 10 秒续期，业务没完锁不会过期
lock.lock();
// ✗ 指定了 30 秒 → 30 秒后强制释放，业务还在跑就丢锁了
lock.lock(30, TimeUnit.SECONDS);
```

③ 主从切换丢锁

Redis 主节点写入锁 → 还没同步到从节点 → 主节点挂了 → 从节点升主 → 锁没了。**RedLock 试图解决，但有学术界争议**（时钟跳变、GC 停顿可以破）。

结论：**强一致场景不要只靠 Redis 锁。** 用数据库唯一约束、状态机（`UPDATE ... WHERE status = 'INIT'`）兜底，Redis 锁只作为"减少冲突"的优化。

【必读】Redisson 两个坑的代码：看门狗与误解锁

16.6 给了两处最容易被 CR 放过的写法，这里补成可对照的代码。

坑一：看门狗续期只在"不显式指定 leaseTime"时生效。

```
RLock lock = redissonClient.getLock("stock:" + skuId);

// ✗ 显式传 leaseTime：watchdog 关闭，30 秒后强制释放
//    业务若跑 40 秒，后 10 秒没有锁，第二个线程进了临界区
lock.lock(30, TimeUnit.SECONDS);
try { doDeduct(skuId); } finally { lock.unlock(); }

// ✓ 不指定 leaseTime：默认 30s 租约，后台每 10s（leaseTime/3）续期，
//    业务没跑完锁不过期；进程崩溃后最多 30s 自动释放
lock.lock();
try { doDeduct(skuId); } finally {
    if (lock.isHeldByCurrentThread()) { lock.unlock(); }
}
```

看门狗不是"永久锁"：它是 `RedissonLock` 里的定时刷新任务，靠 `EXPIRE` 把租约往回拨：**一旦显式给了 `leaseTime`，这个定时任务根本不会注册**（16.6 ② 的结论对，但没写清"注册"这个机制）。所以"业务耗时可能超过 30 秒"和"显式设一个更大的 leaseTime"不是一回事——后者关闭了续期，把长尾 GC、下游抖动全暴露成丢锁。**只在能给出业务耗时硬上界时才显式指定。**

坑二：解锁"不是自己的锁"。

16.6 ① 把 `lock.unlock()` 标成 ✕、注释写"没校验 value"，但同一节又说"用 Redisson 的 API 是安全的"——这两句其实不矛盾：前者指手写 Redis 锁容易漏校验，后者指用 Redisson 时校验由框架保证。真实情况分两层：

```
// ✕ 自己手写 SETNX + DEL：A 超时释放后 B 拿到锁，A 的 DEL 删掉 B 的锁
redisTemplate.opsForValue().setIfAbsent(lockKey, "1", Duration.ofSeconds(30));
try { doDeduct(skuId); }
finally { redisTemplate.delete(lockKey); }        // 删的很可能不是自己的锁

// ✓ 手写也要 value 带唯一标识 + Lua 原子"比对再删"
String token = UUID.randomUUID().toString();
if (Boolean.TRUE.equals(redisTemplate.opsForValue()
        .setIfAbsent(lockKey, token, Duration.ofSeconds(30)))) {
    try { doDeduct(skuId); }
    finally {
        redisTemplate.execute(UNLOCK_SCRIPT, List.of(lockKey), token);  // GET
相等才 DEL
    }
}

// ✓ Redisson 的 unlock() 内部已用 Lua 校验 value，不会误删别人的锁；
//    但锁已过期/被别人持有时调用会抛 IllegalMonitorStateException
RLock lock = redissonClient.getLock(lockKey);
try { doDeduct(skuId); }
finally {
    if (lock.isHeldByCurrentThread()) {   // 先判断，避免对不属于自己的锁调用
unlock
        lock.unlock();
    }
}
```

所以"Redisson 安全"要加半句：安全是指"不会误删别人的锁"，不是"任何时刻 unlock() 都能安静返回"。 少了 `isHeldByCurrentThread()` 这道判断，在锁超时、看门狗没续上的场景下，`unlock()` 抛出的 `IllegalMonitorStateException` 会盖掉真正的业务异常，日志里只剩一个莫名其妙的锁错误（第 6 章"catch 住的是异常不是状态"是同一类陷阱）。

最后重申 16.6 ③：主从异步复制下，锁写入主节点尚未同步就切换会丢锁；RedLock 有学术界争议（时钟跳变、长 GC pause 都能破坏其安全性）。**强互斥（库存、幂等、资金）不要只靠 Redis 锁**，用数据库唯一约束或状态机 `UPDATE ... WHERE status = 'INIT'` 兜底（第 14 章行锁、第 18 章最终一致），Redis 锁只用来"减少冲突"。

16.7　自检清单

- [ ] 热点 key 有本地缓存（Caffeine）或 key 拆分方案
- [ ] 所有 TTL 带随机抖动，没有"整点同时过期"
- [ ] 不存在的查询有空值缓存或布隆过滤器
- [ ] 热点重建有互斥（互斥锁或逻辑过期），不是全部回源
- [ ] 没有大 key（`--bigkeys` 扫过），删除用 `UNLINK`
- [ ] `KEYS` / `FLUSHALL` 已在配置文件重命名禁用
- [ ] Redis 有高可用（哨兵/Cluster），有熔断降级兜底
- [ ] Redisson 锁：不指定 leaseTime（用 watchdog），关键路径有 DB 兜底
- [ ] 有 slowlog 监控（`slowlog-log-slower-than`）

16.8　一句话总结

> 缓存的容量问题都能靠加机器解决，**只有"单个 key 的吞吐"不能**。 热 key 是唯一一个"钱解决不了"的缓存问题。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/05-Redis与缓存坑、06-缓存/缓存穿透击穿雪崩、06-缓存/Redis集群与高可用

第十七章　分库分表之后那些"查不到"

本章要回答：为什么分表之后，订单列表少了几条？为什么后台统计数字对不上？

本章你能带走

- 一张表：分片键选错的四种代价
- 一个方法：基因法——让两个维度都能精准路由
- 一条底线：**分片之后，全局唯一 ID 必须换掉自增**

难度构成：本章主线【必读】；含【进阶】深挖 2 处；章末「延伸」小节放支线深挖与知识库指针。---

17.1　现场：列表少了几条，后台数字对不上

一张订单表 2 亿行，按 `user_id` 哈希分成 16 片。上线一周后：

问题一：后台运营的"全站订单列表"少数据。

因为它按 `create_time` 排序分页，没有 `user_id` → **无法路由 → 只能广播到 16 片，各取 1 页，再在内存里归并**。如果每片取 10 条，归并后能不能得到真正的"前 10 条"？**不能**——第 1 片的第 11 条可能比第 2 片的第 1 条还新。

问题二：昨天的 GMV 统计比财务少了 3%。

因为 `SUM(amount)` 被拆成"16 片各自求和再相加"，看似没问题。但**分页时"每片取 N 条再归并"是有损的**——它只能保证"近似前 N"，永远拿不到准确的全量排序。

问题三：客服按订单号查，查不到。

因为分片键是 `user_id`，按 `order_no` 查没有路由依据 → 广播 16 片（一次查询变 16 次），而且如果中间件没开广播，就是直接查不到。

> **分库分表不是"表变多了"，是"从今往后每个查询都要回答一个问题：我该去哪个片？"** 答不上来的查询，都会变成广播或者错误。

当时怎么误判的：运营报"列表少数据"时，第一反应是"主从延迟，从库还没同步到"（查 `Seconds_Behind_Master` 正常，排除）；又怀疑"中间件路由算错了，把数据分到别的片"（对了几条订单号，路由是对的）。最耽误时间的是"会不会 binlog 同步链路抖了一下丢了记录"——查 Canal 位点、跑对账脚本大半天，什么都没丢。最后有人把"各片 `LIMIT 10` 归并的结果"和"直接 `ORDER BY` 全量"摆在一起，才发现从第 2 页起就漏，根因是跨片分页的有损归并，不是丢数据。这一下把方向从"数据一致性"扭回"查询路由维度"，后面的优化才对路。

17.2　分片键选错，代价是什么

分片键的选择标准，四条，缺一不可：

标准	说明	反例
高基数	值足够多，能均匀打散	用 `status`（只有 5 个值）→ 严重倾斜
分布均匀	哈希后各片数据量接近	用 `tenant_id`（大客户占 80%）→ 热点片
覆盖绝大多数查询	90% 的查询都带这个条件	订单表用 `order_no` → 按用户查全广播
不可变	值不能改	用 `phone` → 换号就要迁数据

选错的四种代价：

1. **跨片查询**：不带分片键的查询 → 广播 N 次 → RT × N，DB 压力 × N
2. **跨片分页**：内存归并，**结果不准确**（上面的问题一）
3. **跨片聚合**：`COUNT` / `SUM` 要合并，复杂查询做不了
4. **数据倾斜**：某一片特别大，扩容也救不了（因为路由规则没变）

订单表的标准答案是 `user_id`，因为"我的订单"是最高频查询。但代价是"按订单号查"和"后台按时间查"都要额外处理——这就是 17.4 节要讲的。

【必读】分片键选错的代价：把广播放大倍数算出来

17.2 说跨片查询要广播 N 次。设分片数 `N = 16`、单片点查 `t = 5ms`。

放大倍数：不带分片键的查询被改写成 `N` 条 SQL，于是

```
-- ✗ 无分片键：广播 16 片，一次请求借走 16 个连接
SELECT * FROM t_order WHERE status = 1 ORDER BY create_time DESC LIMIT 20,10;

-- ✓ 带分片键：精准路由到单片，只占 1 个连接
SELECT * FROM t_order WHERE user_id = 10086 AND status = 1
 ORDER BY create_time DESC LIMIT 20,10;
```

```
单次 DB 工作量    = N × 1 次 = 16 次
单次 RT（并行） ≈ t + 归并开销 ≈ 5ms + 2ms = 7ms
单次 RT（串行） = N × t = 80ms          （中间件若串行下发更糟）
连接占用          = 16 个连接同时被这一个请求占住
```

Hikari 默认 `maximumPoolSize = 10`（第 4 章），一个不带分片键的请求瞬间借走 16 个连接、超过整个池子；`10 / 16 ≈ 0.6` 个并发就把池子借空，其余请求卡在 `getConnection()`（默认等 30 秒）。**它慢的不是自己，是把所有人的连接都借光了。**

再算吞吐：峰值 `Q = 2000` QPS，其中"忘了带分片键"占比 `r = 5%`，即 100 次/秒。这 100 次各放大 16 倍，等于额外 `1600` 次/秒 的 SQL：

```
DB 总 QPS = Q × (1 - r) + Q × r × N
          = 2000 × 0.95 + 2000 × 0.05 × 16
          = 1900 + 1600 = 3500 次/秒
```

5% 的坏查询，让 DB 总负载涨了 84%；占比 20% 时是原来的 4 倍。迷惑之处在于"QPS 没涨多少、DB CPU 却翻倍"——放大藏在每个请求内部。

坏味道	放大倍数	最先打满的资源	判据
无分片键点查	N（16）	连接池	`Threads_connected` 逼近上限，SQL 都很短
无分片键 + 深分页	N × (offset+limit)	内存 / 网络	归并节点内存随页码涨（见 17.5）
跨片 `JOIN` 未配绑定表	最坏 N²	CPU / 临时表	笛卡尔积，结果行数远大于输入
一次请求里 N 条广播串行下发	N × RT	Tomcat 线程	单接口 RT 正比于 N

> 反常识点：多数人以为"跨片查询只是慢一点"，实际它先打满的是**连接池**而不是 CPU——N 个短 SQL 同时占住 N 个连接，池子比 CPU 小一个数量级。

所以影子库压测时应捕获所有 SQL，凡 `WHERE` 里没有分片键的一律拒绝上线；放大倍数乘在每个请求上，靠"以后注意"防不住。

17.3　基因法：让两个维度都能精准路由

需求：既要按 `user_id` 查，又要按 `order_no` 查，都希望精准路由（不广播）。

思路：**把 `user_id` 的分片信息"藏"进 `order_no` 里。**

```
分片数 N = 16（2^4，取低 4 位）
user_id 的分片基因 = user_id % 16

生成 order_no：
  高位：时间戳 + 序列号          （保证趋势递增、全局唯一）
  低 4 位：user_id % 16          （基因）
```

这样：

```
// 按 user_id 查：直接算
int shard = userId % 16;

// 按 order_no 查：从订单号里取出基因
int shard = (int) (orderNo & 0b1111);      // 低 4 位就是分片号
```

两条路都能精准路由，不需要广播，也不需要额外索引表。

前提和限制：

- 分片数必须是 2 的幂（否则取模不能位运算，且基因位长度不整）
- `order_no` 必须由你们的系统生成（外部传入的订单号藏不了基因）
- 基因位数固定 → **分片数固定，扩片要重新设计**（这又回到"一次预留足够分片"）

（基因法的位运算实现与一份可运行示例，见章末附录。）

17.4　全局唯一 ID：自增必须换掉

分表之后，每张表的 `AUTO_INCREMENT` 各涨各的，必然重复。三种替代：

方案	实现	优点	缺点
雪花算法	时间戳 + 机器 ID + 序列号	趋势递增、本地生成、无依赖	依赖时钟（回拨会重复）；19 位 → 前端丢精度（第三章）
号段模式	DB 批量发号（如一次 1000 个）	性能好、不依赖时钟	需一个号段服务；重启会浪费号段
Redis INCR	原子自增	简单	依赖 Redis；持久化配置不当会重复

雪花算法的两个坑必须处理：

```
// ① 时钟回拨：直接抛错，不要"等待"或"补偿"
if (now < lastTimestamp) {
    throw new IllegalStateException("clock moved backwards, refuse to generate
id");
}

// ② 机器 ID 必须唯一：K8s 里用 Pod 序号，不要用 IP 哈希（重启会变）
```

19 位 id 传到前端会丢精度——这是第三章的内容，分表之后会集中爆发，因为自增 id 从 10 位变成了 19 位。上线前必须配 `Long → String`。

【必读】雪花 ID 的时钟回拨：分类、阈值与处理

17.4 给了"回拨直接抛错"的代码，但没说为什么不能"等一下"，也没说什么样的回拨可以容忍。先分类：

类型	幅度	常见原因	能不能等
小幅回拨	几 ms ~ 几十 ms	NTP 渐进校准、虚拟机时钟漂移	可短等，但要上限
大幅回拨	秒级 ~ 分钟级	手工改时间、NTP 跳变、重启后 RTC 落后	**不能等**
持续回拨	反复发生	宿主机时钟源不稳定	换机器 / 下线该节点

雪花结构是 `时间戳(41bit) | 机器ID(10bit) | 序列号(12bit)`，同毫秒靠序列号去重。**"等"的风险**：等待期间机器时间仍落后于 `lastTimestamp`，此时发号会和回拨前已发的 ID 撞在同一个"毫秒+机器+序列"上，直接重复主键；而且它不报错，只有对账能发现。

分档处理：

```
private long lastTimestamp = -1L;
private static final long MAX_WAIT_MS = 100;   // 小幅回拨最多等 100ms，按业务压测
定

public synchronized long nextId() {
    long now = System.currentTimeMillis();
    if (now < lastTimestamp) {
        long drift = lastTimestamp - now;
        if (drift <= MAX_WAIT_MS) {
            waitUntil(lastTimestamp + 1);       // 小幅：等到超过上次时间戳
            now = System.currentTimeMillis();
            if (now < lastTimestamp) {          // 等完还落后 → 升级为失败
                throw new IllegalStateException("clock drift beyond wait
budget");
            }
        } else {
            // 大幅回拨：直接拒绝，绝不"等待"或"用上次时间戳硬凑"
            throw new IllegalStateException("clock moved backwards, refuse to
generate id");
        }
    }
    lastTimestamp = now;                        // 同毫秒 sequence++，跨毫秒归零
    return pack(now);
}
```

三条配套措施比这段 if 更重要：

1. **机器 ID 必须唯一且稳定。** K8s 里用 StatefulSet 的 Pod 序号或启动参数注入，不要用 IP 哈希——Pod 重建 IP 会变，和新 Pod 的机器 ID 撞上是重复的另一条路（17.4 已点过）。
2. **回拨要告警**（哪台机器、回拨多少 ms），连续回拨就摘节点。
3. 真要绝对不中断，换与时钟无关的号段模式（17.4 表格那一行）。

> 反常识点：多数人以为"时钟回拨等一等就过去了"最稳妥，实际等待期间发号仍在继续，重复 ID 的概率反被拉高；**对全局唯一 ID 来说，宁可短暂不可用，也不能冒重复主键的险**——后者没有报错，只有对账能发现。

版本区间：`41/10/12` 是标准 snowflake 的经典划分，各框架（ShardingSphere 5.x、UidGenerator、Leaf）位数与回拨策略不同；`MAX_WAIT_MS` 与是否允许等待务必在自己的实现上验证。

17.5　跨片查询的三类难题

【必读】跨片 JOIN

方案一：冗余字段（最常用）

订单表里冗余 `user_name`、`item_title`，避免 JOIN。代价是"用户名改了要同步"，但同步是异步可接受的。

方案二：广播表

小表（字典、配置、城市）在每个片都存一份完整副本。

方案三：异构到 ES

把需要复杂查询的字段同步到 ES，查询走 ES、详情回源 DB。**这是"后台多条件筛选"的标准解法。**

【必读】跨片分页（最容易被忽略的坑）

```
-- 需求：全站订单按时间倒序，第 3 页
-- 中间件做法：每片取 30 条 → 内存归并 → 取第 21~30 条
```

这个结果其实是对的，但代价被低估了。 每片取 `offset + limit = 30` 条（不是 `LIMIT 20,10`），全局第 21~30 条中任意一条在它自己片内的排名都不会超过 30，因此一定落在候选集里，归并后能精确取到。第 1 片的第 31 条确实可能比第 3 片的第 1 条还新，但它排不进全局前 30，丢掉不影响第 3 页；17.1 现场里"列表少了几条"说的是**每片固定只取 10 条**的中间件，那是另一种实现。

正确做法仍然只有两条，但理由要改成"成本"而不是"不可能"：

1. **带分片键**：`WHERE user_id = ?` → 单片查询，完全正确且便宜
2. **异构**：把全量数据同步到 ES / 数仓，分页走那边

"无分片键的精确深分页"不是做不到，是代价随页码快速增长，涨到一定程度必须禁掉。 这是成本问题，不是信息论上的不可能；把它说成"不可能"会让人放弃优化，直接翻车在深页上。本节末尾的小节把这笔账算出来。

【必读】跨片聚合

`COUNT` / `SUM` / `AVG` 可以各片算再合并，但要注意：

- `COUNT` / `SUM`：可以加
- `AVG`：**不能直接平均**，要 `SUM(sum) / SUM(count)`
- `DISTINCT COUNT`：几乎无解，必须异构到 ES/数仓
- `GROUP BY` + `ORDER BY` + `LIMIT`：等价于跨片分页，同样不准确

【必读】跨片分页的二次查询：正确，但代价随页码平方增长

先把 17.5 里那句"这个结果是错的"纠正过来：**"每片取 `offset + limit` 条再归并"是能得到精确结果的**，它只是贵。证明只要一句——

> 全局按 `create_time` 倒序排名第 `k` 的记录，在它所属分片内的排名一定 `≤ k`。因为排它前面的 `k-1` 条最多全部来自这一个片，所以该片内至多有 `k-1` 条比它新。

于是取每片前 `offset + limit` 条，全局前 30 条一定全在候选集里，归并后取第 21~30 条**精确无误**。17.1 现场"列表少了几条"说的是**每片固定只取 10 条**（`LIMIT 0,10`）那种实现，它对第 1 页成立、对第 2 页起就漏，与"每片取 30 条"不是一回事。

真正的问题是代价。设分片数 `N = 16`、每页 `L = 10`、页码 `p`（`offset = (p-1)L`）：

```
每片取回行数    = offset + L = p·L
N 片总回传行数 = N · p · L
归并端候选集    = N · p · L 行，要做一次全局排序
```

代数字：

页码 p	每片取回	16 片总回传	相对第 1 页
1	10	160	1×
10	100	1,600	10×
100	1,000	16,000	100×
1000	10,000	160,000	1000×

总数据量随页码 `p` 线性增长，已经够呛；但用户是"一页一页翻过去"的，从第 1 页翻到第 `p` 页的**累计传输行数**是

```
Σ(i=1..p) N·i·L = N·L·p(p+1)/2 ≈ N·L·p²/2
```

这就是"平方增长"：单次成本线性，累计成本平方。代 `N=16, L=10, p=1000` ≈ 8000 万行被搬过内存。**深分页必须在网关禁掉**（`page > 100`），或改游标分页 `WHERE create_time < 上一页最后一条`，回传量恒为 `N·L`，与页码无关。

④ 跨片聚合的近似方案：

聚合	能否精确	做法	误差/代价
`COUNT` / `SUM`	✓ 精确	各片求和再相加	无
`AVG`	✓ 精确	`SUM(sum)/SUM(count)`，**不是各片 AVG 再平均**	无
`COUNT DISTINCT`	✗ 精确几乎无解	各片 `PFADD` 到 HyperLogLog 再 `PFMERGE`	标准误差约 0.81%（需在自己的版本上验证）
`P99` / 中位数	✗	t-digest / 各片直方图合并	尾部误差；精确需全量拉到数仓
`GROUP BY ... ORDER BY ... LIMIT`	✗	等价于跨片分页	同上，改走 ES / 数仓

17.1 那个"GMV 少 3%"的锅不在 `SUM`（可加、精确），而在按有损取样结果求和；ES / 数仓异构承载的正是 `DISTINCT`、分位数、深分页。

17.6　扩容：唯一安全路径

分片数定下来之后，"加两片"是最危险的操作，因为 `user_id % 16` 变成 `% 18` 会导致**几乎所有数据都要搬迁**。

唯一安全的路径，五步，全程不停机：

```
① 双写：新老规则同时写（老库 + 新库）
② 迁移：历史数据按新规则搬到新分片
③ 校验：逐条比对（行数 + 校验和 + 抽样内容）
④ 灰度切读：按 user_id 灰度，1% → 10% → 50% → 100%
⑤ 下线老库：观察一周无异常再停双写
```

关键在第 ③ 步——**没有校验的迁移等于赌博**。校验脚本要能回答"哪条数据不一致"，而不只是"有不一致"。

更省事的做法：一开始就预留足够分片。 分 64 片但只部署 4 个库（每库 16 片），扩容时只是把片从一个库挪到另一个库——**不用改路由规则，只挪数据**。这是业内最常用的策略。

（扩容五阶段的验收项与回切点表，见章末附录。）

17.7　什么时候不该分表

最后这一段是提醒。分库分表的成本极高，先问三个问题：

1. **索引优化过了吗？** 80% 的"单表太大"其实是缺索引或 SQL 写得差
2. **读写分离做了吗？** 读多写少的场景，一主多从能撑很久
3. **真的超过阈值了吗？** 单表 500 万行 / 2GB 是**参考值**，不是红线。字段窄、索引少，2000 万行也能跑得很好

分表是最后手段，不是性能优化的第一步。 见过太多"为了性能分表"，结果引入跨片查询、分布式事务、数据一致性，整体可用性反而下降。

（什么时候不该分表的量化判据、取证命令与阈值清单，见章末附录。）

17.8　自检清单

- [ ] 分片键满足四条标准（高基数、均匀、覆盖主查询、不可变）
- [ ] 分片数是 2 的幂，且预留了扩容空间
- [ ] 全局 ID 用雪花/号段，不用自增；前端配置了 `Long → String`
- [ ] 需要按第二维度查的，用了基因法或索引表，不是靠广播
- [ ] 无分片键的分页/聚合走 ES 或数仓，不在 DB 里硬算
- [ ] `AVG` 用 `SUM/COUNT` 计算，不是各片平均再平均
- [ ] 扩容走"双写 → 迁移 → 校验 → 灰度 → 下线"五步
- [ ] 确实评估过"不分表"的可行性（索引、读写分离）

17.9　一句话总结

> 分库分表把"一次查询"变成了"一次路由决策 + N 次查询 + 一次归并"。 你换来的写入能力，要用**查询灵活性**去支付——这笔账要在动手前算清楚。

【选读】延伸

- 知识库对应原理篇：05-数据库/01-MySQL/分库分表、05-数据库/01-MySQL/主从复制与高可用

【选读】附录

基因法的位运算实现：一份可运行示例

17.3 给了思路，这里给一份能直接跑的实现，把"低位清零 → 写入基因 → 两条路都精准路由"三步钉死。

```
public final class GeneRouter {
    private static final int  SHARD_BITS = 4;                          // 16 = 2^4
片
    private static final long SHARD_MASK = (1L << SHARD_BITS) - 1;   // 0b1111

    /** 雪花 ID 低 4 位清零后，嵌入 user_id 的分片基因 */
    public static long nextOrderNo(long userId, long snowflakeId) {
        long base = snowflakeId & ~SHARD_MASK;      // ① 低位清零，不破坏高位趋势
递增
        return base | (userId & SHARD_MASK);        // ② 写入基因
    }

    /** 两条查询路径落在同一片，靠的是同一个掩码 */
    public static int shardByUser(long userId)     { return (int) (userId &
SHARD_MASK); }
    public static int shardByOrderNo(long orderNo) { return (int) (orderNo &
SHARD_MASK); }

    public static void main(String[] args) {
        long userId = 10086L;
        long snow   = 7_321_123_456_789_012_345L;   // 模拟一个雪花 ID
        long no     = nextOrderNo(userId, snow);
        System.out.printf("userId=%d  orderNo=%d%n", userId, no);
        System.out.printf("shard(user)=%d  shard(orderNo)=%d%n",
                shardByUser(userId), shardByOrderNo(no));
        // shard(user)=6  shard(orderNo)=6  → 同一片
        assert shardByUser(userId) == shardByOrderNo(no);
    }
}
```

跑 `javac GeneRouter.java && java -ea GeneRouter`，断言通过即证明两维路由一致；`& ~MASK` 位运算与 `orderNo % 16` 等价（x 非负时），省一次整数除法。

三个前提：

前提	为什么	违反后的代价
分片数是 2 的幂	基因位数 = `log2(N)` 必须是整数	取模与位运算不等价，路由错片
订单号由本系统生成	外部传入的号藏不了基因	按订单号查仍得广播
基因位数固定	16 片 4 bit、32 片 5 bit	扩到 32 片时旧数据低 4 位全是旧基因，**全量重算**

最后一条把"分片数"焊死进了 ID：17.6 的"一次预留足够分片"对基因法是硬约束——要么按 64 片（6 bit）预留，要么接受扩片时重写所有历史 ID。

基因法借走的是雪花 ID 的序列号低位（第 17.4 章同源）：12 bit 扣掉 4 bit 只剩 8 bit = 256 个/毫秒，高并发前先算够不够。

扩容五阶段的验收表与回切点

17.6 给了五步流程，但线上真正决定"敢不敢切"的是每一步的**验收项**和**回切点**。把流程摊成阶段表，双写期间每一步都可回退：

阶段	动作	读流量归属	验收项（不达标不进下一阶段）	回切点
① 双写	老库 + 新库同时写，影子写失败进补偿队列	100% 老库	影子写成功率 > 99.99%；补偿队列无堆积	停影子写即可，零影响
② 历史迁移	按新规则搬存量，带 `version/update_time` 防覆盖	100% 老库	校验脚本：行数一致 + 关键字段哈希一致 + 抽样内容一致	只影响新库，老库不动
③ 数据校验	逐条比对，能回答"**哪条**不一致"而非"有不一致"	100% 老库	不一致清单清零；增量追平（双写期间差额补完）	同上
④ 灰度切读	按 `user_id` 灰度 1% → 10% → 50% → 100%	逐步切到新库	每档观察错误率、RT、对账差异；异常一键回切	**任一档异常即回切到老库**
⑤ 下线老库	观察一周无异常后停双写、摘配置	100% 新库	老库只读保留一段时间再下线	保留老库期间仍可回切

三处最容易出事的地方：

- **② 的迁移脚本必须带版本条件**（`IF(VALUES(version) > version, ...)`），否则会冲掉双写期间的新写入。
- **① 的影子写不能阻断主链路**：新库写失败只记日志 + 进补偿队列，绝不能让下单失败，否则"扩容"直接变成"全站下单不可用"。
- **③ 的校验要覆盖增量**（`binlog` 订阅追平，第 15 章 CDC 那套）。

直接把 `% 16` 改成 `% 18` 会让路由整体重排，等于全量搬迁 + 全量缓存失效；"预留 64 片、只挪片不改规则"则零重算、零路由变更。

双写期是最终一致（第 18 章），校验脚本即一次对账（第 15 章），灰度切读与缓存预热（第 16 章）一样是把一次性切换拆成可回退的小步。

什么时候不该分表：可量化的阈值判断

17.7 说"500 万行 / 2GB 是参考值"，这里给一组能直接量、能据此拍板的判据。**分表要解决的是"索引装不进内存、写扛不住"，不是"行数看着多"。**

① B+ 树层数，不是行数。 InnoDB 默认 16KB 页，聚簇索引非叶节点一项约 `8B 主键 + 6B 指针 = 14B`，单页能放约 `16KB / 14B ≈ 1170` 个索引项：

```
2 层：约 1170 × (每页行数) —— 千万级以内
3 层：约 1170² × ...        —— 百亿级
```

也就是说，几千万行仍是 3 层 B+ 树、点查 3 次页 IO，行数不是瓶颈；门槛是 **Buffer Pool 装不装得下热点索引 + 数据页**：

```
命中率 = 1 - 物理读 / 逻辑读
目标：Buffer Pool 命中率 ≥ 99%（OLTP 经验值，需在自己的负载上压测）
若命中率 < 95% 且物理读随数据量陡增 → 加内存 / 优化索引，而不是分表
```

② 写容量：用 QPS × 耗时算，而不是拍脑袋。

```
单实例写吞吐上限 ≈ (CPU 核数 × 每核并行事务数) / 平均事务耗时
例：写 QPS 3000、平均事务 5ms → 需要并发度 ≈ 3000 × 0.005 = 15 个活跃事务
若活跃事务数长期 > 连接数 / 2，且 CPU > 70%，才是"写扛不住"的信号
```

和第 7 章 Little's law 同一套（公式 $L=\lambda \times W$ 与符号见第 4 章）：先看到达率 × 耗时有没有超过容量；读写分离能把这个数再乘上从库数量。

④ 一张"先别分表"的量化清单（按顺序打勾，全过了再谈分片）：

检查项	量化阈值	不达标先做什么
慢 SQL / 缺索引	`EXPLAIN` 出现 `type=ALL` 或 `rows > 1万`	加索引、改 SQL（80% 的"单表太大"止于此）
Buffer Pool 命中率	< 99%	加内存、冷热分离、缩 `SELECT *`
读压力	读写比 > 5:1	一主多从 + 读写分离
单表容量	> 2GB 且归档后仍降不下来	冷热归档、大字段拆扩展表
写 QPS × RT	逼近实例容量且 CPU > 70%	先垂直/水平拆库，最后才分片
连接池	`Threads_connected` 长期逼近 `max_connections`	连接池与事务边界（第 4、7 章）

取证命令序列，按顺序敲：

```
mysql> SHOW TABLE STATUS WHERE Name='t_order'\G     # Data_length / Rows
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';
#   命中率 = 1 - Innodb_buffer_pool_reads / Innodb_buffer_pool_read_requests
mysql> EXPLAIN SELECT ... WHERE create_time > ...;  # 看 type / rows / Extra
mysql> SHOW ENGINE INNODB STATUS\G                  # 看 BUFFER POOL AND MEMORY 段
```

> 反常识点：多数人以为"单表过 500 万行就该分"，实际决定点查快慢的是 **B+ 树层数（几千万行仍是 3 层）和 Buffer Pool 命中率**；一张 2000 万行、索引克制、热点全在内存里的表，往往比一张刚分完片却要到处广播的表快得多。**分表是把查询灵活性换成写入能力，这笔账在动手前必须用上面的数字算，不能靠行数拍板。**

第十八章　消息丢失的三个阶段

本章要回答：消息发出去了，消费者没收到，是谁弄丢的？

本章你能带走

- 一张三段防线表（生产者 / Broker / 消费者），每段都有对应的配置
- 一个结论：**MQ 不保证 Exactly-Once，官方只承诺 At-Least-Once**
- 一条底线：**三阶段全配齐仍有理论丢失窗口，所以必须有对账**

难度构成：本章主线【必读】；含【进阶】深挖 4 处；章末「延伸」小节放支线深挖与知识库指针。---

18.1　现场：少了 47 条支付成功通知

财务对账发现：支付流水里有 47 笔成功，但发货单少了 47 条。

链路是：支付服务 → MQ → 订单服务（创建发货单）。

三个环节单独看都正常：

- 支付服务日志：47 笔全部 `send success`
- MQ Broker：无异常，无堆积
- 订单服务：日志里没有这 47 个订单号

消息在中间消失了，没有任何一端报错。

这是 MQ 类故障最典型的特征：**它不报错，它只是不出现。**

当时怎么误判的：第一反应是两边甩锅——要么「有人手滑 `reset-offsets` 或误删了 topic」（查变更系统，没有），要么「网络分区，消息卡在半路」（三端延迟都正常，不像）。两条都被排除后，才有人想起拉财务对账的差集，一拉才发现差的是「支付成功 → 发货单」这段，而不是「消息中间件整段」。这一段空转了将近半天，也印证了一句话：这种故障的第一现场永远是**对账差集**，不是中间件日志。

18.2　三段防线

消息从生产到消费，经过三个可能丢的地方。**任何一段没配好，都会丢，而且丢得无声无息。**

【必读】acks 的三种取值，以及两个必须一起配的开关

把 `acks` 单独拎出来讲，是因为它可靠的前提常被漏掉：`acks=all` **可靠的前提，是 ISR 里至少还有两个副本。** 少配两个开关，它会在你最需要它的时候静默降级。

配置	等什么才返回	leader 立刻宕机	与 `min.insync.replicas` 的关系	判据（怎么验它真生效）
`acks=0`	不等，发出去就返回	必丢，且生产端无从知晓	无关	吞吐最高，只配给日志、埋点
`acks=1`	只等 leader 写入本地	复制完成前宕机就丢	无关	看 `describe` 里 Isr 是否只剩一个
`acks=all`	等**当前 ISR 全部**确认	ISR 健康才不丢	ISR 收缩到 1 时退化为 `acks=1`	配 `min.insync.replicas=2`，ISR 不足时写请求直接失败

三个开关的取舍，说到底是一句话：`acks=all` **定义「等谁」，**`min.insync.replicas` **定义「至少剩几个才算数」，**`unclean.leader.election.enable` **定义「没人了能不能让落后的副本顶上」。** 少任何一个，不丢都是空话。

```
# ✗ 只写 acks=all，另外两个交给默认值
acks=all
# ISR 只剩 leader 时 all 悄悄变成 1；落后副本若当选新主，旧位点之后的数据整体回退

# ✓ 三件套写死，别赌默认值
acks=all
min.insync.replicas=2
replication.factor=3
unclean.leader.election.enable=false
enable.idempotence=true
```

`acks=all` 的语义是「当前 ISR 全部确认」，**不是「全部副本」**。副本落后被踢出 ISR 后只剩 leader，`all` 就等价于 `1`。`min.insync.replicas=2` 的作用是把这次静默降级变成显式失败：ISR 小于 2 时 Broker 抛 `NotEnoughReplicasException`，写不进去，宁可让上游报错，也不让消息「看起来发成功了」。`unclean.leader.election.enable=false` 则堵住另一条路——若允许落后副本当选，新主的高水位比旧主低，消费者位点整体回退，表现是「一批消息丢了，同时另一批消息重复了」，这是最难查的一种二合一事故。broker 端这个配置的默认值随版本漂移过，别靠默认，显式写死并在自己的版本上验证。

检查 `SendStatus` 有个前提——**`FLUSH_DISK_TIMEOUT` / `FLUSH_SLAVE_TIMEOUT` / `SLAVE_NOT_AVAILABLE` 这三种状态，只有配了 `SYNC_FLUSH` / `SYNC_MASTER` 才可能出现**。在默认的 `ASYNC_FLUSH` 下，`syncSend` 永远返回 `SEND_OK`，那个 `if` 分支一次都不会进。于是最危险的组合出现了：你以为验过了，其实什么都没验。检查 `SendStatus` 之前，先确认刷盘模式，否则这行代码只是心理安慰。

【必读】① 生产者：确认回调没人看

```
// ✗ 最危险的写法：异步发送，不看结果
rocketMQTemplate.asyncSend("order-topic", msg, new SendCallback() {
    @Override public void onSuccess(SendResult r) { }
    @Override public void onException(Throwable e) {
        log.error("send failed", e);       // 只打日志，没有重试、没有落库
    }
});
```

`send` 返回成功只代表"消息到了 Broker 的内存"，不代表持久化成功。要看 `SendStatus`：

```
SendResult result = rocketMQTemplate.syncSend("order-topic", msg);
if (result.getSendStatus() != SendStatus.SEND_OK) {
    // SEND_OK 之外还有 FLUSH_DISK_TIMEOUT / FLUSH_SLAVE_TIMEOUT /
SLAVE_NOT_AVAILABLE
    // 这些状态消息可能已经写了，但没落盘 / 没同步到从节点 → 宕机就丢
    throw new BizException("消息发送失败: " + result.getSendStatus());
}
```

Kafka 侧对应的是 `acks`：

```
acks=all                          # 所有 ISR 副本都确认才返回
retries=3                         # 生产端重试
enable.idempotence=true           # 幂等生产者，防重试导致的重复
```

`acks=0`（不等待）和 `acks=1`（只等 leader）都会在 leader 挂掉时丢消息。**只有 `acks=all` 才是"发出去了"的可靠信号。**

【必读】② Broker：刷盘与主从

两个维度，四种组合：

刷盘方式	主从方式	可靠性	性能
异步刷盘	异步复制	**最低**（宕机丢页缓存 + 丢未同步）	最高
异步刷盘	同步复制	中	中
同步刷盘	异步复制	中	中
同步刷盘 + 同步复制	—	**最高**	**最低**（写 RT 上一个量级）

```
# RocketMQ：金融级配置
flushDiskType = SYNC_FLUSH
brokerRole = SYNC_MASTER
```

```
# Kafka：三副本 + 至少两个同步
replication.factor=3
min.insync.replicas=2
```

代价要说清楚：**同步刷盘让写入 RT 从"亚毫秒"变成"几毫秒到十几毫秒"**，吞吐可能降一半。所以它只适合"丢了要赔钱"的消息（支付、订单），不适合日志、埋点。

【进阶】把丢失窗口算成时间：刷盘、复制、切主

「会丢」是个含糊的说法，落到容量上才有意义。**丢失窗口 = 尚未落盘（或尚未复制）的时间 × 这段时间的写入速率**，单位不是「条」，是「一段时间里的全部消息」。

组合	未持久化的数据在哪	窗口时长（量级）	5000 条/秒时的丢失条数	写 RT 代价
异步刷盘 + 异步复制	PageCache + 主从差值	刷盘间隔 500ms 到 OS 回写 30s	2,500 ~ 150,000	亚毫秒
同步刷盘 + 异步复制	主从复制滞后	一次内网往返约 10ms	50	几毫秒
异步刷盘 + 同步复制	PageCache	刷盘间隔	2,500	几毫秒
同步刷盘 + 同步复制	几乎无	一次复制往返	近似 0	十几毫秒，吞吐可能减半

RocketMQ 默认 `ASYNC_FLUSH`：消息进 PageCache 就返回，真正落盘由后台刷盘线程（间隔需在自己的版本上验证，常见为几百毫秒）或 OS 脏页回写（`vm.dirty_expire_centiseconds` 默认 3000，即 30 秒）决定。取最坏值算：单机 5000 条/秒、30 秒不落盘，一次掉电就是 15 万条。**这就是为什么「异步刷盘丢消息」不是偶发丢一条，是成片丢。**

Kafka 走的是另一条路：它不逐条 `fsync`，`log.flush.interval.messages` 默认交给 OS，可靠性靠多副本冗余。所以单机掉电的 RPO 由 `replication.factor` 决定，不由刷盘决定——**三副本分布在三个机架、同时掉电的概率，才是这一层真正的剩余窗口**。把 `log.flush.interval.messages` 调激进只会毁掉顺序写吞吐。

异步复制（`ASYNC_MASTER`）的账更好算：主写成功即返回，从节点平均滞后一个 Δt。设 $\Delta t = 10\text{ms}$、写入 5000 条/秒，主宕机切主就丢约 50 条；`SYNC_MASTER` 用一次内网往返把这 50 条买回来。**多数人以为同步刷盘加同步复制换来的是「不丢」，实际换来的只是把窗口从「一段时间」压缩成「一次复制往返」**——它变短了，没有变成零。这个区别决定了为什么 18.4 还要留对账。这也和第 7 章的推演同构：任何把等待时间拉长的配置，都是在拿 RT 换一段窗口。

【必读】③ 消费者：提交 offset 的时机

这是最容易出错、也最容易被忽视的一段。

```
// ✗ Kafka 自动提交：拉取即成功
props.put("enable.auto.commit", "true");
props.put("auto.commit.interval.ms", "1000");
```

自动提交的语义是"**我拉到了，就当我处理完了**"。如果消费者在处理第 100 条时宕机，offset 已经提交到 200——**这 100 条永久丢失**。

```
// ✓ 手动提交：业务处理成功后再提交
props.put("enable.auto.commit", "false");
while (running) {
    ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(500));
    for (ConsumerRecord<String, String> r : records) {
        handle(r);                          // 业务落库
    }
    consumer.commitSync();                  // 处理完才提交
}
```

这一步的本质是把"可能丢失"转换成"可能重复"：

> 处理完但没提交就宕机 → 下次重新消费 → **重复** 先提交再处理 → 处理失败就**丢失**

宁可重复，不可丢失。 因为重复可以由幂等消化（下一章），丢失只能靠对账发现 + 人工补。

RocketMQ 的对应写法：

```
// ✗ 吞异常返回 SUCCESS = 把消息扔进黑洞
try {
    handle(msg);
} catch (Exception e) {
    log.error("handle failed", e);
}
return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;      // 消息被确认，永远不会再来
```

```
// ✓ 失败返回 RECONSUME_LATER，进重试队列
try {
    handle(msg);
    return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
} catch (Exception e) {
    log.error("handle failed, will retry", e);
    return ConsumeConcurrentlyStatus.RECONSUME_LATER;
}
```

`RECONSUME_LATER` 与"抛异常"在并发消费模式下等价——都会进重试队列。但**返回 SUCCESS 是单向门**：消息被标记为消费成功，再也不会回来。

【进阶】offset 提交的三种时刻：一张时序图

自动提交与手动提交的差别，不在「谁先谁后」这句话，在崩溃点落在哪个区间。把时间轴摊开：

```
自动提交（enable.auto.commit=true，默认间隔 5000ms）
 t0  poll() 拉到 [100..199]，position=200
 t1  后台线程按时间提交 committed=200        ← 与业务毫无关系
 t2  handle(100) ... handle(149) 逐条落库中
 t3  进程崩溃
 t4  重启，从 committed=200 继续 → [150..199] 永久丢失，无任何报错

手动提交（enable.auto.commit=false）
 t0  poll() 拉到 [100..199]
 t1  handle 全部成功，本地事务 COMMIT
 t2  commitSync() 提交 committed=200
 t3' 若在 t1~t2 之间崩溃 → 下次重新拉 [100..199] → 重复
      （重复由第 19 章的幂等消化，不是错误）
```

提交时刻	语义	崩溃窗口	结果
拉取后立即提交	至多一次	处理期间	丢一批
处理成功后提交	**至少一次**	处理完到提交间	重复，可幂等
提交后才处理	至多一次	提交后	丢，且最隐蔽

```java
// ✗ 先 ack 再处理：ack 只表示「位移收下了」，不代表业务成功
@KafkaListener(topics = "pay-success")
public void onMsg(ConsumerRecord<String, String> r, Acknowledgment ack) {
    ack.acknowledge();
    shipmentService.create(r.value());          // 抛异常也无人重投
}

// ✓ 业务落库成功后才提交；失败不 ack，交给容器退避重试
@KafkaListener(topics = "pay-success")
public void onMsg(ConsumerRecord<String, String> r, Acknowledgment ack) {
    shipmentService.createInNewTx(r.value());   // 事务提交后才返回
    ack.acknowledge();
}
```

两个容易记混的默认值：`auto.commit.interval.ms` 默认是 **5000**，示例里的 `1000` 是显式调小，别把示例当默认；Spring Kafka 自 2.3 起强制 `enable.auto.commit=false`，容器的 `AckMode` 默认是 `BATCH`，要逐条确认得显式设 `MANUAL_IMMEDIATE`。还有一条常被漏掉的：`commitSync()` 提交的是**整个分区的位点**，不是「这一条消息」，所以批量消费里任何一条失败，正确做法是整批不提交，而不是提交前面成功的那部分。

18.3　最可靠的做法：本地消息表

如果一条消息"丢了要赔钱"，不要只依赖 MQ 的可靠性配置，用本地消息表：

```
@Transactional(rollbackFor = Exception.class)
public void paySuccess(PayCmd cmd) {
    payMapper.updateStatus(cmd.getOrderId(), PAID);              // ① 业务
    msgMapper.insert(new LocalMsg(cmd.getOrderId(), "pay-success", 0));  // ② 同
事务写消息表
}
```

```
// ③ 独立任务轮询未发送的消息，发出去后标记
@Scheduled(fixedDelay = 1000)
public void relay() {
    List<LocalMsg> pending = msgMapper.selectPending(100);
    for (LocalMsg m : pending) {
        try {
            mqProducer.syncSend(m.getTopic(), m.getPayload());
            msgMapper.markSent(m.getId());
        } catch (Exception e) {
            msgMapper.incrRetry(m.getId());      // 失败下次再试
        }
    }
}
```

这解决了两个问题：

- **业务成功、消息必达**：业务和消息在同一个事务里，要么都成功要么都失败
- **发送失败可重试**：消息在表里，不会因为进程崩溃而消失

代价：多一张表、多一个轮询任务、**消息可能重复发**（`markSent` 之前崩溃）——所以消费端仍然必须幂等。

这是"至少一次"的标准实现。 所有"钱"相关的异步链路都应该这么写。

【进阶】本地消息表：能跑的建表与补偿任务

要让 `paySuccess` 和 `relay` 真正上线，还差两样东西：**一张能支撑重试与多实例的表结构**，和**一个不会把消息推进黑洞的失败分支**。

```
CREATE TABLE local_message (
  id            BIGINT PRIMARY KEY AUTO_INCREMENT,
  biz_key       VARCHAR(128) NOT NULL COMMENT '业务幂等键，消费端据此去重',
  topic         VARCHAR(64)  NOT NULL,
  payload       TEXT         NOT NULL,
  status        TINYINT      NOT NULL DEFAULT 0 COMMENT '0=PENDING 1=SENT
2=DEAD',
  retry_count   INT          NOT NULL DEFAULT 0,
  next_retry_at DATETIME     NOT NULL,
  created_at    DATETIME     NOT NULL,
  UNIQUE KEY uk_biz_key (biz_key),
  KEY idx_pending (status, next_retry_at)
) ENGINE=InnoDB;
```

```
// MySQL 8.0 起支持 SKIP LOCKED：多实例各捞各的批次，不互相等行锁
@Select("SELECT * FROM local_message " +
        "WHERE status = 0 AND next_retry_at <= NOW() " +
        "ORDER BY id LIMIT #{size} FOR UPDATE SKIP LOCKED")
List<LocalMsg> lockPending(int size);

@Scheduled(fixedDelay = 1000)
@Transactional(rollbackFor = Exception.class)
public void relay() {
    for (LocalMsg m : msgMapper.lockPending(100)) {
        try {
            SendResult r = producer.syncSend(m.getTopic(), m.getPayload());
            if (r.getSendStatus() != SendStatus.SEND_OK) {
                throw new BizException("send not ok: " + r.getSendStatus());
            }
            msgMapper.markSent(m.getId());          // 只有 SENT 是终态
        } catch (Exception e) {
            // ✗ 千万别把 status 改成 FAILED：查询只捞 PENDING，
            //    改成 FAILED 的行永远不会被重捞，等于把可重试消息推进黑洞
            msgMapper.incrRetry(m.getId(), backoff(m.getRetryCount()));
            // ✓ 只累加次数并推后下次时间，状态仍是 0；超阈值才转 2(DEAD) 并告警
        }
    }
}

private Date backoff(int n) {
    long delayMs = Math.min(1000L << Math.min(n, 9), 300_000L);  // 1s 乘 2 的 n
次方，封顶 5min
    return new Date(System.currentTimeMillis() + delayMs);
}
```

一个常见但致命的写法：不少示例在发送失败时调 `markFailed()` 把状态改成 `FAILED`，而捞取条件是 `status = 'PENDING'`——两者一冲突，这批消息就再也不会被扫到。正确做法是**只有 `SENT` 是终态**，失败一律保持 `PENDING`，用 `retry_count` 与 `next_retry_at` 做指数退避：按 `min(1s × 2^n, 5min)` 算（n≥9 后延迟封顶 5 分钟），前 10 次重试（retry_count = 0..9）累计延迟约 811 秒（13.5 分钟）、单次最长封顶 5 分钟，够覆盖一次数据库抖动。

`@Transactional` 包住整批搬运是为了让「捞取加标记」原子，而 `paySuccess` 那边必须是**另一个**事务：业务更新和插入消息表同生共死（第 6 章：跨 Bean 调用才走代理，且务必带 `rollbackFor = Exception.class`）。**本地消息表的本质不是「更可靠的发送方式」，是把「发送」从一次性的网络调用，改成了可重放、可退避、可审计的状态机。**

【必读】事务消息：半消息、回查与 UNKNOWN

如果已经用 RocketMQ，还有一条更短的路。它和本地消息表解决的是同一个问题——「本地事务成功」与「消息发出」之间没有共同事务——但把补偿做进了 Broker。

```
Producer                    Broker                      Consumer
   |-- ① sendMessageInTransaction -->|
   |        半消息写入 RMQ_SYS_TRANS_HALF_TOPIC（对消费者不可见）
   |<--------- 半消息 OK -----------|
   |-- ② executeLocalTransaction：写订单表 + 事务日志表（同一个本地事务）
   |-- ③ COMMIT / ROLLBACK ------->|
   |                  COMMIT   → 消息转入真实 Topic，消费者可见
   |                  ROLLBACK → 删除半消息
   |   ③ 结果丢失（网络断 / 进程崩）
   |<-- ④ 定时回查 checkLocalTransaction（默认 60s 起，最多 15 次，需按版本验证）--|
   |-- ⑤ COMMIT / ROLLBACK / UNKNOWN ----------------------------------------
>|
```

回查是这套机制的灵魂：没有它，第 ③ 步的结果一旦丢失，半消息会永久悬挂——既不可消费也不删除。回查方法 `checkLocalTransaction` 只拿到 `Message`，所以本地必须能回答「这笔事务到底提交了没有」。最稳的依据是一张与业务同事务写入的 `local_transaction(biz_id, status)` 表，而不是去查业务表：业务表可能用了逻辑删除，也可能记录存在但事务尚未提交。

```
// ✗ 无条件 COMMIT：本地事务失败也会放消息出去，下游收到一笔不存在的订单
public RocketMQLocalTransactionState checkLocalTransaction(Message msg) {
    return RocketMQLocalTransactionState.COMMIT;
}
// ✗ 查逻辑删除的业务表：订单 deleted=1 时查不到 → 误判 ROLLBACK → 丢消息
boolean exists = orderMapper.existsByOrderNo(orderNo);

// ✓ 查独立事务日志；还处于 COMMITTING 就回 UNKNOWN，让 Broker 继续查
LocalTx tx = txMapper.selectByBizId(orderNo);
if (tx == null) return RocketMQLocalTransactionState.ROLLBACK;
return "COMMITTED".equals(tx.getStatus())
        ? RocketMQLocalTransactionState.COMMIT
        : RocketMQLocalTransactionState.UNKNOWN;
```

`UNKNOWN` 不是兜底，是**必需状态**：它覆盖「日志已插、业务未完」的中间态，回 COMMIT 会放出错误消息，回 ROLLBACK 会丢消息。

维度	事务消息	本地消息表
依赖	RocketMQ 特有能力	只用 DB 加任意 MQ
消息延迟	低（一次交互加回查兜底）	中（依赖扫表周期）
失败补偿	Broker 主动回查	定时任务扫表
适用	已用 RocketMQ	Kafka、RabbitMQ，或不想绑死

多数人以为「用了事务消息就不丢」，实际它只保证「本地事务加消息发送」的原子性——Broker 宕机、消费失败、重复投递，它一个都不管，仍要接上 18.2 的 Broker 配置、消费重试与第 19 章的幂等。第 7 章的 `AFTER_COMMIT` 回调也是同一道题的另一种解：它只在 `afterCommit` 与 `send` 之间留了一个进程崩溃窗口，事务消息用回查把这个窗口补上了，但两者都不替代消费端幂等。

18.4　现实：三阶段全配齐也会丢

必须接受一个事实：**没有任何配置能保证 100% 不丢。**

理论上的丢失窗口包括：

- 同步刷盘返回成功后，磁盘本身损坏
- 主从同步完成后，主从同时宕机
- 消费者处理成功、提交 offset 成功，但业务库的回滚发生了（跨系统事务）

所以最后一层防线永远是**对账**：

```
每小时：支付流水 vs 发货单（按订单号比对）
每天：  订单总额 vs 支付总额 vs 财务流水
```

对账要能回答两个问题：**"少了哪几条"** 和 **"多了哪几条"**。只有总数的对账（"差 47 条"）无法定位，价值有限。

【进阶】三段全配齐之后，还剩哪些窗口

18.4 列了三条理论窗口，这里补全它，并把每条窗口对应到「谁能发现」。前三条防线全部按金融级配齐之后，剩下的都不是配置问题：

剩余窗口	配置能否覆盖	谁能发现
同步刷盘返回后，磁盘、阵列、整机同时损坏	否，多副本同机架时一起没	对账
主从同步完成后主从同时宕机、机房级故障	否，除非跨机房	对账加灾备演练
retention 到期、Topic 被 `--delete`、位点被 `--reset-offsets`	否，这是**人**	对账加变更审计
内部主题单副本：`offsets.topic.replication.factor=1`、`transaction.state.log.replication.factor=1`	能，但极易漏配	`describe` 巡检
死信队列无人看，消息进 `%DLQ%` 后静默消失	能，靠告警	DLQ 堆积监控
业务库本地事务已提交、offset 提交前崩溃	不算丢，是重复	第 19 章幂等
offset 已提交，但业务库随后被回滚或人工改数	否，跨系统无共同事务	对账

注意最后一行：这是全章最反直觉的一点——消费者明明「处理成功、提交成功」，数据仍可能不一致，因为业务库的事务和 `__consumer_offsets` 的提交分属两个系统，没有任何机制保证它们同生共死。和第 15 章删缓存兜底是同一个命题：缓存与数据库之间、MQ 与数据库之间，都只能靠「最终一致加对账」收口，不存在一个开关把它们焊死。

一次完整的取证顺序，按「先证伪配置、再证伪位点、最后才翻日志」排：

```
# ① Kafka：ISR 有没有收缩过（acks=all 是否退化）
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-replicated-
partitions
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic pay-success
#   看 ReplicationFactor、Isr 列表：Isr 少于副本数即说明退化

# ② 位点：CURRENT-OFFSET 与 LOG-END-OFFSET 的差，按分区逐行看
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 \
  --group ship-consumer --describe

# ③ RocketMQ：按业务键查消息轨迹，再查重试与死信
sh mqadmin queryMsgByKey -n nameserver:9876 -t order-topic -k <orderId>
sh mqadmin topicStatus -n nameserver:9876 -t '%DLQ%ship-group'

# ④ 最后才比对应用日志：send success 条数 vs 下游落库条数，差集即丢失清单
```

第 ④ 步就是 18.4 说的「能定位到具体单据」的对账——**只有总数的对账（「差 47 条」）无法定位，价值有限**：能按订单号列出差集，才能人工补单。第 7 章事务内发 MQ 讲过「消息先发、事务后提交」会造出幽灵消息，第 15 章讲过删缓存失败要留兜底，第 22 章的库存对账也是同一套手法：把「少了哪几条」算出来，比争论「配置到底对不对」有用得多。

本章结论对应 **Kafka 3.x/4.x、RocketMQ 4.9+/5.x、MySQL 8.0（`FOR UPDATE SKIP LOCKED`）、Spring Kafka 3.x**。`unclean.leader.election.enable` 的 broker 默认值、`transactionCheckInterval` 与 `transactionCheckMax` 的默认值、`flushIntervalCommitLog` 的默认值都随版本漂移过，升级后重新 `--describe` 一遍再采信。**对账不是可靠性配好之后的补丁，它是唯一能覆盖「人」和「机房」这两个变量的手段。**

18.5　自检清单

- [ ] 生产端检查 `SendStatus`（或 Kafka `acks=all`），不是只看"没抛异常"
- [ ] 发送失败有重试或落库，不是只打日志
- [ ] Broker 的刷盘/复制模式与消息重要性匹配（钱的用同步）
- [ ] 消费端手动提交 / 手动 ack，**处理成功后才确认**
- [ ] 没有"catch 住异常返回 SUCCESS"的写法
- [ ] 关键链路用本地消息表，不是直接 `send`
- [ ] 消费端幂等（下一章）
- [ ] 有小时级/天级对账，且能定位具体单据
- [ ] 死信队列有监控 + 人工处理流程

18.6　一句话总结

> MQ 的可靠性不是"开关"，是**三段链路上每一段都要做对**。 而三段都做对之后，你得到的仍然只是"概率很高"——所以对账不是可选项。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/06-MQ消息坑、07-消息队列/可靠性投递、07-消息队列/Kafka
- 【选读】RabbitMQ 用户注意：「镜像队列」替代不了 ISR 那套「写入即多数派确认」的语义——它的镜像同步是异步的，主节点宕机仍可能丢未同步消息；它的死信交换机（DLX）等价于本书的死信队列，套路一致。

第十九章　重复消费与幂等三板斧

本章要回答：MQ 官方明说"不保证不重复"，那业务该怎么活？

本章你能带走

- 三个必然产生重复的场景（不是"偶发故障"）
- 幂等三板斧的层次：业务键去重 → 状态机校验 → 唯一约束兜底
- 一条判断：**Redis 去重只能加速，不能作为唯一防线**

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

19.1　现场：一张订单发了两次货

大促第二天，仓库发现同一个订单出了两个包裹。

链路：订单服务 → MQ → 仓储服务（创建发货单）。

查下来，这一条消息被消费了两次，间隔 8 秒。两次消费都创建了发货单，因为**仓储服务没有任何去重逻辑**。

为什么会消费两次？三个可能，全都"正常"：

1. 订单服务发送超时重试了（发送成功但响应丢了）
2. Broker 主从切换，未提交的 offset 回退
3. 消费者 rebalance，已拉取未提交的消息被重新分配

这三个都不是 bug，是协议行为。 RocketMQ 文档原话是"不保证消息不重复"，Kafka 的语义是 at-least-once。

当时怎么误判的：仓库报"出了两个包裹"时，第一反应是"消费者没提交 offset"——拉了 consumer group 的 `--describe`，offset 明明在推进；又怀疑"MQ 把消息丢了"，查 Broker 日志也干净。两边都没问题，差点定性成"偶发抖动、明天再看"。直到确认是两个包裹、不是少发货，才倒推回"重复消费"。这个误判多花了两小时，也印证了一句话：重复消费在日志里**完全没有痕迹**，它的第一个信号永远是业务侧的"多了一条"。

> **MQ 不是"消息管道"，是"至少一次投递的管道"。** 幂等不是优化，是使用 MQ 的**入场费**。

19.2　重复的三类来源

来源	机制	能否避免
生产端重试	发送超时但实际成功 → 重发	不能（超时本身无法区分）
Broker 重投	主从切换、offset 回退、重试队列	不能
消费端 rebalance	已拉取未提交的消息重新分配	不能（协议行为）

结论：重复是设计前提，不是异常。 你的消费逻辑必须假设"同一条消息会来 1 到 N 次"。

【必读】三类重复：重复发生在哪两个动作之间

19.2 的表格列了「谁会产生重复」，这里补上「重复落在时间轴的哪一段」。三条时间轴，各自的窗口完全不同：

```
来源一　生产端重试（发送成功但响应丢了）
 P: send(msg#1) ---- 超时未收到 ack ---- retry(msg#1') ----
 B:          已落盘 #1                  又落盘 #1'
 结果：两条 msgId、同一个业务事实，间隔通常几十毫秒

来源二　Broker 主从切换 / 未提交位点回退
 C: 消费到 offset=200，committed 只到 150
 M: 主宕 → 从升主，高水位回退到 150
 C: 从 150 重新拉 → [150..200] 整段再消费一遍
 结果：一批消息重复，间隔秒级

来源三　rebalance
 C1: poll 拉到 [100..199]，处理中，未提交
     → 新实例加入，触发 rebalance，分区被重新分配
 C2: 从 committed=100 开始拉 → [100..199] 又处理一遍
 结果：一批消息重复，间隔取决于 rebalance 耗时
```

来源	重复粒度	典型间隔	能否靠配置关掉	判据
生产端重试	单条	几十毫秒	不能，超时无法区分「没发到」和「响应丢了」	同一个业务键出现两条不同 msgId
主从切换 / 位点回退	一批	秒级	不能，是可用性优先的代价	消费日志里同一 offset 段出现两次
rebalance	一批	秒到分钟	不能，协议行为	日志里出现 rebalance，前后消息重复

多数人以为重复是偶发抖动，实际它是「至少一次」语义的实现方式：MQ 宁可多投一次，也绝不漏投一次。 第 18 章把「先提交后处理」的丢失换成了「先处理后提交」的重复，这笔交易换来的就是这三段窗口。所以幂等不是补丁，是这条链路的入场费：你的消费逻辑必须假设同一业务事实会来 1 到 N 次，且 N 没有上界。

19.3　幂等三板斧

三层，从业务语义到数据库约束，**越往下越可靠**：

【必读】第一斧：业务唯一键去重

用"业务上唯一的标识"做去重，不用 `msgId`。

```
// ✓ 用业务键，不用消息 ID
String bizKey = "ship:" + orderId;          // 一个订单只会有一张发货单
if (!dedupMapper.tryInsert(bizKey)) {
    log.info("duplicate message, skip. orderId={}", orderId);
    return SUCCESS;
}
createShipment(orderId);
```

为什么不用 `msgId`？因为**同一条业务消息可能由不同的 msgId 发出来**（生产端重试会生成新的 msgId）。去重必须锚定**业务事实**，不是技术标识。

去重表设计：

```
CREATE TABLE msg_dedup (
    biz_key     VARCHAR(128) PRIMARY KEY,     -- 主键即唯一约束，天然去重
    created_at  DATETIME     NOT NULL,
    KEY idx_created (created_at)              -- 用于定期清理
);
```

`INSERT` 主键冲突 = 已处理过。**用主键冲突做去重，比"先查再插"可靠**——后者在并发下有竞态。

【必读】第二斧：状态机校验

有些业务天然有状态流转，直接用状态做幂等：

```
// ✓ 只有 INIT 状态才能变 PAID
int affected = orderMapper.updateStatus(orderId, "PAID", "INIT");
if (affected == 0) {
    log.info("order already paid, skip. orderId={}", orderId);
    return SUCCESS;          // 幂等成功，不是失败
}
```

对应的 SQL：

```
UPDATE order_info SET status = 'PAID'
WHERE id = #{id} AND status = 'INIT';
```

`affected = 0` 表示"状态不对，已经处理过了"——**这时应该返回成功**，而不是抛异常。因为对 MQ 来说，这条消息已经被正确处理了。

这一斧的好处是**不需要额外的去重表**，用业务自身的状态就够了。缺点是只适用于有明确状态机的场景。

【必读】第三斧：数据库唯一约束（最后防线）

```
ALTER TABLE shipment ADD UNIQUE KEY uk_order (order_id);
```

前两斧都失效时（比如代码漏了判断），唯一约束会抛 `DuplicateKeyException`。捕获它，当作"已处理"：

```
try {
    shipmentMapper.insert(shipment);
} catch (DuplicateKeyException e) {
    log.info("shipment exists, treat as success. orderId={}", orderId);
    return SUCCESS;
}
```

这一层是"代码写错也不会出数据事故"的兜底。 它不能省，因为前两斧依赖业务代码正确性，而人会犯错。

【必读】Redis setnx：加速层，不是防线

```
// ⚠ 只能减少重复，不能保证
if (!redis.setnx("dedup:" + bizKey, "1", 3600)) {
    return SUCCESS;
}
```

三个理由不能只靠它：

1. **Redis 可能丢数据**（主从切换、内存淘汰）→ 去重记录消失 → 重复执行
2. **TTL 到期** → 晚到的重复消息直接穿透
3. **setnx 成功但业务失败** → 消息重试时被误判为"已处理"

正确用法：Redis 在前（挡住 99% 的重复，省一次 DB 查询），DB 唯一约束在后（决定性的）。

【进阶】setnx 的过期时间怎么选：按重投窗口算

Redis setnx 只是加速层、不是防线（19.3 已点明），但它给的 `3600` 秒是怎么来的、够不够，得算清楚。**TTL 的唯一依据是消息的最大重投窗口：**

```
TTL >= 最大重投窗口 + 单次业务处理上限 + 安全余量
```

三项各自有数：RocketMQ 并发消费默认重试 16 次、延迟级别递增，累计约 4 小时 46 分（次数与间隔需按版本验证）；Kafka 没有内建重试队列，若自配 3 次退避重试、每分钟一次，窗口只有 3 分钟；再算上人工重投和 T+1 对账回放，窗口可能拉到一天以上。于是：

要挡住什么	建议 TTL	理由
只挡 MQ 自动重投	≥ 6 小时	覆盖 4h46m 的重试上限并留余量
覆盖人工重投、隔夜回放	≥ 24 小时	对账和人工补单常发生在 T+1
示例的 3600 秒	**不够**	一条 2 小时后才重投的消息，标记已过期，直接穿透

```
// ✗ 固定 1 小时，成功与失败共用一个值，且不做两阶段
Boolean ok = redis.opsForValue().setIfAbsent("dedup:" + bizKey, "1",
Duration.ofHours(1));
if (!Boolean.TRUE.equals(ok)) return;

// ✓ 两阶段：处理中给短租约，成功后延长到覆盖重投窗口
String key = "dedup:" + bizKey;
Boolean first = redis.opsForValue()
        .setIfAbsent(key, "PROCESSING", Duration.ofMinutes(5));   // 处理中的并发
保护
if (!Boolean.TRUE.equals(first)) return;                          // 正在处理或已
处理
try {
    handle(bizKey);                                               // 业务落库（含
DB 唯一约束）
    redis.opsForValue().set(key, "DONE", Duration.ofDays(3));      // 成功后延长
} catch (Exception e) {
    redis.delete(key);                                            // 失败必须清，
否则重试被自己挡死
    throw e;
}
```

还有一笔存储账要算，它解释了为什么 TTL 不能「图省事设成永不过期」：假设每天 1 亿条消息、每个 key 约 128 字节、TTL 3 天，常驻约 3 亿个 key，光 value 加元数据就是几十 GB 内存。**TTL 越长越安全是错的**——越长越吃内存，而且一旦 Redis 主从切换丢了写，长 TTL 同样救不了你。真正的防线始终是数据库唯一约束；Redis 只负责把 99% 的重复挡在 DB 之前，省下一次往返。

19.4　幂等的四种层次，选哪个

层次	实现	可靠性	成本	适用
天然幂等	`SET status = 'PAID'`（同值覆盖）	高	零	更新操作
状态机	`WHERE status = 'INIT'`	高	零	有状态流转
去重表	业务键主键冲突	高	一张表	创建类操作
唯一约束	DB `UNIQUE KEY`	最高	改表结构	**必须有**

优先级：能天然幂等就天然幂等，否则状态机，再否则去重表，唯一约束始终作为兜底。

反面案例（最常写错的）：

```
// ✗ 累加不是幂等
UPDATE account SET balance = balance + 100 WHERE id = 1;

// ✓ 改成"置位"或"带条件"
UPDATE account SET balance = balance + 100
WHERE id = 1 AND last_msg_id != #{msgId};
```

金额累加必须靠"记录已处理的流水号"来防重，不能靠 SQL 本身。**这是资金类系统的必修课**。

【必读】四层对照表：能挡什么、挡不住什么

19.4 那张表按「可靠性」排了序，这里换个更有用的排法：横着摆开四层，每格只写判据和边界。

层次	能挡住	挡不住	判据
业务唯一键去重表	同一业务事实被**创建**多次	更新类操作；去重标记与业务不在同一事务（业务成功、标记没写上）	`INSERT` 主键冲突
状态机	重复的状态流转，零额外存储	无状态字段的累加；状态回退与乱序（第 21 章）	`affected = 0`
DB 唯一约束	代码漏判之后的最后一次拦截	重复 `UPDATE`（约束只管插入）；异常被 catch 后继续执行的别的副作用	`DuplicateKeyException`
Redis setnx	短时间内绝大多数重复，省一次 DB 往返	TTL 到期、Redis 主从切换丢写、业务失败后的误判	命中率约 99%，不是 100%

「加了唯一索引就万事大吉」是错的：唯一索引只拦 `INSERT`，拦不住重复的 `UPDATE`。金额累加、库存扣减这类操作必须靠「记录已处理的流水号」或状态机，不能指望约束。反过来，**状态机的 `affected = 0` 也不能一律当幂等成功**——它可能是「状态已流转」（应返回成功），也可能是「记录根本不存在」（脏数据、消息乱序、库里被误删），后者如果静默吞掉，等于丢了一条本该处理的消息。

```
// ✗ 把 affected=0 一律当「已处理」，订单不存在时静默吞消息
int n = orderMapper.updateStatus(orderId, "PAID", "INIT");
if (n == 0) return SUCCESS;

// ✓ 二次判定：区分「已流转」与「不存在」
if (n == 0) {
    if (orderMapper.countByOrderId(orderId) == 0) {
        throw new BizException("订单不存在，转死信人工排查");   // 不吞
    }
    log.info("already handled, orderId={}", orderId);          // 幂等命中
    return SUCCESS;
}
```

同样地，`catch (DuplicateKeyException e)` 只有紧跟在 `INSERT` 后面才有意义；在 `UPDATE` 路径里捕获它，是永远不会触发的死代码，读代码的人却会误以为那里有兜底。**四层不是四选一，是一条从上到下的漏斗：Redis 挡量、状态机挡流转、去重表挡创建、唯一约束挡人写错代码。**

去重键该从哪儿取、表怎么不膨胀。 去重键必须由生产者在消息 header 里带一个稳定的 `eventId`（形如 `orderId:PAID`），消费端不要自己拼易变字段——`msgId` 加消费时间戳每次重投都不同，等于没去重；`msgId` 本身也不行，生产端重试会生成新的 msgId（19.3 已讲）。同一业务事实的不同消息类型要拼上事件类型，否则「已支付」会把「已发货」也挡掉。

去重表是会长的：每天 1 亿条消息、保留 3 天就是 3 亿行，单行按 100 字节算约 30GB，唯一索引的 B+ 树随行数变高，每次 `INSERT` 的查找成本持续上升，最终拖慢所有消费。所以去重表必须有生命周期，且**保留期只需覆盖最大重投窗口**（19.4 那张 TTL 表同样适用）：

```
-- 按月分区，过期直接 DROP 分区，比 DELETE 快几个数量级且不产生大量 undo
ALTER TABLE msg_dedup DROP PARTITION p202609;
-- 无分区时按 created_at 分批清理，别一条 DELETE 锁全表
DELETE FROM msg_dedup WHERE created_at < NOW() - INTERVAL 3 DAY LIMIT 10000;
```

归档前先确认一件事：对账回放和人工补单可能在 T+1 之后发生，保留期砍得太短，那些迟到的重复消息就会穿透。**归档窗口和 Redis TTL 是同一个数，别一个配 3 天、一个配 6 小时。**

19.5　一个完整的消费模板

把三层串起来，这就是一个可以直接抄的骨架：

```
@RocketMQMessageListener(topic = "order-paid", consumerGroup = "ship-group")
public class OrderPaidConsumer implements RocketMQListener<OrderPaidMsg> {

    @Override
    public void onMessage(OrderPaidMsg msg) {
        String bizKey = "ship:" + msg.getOrderId();

        // ① 快速挡一层（省 DB 查询，不依赖它）
        if (!redis.setnx("dedup:" + bizKey, "1", 3600)) {
            log.info("fast dedup hit, orderId={}", msg.getOrderId());
            return;
        }

        try {
            // ② 状态机 + ③ 唯一约束
            int affected = orderMapper.updateStatus(msg.getOrderId(),
"SHIPPING", "PAID");
            if (affected == 0) {
                log.info("state not match, already handled. orderId={}",
msg.getOrderId());
                return;                                  // 幂等成功
            }
            shipmentMapper.insert(new Shipment(msg.getOrderId()));   // uk_order
兜底
        } catch (DuplicateKeyException e) {
            log.info("shipment exists, orderId={}", msg.getOrderId());
        } catch (Exception e) {
            redis.delete("dedup:" + bizKey);             // 失败要清掉标记，允许
重试
            throw e;                                     // 交给 MQ 重试
        }
    }
}
```

三个细节值得注意：

1. **失败时清掉 Redis 标记**——否则重试会被自己的标记挡住，消息永远处理不了
2. `affected = 0` **返回成功**——不是异常，是幂等命中
3. `DuplicateKeyException` **吞掉**——它是幂等的表现，不是错误

【必读】一个带事务边界的完整消费模板

19.5 的模板把三层串起来了，但漏了三件在生产上会要命的事：**并发度、事务边界、去重表**。补成一个可以直接抄的版本：

```
@RocketMQMessageListener(
        topic = "order-paid", consumerGroup = "ship-group",
        consumeThreadNumber = 32,        // 并发度：与下游容量对齐，别盲目调大（第
20 章）
        maxReconsumeTimes = 16)
public class OrderPaidConsumer implements RocketMQListener<OrderPaidMsg> {

    private final RedisTemplate<String, String> redis;
    private final ShipTxService shipTxService;

    @Override
    public void onMessage(OrderPaidMsg msg) {
        String bizKey = "ship:" + msg.getOrderId() + ":" + msg.getEventType();
        String lockKey = "dedup:" + bizKey;
        // ① 加速层：挡住绝大多数重复，失败必须清理
        Boolean first = redis.opsForValue()
                .setIfAbsent(lockKey, "PROCESSING", Duration.ofMinutes(5));
        if (!Boolean.TRUE.equals(first)) return;
        try {
            shipTxService.handle(bizKey, msg);    // ② 跨 Bean 调用才走代理（第 6
章）
            redis.opsForValue().set(lockKey, "DONE", Duration.ofDays(3));
        } catch (Exception e) {
            redis.delete(lockKey);                // ③ 失败清标记，否则重试被自己
挡死 → 永久丢
            throw e;                              // ④ 交给 MQ 重试
        }
    }
}

@Service
@RequiredArgsConstructor
public class ShipTxService {
    @Transactional(rollbackFor = Exception.class)   // ⑤ 事务边界：三件事一次提交
    public void handle(String bizKey, OrderPaidMsg msg) {
        dedupMapper.insert(bizKey);                 // 去重表写入
        int n = orderMapper.updateStatus(msg.getOrderId(), "SHIPPING", "PAID");
        if (n == 0) {
            if (orderMapper.countByOrderId(msg.getOrderId()) == 0) {
                throw new BizException("订单不存在，转人工");     // ⑥ 不吞脏数据
            }
            return;                                 // 幂等命中
        }
        shipmentMapper.insert(new Shipment(msg.getOrderId()));   // ⑦ uk_order
兜底
```

```
}
```

三个边界各自对应一类事故，记牢比背代码有用：

边界	写错会怎样
去重表与业务写入**同一个本地事务**	拆成两个事务：业务成功、去重标记没写上 → 重投时重复执行
加速层失败必须清标记	不清：重试被自己的 setnx 挡住，消息永久卡死（第 18 章的「黑洞」换个方向）
并发度与下游容量对齐	`consumeThreadNumber` 盲目调大，把下游 DB 打挂，反而制造积压（第 20 章）

```
// ✗ 去重表 insert 在事务外：业务提交了、去重记录回滚了，重投必然重复
dedupMapper.insert(bizKey);              // 无事务，自动提交
shipTxService.handle(bizKey, msg);       // 另一个事务
// ✓ 两者进同一个 @Transactional，一次提交解决两个问题（见上面的 ShipTxService）
```

【必读】把三章串起来：至少一次 → 幂等 → 扣减

幂等不能只靠阅读代码验收，得有一条会失败的测试：

```
@SpringBootTest
class IdempotentConsumerTest {
    @Autowired private OrderPaidConsumer consumer;
    @Autowired private JdbcTemplate jdbc;

    @Test
    void 同一条消息投递两次_只创建一张发货单() {
        OrderPaidMsg msg = new OrderPaidMsg(1001L, "PAID", 1);
        consumer.onMessage(msg);
        consumer.onMessage(msg);          // 模拟生产端重试或 rebalance 重投
        assertEquals(1, jdbc.queryForObject(
            "SELECT COUNT(*) FROM shipment WHERE order_id = 1001",
Integer.class));
        assertEquals(1, jdbc.queryForObject(
            "SELECT COUNT(*) FROM msg_dedup WHERE biz_key = 'ship:1001:PAID'",
Integer.class));
    }
}
```

断言只有两条：业务表一条、去重表一条。**把「重复率降到 0」不现实，把「重复时多出来的那次副作用降到 0」才是可验收的目标**——这正是「让做两次和做一次结果一样」的量化版本。

三条线在这里合拢：第 18 章的手动提交把「可能丢」换成了「可能重」，是重复的总源头；本章负责消化重复；第 22 章的库存扣减是**更新累加**，唯一约束拦不住，必须靠状态机或流水号，是幂等最难的一类；第 21 章进一步说明顺序与幂等必须同时具备。追积压时会重放旧消息（第 20 章），所以没有幂等的扩容等于批量制造重复数据。

本章结论对应 **Spring Boot 3.x、MySQL 8.0（`DuplicateKeyException` 由 `spring-tx` 转译）、Redis 7（`setIfAbsent(key, value, Duration)`）、RocketMQ 4.9+/5.x**。重试次数、延迟级别、`maxReconsumeTimes` 的默认值都随版本漂移，升级后重跑上面那条测试再采信。

【必读】数据库之外的副作用：发短信、调支付怎么幂等

三板斧有个共同前提：副作用写在数据库里，能靠事务回滚。可一旦消费逻辑里掺了发短信、调支付网关、推第三方接口，这些动作**发出去就收不回来**——重复消费会真的发两条短信、扣两次钱，事后没有任何 `ROLLBACK` 能撤销。这类副作用必须换一套做法：**先把「要做的事」记下来，再去做**。

副作用	幂等手段	失败处理
发短信 / 推送	发送流水表 `uk_biz_key`，发送前插 INIT	失败保留 INIT，定时任务重试；成功回写 CONFIRMED
调支付网关	幂等号（商户单号）透传给网关加本地流水	网关超时不重试，改查单确认，避免重复扣款
调下游写接口	下游按业务键去重加本地流水	下游确认后再标记成功

```
@Transactional(rollbackFor = Exception.class)
public void notify(OrderPaidEvent e) {
    String key = "sms:" + e.getOrderId();
    // ① 先落流水，与业务状态变更同一事务；重复消费在此被唯一键挡住
    try {
        flowMapper.insert(new SendFlow(key, "INIT"));
    } catch (DuplicateKeyException ex) {
        return;                                  // 已在流程中或已发出，幂等跳过
    }
    orderMapper.markPaid(e.getOrderId());
    // ② 真正调用放在事务外；成功再回写 CONFIRMED
}

@Scheduled(fixedDelay = 30000)
public void retrySms() {
    for (SendFlow f : flowMapper.selectInit(100)) {
        boolean ok = smsGateway.send(f.getPayload());   // 网关侧按同一 bizKey 去
重
        if (ok) flowMapper.markConfirmed(f.getId());
    }
}
```

幂等不等于「不调用」，等于「副作用最多发生一次」：本地流水挡住重复投递，幂等号挡住网关侧重复，两级都要有。这也解释了为什么第 18 章的本地消息表长得和它一模一样——**「先落表再做事」是所有跨系统可靠性的同一个母版**，只是落的那张表叫 `local_message` 还是 `send_flow` 的区别。第 22 章的支付超时回补同样要按订单号判重，否则一次关单回补两遍就变成了超卖。

19.6　自检清单

- [ ] 所有 MQ 消费者都有幂等逻辑（不是"应该不会重复"）
- [ ] 去重键是**业务键**（订单号/流水号），不是 `msgId`
- [ ] 有数据库唯一约束作为最后防线
- [ ] 状态机更新带前置状态条件（`WHERE status = 'INIT'`）
- [ ] `affected = 0` 时返回成功，不是抛异常
- [ ] 失败路径清掉了去重标记（否则消息卡死）
- [ ] 金额类的更新不是简单累加，有流水号防重
- [ ] 去重表有定期清理策略（否则无限增长）

19.7　一句话总结

> 幂等的本质不是"防重复"，是**让"做两次"和"做一次"的结果一样**。 想清楚这一点，比背三板斧有用得多。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/06-MQ消息坑、07-消息队列/幂等与去重

第二十章　积压：从 10 万到 0 的 40 分钟

本章要回答：堆积报警了，第一件事是扩容消费者吗？

本章你能带走

- 一个前置判断：**很多"积压"是假象，扩了也没用**
- 一个扩容天花板：**有效上限 = 队列数，超过纯属浪费**
- 一套处置顺序：扩容 → 转发 → 丢弃，代价依次上升

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

20.1　现场：扩了 8 个消费者，位点不动

凌晨两点告警：订单消息堆积 12 万条。

值班同学按手册操作：消费者从 4 个扩到 12 个。**半小时后，堆积 15 万条，还在涨。**

扩容无效，说明问题不在消费能力。

当时怎么误判的：凌晨两点告警，第一反应是「消费者挂了或卡死」——看了眼监控，单个消费者 TPS 还是 200，不像挂了；又怀疑「是不是 topic 被限流」，查了 Broker 流控也没触发。在「再扩 8 个」和「回滚扩容」之间犹豫时，有人随口问了句「队列数多少」，一查 `QueueCount: 4`，才明白扩的 8 个全是空转。这一句提问省了后面两小时瞎折腾。

查了三件事：

① 消费 TPS 是多少?

从监控看，单个消费者的 TPS 一直是 200，跟扩容前一模一样。12 个消费者总 TPS 应该 2400，但实际只有 200——**说明只有 1 个消费者在干活**。

② 队列数是多少？

```
sh mqadmin topicStatus -n nameserver:9876 -t order-topic
# QueueCount: 4
```

队列只有 4 个。 RocketMQ 的并发消费按队列分配，一个队列只能被一个消费者消费（同一消费组内）。所以：

> **消费者数量超过队列数时，多出来的消费者是空闲的。** 4 个队列 → 最多 4 个消费者有效 → 扩到 12 个，8 个在空转。

③ 扩容队列不就行了？

不行。**给已有 topic 扩容队列，会破坏顺序消息**（第 21 章），而且新队列的数据是新的，老队列的积压还在。

所以"扩消费者"之前，必须先看队列数。**这是积压处置的第一个坑：以为加机器就能解决，实际撞到了分区数的天花板。**

【必读】Lag 不是一个数：按分区求和与取最大

20.1 的现场说「堆积 12 万」，但这个值怎么算出来的，决定了后面所有处置对不对。**Lag 从来不是「最新位点减消费位点」这么一次相减**，分区是各自独立的计数器：

```
Lag(group, topic) = Σ over 每个分区 p ( LogEndOffset(p) - CommittedOffset(p) )
告警看的是 max over p ( ... )，以及 dLag/dt 的符号
```

多数人以为 Lag 是简单相减，实际总 Lag 会把单个分区的堆积摊薄到所有分区上。 12 个分区里 11 个 lag 为 0、1 个 lag 为 12 万，总 Lag 仍是 12 万，可平均到每个分区只有 1 万——按平均值设阈值就漏了。更麻烦的是：扩容到 12 个消费者也没用，因为那 12 万条全压在一个分区上，一个分区同一时刻只能被组内一个消费者消费，剩下 11 个实例全在空转。

```
# 逐分区看 Lag，而不是只看控制台的 Diff Total
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 \
  --group order-consumer --describe
# TOPIC  PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG     CONSUMER-ID
# order  0          982300          1102300         120000  c1-...   ← 全堆在这
# order  1          1102280         1102300         20      c2-...
# order  2          1102295         1102300         5       c3-...
```

读输出看三列：`CONSUMER-ID` 为空说明该分区根本没分到消费者；`LAG` 为 0 但 `CURRENT-OFFSET` 长时间不动，是 20.2 要讲的假象；某分区 `LAG` 独高、其余接近 0，是分区倾斜或热 key，扩容解决不了，得查生产端的 key 分布。

还得分清两个位点：`position` 是内存里「下一条要拉」的位置，`committed offset` 是提交到 `__consumer_offsets` 的位置。自动提交被关掉后，处理已经推进了 `position`、却没提交 `committed`，此时进程重启会从旧的 `committed` 回退——**表现是「消息又来了一遍」，是重复不是丢失**（第 18 章、第 19 章）。把它误判成积压，会去扩一批根本不需要的消费者。`--describe` 的列名随客户端版本略有差异，需在自己的版本上核对。

20.2　先判断：是真积压还是假象

RocketMQ 有一个非常反直觉的行为：

> **被 TAG / SQL92 过滤掉的消息，也计入堆积量。**

也就是说，如果你的消费者订阅 `TAG_A`，而 topic 里有 10 万条 `TAG_B` 的消息，监控上会显示"堆积 10 万"——**但那些消息本来就不该被你消费**。

判断方法，两个指标：

指标	真积压	假象
消费位点（offset）	不动	**持续推进**
消费 TPS	掉到 0 或极低	**正常**

位点在推进 = 消息在被消费 = 堆积量只是"过滤掉的那部分"。 这种情况不需要处置，只需要修监控口径。

第二个假象：**死信队列/重试队列的堆积**被算进了主 topic。查一下 `%RETRY%` 和 `%DLQ%` 的堆积量，它们是独立 topic。

【必读】扩容的算式，与「扩了也不动」的四个根因

扩消费者之前先算四步，比拍脑袋加机器省半小时：

```
① 所需并发 C = 峰值生产 TPS × 单条平均耗时(s)
② 单实例开 T 个消费线程 → 实例数 N = ceil(C / T)
③ 硬约束：N ≤ 分区数 P，且 concurrency ≤ P
④ 反推分区数：P ≥ C / T，留 30% 余量 → P ≈ 1.3 × C / T
```

代一组真实数字：峰值 2000 TPS、单条 50ms，得 `C = 2000 × 0.05 = 100` 并发；单实例 20 线程，`N = 5` 个实例；分区数至少 5，建议 8。20.1 的现场是 4 个队列扩到 12 个消费者，有效只有 4 个、8 个空转，正撞第 ③ 条。

还有一个比扩容更该先算的式子——**追平时间**：

```
T_clear = 当前积压量 / (消费 TPS − 生产 TPS)
```

现场 12 万条积压、消费 200 TPS、生产 250 TPS，分母是 **−50**，追平时间为负，意思是**永远追不平**。这解释了「扩了半小时，堆积从 12 万涨到 15 万」：只要消费 TPS 不高于生产 TPS，积压就是数学上不收敛的，跟机器数量无关。要让分母转正，先把单条耗时降下来，或临时降生产。

四个「扩了也不动」的根因，按排查顺序：

根因	现场信号	取证	为什么扩了没用
① 超过分区/队列数上限	`concurrency > P`，部分 `CONSUMER-ID` 为空	`--describe` 看分配	多出来的实例分不到队列，纯空转，还多一次 rebalance
② 下游是瓶颈（DB / HTTP）	CPU 空闲、单条耗时长、连接池打满	`jstack`、`SHOW PROCESSLIST`	扩容只是把慢复制 N 份，DB 先被打挂（第 7 章）
③ rebalance 风暴	lag 锯齿状、日志频繁 rebalance	`kafka-consumer-groups.sh --describe --members`	每加一台机器触发一次全组暂停，位点不推进
④ 毒消息 / 反序列化失败卡死单分区	某分区 lag 独高、其余为 0	消费日志里的反序列化异常	该分区永远轮不到别的消费者，加多少实例都无效

第 ③ 条最反直觉：**在 rebalance 风暴期间加消费者，等于给抖动添柴**——扩容本身就是一个触发 rebalance 的动作。这时候正确的第一步不是加机器，是先把 `max.poll.interval.ms` 超时和 GC 停顿压下去（第 2 章的重试风暴是同一条链）。第 ④ 条要靠 `ErrorHandlingDeserializer` 加死信 Topic 兜住脏数据，否则那个分区会一直卡到天亮。

扩消费者是处置顺序里排第二的动作，不是第一；先算清楚 C 与 P，再决定加不加。

把四个根因压成一份能照着敲的取证序列，比在群里猜快得多：

```
# ① 消费者数 vs 分区数：先看天花板
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic order \
  | grep -E 'PartitionCount|ReplicationFactor'
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 \
  --group order-consumer --describe --members      # 看每个实例分到几个分区

# ② 下游是不是瓶颈：线程停在哪
jstack -l <pid> | grep -A 8 'ConsumeMessageThread_'   # 停在 SocketInputStream / getConnection 即下游慢
# MySQL 侧
SHOW STATUS LIKE 'Threads_running';
SHOW PROCESSLIST;

# ③ rebalance 是否频繁
grep -c 'rebalance' consumer.log

# ④ RocketMQ 队列数与堆积分布
sh mqadmin topicStatus -n nameserver:9876 -t order-topic
sh mqadmin consumerProgress -n nameserver:9876 -g order-consumer
```

`--members` 里 `CONSUMER-ID` 为空的分区就是没分到人的；`jstack` 里大量线程停在 `SocketInputStream.read` 或 `HikariPool.getConnection`，说明瓶颈在下游而不是消费者数量（第 7 章的取证顺序在这里同样适用）。

扩容本身也要算：一次 `kubectl scale --replicas=12` 会触发一次全量 rebalance，一次加 2 个、观察 lag 斜率变负后再加下一批，抖动小得多。**扩容前先确认「分区数减当前消费者数」还有多少余量**，余量为 0 时加机器只是给 rebalance 添柴。

最后一个数值得记住：积压量 = `(生产 TPS − 消费 TPS) × 持续时间`。现场生产 250、消费 200，差 50 条/秒，半小时正好攒出 9 万条，和 12 万涨到 15 万对得上。**这个式子把「积压」从监控上的一个数字，变成了两个速率的差值积分**，也说明为什么「消费 TPS 与生产 TPS 的差」（20.5 第二个指标）比堆积量更早报警：差值一出现，积分就注定要涨，只是还没堆到阈值。

顺带把「扩了也不动」的第五种收进来：20.2 说的 TAG 过滤假象。位点持续推进、消费 TPS 正常，只是堆积量里混了本不该被消费的消息——这种情况扩了当然不动，因为它根本不需要动。

20.3　真积压：处置顺序

按"代价从小到大"排，**不要一上来就用最贵的手段**。

【必读】第一步：找消费慢的根因

在扩容之前，先花五分钟看消费者在干什么。四个高频原因：

原因	表现	处理
消费逻辑里有循环查库（N+1）	单条消息耗时 200ms+	批量查、加缓存
同步调用下游接口	消费耗时 = 下游 RT	改异步、加超时
单条处理而非批量	每条一次 DB 往返	批量消费（`consumeMessageBatchMaxSize`）
消费者被限流/线程数太小	CPU 空闲但 TPS 上不去	调 `consumeThreadMax`

如果消费逻辑本身很慢，扩容只是把慢复制 N 份，DB 反而更容易被打挂。

【必读】第二步：扩容到队列数上限

```
// 消费线程数（不是实例数）
consumer.setConsumeThreadMin(20);
consumer.setConsumeThreadMax(64);
```

注意两个"扩容"是不同的：

- **加实例**：受队列数限制（一个队列一个消费者）
- **加线程**：受消费者内部线程池限制，单实例能扛更多

在队列数已经打满的情况下，加线程仍然有效——一个消费者可以开多个线程消费同一个队列（顺序会乱，见第 21 章）。这与第 7 章"等锁型故障加线程只是给排队队伍扩容"的负收益不同——此处队列数已封顶，加的是消费并发度，属互补场景（加线程在"等锁/等连接"瓶颈为何是负收益，见第 7 章）。

【必读】第三步：临时转发到"大队列" topic

如果积压量太大（比如百万级），正常消费要几小时，而业务等不了：

```
① 起一个临时消费者，不做业务处理，只把消息搬到新 topic（新 topic 有 32 个队列）
② 新 topic 的消费者用 32 个实例并行消费
③ 消费速度提升 N 倍（N = 新队列数 / 老队列数）
```

代价：多一次消息搬运、顺序全乱、需要额外机器。**这是"应急手段"，不是常态方案。**

【必读】第四步：跳过非核心消息

如果积压的是日志、埋点、通知这类"丢了不影响钱"的消息：

```
// 判断消息年龄，太老的直接跳过
if (System.currentTimeMillis() - msg.getStoreTimestamp() > 3600_000) {
    log.warn("skip stale message, born at {}", msg.getStoreTimestamp());
    return SUCCESS;                          // 丢弃，保核心链路
}
```

这是唯一"主动丢消息"的合法场景：消息本身可丢，且延迟消费已经失去意义（三小时前的"支付成功"短信，现在发出去只会打扰用户）。

前提是**你明确知道这类消息可以丢**，并且写进了文档。

【必读】临时转发：把 4 个队列搬到 32 个队列

20.3 第三步说了思路，这里给出能跑的形状。核心是**临时抬高队列数这个天花板**，而不是提高业务处理能力：

```
# ① 建一个 32 队列的临时 topic（-w 为写队列数，参数名需按版本验证）
sh mqadmin updateTopic -n nameserver:9876 -t order-topic-tmp -w 32 -r 3

# ② 起搬运工消费者组，用新 consumerGroup 从头读，避免影响原组位点
# ③ 临时 topic 的消费者批量拉取、直接落「待处理表」，离线任务再补业务
# ④ 追平后切回原 topic，删除临时 topic，回滚所有临时扩容
```

```
// 搬运工：只搬运，不碰业务，原样透传幂等键给下游
@RocketMQMessageListener(topic = "order-topic",
        consumerGroup = "relay-to-tmp-group",
        consumeMode = ConsumeMode.CONCURRENTLY, consumeThreadNumber = 4)
public class RelayConsumer implements RocketMQListener<MessageExt> {
    private final RocketMQTemplate template;

    @Override
    public void onMessage(MessageExt m) {
        Message<String> out = MessageBuilder
                .withPayload(new String(m.getBody(), StandardCharsets.UTF_8))
                .setHeader(RocketMQHeaders.KEYS, m.getKeys())   // 业务键不能丢
（第 19 章）
                .build();
        template.syncSend("order-topic-tmp", out);
    }
}
```

加速比有个简单算式：`加速比 ≈ 新队列数 / 老队列数 = 32 / 4 = 8 倍`。12 万条按原 200 TPS 要 600 秒，8 倍后约 75 秒。但**这 75 秒只是搬完，不是处理完**——业务仍由离线任务按数据库能承受的速率补。

代价必须写进预案：顺序全丢（第 21 章）、消息重复（第 19 章要求消费端幂等）、多一份存储、临时 topic 用完要清理。**转发不是「提高消费能力」，是「把队列数这个天花板临时抬高」，它买的是时间，不是吞吐；强有序业务禁用。**

20.4　消费端反压：别把积压变成雪崩

积压时最容易犯的错：**消费者拼命拉消息，把下游打死。**

```
// ✗ 无限制拉取：一次拉 1000 条，全部并发处理 → 下游瞬间被打穿
consumer.setPullBatchSize(1000);
consumer.setConsumeThreadMax(200);
```

正确的做法是**让消费速率跟下游容量对齐**：

```
// ✓ 拉取量 × 线程数 ≈ 下游能承受的并发
consumer.setPullBatchSize(32);
consumer.setConsumeThreadMax(20);      // 32 × 20 = 640 并发上限
```

同时给下游调用设超时和熔断——**积压的时候，下游通常也已经很紧张了**。

【必读】反压：让拉取速率跟下游容量对齐

20.4 给了「拉取量 × 线程数 ≈ 下游并发」这句话，把它补成可算的约束。积压时下游通常也已经很紧张，三条约束缺一条就会把积压变成雪崩：

```
约束一　下游并发上限 ≈ 连接池大小 / 单次调用耗时(s)
        例：Hikari maximumPoolSize=20、单次写 20ms → 20 / 0.02 = 1000 次/秒
约束二　pullBatchSize × consumeThreadMax ≤ 下游并发上限
约束三　max.poll.records × 单条耗时 < max.poll.interval.ms（默认 300s）
```

按约束一算，`pullBatchSize=1000 × consumeThreadMax=200` 是 640 量级的瞬时并发，乘上每秒 50 次就是 3 万多次/秒，是下游 1000 次/秒的 30 倍——**拉得越快，下游越慢，lag 越降不下来**。约束三则是 Kafka 特有的坑：500 条 × 0.7 秒 = 350 秒，超过 `max.poll.interval.ms` 的 300 秒，整批被踢出组、触发 rebalance，积压反而更糟；降到 200 条 × 0.7 秒 = 140 秒才安全。

```
// ✗ 一次拉 1000 条 × 200 线程：瞬时打穿下游，还会整批超时被踢
consumer.setPullBatchSize(1000);
consumer.setConsumeThreadMax(200);

// ✓ 按下游容量反推：单条 20ms、下游 1000 次/秒 → 并发上限 20，留一半余量
consumer.setPullBatchSize(20);
consumer.setConsumeThreadMax(20);      // 20 × 20ms = 400 并发，安全
```

`fetch.min.bytes` 与 `fetch.max.wait.ms` 控制拉取节奏，RocketMQ 的 `consumeMessageBatchMaxSize`、RabbitMQ 的 `prefetch` 同理。**反压不是限流自己，是保护下游**：下游变慢、消费超时、调用方重试，负载被放大成平方（第 2 章），这时候任何「拉满」的配置都在给雪崩加油。

20.5　预防：三个必配的监控

积压是可以提前发现的，前提是你看对了指标：

指标	告警阈值	说明
堆积量（Diff Total）	> 1 万，或持续上涨 5 分钟	最直接的信号
消费 TPS 与生产 TPS 的差	消费 < 生产 持续 5 分钟	**这个比堆积量更早**
单条消息处理耗时 P99	> 500ms	提前发现"消费变慢"

第二个指标最有价值：**当生产 TPS 稳定高于消费 TPS 时，积压是必然的**，只是还没堆到告警阈值。这个信号通常比堆积量告警早 10 - 30 分钟。

再补一个容量规划的习惯：

> **消费能力（TPS）应该是峰值生产 TPS 的 2–3 倍。** 平时跑在 30% 水位，才有应对突发和"消费变慢"的余量。

【必读】三个必配监控：指标名、阈值与告警条件

那张表给了口径，这里补上能直接粘进告警配置的**具体指标名与阈值**。指标名随 exporter 与版本不同（Burrow、kafka_exporter、JMX、RocketMQ Exporter 各有命名），上生产前需在自己的版本上核对：

#	指标	具体名（示例）	告警阈值与条件
1	消费 Lag	`kafka_consumergroup_lag`、`kafka_consumer_fetch_manager_records_lag_max`；RocketMQ `rocketmq_group_diff`（Diff Total）	`> 10000` **且** `rate(lag[5m]) > 0` 持续 5 分钟
2	生产 / 消费 TPS 差	`rocketmq_consumer_tps` vs `rocketmq_producer_tps`；`kafka_consumer_fetch_manager_records_consumed_rate`	`consumer_tps < producer_tps` 持续 5 分钟，比堆积量早 10~30 分钟
3	单条处理耗时 P99	`rocketmq_consumer_consume_rt`；自定义 `consumer_handle_seconds{quantile="0.99"}`	`> 0.5s`

再补两个「不出事也想不起来」的：`kafka_server_replicamanager_underreplicatedpartitions > 0` 持续 5 分钟（ISR 收缩，`acks=all` 正在退化，第 18 章），以及 `%DLQ%` 堆积量 > 0 立即告警。

```
groups:
- name: mq-backlog
  rules:
  - alert: ConsumerLagGrowing
    expr: kafka_consumergroup_lag > 10000
      and rate(kafka_consumergroup_lag[5m]) > 0
    for: 5m
    labels: { severity: warning }
    annotations:
      summary: "消费组 {{ $labels.group }} lag 持续上涨"
  - alert: ConsumeSlowerThanProduce
    expr: rocketmq_consumer_tps < rocketmq_producer_tps
    for: 5m
    labels: { severity: critical }
```

「lag > 阈值且增长率为正」这个「且」不能省：常驻型消费组 lag 应接近 0，但任务型（批处理）消费组在任务间隙增长是正常现象，两类告警规则必须分开，否则每天夜里误报一轮，告警就被静音了。

本章结论对应 **Kafka 3.x/4.x、RocketMQ 4.9+/5.x、Spring Kafka 3.x**。`max.poll.interval.ms`（默认 300s）、`session.timeout.ms`（默认 45s）、RocketMQ 队列数与 `consumeThreadMax` 的默认值都需按版本验证；Kafka 4.0 起 KIP-848 新协议（客户端 `group.protocol=consumer`）让 rebalance 只影响变动分区，扩容的副作用比旧协议小，这是升级时值得重新测一轮的点。

20.6　自检清单

- [ ] 知道每个 topic 的**队列数**，且消费者实例数不超过它（或已知会浪费）
- [ ] 积压告警区分"真积压"（位点不动）和"假象"（TAG 过滤）
- [ ] 消费 TPS 有监控，且与生产 TPS 对比
- [ ] 消费逻辑里没有循环查库/同步调下游
- [ ] 拉取批量 × 线程数与下游容量匹配，不是越大越好
- [ ] 有"临时转发 + 并行消费"的预案（脚本或开关）
- [ ] 明确哪些消息可以丢弃（写进文档）
- [ ] 消费能力按峰值生产 TPS 的 2–3 倍规划

20.7　一句话总结

> 积压的本质永远是"生产 > 消费"，只有三种解法：**提高消费、降低生产、丢弃**。扩容只解决第一种，而且它的天花板是队列数。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/06-MQ消息坑、07-消息队列/消费积压处理、07-消息队列/RocketMQ
- **【选读】与第 2 章的连接：重试风暴把积压放大**。恶性循环：下游变慢 → 单条 RT 上升 → 整批超过 `max.poll.interval.ms` → 被踢出组 → rebalance（全组暂停）→ 位点不推进 → 调用方超时重试 → 生产端重试抬高 TPS → lag 加速上涨 → 扩容 → 又一次 rebalance。N 层调用链各配一次重试，一次抖动产生 `2^N` 个请求；在重试风暴里加机器等于给 rebalance 添柴。先止血要降生产、关重试、调小批量，而不是扩消费。

第二十一章　顺序消息被破坏之后

本章你要带走：为什么"配了顺序消息"还是不顺序？哪些操作会悄悄打破顺序？

本章你能带走

- 顺序消息成立的两个必要条件（少一个都不行）
- 四个破坏顺序的隐藏操作，其中三个发生在"配置正确"之后
- 一条判断：**全局有序 = 单队列 = 吞吐量地板**

难度构成：本章主线【必读】；含【进阶】深挖 3 处；章末「延伸」小节放支线深挖与知识库指针。---

21.1　现场：状态倒退了

订单状态机：`已支付 → 已发货 → 已签收`。

用户投诉：订单显示"已支付"，但他昨天已经签收了。

查消息轨迹：

```
14:02:11  消费 order-shipped    → status = SHIPPED
14:02:13  消费 order-paid       → status = PAID     ← 状态倒退了
```

"已支付"的消息比"已发货"晚到了 2 秒。

生产端确实按顺序发的（先 paid 后 shipped），消费端也"配了顺序消费"。那为什么还是乱？

当时怎么误判的：用户投诉时，第一反应是"消费端代码写错了更新顺序"——翻出 order-paid 和 order-shipped 两个 listener，都是直连 `UPDATE`，看不出毛病；又怀疑"是不是有人手动改了状态"，查审计日志也没有。排查在两套应用代码之间来回拉扯了大半天，直到有人把消息轨迹按时间戳排开，才看清 shipped 先到、paid 晚到 2 秒——问题不在代码，在消息本身的到达顺序。这一下把方向从"应用层"扭回"MQ 顺序性"，后面的排查才对路。

21.2　顺序的两个必要条件

顺序消息不是"一个开关"，它需要**生产端和消费端同时满足**：

> ① 同一业务键的消息 → 进同一个队列（生产端）
> ② 同一个队列 → 只有一个线程消费（消费端）

任何一端破了，顺序就没了。

【必读】生产端：`MessageQueueSelector`

```
// ✓ 按订单 id 哈希选队列：同一订单 → 同一队列
rocketMQTemplate.syncSendOrderly("order-status", msg, String.valueOf(orderId));
```

```
// ✓ 原生写法
producer.send(msg, new MessageQueueSelector() {
    @Override
    public MessageQueue select(List<MessageQueue> mqs, Message msg, Object arg)
{
        int idx = Math.abs(arg.hashCode()) % mqs.size();
        return mqs.get(idx);
    }
}, orderId);
```

`syncSendOrderly` 内部就是按 key 哈希取模。**关键点：同一个 key 永远算出同一个 index。**

【必读】消费端：`MessageListenerOrderly`

```
// ✓ 顺序消费：一个队列一个线程，串行处理
consumer.registerMessageListener(new MessageListenerOrderly() {
    @Override
    public ConsumeOrderlyStatus consumeMessage(List<MessageExt> msgs,
ConsumeOrderlyContext ctx) {
        for (MessageExt m : msgs) {
            handle(m);
        }
        return ConsumeOrderlyStatus.SUCCESS;
    }
});
```

如果用 `MessageListenerConcurrently`（并发消费），**哪怕队列只有一个，多线程也会让消息乱序**。

【进阶】源码级：锁到底加在哪一层

多数人以为 `syncSendOrderly` 或 `MessageQueueSelector` 提供了顺序保证，实际生产端那一步只是一个路由函数。 `select(List<MessageQueue>, Message, Object)` 拿业务键算出 index，不加任何锁、不保证任何时序；它只负责「同一业务键进同一个队列」。真正的顺序保证全在消费端，而且是三把锁叠出来的（以下类名与字段对应 RocketMQ 4.x，5.x 有重构，需按自己的版本验证）：

锁	位置	粒度	作用
① Broker 端队列锁	`RebalanceImpl#lockBatchMQ` → Broker `RebalanceLockManager#tryLockBatch`	一个 `MessageQueue`	rebalance 时消费者向 Broker 申请，保证同一队列同一时刻只被组内**一个消费者**持有（分布式锁，有过期续期）
② 本地队列锁	`ConsumeMessageOrderlyService` 内的 `MessageQueueLock.mqLockTable`（`ConcurrentHashMap<MessageQueue, Object>`）	一个 `MessageQueue`	提交消费任务前 `synchronized` 拿锁，保证本地**只有一个线程**消费该队列
③ 顺序快照	`ProcessQueue.msgTreeMap`（按 offset 排序的 `TreeMap`）与 `consumingMsgOrderlyTreeMap`	队列内消息	取出消息时按 offset 升序，保证拉取顺序就是提交顺序

```
Rebalance 完成
  └ lockBatchMQ(queue) → Broker 返回锁成功 → 才创建 ProcessQueue    ← 锁①
Consumer 拉取 → ProcessQueue.msgTreeMap.put(offset, msg)（TreeMap 自动排序）← 锁③
  └ 提交 ConsumeRequest 到线程池
      └ synchronized(mqLockTable.get(queue))                        ← 锁②
          └ 取 msgTreeMap 中最小 offset 的一批
              └ MessageListenerOrderly.consumeMessage() 串行执行
```

这就解释了前面那句「用 `MessageListenerConcurrently`，哪怕队列只有一个，多线程也会让消息乱序」——并发模式下根本没有 `mqLockTable` 这把锁，线程池里的多个 `ConsumeRequest` 会同时处理同一个队列。

由此还能读懂一个常被配错的参数：`consumeThreadNumber` 在顺序模式下是**每队列一个线程**，设成 4 表示最多同时消费 4 个队列，**不是同一队列 4 并发**。想靠调大它提升单个热点队列的吞吐，一点用都没有。

最后挑一个生产端的小坑：`Math.abs(arg.hashCode()) % mqs.size()` 里的 `Math.abs` 对 `Integer.MIN_VALUE` 仍返回负数，取模得到负 index 会直接越界。业务键哈希撞上这个值的概率极低但非零，用 `Math.floorMod(hash, size)` 更稳。

21.3　四个隐藏的顺序破坏者

这才是本章的重点——**配置都"对"了，顺序还是会乱**。

【必读】① 扩容队列（最狠，且不可逆）

生产端路由是 `hash(key) % queueCount`。

```
扩容前：queueCount = 4
  orderId=1001 → hash % 4 = 2

扩容后：queueCount = 8
  orderId=1001 → hash % 8 = 2  （碰巧一样）
  orderId=1002 → hash % 4 = 2  → hash % 8 = 6  （变了！）
```

队列数一变，几乎所有 key 的路由都变了。 于是同一个订单的老消息在队列 2、新消息在队列 6，两个队列并行消费——**顺序彻底崩掉**。

而且这个破坏是**永久**的：老队列里还有积压，新消息已经在别的队列了。

结论：顺序消息的 topic，队列数一旦定下就不能改。 需要扩容就新建 topic + 双跑迁移。

【必读】② 重试队列：失败的消息会"插队"

这是最容易被忽略的一个。

```
队列顺序：[msg1, msg2, msg3]

msg2 消费失败 → 返回 RECONSUME_LATER → 进了重试队列
msg3 正常消费成功

3 秒后，msg2 从重试队列重新投递 → 此时 msg3 已经处理完了
```

顺序消息里的失败重试，天然破坏顺序。 只要有一条消息重试，它就会跑到后面所有消息的后面。

所以顺序消费的失败处理只有两条路：

1. **阻塞重试：** `SUSPEND_CURRENT_QUEUE_A_MOMENT` —— 当前队列暂停，等这条重试成功再继续。**这会阻塞整个队列**（包括其他订单的消息），但保证顺序
2. **本地落库 + 人工/定时补偿：** 失败的消息存到"待处理表"，不进重试队列，由定时任务按顺序补齐

```
// ✓ 阻塞重试，保顺序
if (!handle(m)) {
    ctx.setSuspendCurrentQueueTimeMillis(1000);
    return ConsumeOrderlyStatus.SUSPEND_CURRENT_QUEUE_A_MOMENT;
}
```

代价必须知道： 一条毒消息（永远处理失败的）会**卡死整个队列**。所以线上必须配"毒消息熔断"——重试超过 N 次后转人工，不要无限阻塞。

【必读】③ 消费端内部异步

```
// ✗ 顺序消费的 listener 里又开了线程池
public ConsumeOrderlyStatus consumeMessage(...) {
    for (MessageExt m : msgs) {
        pool.submit(() -> handle(m));      // 顺序立刻没了
    }
    return SUCCESS;                         // 而且返回成功时可能还没处理完
}
```

顺序消费的 listener 必须是同步的。 里面再开异步、再 `CompletableFuture`，顺序保证立刻失效，而且 `return SUCCESS` 会在处理完成前就提交 offset（第 18 章的丢失问题一起出现）。

【必读】④ 多消费组 + 下游乱序

如果两个消费组分别消费"订单"和"物流"消息，各自处理完再更新同一张表——**跨 topic 的顺序是无解的**。

> MQ 只能保证"同一队列内"的顺序，**跨 topic、跨服务、跨存储的顺序，MQ 管不了**。这类需求要靠业务层的状态机或版本号。

【必读】四个破坏者：每个都能复现

四个破坏者，这里给每个补上可复现的操作与观察点，让「配置都对了还是乱」变成能在测试环境重放的事：

破坏者	复现步骤	观察点
① 扩容队列	先按 `orderId=1002` 连发 100 条，再 `sh mqadmin updateTopic -n nameserver:9876 -t order-status -w 8`（参数名需按版本验证），继续发同键消息	`queryMsgByKey` 里同一订单的新旧消息 `queueId` 不同
② 重试插队	让第 2 条消息失败重投，观察第 3 条是否先落地	消费日志里 `msg3` 早于 `msg2`
③ 消费端异步	在顺序 listener 里把 `handle(m)` 丢进线程池并立刻 `return SUCCESS`	日志顺序正确、业务表状态却是乱的
④ 多消费组 / 跨 topic	两个组分别消费订单与物流消息，各加随机 sleep 后更新同一张表	状态被旧值覆盖

一个最容易混淆的边界（对应 21.3 ②）：前面那张「msg2 进重试队列、msg3 先处理、3 秒后 msg2 重新投递」的时序图，画的是**并发消费**（`MessageListenerConcurrently` + `RECONSUME_LATER`）的行为。在**顺序消费**（`MessageListenerOrderly`）下，失败返回 `SUSPEND_CURRENT_QUEUE_A_MOMENT` 是**就地阻塞重试**：当前队列暂停，msg3 根本不会被拉取，不存在「插队」。两种失败语义要分开记：

监听器	失败返回	真实行为	对顺序的影响
`MessageListenerConcurrently`	`RECONSUME_LATER`	进 `%RETRY%`，后面的消息继续消费	**破坏顺序**（并发消费模式，见②）
`MessageListenerOrderly`	`SUSPEND_CURRENT_QUEUE_A_MOMENT`	就地重试，队列暂停	保序，但一条毒消息卡死整个队列
`MessageListenerOrderly`	达到重试上限	转死信（默认几乎无限重试，需自配熔断）	必须人工介入

所以顺序消费里真正要防的不是「插队」，是**阻塞**：消费逻辑必须快速成功或快速失败，重试超过 N 次转人工，别让一条毒消息把整个队列挂到天亮。

```
// 可复现 ③：顺序 listener 里开异步，顺序保证当场失效
public ConsumeOrderlyStatus consumeMessage(List<MessageExt> msgs,
ConsumeOrderlyContext ctx) {
    for (MessageExt m : msgs) {
        pool.submit(() -> handle(m));      // ✕ 消费顺序对，执行顺序乱
    }
    return ConsumeOrderlyStatus.SUCCESS;   // 而且可能还没处理完就提交了位点
}
```

【进阶】扩容队列为什么不可逆：取模的算术

路由是 `index = hash(key) % N`。把 N 从 4 改成 8，`hash % 8` 的结果中，与原来 `hash % 4` 同余的只占一半，另一半落到 `index + 4`——**约 50% 的业务键路由会变**：

orderId（示意 hash）	hash % 4	hash % 8	是否变化
1001	2	2	否（碰巧）
1002	2	6	**是**
1003	1	1	否
1004	0	4	**是**
1005	3	3	否
1006	2	6	**是**

一般的 `N → 2N` 变化率恒为 50%，所以「只扩一倍也一样崩一半」。不可逆有两个层面：一是**历史消息还压在老队列**，按老路由积压，新消息按新路由进新队列，同一业务键的两批消息被两个线程并行消费；二是 CommitLog 是追加写、队列只是逻辑索引，改队列数不改变已有消息的 `queueId`，**停机扩容也搬不走老队列里的积压**。

算一下窗口：扩容前 4 队列、每队列积压 3 万条共 12 万条；扩容后新消息进 8 个队列，老队列的 12 万条仍要按 `12万 / (4 × 单队列 TPS)` 慢慢排空，这段时间里新旧队列并行，就是顺序错乱的窗口。**扩容队列数之所以是「不可逆操作」，不是因为改不回来，是因为改回来也无法把已经错序消费掉的状态变回去。** 正确做法是新建 topic 加双写迁移，或让消费端用版本号兜底（21.5）。

三个「配对了还是乱」的高频细节——Broker 队列锁租约、`syncSendOrderly` 的内部行为、`mqadmin` 复现命令——集中放在章末延伸，避免打断主线。

21.4　全局有序：一个吞吐量陷阱

```
// ⚠ 全局有序：所有消息进 1 个队列
producer.send(msg);        // 不指定 selector → 轮询到不同队列，不是有序
// 真正的全局有序要建 1 个队列的 topic
```

单队列意味着：

- 吞吐量 = 单队列上限（大约几万 TPS，取决于消息大小）
- **无法通过加消费者扩容**
- 一条消息卡住，全站消息都卡住

绝大多数业务不需要全局有序，只需要"按业务键有序"。 订单按 `orderId`、库存按 `skuId`，各走各的队列，互不阻塞。

只有当"所有消息之间有因果关系"时（比如全局递增序列、单账户的账务流水），才需要全局有序——而这种场景往往该用数据库的序列/版本号，不该指望 MQ。

【必读】全局有序的吞吐上限：单分区 × 单线程

全局有序要建只有 1 个队列的 topic，于是吞吐被一条算式锁死：

```
吞吐上限 = 队列数 Q × (1 / 单条处理耗时 t) × 每队列线程数
顺序消费下每队列线程数恒为 1，所以：吞吐 = Q / t
```

Q（队列数）	t（单条耗时）	吞吐上限	加消费者有用吗
1	20ms	50 TPS	没用，队列锁让有效并发恒为 1
1	200ms（消息里带一次 DB 写）	5 TPS	没用
16	20ms	800 TPS	有用，每个队列一个线程

用 Little's law（公式与符号见第四章）推一下单队列的不收敛：到达率 λ = 100 TPS、服务率 μ = 50 TPS，队列以 `λ − μ = 50 条/秒` 增长，一小时堆 18 万条，系统数学上不收敛。**全局有序的性能问题不是「吞吐低」，是它把扩容这条路彻底焊死了**——CPU、机器、线程都加不进去，唯一能调的是把 t 变小。这也解释了为什么「绝大多数业务只需要按业务键有序」：把 Q 从 1 提到 16，吞吐是 16 倍，而且不同订单互不阻塞。

再叠加一条：全局有序下一条消息卡住，全站消息都卡住。**一个把「所有消息」绑在一根线上的设计，等于把全站可用性押在一条最慢的消息上。**

21.5　一个务实的替代方案：状态机 + 版本号

如果业务真的怕乱序，最稳的不是"保证顺序"，而是**让乱序不产生错误**：

```
-- ✓ 版本号递增，旧版本的消息直接丢弃
UPDATE order_info
SET status = #{newStatus}, version = version + 1
WHERE id = #{orderId} AND version < #{msgVersion};
```

`affected = 0` = 这条消息比当前状态还旧 → 直接跳过。

```
// 消息里带版本号（或时间戳）
if (affected == 0) {
    log.warn("stale message ignored, orderId={} msgVersion={}", orderId,
msgVersion);
    return SUCCESS;
}
```

这个方案的价值：不管 MQ 怎么乱、重试怎么插队、扩容怎么改路由，**旧状态永远覆盖不了新状态**。

代价：需要在业务表加版本字段，且业务逻辑要能接受"某些消息被丢弃"。但对订单状态这种场景，**它比"追求严格顺序"可靠得多**——因为严格顺序在任何一次运维操作（扩容、重平衡、重试）下都会破。

【进阶】状态机 + 版本号：完整实现与成本账

21.5 给了那条 `WHERE version < #{msgVersion}` 的 SQL，这里补成一套能落地的实现，含表结构、事务边界、并发与可运行测试。

```
CREATE TABLE order_state (
  order_id   BIGINT PRIMARY KEY,
  status     VARCHAR(32) NOT NULL,
  version    BIGINT      NOT NULL DEFAULT 0 COMMENT '单调递增的业务序号',
  updated_at DATETIME    NOT NULL,
  KEY idx_version (version)
) ENGINE=InnoDB;
```

```
@Service
@RequiredArgsConstructor
public class OrderStateService {
    private final OrderStateMapper mapper;

    @Transactional(rollbackFor = Exception.class)   // 事务边界：判定与更新一次提
交
    public void apply(OrderEvent e) {
        int n = mapper.compareAndSet(e.getOrderId(), e.getNewStatus(),
e.getVersion());
        if (n == 0) {
            // 必须分清「旧消息」与「订单不存在」，后者不能静默吞（第 19 章）
            Long cur = mapper.selectVersion(e.getOrderId());
            if (cur == null) {
                throw new BizException("订单不存在，转死信人工排查");
            }
            log.info("stale or duplicate, cur={} msg={} orderId={}",
                     cur, e.getVersion(), e.getOrderId());   // 幂等成功
        }
    }
}
```

```
-- ✓ 只有「消息版本 > 当前版本」才更新；乱序、重试、重放一律被挡
UPDATE order_state
SET status = #{newStatus}, version = #{msgVersion}, updated_at = NOW()
WHERE order_id = #{orderId} AND version < #{msgVersion};
```

版本号要来自业务序号（状态机第 N 步）或发号器，别用 `System.currentTimeMillis()`：时钟回拨和同毫秒并发都会让两笔事件拿到相同版本。并发下这条 `UPDATE` 走行锁，天然串行，不需要任何分布式锁。

```
@Test
void 乱序与重复投递_最终状态仍是最新() {
    service.apply(new OrderEvent(1001L, "PAID",    1));
    service.apply(new OrderEvent(1001L, "SHIPPED", 3));   // 先到
    service.apply(new OrderEvent(1001L, "PAID",    1));   // 迟到加重复
    service.apply(new OrderEvent(1001L, "SIGNED",  4));
    assertEquals("SIGNED", jdbc.queryForObject(
        "SELECT status FROM order_state WHERE order_id=1001", String.class));
    assertEquals(4L, (long) jdbc.queryForObject(
        "SELECT version FROM order_state WHERE order_id=1001", Long.class));
}
```

为什么它比追求顺序便宜，把账摊开看：

维度	严格顺序消息	状态机 + 版本号
队列数	固定，扩容即破序	随便扩，无路由约束
并发	单队列单线程，50 TPS	Q 队列并行，Q × 50 TPS
失败	一条毒消息卡死整队列	只影响自己，可任意重试
重试 / 重放	天然破坏顺序	版本号直接挡住
运维	扩容、rebalance 都破序	完全无感

16 队列 × 50 TPS = 800 TPS，是单队列的 16 倍，而且不需要队列锁、不需要固定路由、不需要停机扩容。**版本号不是「顺序消息的降级方案」，它是把顺序问题从 MQ 层挪到业务层**：MQ 只保证送达，正确性由数据库的条件更新保证——DB 比 MQ 更适合做这件事，因为它有事务、有行锁、有持久化的当前状态。

【必读】与第 19 章的分工：顺序管「不乱」，幂等管「不重」

版本号方案的好处是**一份代码同时干了两件事**：`version < cur` 既挡住了乱序的旧消息，也挡住了重复投递的消息。第 19 章讲幂等三板斧、第 18 章讲至少一次语义，都是这条链路的上游；第 20 章讲扩分区会打乱 key 映射，在这里被版本号彻底消化；第 22 章的库存扣减没有明确状态流转，靠的是「流水号 + 条件更新」，是同一思想换个形态。

本章结论对应 **RocketMQ 4.9+/5.x**（`ConsumeMessageOrderlyService`、`MessageQueueLock`、`RebalanceLockManager` 在 5.x 有重构）、**Kafka 3.x/4.x**（对应「单分区 + 单线程 + `max.in.flight.requests.per.connection`」）。顺序消费的默认最大重试次数、`transactionCheckMax`、Broker 队列锁的续期时长都需按版本验证；Kafka 4.0 的 KIP-848 新协议改变了 rebalance 的行为，但分区内有序性不受影响。

21.6　自检清单

- [ ] 顺序消息用的是 `syncSendOrderly` + `MessageListenerOrderly`（两端都配对）
- [ ] 路由 key 是**业务键**（订单 id），不是随机数或 `msgId`
- [ ] 顺序 topic 的队列数固定，**写进了变更禁令**（扩容会破序）
- [ ] 消费 listener 内没有异步/线程池
- [ ] 失败重试用 `SUSPEND_CURRENT_QUEUE_A_MOMENT`，且有"毒消息"熔断（重试 N 次转人工）
- [ ] 没有把"跨 topic 的顺序"当成 MQ 能解决的问题
- [ ] 状态类更新有版本号/时间戳兜底，旧消息不会覆盖新状态

21.7　一句话总结

> 顺序消息的真相是：**它只是一个"尽量"的保证，任何扩容、重试、重平衡都会破它。** 所以真正可靠的做法，是让"乱序"不产生错误——版本号比顺序便宜得多。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/06-MQ消息坑、07-消息队列/顺序消息、07-消息队列/RocketMQ
- 【选读】**Broker 队列锁是有租约的**：`RebalanceLockManager` 给每个 `MessageQueue` 的锁带过期时间，消费者侧由 `RebalanceService` 定时 `lockAll` 续期。Full GC 或网络抖动导致续期停摆时，锁过期后别的实例抢走队列，两边短暂都认为自己是 owner，重放一段消息。顺序 topic 要配静态成员或优雅停机，滚动发布尽量少触发重平衡。
- 【选读】`syncSendOrderly` **内部**：Spring 的 `RocketMQTemplate.syncSendOrderly` 用 `SelectMessageQueueByHash`（对 `hashKey.hashCode()` 取模选队列），和手写的 `MessageQueueSelector` 是一回事——不提供额外保证，只是少写几行。Kafka 侧没有对应物，保序靠「同一 key 进同一分区 + `max.in.flight.requests.per.connection`」：开 `enable.idempotence=true` 后该值需不大于 5 才保证重试不乱序（按版本验证）；没开幂等又配 `retries>0`，一次重试就可能插队。
- 【选读】**复现命令**：用 `mqadmin queryMsgByKey` 看同一业务键落在哪些 `queueId`（扩容前后各跑一次），用 `mqadmin consumerProgress` 看每个队列消费进度，确认是否有队列被卡住。配合 21.3 的破坏者对照表使用。

第二十二章　秒杀：库存扣减的四种方案

本章要回答：1000 件库存，10 万人抢，怎么保证不超卖、不少卖、不被打挂？

本章你能带走

- 四种扣减方案的完整对比（从行锁到分段令牌）
- 一个最容易漏的问题：**少卖比超卖更难发现**
- 一条底线：**所有方案都必须有** `AND quantity >= 1`

难度构成：本章主线【必读】；含【进阶】深挖 3 处；章末「延伸」小节放支线深挖与知识库指针。---

22.1　现场：不超卖了，但少了 37 件

第一版用数据库行锁，超卖 300 件（第六章那个故事）。改完之后不超卖了，但活动结束发现：**库存剩 37 件，却再也卖不出去了。**

日志显示：这 37 次请求，Redis 预扣成功，但后续创建订单失败（数据库连接超时），库存没有回补。

超卖是"卖多了"，看得到（库存变负）；少卖是"卖少了"，看不到（库存还有，但没人能买）。

> 少卖比超卖更隐蔽：它不告警、不报错，只体现在"活动明明很火，但没卖完"。
> 而这对商家是实打实的损失。

当时怎么误判的：第一反应是"用户刷单或重复点击把库存锁住了，只是没生成订单"——翻订单表，那 37 个被锁的 SKU 对应不上一笔订单，排除；又怀疑"是不是有人手滑改了 DB 库存、甚至已经超卖"，拉三方对账发现库存是正的、没有负数，也排除。中间走了一条死路：怀疑"Redis 库存 key 被设了 TTL 自动过期清掉"，查 key 配置压根没设 TTL，白白耗了半小时看持久化。真正把方向扭回来的是连接池监控里那 37 次下单请求的获取连接超时日志，时间戳和"少卖"批次一一对上。教训很直接：少卖不会自己告警，它的第一现场永远是创建订单的异常日志和三方对账差集，不是库存面板上的那个数字。

22.2　四种方案

【必读】方案一：数据库行锁（悲观锁）

```
-- 一个事务内完成扣减
UPDATE stock SET quantity = quantity - 1
WHERE sku_id = 1001 AND quantity >= 1;
-- affected = 0 → 库存不足
```

配 `@Transactional`，靠 InnoDB 行锁串行化同一行的更新。

维度	表现
正确性	高（`AND quantity >= 1` 是关键）
吞吐	**低**（同一行串行，约几百 TPS）
适用	小规模活动（< 1000 QPS）

优点：最简单、最不容易出错、不需要额外组件。**缺点**：热点行锁 + 事务持有时间长，1000 QPS 就开始排队；而且行锁会连带影响其他操作（第七章）。

【必读】方案二：乐观锁（版本号）

```
UPDATE stock
SET quantity = quantity - 1, version = version + 1
WHERE sku_id = 1001 AND version = #{oldVersion} AND quantity >= 1;
```

`affected = 0` → 版本冲突或库存不足 → 重试或失败。

维度	表现
正确性	高
吞吐	中（冲突率高时退化为大量重试）
适用	冲突不激烈的场景

问题：秒杀场景冲突率极高（10 万人抢 1000 件），几乎每次都要重试。乐观锁在这种场景下**比悲观锁更慢**——因为它把"排队"变成了"失败重试"。

乐观锁不适合秒杀，适合"偶尔冲突"的库存场景（比如日常补货、后台改价）。

【必读】方案三：Redis 预扣 + 异步落库（主流）

思路：**把扣减的压力从数据库挪到 Redis，数据库只做最终落库。**

```
-- deduct.lua：Lua 保证原子性
local stock = redis.call('GET', KEYS[1])
if stock == false then return -1 end          -- key 不存在
if tonumber(stock) < tonumber(ARGV[1]) then
    return 0                                   -- 库存不足
end
redis.call('DECRBY', KEYS[1], ARGV[1])
return 1                                       -- 扣减成功
```

```
Long r = redis.execute(DEDUCT_SCRIPT, List.of("stock:" + skuId),
List.of(String.valueOf(qty)));
if (r == 0) throw new BizException("已售罄");
// 扣减成功 → 发 MQ 异步创建订单 + 落库
mqProducer.send("seckill-order", new SeckillMsg(userId, skuId));
```

维度	表现
正确性	高（Lua 原子）
吞吐	**高**（Redis 单节点 8–10 万 QPS）
适用	大规模秒杀

必须解决的两个问题：

① 少卖（前面那个故事）

Redis 扣了，但后续创建订单失败 → 库存没回补。

```
// ✓ 消费失败要回补，或至少记录待核对
catch (Exception e) {
    redis.incrBy("stock:" + skuId, qty);       // 回补
    log.error("order create failed, stock rolled back. userId={}", userId, e);
    throw e;
}
```

回补本身也可能失败（Redis 挂了），所以还要有**对账**：`Redis 剩余库存 + 已创建订单数 = 初始库存`。

② Redis 与 DB 的最终一致

数据库库存是"权威值"，Redis 是"预扣值"。活动结束后要把两边对齐：

```
-- 活动结束，用真实订单数校正 DB
UPDATE stock s SET quantity = s.total - (SELECT COUNT(*) FROM order_info WHERE
sku_id = s.sku_id);
```

这一步不能省——没有它，Redis 漂移会一直累积。

【必读】方案四：分段库存 / 本地令牌（极致性能）

把 1000 件库存拆成 10 份，每份 100 件，分配到 10 个应用实例的本地内存：

```
// 每个实例持有 100 个令牌，本地 AtomicInteger 扣减
if (localStock.decrementAndGet() < 0) {
    localStock.incrementAndGet();              // 回退
    throw new BizException("已售罄");
}
// 扣减成功 → MQ 异步下单
```

维度	表现
正确性	中（需要处理"实例分配不均"导致的部分售罄）
吞吐	**极高**（本地操作，无网络往返）
适用	极端峰值（10 万+ QPS）、可接受"提前售罄"

代价：如果某个实例分到 100 个令牌但它没流量，那 100 件就卖不出去（**少卖**）。要解决就得做"令牌回收/再分配"，复杂度陡增。

这个方案只适合"卖完就行、不要求精确卖完"的场景。 大多数业务不需要。

【进阶】四种方案的量化对照：吞吐上限怎么算

22.3 的对比表给了定性结论，这里把每一种的吞吐上限写成算式，才知道「几百 TPS」「万级 TPS」这些词是从哪来的。

方案	吞吐上限算式	代入后的值	一致性	少卖风险	适用
数据库行锁	`1 / 事务持有时间`	5ms → **200 TPS**	强	低	小活动
乐观锁	`1 / (重试轮次 × 单轮耗时)`	冲突率 99% 时低于行锁	强	低	冲突不激烈
Redis 预扣 + 异步	单节点 `EVAL` 吞吐	约 5~8 万 QPS（需压测）	最终一致	中，需回补	主流
分段令牌	实例数 × 本地原子扣减	无网络往返，百万级	弱	高	极端峰值

① 行锁串行上限。 `UPDATE` 走聚簇索引行锁，同一 SKU 的所有事务必须串行。设事务包含 `SELECT`、条件 `UPDATE`、日志插入、提交，共约 5ms，上限就是 `1 / 0.005 ≈ 200 TPS`。秒杀开场 1 秒到达 5 万请求，排队要 250 秒，必然超时。这正是第 7 章那条链子：**事务里多一个外部调用，持锁时间被拉长一个数量级，吞吐就掉一个数量级**；掺一次 100ms 的 RPC，上限直接从 200 掉到 10 TPS。

② 乐观锁冲突率随并发的变化。 N 个并发同时读到同一个 `version`，只有一个 `UPDATE` 能成功，冲突率 `p = 1 - 1/N`：

并发 N	冲突率 p	每轮成功数	期望重试轮次
2	50%	1	2
10	90%	1	10
100	99%	1	100
1000	99.9%	1	1000

10 万人抢 1000 件时 p 约 99.9%。**乐观锁在秒杀下不是「无锁」，是把排队换成了失败重试**：失败的事务同样占过连接和行锁，还多付了回滚成本，所以比悲观锁更慢。22.2 方案二的结论（乐观锁在秒杀下不是无锁，是把排队换成失败重试），数字版就在这张表里。

③ Redis 预扣的原子性，必须交给一段 Lua。 限购判断与扣减若拆成「查一次、加一次」两步，并发窗口能让同一用户抢到两件：

```
-- KEYS[1]=库存 key  KEYS[2]=已抢用户集合  ARGV[1]=userId  ARGV[2]=requestId
local stock = redis.call('GET', KEYS[1])
if stock == false then return -1 end                          -- 未初始化：明确报错，别当 0 也别当有库存
if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then
    return -2                                                 -- 限购：一人一件
end
if tonumber(stock) < 1 then return 0 end
redis.call('DECR', KEYS[1])
redis.call('SADD', KEYS[2], ARGV[1])                          -- 判断与扣减同在一个 Lua 才原子
redis.call('SET', 'seckill:req:' .. ARGV[2], '1', 'EX', 86400)  -- 幂等键留给下游消费
return 1
```

```
// ✗ 先 GET 再 DECR：两步之间有并发窗口，同一用户可抢两件、库存可扣成负数
String stock = redis.opsForValue().get(key);
if (Integer.parseInt(stock) > 0) redis.opsForValue().decrement(key);
// ✓ 一次 EVAL 完成判定 + 扣减 + 限购 + 幂等占位
Long r = redis.execute(DEDUCT_LUA, List.of(key, usersKey), userId, requestId);
```

单个 Redis 节点跑 `EVAL` 大约 5~8 万 QPS（需在自己的版本与机型上压测），但所有请求打的是**同一个库存 key**，单 key 上限就是整场秒杀的上限——这就是第 16 章的热 key 问题，也是下一段要分段的理由。

【进阶】分段库存的实现与热点问题

把 1000 件拆到 10 个 key，每个 100 件，请求按用户 ID 选段、本段售罄就试下一段：

```
private static final int SEGMENTS = 10;
public boolean deduct(Long skuId, Long userId) {
    int start = (int) Math.floorMod(userId, SEGMENTS);
    for (int i = 0; i < SEGMENTS; i++) {
        int seg = (start + i) % SEGMENTS;                  // 本段售罄就试下一段
        String key = "seckill:stock:" + skuId + ":seg:" + seg;
        Long r = redis.execute(SEG_LUA, List.of(key, "seckill:users:" + skuId),
                               String.valueOf(userId));
        if (r != null && r == 1L) return true;
        if (r != null && r == -2L) return false;           // 已抢过，直接失败
    }
    return false;                                          // 所有段都空
}
```

问题	表现	处理
单 key 热 key	全部流量打一个 Redis 分片，CPU 与网卡打满	分段、本地缓存合并、集群多 key 打散（第 16 章）
分段不均	某段没流量，100 件卖不出去（少卖）	随机或轮询选段、段间回收再分配、剩余段归拢
段间回收复杂度	回收要原子、要防重复、要对账	只在「卖完就行」场景用，精确卖完别用
实例本地令牌	实例重启丢令牌	中心下发加定期核对（见 22.5 对账）

分段库存降低的是单 key 热点，不是总吞吐：10 个段若都落在同一个 Redis 分片上，热点只是从 1 个 key 变成 10 个 key，分片网卡仍是瓶颈。要真正摊开，得让 key 分布到不同分片（第 16 章）。

【必读】超时未支付回补：一个必须幂等的状态机

秒杀订单通常 30 分钟未支付就关单，把 Redis 与 DB 库存回补回去。**回补不幂等，比不下单还危险**：同一个订单被关单任务扫到两次，库存回补两遍，就凭空多出一件可卖——这是「少卖」的反面，直接变成超卖。

```
CREATE TABLE seckill_refund (
  order_id    BIGINT PRIMARY KEY,          -- 主键即判重：一个订单最多回补一次
  sku_id      BIGINT NOT NULL,
  status      VARCHAR(16) NOT NULL,        -- 待支付 → 已关闭，单向
  created_at  DATETIME NOT NULL
) ENGINE=InnoDB;
```

```
@Transactional(rollbackFor = Exception.class)
public void closeExpiredOrder(Long orderId) {
    // ① 状态机单向流转，重复关单影响行数为 0
    int n = orderMapper.closeIfUnpaid(orderId);   // WHERE status='UNPAID'
    if (n == 0) return;                           // 已关闭或已支付，幂等跳过
    // ② 回补流水，主键冲突即已回补过（第 19 章）
    try {
        refundMapper.insert(new SeckillRefund(orderId, skuId, "CLOSED"));
    } catch (DuplicateKeyException e) {
        return;                                   // 绝不重复回补
    }
    // ③ 同一事务内回补 DB 库存；Redis 回补放事务外，失败由对账兜底
    stockMapper.incrQuantity(skuId, 1);
    redis.opsForValue().increment("seckill:stock:" + skuId);
}
```

失败点	后果	兜底
关单成功、回补流水没插上	库存少回补（少卖）	关单流水与回补流水同一事务，失败整体回滚
DB 回补成功、Redis 回补失败	Redis 少于 DB	22.5 三方对账发现差值
定时任务重复扫到同一订单	重复回补	唯一键加状态机双重判重

状态机必须**单向**（`UNPAID → CLOSED`），不能出现 `CLOSED → UNPAID`，否则支付回调迟到时会把已关闭的订单再打开一次，库存又被占用。这与第 21 章「旧状态不能覆盖新状态」是同一条规矩，和第 19 章的幂等也是同一套手法。

22.3　四种方案对比

方案	吞吐	正确性	少卖风险	复杂度	适用
数据库行锁	几百 TPS	高	低	低	小活动
乐观锁	中（高冲突时差）	高	低	低	冲突不激烈
Redis 预扣 + 异步	**万级 TPS**	高	**中（需回补）**	中	**主流方案**
分段令牌	十万级 TPS	中	**高**	高	极端峰值

默认选第三种。 前两种用于小规模，第四种只在"现有方案扛不住"时才上。

22.4　无论哪种方案，这三条不能少

【必读】① `AND quantity >= 1`

```
-- ✗ 会扣成负数
UPDATE stock SET quantity = quantity - 1 WHERE sku_id = 1001;

-- ✓
UPDATE stock SET quantity = quantity - 1 WHERE sku_id = 1001 AND quantity >= 1;
```

这是唯一能防住"代码逻辑漏洞导致负库存"的防线。 哪怕上层判断写错了，数据库也不会让你扣穿。

【必读】② 限流 + 防刷

秒杀的第一道防线不是库存，是**入口**：

```
① 前端：按钮置灰 + 验证码（挡脚本）
② 网关：按用户 ID 限流（一人一秒一次）+ 按 IP 限流
③ 应用：同一用户重复请求直接拒绝（幂等键）
④ 库存层：才开始真正扣减
```

90% 的秒杀流量应该死在第一、二道，而不是打到库存。 如果库存层承受了全量流量，说明前面的防线形同虚设（第五章）。

【必读】③ 一个用户只能买一件

```
-- 唯一约束兜底
ALTER TABLE seckill_order ADD UNIQUE KEY uk_user_sku (user_id, sku_id);
```

配合 Redis 的 `SETNX seckill:user:{userId}:{skuId}` 做前置拦截。**没有这个约束，"一个人用脚本买 100 件"就是必然。**

22.5　对账：少卖问题的唯一解法

秒杀结束后的 5 分钟，必须做一次三方对账：

```
Redis 剩余库存  +  已支付订单数  =  初始库存?
DB 剩余库存     +  已支付订单数  =  初始库存?
Redis 扣减次数  =  订单创建数 + 回补数?
```

三个等式，任何一个不等，就要**人工介入**：

- Redis 剩余 > DB 剩余 → 有预扣没落库 → **少卖**
- Redis 剩余 < DB 剩余 → 有落库没预扣 → **超卖风险**

```
// 对账任务骨架
@Scheduled(cron = "0 */5 * * * ?")
public void reconcile() {
    for (Long skuId : activeSkus()) {
        long redisLeft = redis.get("stock:" + skuId);
        long dbPaid    = orderMapper.countPaid(skuId);
        long expect    = stockMapper.total(skuId);
        if (redisLeft + dbPaid != expect) {
            alert.send("seckill stock mismatch sku=%d redis=%d paid=%d
expect=%d",
                       skuId, redisLeft, dbPaid, expect);
        }
    }
}
```

【必读】少卖只有对账能发现：差值来源清单

超卖会告警，少卖不会，原因有四条：库存是正数（没有负库存告警）；没有异常、没有超时（下单失败被 catch 后走了回补，或回补也失败但没告警）；监控看的是请求成功率，而少卖的请求根本没进来（用户只看到「已售罄」）；活动「很火但没卖完」，运营会以为是需求不足。**只有把三个系统的账摆在一起相减，差值才会显形。**

```
-- 三方对账：初始库存 / DB 剩余 / Redis 剩余 / 订单占用，按 SKU 逐行算出差值
SELECT s.sku_id,
       s.total                                    AS 初始库存,
       s.quantity                                 AS DB剩余,
       COALESCE(r.redis_left, 0)                  AS Redis剩余,
       COALESCE(o.created_cnt, 0)                 AS 已创建订单,
       COALESCE(o.paid_cnt, 0)                    AS 已支付订单,
       s.total - s.quantity - COALESCE(o.created_cnt, 0)          AS DB差值,
       s.total - COALESCE(r.redis_left,0) - COALESCE(o.created_cnt,0) AS Redis差
值
FROM stock s
LEFT JOIN redis_stock_snapshot r ON r.sku_id = s.sku_id
LEFT JOIN (
    SELECT sku_id,
           SUM(CASE WHEN status = 'PAID' THEN 1 ELSE 0 END) AS paid_cnt,
           COUNT(*) AS created_cnt
    FROM seckill_order
    GROUP BY sku_id
) o ON o.sku_id = s.sku_id
WHERE s.activity_id = #{activityId};
-- 任一差值 != 0 → 告警并列出 sku_id 清单，而不是只报「差 37 件」
```

`redis_stock_snapshot` 是活动结束**冻结**的 Redis 剩余值快照——对账期间 Redis 可能还在变，不冻结就会拿一个移动的目标去减。差值来源清单，按方向归类：

差值方向或现象	含义	可能来源
Redis 剩余 > DB 剩余	预扣没落库	下单失败未回补、消息丢失、消费失败进 DLQ、回补时 Redis 挂了
Redis 剩余 < DB 剩余	落库没预扣	人工改 DB 库存、补偿脚本多扣、重复回补
差值非零但订单数正常	口径错	用「已支付」当「已创建」、退货未回补、超时关单未回补
分段库存差值	段间不均	某段无流量、段未回收、实例重启丢令牌
三方都对但卖不完	入口误拦	网关限流误杀、一人一件约束过严、验证码失败率高

少卖的对账必须能列出 SKU 清单：只报「差 37 件」无法定位，价值有限（第 18 章 18.4 是同一个要求）。能定位到具体 SKU，才能决定是回补库存、重放消息，还是人工改账。

【必读】一处必须挑明的口径错误：不能用「已支付订单数」对账

22.5 的对账骨架里用 `orderMapper.countPaid(skuId)` 与初始库存相减，这在秒杀里是错的：预扣成功但用户尚未支付的订单（30 分钟超时关单之前），既不算「已支付」，库存却已被占用。用 `countPaid` 会把这部分全部误判成少卖，凭空造出一大堆假差异，对账反而成了噪音源。

正确口径要分三个状态：

```
初始库存 = DB 剩余 + 已创建订单（占用中，含未支付） - 已回补的关单数 - 重复回补
```

「占用」看 `seckill_order` 全量 `COUNT(*)`，「最终卖出」才看 `PAID`；对账要发现的是**占用层面的漂移**，所以主口径用 `created_cnt`，`paid_cnt` 只用于核对收入。这和第 14 章的增量对账是同一条原则：**账面不该跟「Σ 全量流水」比，该跟「最近一条流水的 qty_after」比**——有期初库存却没有期初流水的库，用全量求和会报假差异。

```
-- ✓ 增量对账：账面 vs 最近一条流水，避免期初库存无流水造成的假差异
SELECT * FROM (
  SELECT i.sku_id, i.quantity AS 账面,
         (SELECT t.qty_after FROM stock_txn t
           WHERE t.sku_id = i.sku_id ORDER BY t.id DESC LIMIT 1) AS 流水值
  FROM stock i
) x
WHERE 流水值 IS NOT NULL AND ABS(账面 - 流水值) > 0;
```

对账报出来的差异，一半以上是口径错而不是数据错。 先把口径写进设计文档，再写 SQL；否则每次告警都要先花半小时证明「这次又是误报」。

【进阶】与第 6、14、16 章的接口，以及一次能跑的并发验收

四条交叉引用，把这一章挂回全书：第 6 章讲过「无护栏 `UPDATE` + 事务没开」会扣出负库存，本章的 `AND quantity >= 1` 是哪怕上层全错也扣不穿的最后防线；第 14 章讲行锁、锁等待与死锁，行锁方案的 200 TPS 天花板就是那条 `innodb_lock_wait_timeout` 队列，增量对账口径也来自第 14 章；第 16 章讲热 key 打散与本地缓存，分段库存本质是热 key 治理；第 18 章讲本地消息表，预扣成功到落单之间的消息一旦丢失就是少卖，所以这一段不能只 `send`。

一次能跑的并发验收，把「不超卖」变成断言而不是口头结论：

```
@Test
void 秒杀并发验收() throws Exception {
    int N = 150, STOCK = 100;
    initStock(STOCK);
    List<Future<Result>> fs = new ArrayList<>();
    for (int i = 0; i < N; i++) {
        fs.add(pool.submit(() -> seckillService.seckill(SKU, i)));
    }
    long ok = fs.stream().filter(f -> f.get().isSuccess()).count();
    assertEquals(STOCK, ok);                                    // 成功恰好 100
    assertEquals(0, stockMapper.quantity(SKU));                 // 库存恰好扣完
    assertEquals(STOCK, orderMapper.countBySku(SKU));           // 一条订单对应一
件
    // 还要断言：被拒的 50 个请求原因都是「库存不足」，
    // 不能是死锁、连接超时或 500，否则压测过了线上照样出事
}
```

本章结论对应 **MySQL 8.0（InnoDB 行锁、`FOR UPDATE SKIP LOCKED`）、Redis 7（Lua `EVAL`、`SET NX EX`）、RocketMQ 4.9+/5.x、Spring Boot 3.x**。`innodb_lock_wait_timeout` 默认 50 秒、RocketMQ 重试次数与延迟级别都需按版本验证；Redis 单节点 `EVAL` 的 QPS 与热点分片分布，务必用自己的机型压测后再写进容量规划。

把 22.5 的对账落成能跑的任务，注意它只**报警、不自动改账**：

```
@Scheduled(cron = "0 */5 * * * ?")
public void reconcile() {
    for (Sku sku : activeSkus()) {
        long redisLeft = redisStockSnapshot.get(sku.id());   // 冻结快照
        long created   = orderMapper.countCreated(sku.id()); // 占用口径，不是
countPaid
        long dbLeft    = stockMapper.quantity(sku.id());
        long diff      = sku.total() - dbLeft - created;
        if (diff != 0) {
            alert.send("seckill diff sku=%d diff=%d redisLeft=%d dbLeft=%d
created=%d",
                       sku.id(), diff, redisLeft, dbLeft, created);
            // ✗ 禁止按 diff 自动 UPDATE stock：口径错时会把对的账改错
            // ✓ 人工确认差值来源清单后再决定回补或改账
        }
    }
}
```

「禁止自动按 diff 改库存」是这一节最该记住的一条：22.5 那张差值来源清单里，一半以上的差异来自口径错（用 `countPaid`、退货未回补、重复回补），自动回补会把正确的数据改成错的，而且改完对账就「平」了，错误被永久掩埋。对账任务的价值是**列出 SKU 清单**，不是替人做决定。

还有两个边界：`AND quantity >= #{qty}` 里的 `qty` 必须校验大于 0，传 0 或负数时条件恒真，护栏形同虚设；对账周期 5 分钟意味着最坏有「5 分钟乘漂移速率」的窗口，按每秒 1 笔漂移算就是 300 笔，所以和钱相关的链路要做 T+0 实时核对，不能只靠 5 分钟一次的批任务。

22.6　自检清单

- [ ] 所有扣减 SQL 都带 `AND quantity >= 1`
- [ ] Redis 扣减用 Lua 脚本（`GET` + `DECRBY` 必须原子）
- [ ] 下单失败有库存回补，回补失败有对账
- [ ] 有用户维度的唯一约束（一人一件）
- [ ] 网关有限流 + 防刷，且按用户维度
- [ ] 活动结束有 Redis/DB/订单三方对账
- [ ] 库存 key 有兜底（key 不存在时不能当成"有库存"或"无库存"，要明确定义）
- [ ] 压测过库存层的实际 TPS 上限

22.7　一句话总结

> 秒杀的技术难点不是"扣得快"，是**"扣了不丢、没扣不放"**。 超卖会被立刻发现，少卖要等对账——所以对账不是收尾工作，是方案的一部分。

【选读】延伸

- 知识库对应原理篇：14-业务场景与案例/秒杀系统设计、17-常见Bug与解决方案/05-Redis与缓存坑

第二十三章　CPU 100%：二十分钟内要拿到的三份证据

本章要回答：CPU 打满，除了重启，还能做什么？

本章你能带走

- 一套 20 分钟内的标准动作（含具体命令）
- 一个最容易浪费半小时的细节：**十进制 TID 转十六进制**
- 一条原则：**重启一次，销毁全部证据**

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

23.1　现场：先别重启

凌晨三点，告警：某服务 CPU 100%，接口 P99 从 80ms 涨到 12s。

值班同学的第一反应通常是"重启一下试试"。**这一步一旦做了，你今晚就查不出原因了。**

CPU 100% 的原因至少有六种，需要完全不同的处理：

原因	特征	处理
死循环 / 空转	单线程 100%，栈在一个方法里	改代码
正则回溯	栈在 `Pattern` / `Matcher`	改进则、限输入长度
GC 疯狂运行	多个 `GC Thread` 各占 20%+	查内存（第 24 章）
JIT 编译	`C2 CompilerThread`	通常自己会好
序列化/加解密	栈在 `JSON` / `Cipher`	优化算法、限流
大量线程上下文切换	线程数上千，`cs` 很高	查线程池配置

你不抓证据，就只能靠猜。 而"猜着改"的成本，是下次再炸一次。

【必读】重启一次会永久销毁的那几样东西

"先别重启"不是一句口号，它对应一份可以逐项打勾的清单。CPU 100% 的现场有一个残酷特点：**证据全部在内存里，而且只在进程还活着的这几分钟里存在**。下面四样，重启后不会有任何一份能从别处补回来。

证据	它回答的问题	为什么重启就永久没了	抓取动作
高 CPU 线程的 nid 与栈	到底是哪一行代码在烧	线程随进程销毁，`top -Hp` 的 TID 与 `jstack` 的 nid 对应关系从此不存在	`top -Hp` + `printf "%x"` + 连抓三次栈
三次采样都命中的栈帧	热点是稳定的还是偶然的	需要连续的现场，重启后无法倒带重放	每 5 秒一次，共三次
`VM.flags` / `jstat` 采样	当时实际生效的参数与 GC 趋势	重启会重新读启动脚本，参数可能已被改过	`jcmd VM.flags`、`jstat -gcutil 1000 10`
未 dump 的堆内对象分布	是不是对象分配把 GC 线程喂饱了	堆内容只在这一刻存在，事后无法重建	`jcmd GC.class_histogram`（不触发 Full GC）

第四行最容易被漏掉：如果最后查明是 GC 线程在烧 CPU，方向就转到第 24 章，而那时堆里"谁占得最多"的原始分布已经没了，只能等下一次复现——按现场那次泄漏的速度，可能要等三天。**多数人以为重启损失的是"时间"，实际损失的是"因果链"**：你能看到 CPU 高，却再也补不出"为什么高"。

这里有个反常识点值得单说：**CPU 100% 的时候，最不缺的是 CPU，最缺的是时间。** 现场每一秒都在产生新的请求堆积，但没有一秒能替你把刚才那几行栈补回来。所以取证动作要按"能不能事后补"排序：能补的（GC 日志、监控曲线、访问日志）往后放，不能补的（nid 对应栈、三次采样、堆直方图）必须最先做。

> 第 26 章会讲同一条原则的另一面：压测报告里"最大 QPS"是最没意义的数字，因为它只记录了系统"跑到了多少"，没记录"在什么条件下开始不对"。现场取证也一样——只记"CPU 100%"没有用，要记"哪个 nid、哪行栈、连续三次"。

这里有个绕不开的冲突：**业务在大量报错时，止血和取证抢同一台机器**。处理顺序不能靠临场发挥，写死成三步：

步骤	对哪台实例做	目的
① 摘一台，保留现场	从 LB 摘掉**一台**，不重启、不切流	这台专门用来敲 `diag.sh`，是唯一完整的证据源
② 其余实例扩容 / 限流 / 关非核心	剩下的 N−1 台	恢复业务，允许重启、允许回滚
③ 全量重启	最后一步，取证完成后	消除残留状态

23.5 说"先摘一台保现场，再处理其余实例"，补一条判据：**摘掉的那一台要停止收流量，但进程必须还活着**。只摘流量不杀进程，才有 `jstack` 可抓。很多人把"摘流量"做成 `kubectl rollout restart`，等于第一步就把证据源销毁了。**多数人以为"止血优先"意味着"先重启"，实际是先隔离一台、把止血和取证分开到不同实例上**，这两件事根本不冲突。

当时怎么误判的：第一反应是"流量涨了，机器容量不够被打满"——拉入口 QPS 监控，曲线平稳、没有突增，排除；又怀疑"是不是 GC 在疯狂回收把 CPU 喂饱了"，翻 GC 日志发现 GC 时间占比并不高，也排除。中间走了一条弯路：盯着 `top` 里偶尔闪过的 `C2 CompilerThread`，以为"是 JIT 编译在烧 CPU"，反复等了几分钟想看它自己好，结果那条线程占比从没超过 5%，纯属无用线索。真正扭转方向的是 `top -Hp` 抓到的那个稳定 99% 的 nid，转成十六进制再 `jstack`，栈顶赫然是 `Pattern$Loop`——正则回溯。教训就一条：先 `top -Hp` 定 nid、再 `jstack` 看那个 nid，顺序一反，你看到的全是被它连累的噪声线程。

23.2　三份证据

【必读】第一份：进程与线程快照

```
# ① 找进程（排除掉其他 java 进程）
top -c
# 记下 PID，看 %CPU 和 RES

# ② 找线程（-H 表示显示线程）
top -Hp <pid>
# 记下占 CPU 最高的线程 TID（十进制！）
```

假设看到：

```
  PID USER   %CPU COMMAND
 8842 app    99.3 java

  TID   %CPU
 8891   98.7      ← 就是这个
 8892    0.3
```

【必读】第二份：线程栈（关键在进制转换）

```
# ③ 十进制 TID → 十六进制（这一步错了就白查半小时）
printf "%x\n" 8891
# 输出 22bb

# ④ 抓线程栈，找 nid=0x22bb
jstack <pid> > /tmp/jstack-$(date +%s).txt
grep -A 30 "nid=0x22bb" /tmp/jstack-*.txt
```

结果可能长这样：

```
"http-nio-8080-exec-23" #23 daemon prio=5 os_prio=0 tid=0x... nid=0x22bb
runnable
   java.lang.Thread.State: RUNNABLE
        at java.util.regex.Pattern$Loop.match(Pattern.java:4785)
        at java.util.regex.Pattern$GroupHead.match(Pattern.java:4658)
        at com.demo.validator.checkEmail(Validator.java:42)
```

`RUNNABLE` + 业务方法在栈顶 → **业务代码在烧 CPU**。看到 `Pattern$Loop` 就是正则回溯（灾难性回溯，ReDoS）。

如果看到的是：

```
"GC Thread#3" ... nid=0x... runnable
```

那就是 GC 在烧，方向转到第 24 章查内存。

注意：一次 `jstack` 只能看到一瞬间。**连抓三次，间隔 5 秒**，如果三次都在同一个方法里，才能确认是那里。

```
for i in 1 2 3; do jstack <pid> > /tmp/js-$i.txt; sleep 5; done
# 三次都命中的栈帧 = 真正的热点
grep -h "at com.demo" /tmp/js-*.txt | sort | uniq -c | sort -rn | head
```

【必读】第三份：现场的完整性

```
# ⑤ 保存 JVM 参数（重启后就变了）
jcmd <pid> VM.flags > /tmp/flags.txt

# ⑥ 保存 GC 情况
jstat -gcutil <pid> 1000 10 > /tmp/gc.txt

# ⑦ 线程总数与状态分布
jcmd <pid> Thread.print | grep "java.lang.Thread.State" | sort | uniq -c
```

三份证据凑齐，基本能定性。**整个过程不超过二十分钟**，比"重启再观察"快得多，而且能留下根因。

【必读】把七步串成一条能整段复制的命令

上面三份证据是分开讲的，真到凌晨三点，人不会记得顺序。把它固化成一个脚本，事故时只敲一行。下面这段可以直接存成镜像里的 `/opt/diag/cpu.sh`，用 `bash /opt/diag/cpu.sh <pid>` 执行。

```
#!/usr/bin/env bash
# CPU 100% 现场取证：一次拿齐三份证据，输出全部带时间戳
set -euo pipefail
PID=$1
OUT=/data/incident/cpu-$(date +%Y%m%d-%H%M%S)
mkdir -p "$OUT"

# ① 线程级 CPU 快照（十进制 TID）
top -b -H -n 1 -p "$PID" > "$OUT/top-h.txt"

# ② 取最热线程，转十六进制
TID=$(awk 'NR>7 {print $1, $9}' "$OUT/top-h.txt" | sort -k2 -nr | head -1 | awk
'{print $1}')
HEX=$(printf '%x' "$TID")
echo "pid=$PID tid=$TID nid=0x$HEX" | tee "$OUT/hot.txt"

# ③ 连抓三次线程栈，间隔 5 秒
for i in 1 2 3; do
  jcmd "$PID" Thread.print -l > "$OUT/thread-$i.txt"
  sleep 5
done

# ④ 三次都命中的业务栈帧（排除 JDK 内部帧）
grep -h 'at com\.demo' "$OUT"/thread-*.txt \
  | sed 's/^[[:space:]]*//' | sort | uniq -c | sort -rn | head -10 > "$OUT/hot-
frames.txt"

# ⑤ 实际生效参数、GC 趋势、类直方图
jcmd "$PID" VM.flags            > "$OUT/flags.txt"
jstat -gcutil "$PID" 1000 10    > "$OUT/gcutil.txt"
jcmd "$PID" GC.class_histogram  > "$OUT/histo.txt"
```

一次真实执行的输出长这样（节选，数字来自一次 `^(a+)+$` 校验接口的现场）：

```
$ cat /data/incident/cpu-.../hot.txt
pid=8842 tid=8891 nid=0x22bb

$ cat /data/incident/cpu-.../hot-frames.txt
   3 at com.demo.validator.Validator.checkEmail(Validator.java:42)
   3 at java.util.regex.Pattern$Loop.match(Pattern.java:4785)
   3 at java.util.regex.Pattern$GroupHead.match(Pattern.java:4658)
   1 at com.demo.order.OrderService.query(OrderService.java:88)

$ grep -A 5 'nid=0x22bb' thread-1.txt
"http-nio-8080-exec-23" #23 daemon prio=5 os_prio=0 tid=0x... nid=0x22bb
runnable
   java.lang.Thread.State: RUNNABLE
        at java.util.regex.Pattern$Loop.match(Pattern.java:4785)
        at java.util.regex.Pattern$GroupHead.match(Pattern.java:4658)
        at com.demo.validator.Validator.checkEmail(Validator.java:42)
```

`3` 这个计数是关键：三次采样都命中同一行，才叫热点；只命中一次的那条 `OrderService.query` 是恰好路过的普通请求，**别拿它当根因**。进制转换这一步也别手工算——`printf '%x' 8891` 得 `22bb`，栈里写的是 `nid=0x22bb`；把十进制 `8891` 直接拿去 `grep nid=0x8891`，永远查不到，这是最容易白丢半小时的一个字符差。

> 这套脚本和 23.2 的七步是同一件事，区别只在于：它把"顺序"和"进制转换"从人脑挪到了机器里。第 27 章会把这个思路再推一步——**能被机器执行的检查，才不会在第三次事故时被人省略**。

23.3　容器环境：命令要换

在 K8s 里，PID 是容器内的，宿主机 `top` 看不到。三个方案：

```
# ① 进容器（最快）
kubectl exec -it <pod> -- bash
top -Hp 1                       # 容器里 Java 进程通常是 PID 1

# ② 用 arthas（最省事，一步到位）
curl -O https://arthas.aliyun.com/arthas-boot.jar
java -jar arthas-boot.jar
thread -n 3                     # 直接列出最忙的 3 个线程 + 栈

# ③ JDK 21+ 用 jcmd（比 jstack 更稳）
jcmd 1 Thread.print -l
```

推荐用 arthas 的 `thread -n 3`：它自动完成"找线程 + 转进制 + 抓栈"三步，还带 CPU 占比。生产环境装一个（或提前打好镜像），排障效率提升一个数量级。

`-l` 参数（`Thread.print -l`）会额外打印 `ownable synchronizers`，能看到 `ReentrantLock` 的持有者——排查死锁必需。

【必读】jstack、jcmd 与强制抓栈：容器里到底差在哪

原文 23.3 把 `jstack` 和 `jcmd` 当成"两个都能用的等价命令"，但它们在不同 JDK、不同容器里的行为差别很具体，选错会卡在"连不上"这一步。

维度	`jstack <pid>`	`jcmd <pid> Thread.print -l`	`jstack -F` / `kill -3`
attach 机制	Attach API，依赖 `/tmp` 下的 socket 文件	同为 Attach API，但是统一子命令入口	`-F` 走 SA/ptrace，`kill -3` 是信号
需要同 UID	是，否则 `Unable to open socket file`	是，同样	是
JDK 8	可用	可用	可用
JDK 17 / 21	仍提供，但官方推荐 `jcmd`	**推荐**，输出稳定、可加 `-l` 看 ownable synchronizers	`-F` 在新版本上能力受限
进程已卡死	可能长时间挂住	同样可能挂住	用 `-F`；`kill -3` 打印到 stdout，容器里常因 stdout 未采集而丢失
容器内 PID	容器内 PID（常为 1）	同左	同左

三点最容易踩：

1. **`jstack` 在 JDK 21 上不是被删了，是被"降级推荐"了。** 官方文档把 `jcmd` 列为首选，`Thread.print` 与 `jstack` 等价、`GC.heap_dump` 与 `jmap -dump` 等价。新脚本统一写 `jcmd`，少一套版本判断。
2. **容器内连不上，先查三件事再怀疑权限：**目标进程是不是 PID 1、执行用户是不是同 UID、`/tmp` 是否可写（attach socket 默认落在这里）。不少精简镜像把 `/tmp` 挂成只读，报错长得和权限错一模一样。
3. **`-l` 不是可选项。** `Thread.print -l` 会额外打印 `ownable synchronizers`，能看到 `ReentrantLock` 的持有者；不加它，死锁现场只剩一片 `WAITING`，看不出谁等谁（第 14 章的锁等待是同一类判据）。

如果进程已经不响应 attach，别在普通 `jstack` 上死等，直接上 `jstack -F`；或者用更低开销的 JFR 补一份"过去一小时"的录像：`jcmd <pid> JFR.start name=diag settings=profile duration=120s filename=/data/jfr/diag.jfr`。JFR 从 JDK 11 起开源（JEP 328），开销通常低于 1%，能长期常开——**它比"事后连抓三次 jstack"更准，因为它不需要你正好在场**。版本区间记一下：JDK 8 没有统一日志和开源 JFR，JDK 9 起 `-Xlog` 取代 `-XX:+PrintGCDetails`，JDK 17/21 上 `jcmd` + `-Xlog` 才是标准组合。

三次 `jstack` 是"抽样"，火焰图是"录像"，后者能补上前者的盲区：抽样只告诉你某一瞬间栈在哪，录像能看出 `Pattern$Loop` 在整个 30 秒里占了多大宽度，还能看到 native 栈（比如压缩、加解密）。低开销的取法：

```
# ✓ async-profiler：采 30 秒 CPU 火焰图，不需要重启、不需要反复 attach
profiler.sh -e cpu -d 30 -f /data/incident/flame.html <pid>

# ✓ 或 JFR 事后聚合，JDK 17+ 自带 jfr 命令
jfr print --events jdk.ExecutionSample /data/jfr/diag.jfr | head -40
```

判据很直接：**火焰图里最宽的那一块栈，就是 `top -Hp` 里那个 99% 的 nid 在干的事**。两者对不上，说明要么 nid 抓错了（进制转换），要么热点在 native 层而 `jstack` 只给了 Java 帧。第 25 章讲分配速率时也会用同一份火焰图找"谁在不停造对象"——**同一份证据，CPU 问题和 GC 问题各看一列**，这也是取证阶段宁可多存一个文件的理由。

23.4　三种高频案例

【必读】① 正则灾难性回溯

```
// ✗ 嵌套量词 + 长输入 = 指数级回溯
Pattern.compile("^(a+)+$").matcher(longInput).matches();
```

输入 `aaaa...a!`（30 个 a 加一个感叹号），回溯次数是指数级。**单核 CPU 直接 100%，而且不报错、不超时。**

```
// ✓ 三条防线
// 1. 不用嵌套量词
// 2. 限制输入长度（先 length() 判断）
// 3. 加超时（Java 8 没有，用 nfa 库或改业务逻辑）
if (input.length() > 64) return false;
```

这类问题在"邮箱/URL/身份证号校验"里最常见，因为从网上抄的正则往往有嵌套量词。

【必读】② 死循环

```
// ✗ 典型：条件永远不满足
while (iterator.hasNext()) {
    if (someCondition) {
        // 忘了 iterator.next() 或 break
        continue;
    }
    iterator.next();
}
```

`RUNNABLE`，栈在一个 `while` 里，CPU 100%，但**不打日志、不报错、不超时**。这类 bug 静态检查发现不了，只能靠 CPU 曲线。

【必读】③ 日志同步刷盘

```
<!-- ✗ 同步 appender + 高频日志：每个请求都 fsync -->
<appender name="FILE" class="ch.qos.logback.core.FileAppender">
```

高并发下，日志刷盘会占据大量 CPU（内核态）。特征：`top` 里 `sy`（系统态 CPU）很高，栈里是 `FileOutputStream.writeBytes`。

改成 `AsyncAppender` + 合理队列大小即可。

【必读】正则回溯：把指数级算出来

`(a+)+` 这类嵌套量词为什么会打满一个核？不是因为 Java 的正则引擎慢，是因为它在**枚举所有可能的划分方式**。对输入 `a^n!`（n 个 a 后跟一个不匹配字符），`(a+)+` 把 n 个 a 切成若干段的方法数是 `2^(n-1)`；每种划分失败后引擎都要回溯到最后再试下一种，总匹配步数近似 `O(2^n)`。

算一组具体数字，别停在"指数级"这四个字上：

n（a 的个数）	划分数 `2^(n-1)`	实测 `matches()` 耗时（单核，JDK 17）
20	524,288	约 8 ms
25	16,777,216	约 0.25 s
30	536,870,912	约 8 s
35	17,179,869,184	数分钟级

n 每加 5，耗时约翻 32 倍——正好是 `2^5`。**输入只长了 5 个字符，代价涨了三十多倍**，这就是"不报错、不超时、CPU 悄悄 100%"的来源：请求没失败，只是永远算不完。

下面这段能直接在自己机器上复现，把 `n` 从 20 调到 30 看曲线：

```
// ✓ 可运行复现：观察 n 每 +5，耗时约 ×32
@Test
void redosCurve() {
    Pattern bad = Pattern.compile("^(a+)+$");
    for (int n = 20; n <= 30; n += 5) {
        String input = "a".repeat(n) + "!";
        long t0 = System.nanoTime();
        bad.matcher(input).matches();       // 恒为 false，但要枚举完才肯返回
        long ms = (System.nanoTime() - t0) / 1_000_000;
        System.out.printf("n=%d  cost=%d ms%n", n, ms);
    }
}
```

```
// ✗ 嵌套量词 + 不限长输入 = 把 CPU 交给外部输入决定
Pattern.compile("^(a+)+$").matcher(userInput).matches();

// ✓ 平铺量词 + 先判长度，复杂度回到 O(n)
boolean ok(String s) {
    return s != null && s.length() <= 64
        && Pattern.compile("^a+$").matcher(s).matches();
}
```

`^a+$` 是线性的：对每个字符只做一次判定，失败即返回，回溯步数是 `O(n)`。**同一段校验逻辑，从 `O(2^n)` 换成 `O(n)`，不需要加机器、不需要调 JVM，只需要改正则。** 这类从网上抄来的邮箱、URL、身份证校验，是第 26 章"压测最容易漏的输入"里最典型的一种：压测数据都是合法短串，永远触发不到回溯分支，只有真实用户粘一段超长字符串才会中招。

【进阶】异步日志为什么总被冤枉

这一段要顺手修正一个常见写法。本章 23.4 的第 ③ 种案例把"同步刷盘"列成 CPU 100% 的原因，并说"改成 `AsyncAppender` 即可"——**方向对，因果要拧过来**。日志写入主要是系统调用和 IO 等待，不是持续占用用户态 CPU 做计算；真正会把一个核烧到 100% 的，是上面那种正则回溯或死循环。同步 appender 的典型症状是 `sy`（内核态）偏高、线程大量 `BLOCKED` 在 appender 的锁上，而不是某一个 `nid` 稳定占着 99% 的用户态 CPU。

更常见的现场是这样的：真正的原因在别处（比如某个接口在做正则回溯），业务线程被拖住后都要写日志，于是栈里几十条都停在 `ch.qos.logback.core.OutputStreamAppender.subAppend` 或 `AsyncAppender` 的队列上。**栈里出现最多的是日志，日志却是结果，不是原因。**

判据只有一个，回到 23.2 的第一份证据：

```
# ✓ 看"哪个 nid 占着 99%"，而不是"哪个类的栈出现次数最多"
top -Hp 8842 | sort -k9 -nr | head -5
# 若一个 nid 稳定 98.7%，看它的栈顶；其余 BLOCKED 在日志锁上的都是被连累的

# ✗ 拿"栈里日志帧最多"当根因，然后去调 AsyncAppender 队列大小
grep -c 'OutputStreamAppender' /tmp/js-*.txt   # 这个计数只说明"大家都在等写日志"
```

而一个**已经配了异步日志**的系统更不该背锅：`AsyncAppender` 把写盘挪到单独的 worker 线程，业务线程只做一次队列入队，几乎不占 CPU；这时若还有单线程 99%，它几乎不可能是日志线程。**多数人以为"栈里全是日志 = 日志把 CPU 打满了"，实际是"所有线程都堵在同一把日志锁上，而真正烧 CPU 的那个 nid 被这堆相同的栈淹没了"。** 这也是为什么 23.2 要求"先 `top -Hp` 定 nid，再 `jstack` 看那个 nid"——顺序反了，看到的就是一片噪声。真要动日志，也该在确认是 IO 等待（`iowait` 高、`sy` 高、`%CPU` 不高）之后再动，而不是在 CPU 100% 的现场里先改队列大小。

23.5　降级：先保命，再查因

如果业务已经在大量报错，先降级再查：

① 摘流量（从 LB 摘掉一台，保留现场继续查）
② 限流（把入口压到系统能承受的水位）
③ 关掉非核心功能（导出、报表、定时任务）
④ 扩容（如果是容量问题）

注意顺序：**先摘一台保现场，再处理其余实例。** 全量重启等于把唯一的证据毁了。

【必读】什么时候该怀疑是压测漏掉的输入

把前面几节连起来看，CPU 100% 里有一类不是"代码写错了"，而是"没见过这种输入"。正则回溯、超大报文序列化、一次性 `SELECT *` 出百万行再 JSON 化，都属于这一类：逻辑本身没问题，是输入规模越过了某条没写下来的上限。

第 26 章给了压测必须满足的四个条件——数据量接近生产、流量模型接近真实、链路完整、读写成比例——**再补一条和本章直接相关的：参数值域要接近生产**。压测脚本里若用同一批合法短串（`test01@example.com` 循环一万次），正则走的是最快路径；真实流量里混进一段 200 字符的畸形邮箱，才走到回溯分支。用 wrk 的 Lua 脚本或 Gatling 的 Feeder 把参数随机化，是让这类问题在压测阶段暴露的最低成本做法：

```
# ✗ 所有请求同一个参数：缓存全命中、正则全走快路径
wrk -t8 -c200 -d60s 'http://svc/check?email=a@b.com'

# ✓ 参数化 + 混入长串与畸形值，把"最坏输入"也压一遍
wrk -t8 -c200 -d60s -s param.lua http://svc/check
```

```
-- param.lua：随机长度 + 一定概率的畸形长串
request = function()
  local n = math.random(1, 64)
  if math.random(1, 100) == 1 then n = 256 end   -- 1% 的超长输入
  local s = string.rep("a", n) .. "@x.com"
  return wrk.format("GET", "/check?email=" .. s)
end
```

一旦压测阶段让这个 1% 的超长请求进来，`top -Hp` 会立刻出现一个 99% 的 nid，栈停在 `Pattern$Loop`——**这正是 23.2 那套取证流程在压测环境里的预演**。第 26 章讲"找拐点"，本章讲"找那一行栈"，两件事在同一条曲线上：拐点是系统开始不对的那个流量，栈帧是系统开始不对的那行代码。把压测参数化脚本和 `diag.sh` 一起放进镜像，故障当天就不需要现想。

> 一个反常识的收尾：**压测没复现的问题，不等于不存在，只等于你的输入不够坏。**大促当天最先打满 CPU 的，往往不是 QPS 最高的接口，是那个从来没人用超长字符串调过的校验接口。

最后把本章最常出现的三种误判摆成一张表，作为值班时的对照卡：

现场看到	第一反应	实际判据	该翻哪一章
`top -Hp` 找不到 99% 的 nid，但 CPU 整体高	"业务死循环"	大量 `BLOCKED` / `iowait`，是排队不是计算	第 26 章容量、第 4 章连接池
栈里几十条都停在日志类	"日志把 CPU 打满"	找一个稳定 99% 的 nid，日志线程通常不是它	本章 23.4 异步日志
栈里全是 `GC Thread`，业务帧很少	"代码热点"	GC 线程在烧，方向转到堆	第 24 章 OOM、第 25 章 GC

三行有个共同结构：**看到的都是"谁出现得多"，该问的却是"谁占着 CPU"**。出现次数是结果，CPU 占比才是原因。把这张卡和 `diag.sh` 一起放进值班手册，凌晨三点就不需要靠记忆判断方向了。

23.6　自检清单（排障时逐条过）

- [ ] **没有重启**（先取证）
- [ ] `top -Hp` 拿到了最高 CPU 的 TID
- [ ] TID 用 `printf "%x"` 转成了十六进制
- [ ] `jstack` 连抓 3 次，确认栈帧稳定
- [ ] 确认是业务线程还是 GC 线程
- [ ] 保存了 `VM.flags`、`jstat`、线程状态分布
- [ ] 容器环境用 arthas `thread -n 3` 或 `jcmd`
- [ ] 有降级预案（摘流量、限流、关非核心）

23.7　一句话总结

> CPU 100% 不是"服务病了"，是**它在很努力地做一件不该做的事**。 你的任务不是让它停下来，是找出那件事。

【选读】延伸

- 知识库对应原理篇：02-Java进阶/01-JVM/OOM与CPU问题排查、02-Java进阶/01-JVM/故障排查工具、17-常见Bug与解决方案/02-并发与线程池坑

第二十四章　OOM：加堆之前先取证

本章要回答： `OutOfMemoryError` 到底缺的是哪块内存？为什么 `-Xmx` 从 4G 加到 8G，第二天又 OOM 了？

本章你能带走

- 六类 OOM 的区分方法（**冒号后面那几个字决定一切**）
- 一个反直觉的事实：`exit code 137` 不是 Java OOM，**不会产生 dump**
- 一条原则：**加堆是最后手段，不是第一反应**

难度构成： 本章主线【必读】；含【进阶】深挖 0 处；章末「延伸」小节放支线深挖与知识库指针。---

24.1　现场：加到 16G，第三天还是 OOM

一个服务 OOM，运维加到 8G，好了三天；又 OOM，加到 16G；第三天又 OOM。

"加内存能缓解三天"这个现象，本身就是最强的证据：你在漏，不是在不够用。

如果是容量不足，加到 16G 之后应该稳定；如果是泄漏，加多少都是同一个结局——只是时间被拉长了。而拉长的时间会让人误以为"问题解决了"，直到下次爆发。

加堆不是解决问题，是延后问题，并且让下次的 dump 更大、MAT 更难打开。

当时怎么误判的：第一反应是"内存真的不够，加堆就行"——加到 8G 好三天、再加到 16G 又 OOM，每次"缓解"都更让人确信是容量问题，直到第三次爆发才起疑。中间走了一条死路：怀疑"是不是热部署或动态代理把 Metaspace 撑爆了"，`jcmd VM.metaspace` 拉出来类数量一直平稳，纯属无用线索，白白加了一次 `-XX:MaxMetaspaceSize`。真正扭转方向的是（本该在事前就配的）`HeapDumpOnOutOfMemoryError` 拿到的那份 dump，MAT 的 Dominator Tree 里一个静态 `HashMap` 的 Retained Heap 接近整堆。教训很硬：加堆只是延后问题、还把下次现场变大，先留证再谈加堆，否则你永远在和三天后的自己打同一场仗。

24.2　先看冒号后面那几个字

`OutOfMemoryError` 只是大类。真正决定排查方向的是 message：

message	缺哪块	典型原因	首查动作
`Java heap space`	堆	泄漏、一次加载大量数据、缓存无界	`jmap -dump` + MAT
`GC overhead limit exceeded`	堆（其实满了）	同上，GC 空转	同上，**别关** `UseGCOverheadLimit`
`Metaspace`	元空间	动态生成类、CGLIB、热部署、ClassLoader 泄漏	`jcmd VM.metaspace`
`Compressed class space`	压缩类空间	类元数据超默认 1G 上限	`-XX:CompressedClassSpaceSize`
`Direct buffer memory`	直接内存	Netty ByteBuf 未释放、`allocateDirect` 无节制	`jcmd VM.native_memory summary`
`unable to create new native thread`	线程栈 / OS	线程数触顶、`ulimit -u`、容器 pids limit	`jcmd Thread.print` 数线程
`Requested array size exceeds VM limit`	堆	数组长度超 `Integer.MAX_VALUE - 2`	查大集合 `toArray()`、大文件一次性读

一个高频误判：`Metaspace` 和 `Compressed class space` 经常被混为一谈，但它们由两个不同参数控制：

```
-XX:MaxMetaspaceSize=512m            # 元空间上限
-XX:CompressedClassSpaceSize=1g      # 压缩类空间上限（默认 1G）
```

只调前者，后者溢出照样报错。看到 `Compressed class space` 就去调第二个。

【必读】冒号后面每个词对应哪块内存、谁在申请它

24.2 那张表给的是"首查动作"，这里把每一类再往下拆一层：**是谁在向哪块内存申请、申请不到时 JVM 走哪条路抛出来**。同一个 `OutOfMemoryError`，冒号后面的词不同，`-Xmx` 加不加得上完全不同。

message	申请方 → 内存区	触发判据	加 `-Xmx` 有用吗
`Java heap space`	`new` / 集合扩容 → 堆	分配时新生代与老年代都放不下，且 Full GC 后仍不够	有时（仅容量不足），效
`GC overhead limit exceeded`	同上 → 堆	GC 时间占比 > 98% 且每次回收 < 2% 堆，连续多次	✗ 它就是堆满，关掉把报错推迟成 `Java h space`
`Metaspace`	类加载器 `defineClass` → 本地内存的类元数据	动态代理、CGLIB、热部署持续造类，`MetaspaceSize` 高水位反复触发 Full GC 仍涨	✗ 完全无关，`-XX:MaxMetaspaceSiz`
`Compressed class space`	类元数据中的 Klass 部分 → 压缩类空间	默认 1G 上限被打满，常与 `UseCompressedClassPointers` 同时出现	✗ 要调 `-XX:CompressedClass`
`Direct buffer memory`	`ByteBuffer.allocateDirect` / Netty `ByteBuf` → 堆外	`Bits.reserveMemory()` 申请不到，默认上限约等于 `-Xmx`	✗ 反而更糟，要 `-XX:MaxDirectMemory`
`unable to create new native thread`	`pthread_create` → OS 线程栈	线程数触顶：`ulimit -u`、容器 pids limit、`-Xss` 过大	✗ 加堆只挤占本就不
`Requested array size exceeds VM limit`	数组分配 → 堆	请求长度 > `Integer.MAX_VALUE - 2`（2,147,483,645）	✗ 这是长度上限，不够

`GC overhead limit exceeded` 值得单独说清，因为它最容易被当成"GC 出问题了"。它的判据是一个算式：`UseGCOverheadLimit` 默认开启，当**最近一段时间 GC 总耗时占比超过 98%、且每次回收释放不到堆的 2%** 时，JVM 主动抛出这个错误，好让进程别把剩下的时间也耗在无效 GC 上。**它不是一种独立的内存区，是"堆已经满了"的提前引爆。** 所以 `-XX:-UseGCOverheadLimit` 这种"关掉就好"的处理，只是把报错换成了稍后更贵的 `Java heap space`，还顺手丢掉了这条最有价值的线索。

`Direct buffer memory` 的链路最容易查错方向：

```
ByteBuffer.allocateDirect(cap) / Netty PooledByteBufAllocator
   → java.nio.Bits.reserveMemory(size, cap)
      → 已用 + size > MaxDirectMemorySize ?
         → 先触发一次 System.gc()，等 Cleaner 回收堆外块
         → 仍不够 → 抛 OutOfMemoryError: Direct buffer memory
```

注意中间那次 `System.gc()`：如果启动时加了 `-XX:+DisableExplicitGC`（第 25 章会讲），这条兜底路径直接断掉，堆外块只能等 Cleaner 被动触发，泄漏会来得更快。**看到 `Direct buffer memory`，先别去调 `-Xmx`——堆外的账不在堆里。**

`unable to create new native thread` 也值得算一笔，因为它最常被误当成"内存不够"从而去加 `-Xmx`。线程栈是每线程一份、堆外的，能吃多少线程由三者里最小的那个决定：

```
线程数上限 ≈ min( 内存余量 / (-Xss),  ulimit -u,  pids limit )
举例：-Xss1m、容器剩余内存 1.5G、pids limit 512、ulimit -u 1024
→ 内存能撑 1.5G/1M ≈ 1536 个，三者取最小 → 最多约 512 个线程
→ 若某次发版把线程池 core 从 200 调到 600，
   第 513 个线程创建时抛 unable to create new native thread
```

判据在 24.3 的 `05-thread.txt`：线程总数贴着 pids limit、`VM.native_memory` 里 `Thread` 的 `committed` 还在涨，就是它。**此时加堆不仅无效，还会挤占本就不够的线程栈预算**——堆和栈抢的是同一个 cgroup limit。这也是 24.4 那条 `limit = -Xmx × 1.3~1.5` 的另一半理由：多留出来的 30%，一部分就是给线程栈和直接内存预留的。第 9 章讲无界线程池会讲到同一条上限，只是视角从"任务堆积"换成了"线程数触顶"。

24.3　取证：重启之前必须做的五件事

```
# ① 抓 heap dump（最重要）
jmap -dump:format=b,file=/tmp/heap.hprof <pid>
# 或 jcmd（更稳，对大堆影响小）
jcmd <pid> GC.heap_dump /tmp/heap.hprof

# ② 类直方图（dump 太大时先用这个快速定性）
jcmd <pid> GC.class_histogram | head -30
# 看哪个类的实例数和占用最多

# ③ JVM 参数
jcmd <pid> VM.flags > /tmp/flags.txt

# ④ 直接内存（如果怀疑 netty/off-heap）
jcmd <pid> VM.native_memory summary     # 需要启动时加 -
XX:NativeMemoryTracking=summary

# ⑤ GC 日志
# 启动时就该配好：-
Xlog:gc*:file=/var/log/gc.log:time,uptime:filecount=5,filesize=50m
```

这五件事做完再重启。 重启一次，`/tmp` 之外的一切证据都会消失，而"下次复现"可能要等三天——三天后你面对的还是一样的信息空白。

【必读】提前配好 dump 参数

```
# 别等出事才想起来
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/data/dump/
-Xlog:gc*:file=/data/log/gc.log:time,uptime:filecount=5,filesize=50m
```

`HeapDumpOnOutOfMemoryError` 是**唯一能保证"OOM 那一刻自动留证"**的配置。没有它，你能拿到的只有"服务挂了"。

注意：大堆（16G+）dump 会暂停应用几十秒，且文件可能十几 G。**要预留磁盘空间**，并把 dump 目录挂到持久卷上——容器销毁后 `/tmp` 里的一切都会消失。

【必读】五件事按这个顺序敲，每条看输出里哪个字段

24.3 列了五件事，但没说"每条命令的输出该盯哪个字段"。凌晨三点对着一屏数字，最费时间的就是这一步。把顺序和字段一起固化进脚本：

```
#!/usr/bin/env bash
# OOM 现场取证：先留证，后重启；全部落到持久卷，别写 /tmp
set -euo pipefail
PID=$1
OUT=/data/incident/oom-$(date +%Y%m%d-%H%M%S)
mkdir -p "$OUT"

jcmd "$PID" VM.flags                            > "$OUT/01-flags.txt"
jcmd "$PID" GC.heap_info                        > "$OUT/02-heap.txt"
jcmd "$PID" GC.class_histogram                  > "$OUT/03-histo.txt"   # 看
#instances 与 #bytes
jcmd "$PID" VM.native_memory summary scale=MB > "$OUT/04-nmt.txt"    # 需启动时开
NMT
jcmd "$PID" Thread.print -l                     > "$OUT/05-thread.txt"  # 数线程，
查 pids limit
jstat -gcutil "$PID" 1000 10                    > "$OUT/06-gcutil.txt"  # 看 O 列
是否贴顶
# 已配 HeapDumpOnOutOfMemoryError 就直接取现成 dump；否则才现场抓
cp /data/dump/*.hprof "$OUT/" 2>/dev/null || \
  jcmd "$PID" GC.heap_dump "$OUT/heap.hprof"
```

各文件里盯哪个字段，比命令本身重要：

文件	盯哪一列	什么值算命中
03-histo.txt	#instances、#bytes	某个业务类实例数百万、或 [B/char[] 累计占堆 70%+
04-nmt.txt	Internal、Class、Thread 各行 committed	Internal（含堆外）远大于堆 → 直接内存嫌疑
05-thread.txt	java.lang.Thread.State 计数、线程总数	线程数接近 ulimit -u 或 pids limit → 第 7 行那类 OOM
06-gcutil.txt	O（老年代%）、FGC、FGCT	O 长期 99%+、FGC 快速增长 → 堆已满

一次真实直方图的开头长这样，`#bytes` 一列直接把嫌疑点排在第一行：

```
 num     #instances         #bytes  class name
----------------------------------------------
   1:       3,214,887   1,542,345,760  [B
   2:       3,208,102     102,659,264  com.demo.cache.Product
   3:           8,901      41,203,456
java.util.concurrent.ConcurrentHashMap$Node
```

`[B`（byte 数组）三千多万个、占 1.5GB，`Product` 三百多万个——**这一屏就能定性是"缓存把 byte 数组攒住了"**，比打开几个 G 的 dump 快得多。也正因如此，24.3 把直方图排在第二而不是最后：它是"便宜且先给方向"的那一步。

五件事的顺序不能颠倒，理由各自都很具体：

顺序	为什么必须在这个位置	颠倒的代价
`VM.flags` 第一	后面所有命令的解释都依赖"当时实际生效的参数"	重启后参数可能被改过，先抓才对得上
直方图在 dump 前	不触发 Full GC、几秒出结果，先给方向	一上来就 dump，几个 G 的文件打开前不知道找什么
`Thread.print` 在数线程前	顺带把线程栈和锁信息一起留下	只数数量，事后无法判断线程在等什么
dump 最后	会 STW，线上要挑摘流实例	高峰期全量 dump 能把服务冻结数分钟

一个真实反面案例：某次值班先跑了 `jmap -histo:live`（带 `:live` 会先触发一次 Full GC），8G 堆停了 6 秒，接口超时一片，dump 还没抓到就被要求"先恢复服务"，最后只剩一份被 GC 改过现场的直方图。**`:live` 版更聚焦泄漏，但它不是取证第一步**；要看存活对象分布，用不带 `:live` 的 `jcmd GC.class_histogram`，把"先触发 Full GC"留到已经摘流的实例上。

【必读】HeapDumpOnOutOfMemoryError 在容器里静默失败的四个原因

`HeapDumpOnOutOfMemoryError` 是唯一能在 OOM 那一刻自动留证的配置，但它有四个"配了却没 dump"的坑，全在容器里：

坑	现象	判据与修法
路径在 `/tmp` 或 emptyDir	Pod 重启后 dump 随容器一起消失	`HeapDumpPath` 必须指向 PVC/持久卷；`-XX:HeapDumpPath=/data/dump/` 要带尾斜杠，指向目录而非文件
目录不存在或只读	JVM 写 dump 失败但**只打一行日志**，不阻断 OOM	镜像里 `mkdir -p` 并 `chmod`；上线前用 `jcmd <pid> GC.heap_dump /data/dump/probe.hprof` 预演一次
磁盘 quota 小于堆	写到一半失败，得到的 hprof 打不开	预留 ≥ 堆大小 × 1.5；`df -h /data/dump` 进启动检查
进程被 SIGKILL	参数根本来不及执行	这是 exit code 137，见 24.4，属于另一类问题

第四行要和 `-XX:+ExitOnOutOfMemoryError` 一起看：容器内常配它让 JVM 快速失败、交给 K8s 重启，但它与"留 dump"是有顺序竞争的两件事——先写 dump 再退出才对，若 dump 目录不可写，快速失败反而让现场丢得更干净。**上线前那次 `GC.heap_dump` 预演，是唯一能提前证明"这条链路通"的动作**，别等真 OOM 才发现目录是只读的。

24.4 `exit code 137`：不是 Java 的错

K8s 里 Pod 突然重启，`kubectl describe` 显示：

```
Last State: Terminated
  Reason: OOMKilled
  Exit Code: 137
```

这不是 Java 的 `OutOfMemoryError`，是内核的 OOM Killer 用 SIGKILL 杀掉了进程。

区别很关键：

	Java OOM	内核 OOMKilled
触发者	JVM	Linux 内核
异常	有 `OutOfMemoryError`	**无任何 Java 异常**
dump	`HeapDumpOnOutOfMemoryError` 会生成	**不会生成**
证据	堆 dump、GC 日志	`dmesg`、kubelet events、NMT

所以"配了 `HeapDumpOnOutOfMemoryError` 却没 dump"的经典困惑，答案通常是 137。

排查 137 要看三个地方：

```
# ① 内核日志（宿主机或节点上）
dmesg -T | grep -i "killed process"

# ② K8s 事件
kubectl describe pod <pod> | grep -A5 "Last State"

# ③ 容器内存构成
# JVM 总占用 = 堆 + Metaspace + 直接内存 + 线程栈 + 代码缓存 + GC 结构
# 容器 limit 必须 > 这个总和（不是 > -Xmx）
```

最常见的原因： `-Xmx` 设成了容器 limit 的 100%。比如 limit 4G、`-Xmx4g`，那么堆能用到 4G，加上元空间、直接内存、线程栈，必然超 4G → 内核杀。

经验配置：

```
容器 limit = -Xmx × 1.3 ~ 1.5
```

或者用 `-XX:MaxRAMPercentage=70`，让 JVM 按容器 limit 自己算。

【必读】137 = 128 + 9：把 cgroup OOM Killer 判死

`Exit Code: 137` 不是一个需要背的错误码，它是一句算术：`137 = 128 + 9`，`9` 是 `SIGKILL` 的信号编号，**Shell 用 128+信号号 表示"被信号杀死"**。同理 `143 = 128 + 15` 是 `SIGTERM`。看到 137，第一结论就该是：**没有人调用 `System.exit()`，是内核直接把进程掐了**，所以 Java 层不会有任何异常、任何 finally、任何 dump。

把 24.4 那张判别表补成一条可执行的决策树：

```
# ① 内核日志：谁被杀、为什么
dmesg -T | grep -i -E 'killed process|oom-kill|memory cgroup'
# 典型行：Killed process 8842 (java) total-vm:..., anon-rss:4194304kB

# ② cgroup 账本（v2；v1 路径是 /sys/fs/cgroup/memory/）
cat /sys/fs/cgroup/memory.current     # 当前用量
cat /sys/fs/cgroup/memory.max         # 上限
cat /sys/fs/cgroup/memory.events      # oom_kill 计数，>0 即实锤

# ③ K8s 侧交叉印证
kubectl describe pod <pod> | grep -A5 'Last State'
```

判别顺序只有三问：**有没有 `OutOfMemoryError` 异常栈？有没有 `.hprof` 文件？`memory.events` 的 `oom_kill` 是否 > 0？** 三问里若前两问都"没有"、第三问"是"，就是 cgroup OOM Killer，不用再翻 JVM 参数。反过来，有异常栈、有 dump 却也显示 137，通常是"先被 kill、随后编排系统重启时又记录了一次"的叠加事件，按时间线分开读，别合成一个原因。

最常见的成因可以算得很清楚。容器 `limit = 4Gi`，却配了 `-Xmx4g`：

```
堆 4.0G + Metaspace 0.5G + 线程栈 300×1M≈0.3G + 直接内存 0.5G + CodeCache 0.25G
= 约 5.55G  >  4Gi limit
```

堆还没满，RSS 已经越界，内核先动手。**多数人以为"堆没到 -Xmx 就不会 137"，实际是"堆只是 RSS 的一部分"**：`limit` 管的是整个 cgroup，不是 `-Xmx`。所以经验值 `limit = -Xmx × 1.3 ~ 1.5`，或者干脆用 `-XX:MaxRAMPercentage=70` 把算账交给 JVM（JDK 10 起 `UseContainerSupport` 默认开启，8u191 也回移了）。这两个写法别混用——显式 `-Xmx` 会覆盖 `MaxRAMPercentage`，混用时以 `-Xmx` 为准，容易以为自己在按比例、其实写死了。

cgroup 两代的取证路径不一样，抄错会一无所获：

版本	用量 / 上限	OOM 计数	判据
cgroup v1	`memory.usage_in_bytes` / `memory.limit_in_bytes`	`memory.failcnt`、`memory.oom_control`	`failcnt` 持续增长
cgroup v2	`memory.current` / `memory.max`	`memory.events` 的 `oom` / `oom_kill` 行	`oom_kill > 0` 即实锤

```
# ✓ v2 一眼判死：三个数就够
cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/memory.current
/sys/fs/cgroup/memory.events
```

`dmesg` 还要和节点对上：容器里往往没有权限读宿主机内核日志，得在**节点上**执行，或者看 kubelet 的 event。三处证据（内核日志、cgroup 账本、Pod Last State）齐了才写结论——只凭 `Exit Code: 137` 就断定"内存不够"，会漏掉另一种可能：**进程是被存活探针失败后以 `SIGTERM`（143）主动终止、再叠加一次重启记录的**，那是一条完全不同的链。

24.5　MAT：只看三个视图就够了

dump 拿到之后，MAT 打开（大堆记得调 `MemoryAnalyzer.ini` 的 `-Xmx`），按顺序看三件事：

① Leak Suspects（一键泄漏报告）

MAT 自动分析，直接给出"疑似泄漏点"和占比。**80% 的情况这里就有答案。**

② Dominator Tree（支配树）

按"持有内存"排序，看谁占得最多。注意看的是**支配关系**，不是引用关系——一个 `HashMap` 引用 100 万个对象，支配树里会显示 `HashMap` 占 100MB。

③ Path to GC Roots（引用链）

找到可疑对象后，右键 → `Path to GC Roots` → **排除弱引用/软引用**，看它是被谁强引用住的。

三条最常见的引用链：

```
Thread → ThreadLocalMap → Entry → value          ← ThreadLocal 泄漏
ClassLoader → 静态字段 → 集合                    ← 静态缓存无界
连接池 → Connection → Statement → ResultSet      ← 资源未关
```

`Path to GC Roots` **是 MAT 里最有价值的功能**，因为它回答的是"为什么它还没被回收"——而这正是泄漏的定义。

【必读】三个视图各回答一个问题，别串着用

24.5 给了三个视图，但没说清它们各自**只回答一个问题**。混着用最常见的后果是：在 Histogram 里挑了一个实例数最多的 `String`，追了半天发现它只是常量池里的，跟泄漏无关。

视图	它回答的问题	该看哪一列	什么时候会误导
Leak Suspects	"有没有一处明显的大块头？"	自动报告的 `Retained Heap` 占比	只看单个 dominator，多个中等泄漏各占 10% 时它会说"未发现明显泄漏"
Dominator Tree	"如果回收掉这个对象，能释放多少？"	`Retained Heap`	`HashMap` 支配 100 万个小对象时它排第一，但根因在"谁往里 put"
Histogram	"哪一类对象数量/体积反常？"	`#instances`、`Shallow Heap`	只按数量排，`String`/`byte[]` 永远第一，看不出谁持有

两个必须先分清的概念，MAT 里所有列都建立在它们之上：

```
Shallow Heap  = 对象自身占用（不含它引用的对象）
Retained Heap = 该对象被回收后，能连带释放的总量
              = Shallow(自身) + Σ Retained(仅被它支配的子对象)
```

一个 `ConcurrentHashMap` 的 Shallow Heap 可能只有 64 字节，Retained Heap 却是 3.2GB——**因为支配树算的是"它活着，这 3.2GB 就下不去"**。所以 24.5 说"看支配关系不是引用关系"，指的就是 Retained Heap：引用只说明"能找到"，支配才说明"卡住了谁"。

正确的顺序应当是：**Leak Suspects 先给方向 → Dominator Tree 按 Retained Heap 排序锁定那个大块头 → 对它右键 `Path to GC Roots`，排除弱引用/软引用/虚引用，看是谁强引用着它**。第三步是唯一能回答"为什么它还没被回收"的，也是 24.5 里最有价值的一步。补一个实操坑：打开几个 G 的 dump 前先调 `MemoryAnalyzer.ini` 里的 `-Xmx`，MAT 自己也是个 Java 程序，默认配置常常比 dump 还小，一打开就先 OOM 一次。

24.6　三种根因与对策

根因	表现	对策
真泄漏	每次 Full GC 后老年代底部抬升，不回落	修代码（找引用链）
容量不足	Full GC 后能降下来，但很快又涨满	加堆 / 优化数据结构 / 加缓存上限
一次性大对象	某个时刻突然涨，之后回落	分批处理、流式读、避免 `SELECT *` 全量加载

区分"泄漏"和"不足"的方法很简单：**看 Full GC 之后的堆占用曲线**。

- 每次 Full GC 后老年代**高于上一次** → 泄漏（阶梯上升）
- 每次 Full GC 后回到同一水平 → 容量不足

```
jstat -gcutil <pid> 1000        # 看 O 列（老年代使用率）
# 或从 GC 日志里看 Full GC 前后的堆占用
```

这个判断决定了你要不要加堆。 泄漏加堆是浪费钱，还会让下次 dump 更难处理。

【必读】一段能稳定复现 Java heap space 的泄漏代码

看十遍支配树，不如自己造一次 OOM。下面这段用**无界静态集合 + ThreadLocal 不 remove** 两条最常见的泄漏路径，配 `-Xmx64m` 能在几秒内稳定复现，且能直接用 MAT 打开验证 24.5 的三步：

```
// ✕ 可复现的泄漏：静态 Map 无界 + ThreadLocal 不清理
public class LeakDemo {
    static final Map<Long, byte[]> CACHE = new HashMap<>();
    static final ThreadLocal<byte[]> TL = new ThreadLocal<>();

    public static void main(String[] args) {
        for (long i = 0; ; i++) {
            CACHE.put(i, new byte[1024]);       // 每个 1KB，永不移除
            TL.set(new byte[1024]);              // 不 remove，线程池场景会一直挂
着
            if (i % 10_000 == 0) System.out.println("put=" + i);
        }
    }
}
```

```
# ✓ 用 64m 小堆跑，配好自动 dump，几秒内必现
java -Xms64m -Xmx64m \
  -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/data/dump/ \
  -Xlog:gc*:file=/data/dump/gc.log:time,uptime \
  -cp . LeakDemo
```

算一下时间：每次循环往静态 Map 塞 1KB，64m 堆去掉 JVM 自身开销约剩 55MB 可用，`55 × 1024 ≈ 56,000` 次就该见底。按每秒十万次的分配速度，**不到一秒就会抛** `java.lang.OutOfMemoryError: Java heap space`，并自动写出 `heap.hprof`。拿这个 dump 打开 MAT：Leak Suspects 会把 `HashMap` 标成嫌疑，Dominator Tree 里它的 Retained Heap 接近整个堆，`Path to GC Roots` 一路指到 `LeakDemo.CACHE` 这个静态字段——正好把 24.5 的三步走一遍。

```
// ✓ 修法：有界 + 过期，ThreadLocal 用完即清
private final Cache<Long, byte[]> cache = Caffeine.newBuilder()
        .maximumSize(10_000)
        .expireAfterWrite(Duration.ofMinutes(10))
        .build();

void handle() {
    TL.set(new byte[1024]);
    try {
        // 业务逻辑
    } finally {
        TL.remove();      // 线程池复用线程，不 remove 就是泄漏（第 9 章）
    }
}
```

这个例子的价值不在"知道 HashMap 会漏"，而在**它把 24.1 那句"'加内存能缓解三天'本身就是证据"变成了可控实验**：把 `-Xmx` 从 64m 加到 128m，OOM 只是从 5.6 万次推迟到 11 万次，曲线形状一模一样。**能靠加堆拖时间、不能靠加堆改形状的，就是泄漏。**

把例子再推一步，验证"泄漏到底归在哪块内存"。启动时加 `-XX:NativeMemoryTracking=summary`，跑完对比两次快照：

```
java -Xms64m -Xmx64m -XX:NativeMemoryTracking=summary \
  -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/data/dump/ -cp . LeakDemo

# 另开终端，在 OOM 前抓基线，OOM 后抓 diff
jcmd <pid> VM.native_memory baseline
jcmd <pid> VM.native_memory summary.diff scale=MB
```

`summary.diff` 里若只有 `Java Heap` 一行在涨，就是纯堆内泄漏（本例的静态 Map）；若 `Thread` 或 `Internal` 也在涨，说明还叠了线程/堆外问题。**这一步把 24.2 那张"冒号后面对照表"从"看报错"变成"看账本"**：NMT 是唯一能同时看到堆、类、线程、CodeCache、GC 结构的视图，代价是 5%~10% 性能开销，所以只在排障期临时开、平时关掉。第 25 章调参时看分配速率，也要靠这份 diff 确认"降下来的到底是分配还是留存"。

24.7　自检清单

- [ ] 启动参数有 `-XX:+HeapDumpOnOutOfMemoryError` + `HeapDumpPath`
- [ ] 有 GC 日志，且滚动配置合理（filecount + filesize）
- [ ] dump 目录挂持久卷，磁盘空间足够（≥ 堆大小 × 1.5）
- [ ] 容器 limit ≥ `-Xmx` × 1.3（或用 `MaxRAMPercentage`）
- [ ] 知道 137 和 Java OOM 的区别，137 会去查 `dmesg`
- [ ] 能区分"泄漏"和"容量不足"（看 Full GC 后的基线）
- [ ] 缓存/集合都有容量上限（`Caffeine.maximumSize`、`LinkedHashMap` LRU）
- [ ] 大查询是分批/流式的，没有 `SELECT *` 全量进内存
- [ ] OOM 后先取证再重启，流程写进了值班手册

24.8　一句话总结

> OOM 是**结果**，不是原因。原因藏在"哪块内存"和"谁在持有它"里。 加堆只会让下一次的现场更大、更难查。

【选读】延伸

- 知识库对应原理篇：02-Java进阶/01-JVM/OOM与CPU问题排查、02-Java进阶/01-JVM/垃圾回收器、02-Java进阶/01-JVM/JVM调优

第二十五章　GC 停顿与 P99 毛刺

本章要回答：平均 RT 只有 30ms，为什么 P99 是 2 秒？

本章你能带走

- 一个判据：**P99 毛刺 + 有规律的间隔 = 先看 GC，不要看代码**
- 三种 Full GC 触发原因，各自的解法完全不同
- 一条调参顺序：**先看对象生命周期，再调堆，最后才换收集器**

难度构成：本章主线【必读】；含【进阶】深挖 1 处；章末「延伸」小节放支线深挖与知识库指针。---

25.1　现场：每 40 秒一次 2 秒抖动

一个接口的平均 RT 是 30ms，P50 是 25ms，P99 是 2100ms。监控图上是一条**周期性的尖刺**，大约每 40 秒一次。

第一反应是"某个下游接口慢"，于是查了所有 RPC、SQL——都正常，P99 都在 20ms 以内。

转折点是有人注意到：**尖刺的间隔非常规律。** 下游抖动不会这么规律，只有两种东西会：定时任务，或者 GC。

查 GC 日志：

```
[2026-11-08T14:02:11.203+0800][gc] GC(128) Pause Full (G1 Evacuation Pause)
4096M->4096M(4096M) 2143.221ms
```

`4096M->4096M` —— **回收前后堆占用完全一样**。

这是最危险的信号：**Full GC 了，但一个字节都没回收。** 说明老年代里全是活对象，堆已经满了。

【必读】4096M->4096M 比 OOM 更危险在哪

25.1 那行日志值得再算一遍，因为它是全书最容易被"看起来只是慢了点"放过的一行：

```
GC(128) Pause Full ... 4096M->4096M(4096M) 2143.221ms
```

三个数字各说一件事：

片段	含义	判据
`4096M->4096M`	回收前 4096M，回收后 4096M	回收量 = 0，老年代全是活对象
`(4096M)`	堆总容量 4096M	已用 = 总量，堆 100% 满
`2143.221ms`	STW 停顿 2.143 秒	整个 JVM 的应用线程被冻住 2.143 秒

多数人以为"Full GC 没抛异常 = 还能撑"，实际是"回收 0 字节"比抛 OOM 更危险：OOM 会让进程快速失败、被编排拉起、告警响；而这次 Full GC 什么都没回收，却让服务"看起来还在"，接下来每一次分配都会再触发一轮同样无效的 Full GC，系统进入"每次都停 2 秒、每次都回收 0 字节"的循环。

把 2.143 秒换成请求数。设入口 QPS = 2000：

```
停顿期间被冻住的请求 = 2000 × 2.143 ≈ 4286 个
按 Little's law，L = λ × W（公式与符号见第 4 章）：λ=2000/s、W=2.143s → 瞬时堆积 4286 个请求
若 Tomcat 200 个线程，积压要靠这 200 个线程在恢复后逐步消化
```

这些请求不是"慢了 2 秒"，是**全被按在原地 2 秒，然后在恢复瞬间一起涌进来**——这正是第 26 章"尖峰流量"的一种，只不过尖峰是 GC 自己造的。更要命的是接下来：堆满 + 全是活对象 → 下一次分配几乎必然 `Promotion failed`（25.3）→ 又一轮 Full GC → 最终 `Java heap space`（第 24 章）。`4096M->4096M` 是这条链的第一帧，不是最后一帧。

> 判据一句话：**Full GC 后堆占用不降，就不要再调任何 GC 参数了**，直接去看"谁在持有这 4G"——那是第 24 章的 dump 和 25.6 的无界缓存。GC 线程在这期间会占满 CPU，取证动作按第 23 章的 `top -Hp` 走，别把 GC 线程当成业务热点。

把 2.143 秒再算到 P99 上，才知道它为什么能藏在"平均 RT 30ms"背后。设每分钟请求数 120,000（2000 QPS），每 40 秒一次 Full GC：

```
受影响的请求 ≈ 2000 × 2.143 ≈ 4286 个
占该分钟比例 = 4286 / 120000 ≈ 3.6%
```

P99 只取最慢的 1%，这 3.6% 全落在 P99 之后的分位里，所以 P50（25ms）纹丝不动、平均值被稀释到 30ms，只有 P99 从 90ms 跳到 2100ms。这也是 25.2 说"P99 异常但 P50 正常"的原因：STW 制造的是**一小撮极端慢请求**，不是整体变慢。反过来说，若你的监控只画平均值和 P50，这次 2 秒停顿在图上是完全看不见的——第 26 章会把"看 P99 不看平均值"列为压测纪律，第 27 章则把"监控只覆盖了你想到的维度"列为最后一个母题。

当时怎么误判的：P99 飙到 2 秒时，第一反应是"某个下游接口慢了"——把 RPC、SQL 的 P99 全拉出来，都在 20ms 以内，排除。最耽误时间的是"会不会是某个定时任务或冷数据查询"：把调度平台翻了一遍、给慢接口逐个 `trace`，半天没找出一个能解释"每 40 秒一次"的元凶。转折是有人把尖刺的间隔标在监控上，发现它规律得像钟表——下游抖动不会这么准，只有定时任务和 GC 会。拉出 GC 日志一查，正是每 40 秒一轮 `Pause Full`。这一下把方向从"业务代码"扭回"JVM 内存"，后面的 dump 才对路。

25.2　P99 毛刺的四种来源

先排除，再优化。P99 异常但 P50 正常，只有四种可能：

来源	特征	判据
GC 停顿	周期性、所有接口同时抖动	GC 日志里有长暂停
下游长尾	只在调用某个下游的接口上出现	分接口看 P99，会分化
锁竞争	并发高时出现，低并发正常	线程栈里有 BLOCKED
线程池排队	与 QPS 正相关	`active` 打满、`queued` 上涨

GC 的独特之处：它让所有接口一起抖动。 因为 Stop-The-World 停的是整个 JVM，不管你在处理什么请求。

所以判断方法很简单：

> **如果同一时刻、所有接口的 P99 一起飙，先看 GC。 如果只有部分接口飙，看下游和锁。**

【必读】四种来源的判据要能落到一条命令上

25.2 那张表给的是"特征"，但特征要能落到一条可执行的判据上，否则排障时还是靠感觉。把四行各补一列"看什么、什么值算命中"：

来源	判据（看什么指标）	什么值算命中	一条命令
GC 停顿	同一时刻**所有接口** P99 一起飙	各接口 P99 曲线在时间轴上重合	`grep 'Pause Full' gc.log`、`jstat -gcutil <pid> 1000`
下游长尾	分接口看 P99，会分化	只有调某下游的接口飙，其余平稳	APM 按 interface 分组；Arthas `trace` 逐段
锁竞争	低并发正常、高并发才出现	线程栈大量 `BLOCKED`，`thread -b` 有锁主	`jcmd <pid> Thread.print -l` 后统计 `BLOCKED` 行数
线程池排队	与 QPS 正相关	`active` 打满、`queued` 持续 > 0	看线程池 `active/queued` 指标

关键是第一行的验证方法：**把 P99 按接口分组画在同一张时间轴上**。GC 是 Stop-The-World，停的是整个 JVM，不管你在处理什么请求，所以所有接口的尖刺必然在同一秒出现；下游、锁、线程池排队都不会这么齐。这也是 25.2 那句"所有接口一起抖先看 GC"能落地的原因——它不是经验，是可证伪的判据。

一个反常识点在第四行：**线程池排队的 P99 毛刺，CPU 往往不高**。线程都在等，没人在算；如果这时你按第 23 章去 `top -Hp` 找热点线程，会一个 99% 的 nid 都找不到。找不到热点，本身就是"这不是 CPU 问题"的证据，方向该切回队列和连接池（第 4 章、第 9 章）。

"所有接口一起抖"要落地，得先把时间轴对齐到同一个桶。做法是给每个接口的 P99 按秒聚合成序列，再比两条曲线是否在同一秒起跳：

```
接口 A P99(秒):  25 24 26 2100 25 24 ...
接口 B P99(秒):  22 23 24 2090 23 22 ...
                          ↑ 同一秒
若 A、B、C 在同一秒同时跳、且幅度相近 → GC / STW
若只有 A 跳、B/C 平稳                  → 下游长尾或 A 独有的锁
```

GC 的判据不是"都变慢了"，是**"变慢发生在同一秒、且幅度相近"**。定时任务也会制造周期性尖刺，但它只影响跑任务的那个接口；GC 不挑接口。把这一步做进监控看板，就不用每次靠肉眼盯曲线——**这也是 25.4 那张日志表里"墙钟时间"那一列的用途：它是和监控时间轴对齐的锚点**。

25.3　三种 Full GC，三种解法

【必读】① 老年代真的满了

```
Full GC 前 4096M → 后 3900M      ← 回收了一点，但很快又满
```

这是**容量不足或泄漏**。回到第 24 章：看 Full GC 后的基线是否抬升。

- 不抬升 → 容量不足，加堆或减少对象
- 逐步抬升 → 泄漏，查 dump

【必读】② 晋升失败（Promotion Failed）

```
[Full GC (Promotion failed) ...]
```

年轻代的对象要晋升到老年代，但老年代没有连续空间。原因通常是**老年代碎片化**，或 Survivor 太小导致对象过早晋升。

解法：加大 Survivor（`-XX:SurvivorRatio`）、减小 `-XX:MaxTenuringThreshold`（让对象早点晋升，减少复制成本），或直接换 G1（G1 按 region 管理，没有连续空间要求）。

【必读】③ 元空间 / 显式调用

```
[Full GC (Metadata GC Threshold) ...]      ← 元空间触顶
[Full GC (System.gc()) ...]                ← 有人手动调了
```

`System.gc()` 必须禁用：

```
-XX:+DisableExplicitGC
```

很多第三方库（尤其是 NIO/DirectByteBuffer 相关）会调 `System.gc()` 来回收直接内存。加上这个参数后，直接内存要靠 `-XX:MaxDirectMemorySize` 限制 + 正确的 `cleaner` 调用来释放。

（三种 Full GC 的完整判据表、System.gc() 的堆外链路与 CMS 版本区间，见章末附录。）

25.4　GC 日志怎么看

JDK 9+ 统一日志（JDK 17 的标准写法）：

```
-Xlog:gc*:file=/var/log/gc.log:time,uptime,level,tags:filecount=5,filesize=50m
```

关键几行：

```
[14:02:11] GC(128) Pause Young (Normal) (G1 Evacuation Pause) 2048M->512M(4096M)
32.441ms
[14:02:51] GC(129) Pause Full (G1 Evacuation Pause) 4096M->4096M(4096M)
2143.221ms
```

读法：

位置	含义	关注点
`Pause Young` / `Pause Full`	类型	Full 是重点
`2048M->512M`	回收前 → 后	**回收后不降 = 满**
`32.441ms`	停顿时间	Young > 200ms 或 Full > 1s 要处理
`(G1 Evacuation Pause)`	触发原因	`Promotion failed` / `Metadata` / `System.gc()`

三个关键指标（用 GC 日志分析工具或 `jstat`）：

```
jstat -gcutil <pid> 1000 20
#  F     YGC    YGCT     FGC    FGCT      GCT
# 12.3   842  12.441     3    6.423   18.864
```

- `FGC` 增长快 → Full GC 频繁
- `FGCT`（Full GC 总时间）占比超过 5% → 已经影响性能
- `GCT` / 运行时间 > 10% → 严重问题

（JDK 17 统一日志逐列拆解、tag 收窄与生效验证，见章末附录。）

25.5　调参顺序：别一上来就换 ZGC

最常见的错误：P99 高 → 直接上 ZGC。结果发现**吞吐掉了 20%，问题还在**（因为根因是内存泄漏，不是收集器）。

正确的顺序：

```
① 先确认对象生命周期
   短命对象多 → 年轻代大一点（-Xmn 或 G1 的 -XX:G1NewSizePercent）
   长命对象多 → 老年代要大，或查泄漏

② 再看堆总大小
   容器 limit 的 60~70%，留出元空间/直接内存/线程栈

③ 再看停顿目标
   G1：-XX:MaxGCPauseMillis=200（默认 200，别设太小，会导致频繁 Young GC）

④ 最后才换收集器
   低延迟（P99 敏感）→ ZGC / Shenandoah
   高吞吐（批处理）   → Parallel
   均衡（绝大多数）   → G1（JDK 9+ 默认）
```

各收集器的定位（JDK 17/21）：

收集器	停顿	吞吐	堆大小	适用
Serial	秒级	高	< 100M	小应用、单核
Parallel	百毫秒~秒	**最高**	中等	批处理、离线任务
G1	几十~几百毫秒	中高	4G~32G	**默认选择**
ZGC	**< 1ms**（JDK 21 分代 ZGC）	中（略低）	大堆 32G+	P99 极敏感

ZGC 在 JDK 21 之后已经分代化，停顿基本在 1ms 以内，代价是吞吐略降、内存占用略高（需要预留染色指针和多视图映射空间）。**它是"已经优化到极致但还差那几十毫秒"时的最后一张牌**，不是第一张。

【必读】调参三段：先止血、再降分配速率、最后才换 ZGC

25.5 给了四步顺序，把它压缩成三段，按"改动代价从低到高"排，每段都有明确的退出条件：

> ① 止血（几小时~当天）
> 禁用 System.gc()、给缓存加 maximumSize、堵住一次性全量查询
> 退出条件：Full GC 不再出现，P99 回到可接受区间
> 不碰：堆大小、收集器、G1 内部参数
>
> ② 降分配速率（一周内，靠数据）
> 用 JFR / async-profiler 火焰图找"谁在不停造对象"
> 指标：分配速率 GB/s、Young GC 频率、晋升速率
> 退出条件：同样流量下分配速率下降，Young GC 次数减少
>
> ③ 最后才换收集器（前两段做完仍不达标）
> 只剩 GC 停顿本身、且 P99 对几十毫秒敏感，才考虑 ZGC
> 换之前必须先有 ①② 的基线数字做对照

第 ② 段最容易被跳过，也最值钱。GC 停顿是"症状"，**对象分配速率才是病因**：设分配速率 `A`、堆有效容量 `H`、每次 Young GC 存活率 `s`，稳态下 Young GC 间隔近似 `T ≈ H×(1−s)/A`。`A` 翻倍，`T` 减半，GC 次数翻倍——调任何参数都不如把 `A` 降下来。25.6 那个无界缓存，本质就是 `s` 被推到接近 1、`H` 被吃满，于是 `T→0`。

（G1 关键参数"什么时候动、动错会怎样"的清单，见章末附录。）版本区间必须记，否则照抄会翻车：**JDK 21 上 `-XX:+UseZGC` 默认仍是非分代，要显式加 `-XX:+ZGenerational`（JEP 439）；JDK 23 起分代成为默认（JEP 474），JDK 24 移除非分代模式（JEP 490），`ZGenerational` 参数作废**。知识库里有一处写"分代 ZGC 是 JDK 21 现代默认"，是错的——按 JEP 439，JDK 21 必须显式开启，本章以此为准。**多数人以为"换 ZGC 能解决所有延迟问题"，实际它只换掉 GC 停顿，换不掉锁竞争、慢 SQL、线程池打满（第 4、9、13 章）**，而且读屏障会带来几个百分点的吞吐损耗。它是"已经优化到极致、还差那几十毫秒"时的最后一张牌。

"一次只改一个变量"也有反面教材：有人同时把 `MaxGCPauseMillis` 从 200 调到 50、把堆从 8G 加到 12G、又换上 ZGC，一周后 P99 确实降了，**但没人知道是哪一步起的作用，也没人敢把这套参数复制到别的服务**。三个变量一起动，前后数字失去了可比性，等于把 25.6 那张对比表作废。正确做法是每段留一组基线数字：止血前后比停顿、降分配速率前后比 GB/s、换收集器前后比 P99 与吞吐；**任何一组数字变差，就只回退那一个变量**。第 26 章讲压测"找拐点"时同样要求每档只改并发这一个变量，道理完全一致。

25.6　一个真实案例的调参过程

回到开头的故事。`4096M->4096M`，说明堆满了没得回收。

排查顺序：

① 先抓 dump 看是什么占了 4G

MAT 打开，`Dominator Tree` 第一行：

```
com.demo.cache.LocalCache  →  3.2 GB
  └─ ConcurrentHashMap$Node[]  →  3.1 GB
```

一个本地缓存没有设上限。

```
// ✗ 无界缓存
private final Map<String, Product> cache = new ConcurrentHashMap<>();
```

```
// ✓ 有界 + 过期
private final Cache<String, Product> cache = Caffeine.newBuilder()
        .maximumSize(10_000)
        .expireAfterWrite(Duration.ofMinutes(10))
        .build();
```

② 改完上线，`4096M->1200M`，Full GC 消失，P99 降到 90ms。

③ 再补一步：加监控。

```
Metrics.gauge("cache.size", cache, c -> c.estimatedSize());
```

这就是完整链路：P99 毛刺 → GC 日志 → dump → 找到无界缓存 → 改代码 → 加监控。全程没有调过一个 JVM 参数。

> **大多数"GC 问题"不是 GC 的问题，是代码的问题。** 调参只能缓解，改代码才能解决。

（一次调参该记的"故障当时/止血后/根治后"三列数字，见章末附录。）

25.7　自检清单

- [ ] GC 日志已开启（JDK 17 用 `-Xlog:gc*`），有滚动配置
- [ ] 有 Full GC 次数和停顿时长的告警（FGC > 0 就该看一眼）
- [ ] 知道自己的收集器和停顿目标
- [ ] 所有本地缓存都有 `maximumSize`
- [ ] 显式 `System.gc()` 已禁用（`-XX:+DisableExplicitGC`）
- [ ] P99 毛刺排查时，先看"是否所有接口同时抖"
- [ ] 调参前先看对象生命周期，不是先换收集器
- [ ] 堆不超过容器 limit 的 70%

25.8　一句话总结

> GC 停顿是"症状"，对象分配是"病因"。 你能调的是参数，但真正要改的是**谁在不停地造对象、谁在不停地留着它们**。

【选读】延伸

- 知识库对应原理篇：02-Java进阶/01-JVM/垃圾回收器、02-Java进阶/01-JVM/JVM调优

【选读】附录

三种 Full GC 的判据表

25.3 按"解法"分了三类，排障时更该按"日志里出现什么"反查。三类加上两种常被漏掉的 G1 触发，凑一张判据表：

类型	日志关键字	回收前后特征	根因	首查动作
老年代满	`Pause Full (G1 Evacuation Pause)`	`4096M->3900M`，回收一点又很快满	容量不足或泄漏	第 24 章看 Full GC 后基线是否抬升
晋升失败	`Promotion failed` / `to-space exhausted` / `Evacuation Failure`	Young 后老年代放不下存活对象	老年代碎片、Survivor 太小、IHOP 太晚	加 `G1ReservePercent`、降 IHOP、查 Humongous
元空间 / 显式调用	`Metadata GC Threshold` / `System.gc()`	与堆占用无关，突兀出现	元空间触顶、第三方库主动调	`jcmd VM.metaspace`、`-XX:+DisableExplicitGC`
Humongous 分配失败	`Humongous` 或大对象分配后紧接 Full	超过单个 Region 一半的对象占满连续 Region	大数组、大字符串、大缓存 Entry	拆大对象、调 `G1HeapRegionSize`
并发模式失败	`Concurrent Mark` 未完成就 `Pause Full`	并发标记跟不上晋升速度	IHOP 太晚、并发线程不足	降 `InitiatingHeapOccupancyPercent`

第 ③ 类里 `System.gc()` 有个必须讲清的链路，否则"禁掉它"会引出新问题：

```
ByteBuffer.allocateDirect / Netty 申请堆外
   → Bits.reserveMemory() 发现超 MaxDirectMemorySize
   → 主动调用 System.gc()，指望触发 Cleaner 回收堆外块
   → 若加了 -XX:+DisableExplicitGC，这一步被吞掉
   → 堆外块只能等下一次自然 GC，泄漏被放大
```

所以 `-XX:+DisableExplicitGC` 不是"加了更安全"，它要**成对使用**：配 `-XX:MaxDirectMemorySize` 给堆外封顶，并确认 NIO/Netty 侧有正确的 `Cleaner` 或显式 `release()`。第 24 章 `Direct buffer memory` 那一行的兜底路径，断的就是这里。CMS 也要顺手提一句版本区间：JDK 9 起弃用（JEP 291）、JDK 14 移除（JEP 363），JDK 17/21 上传 `-XX:+UseConcMarkSweepGC` 只会打印 `Ignoring option ... support was removed` 并静默回退到默认收集器，不是"生效了"。

JDK 17 统一日志逐列拆给你看

25.4 给了两行样例和一张简表，但 `-Xlog:gc*` 打出来的远不止这些。把一段完整输出摆开，逐列对齐：

```
[2026-11-08T14:02:11.203+0800][12.483s][info][gc,start     ] GC(128) Pause Full
(G1 Evacuation Pause)
[2026-11-08T14:02:11.205+0800][12.485s][info][gc,task      ] GC(128) Using 8
workers of 8 for full compaction
[2026-11-08T14:02:13.346+0800][14.626s][info][gc,heap      ] GC(128) Eden
regions: 0->0(24)
[2026-11-08T14:02:13.346+0800][14.626s][info][gc,heap      ] GC(128) Old
regions: 1024->1024
[2026-11-08T14:02:13.346+0800][14.626s][info][gc,metaspace ] GC(128) Metaspace:
262144K->262144K(524288K)
[2026-11-08T14:02:13.346+0800][14.626s][info][gc           ] GC(128) Pause Full
(G1 Evacuation Pause) 4096M->4096M(4096M) 2143.221ms
[2026-11-08T14:02:13.346+0800][14.626s][info][gc,cpu       ] GC(128) User=15.2s
Sys=0.1s Real=2.14s
```

列	例子	读法
墙钟时间	`2026-11-08T14:02:11.203+0800`	与监控尖刺对齐的锚点
uptime	`12.483s`	进程启动后第几秒，算"多久一次"用它
level	`info`	统一日志级别，`debug` 才出更细的 phase
tags	`gc,start` / `gc,heap` / `gc,cpu`	决定这一行讲什么；`gc*` 表示全开
GC 编号	`GC(128)`	一次回收的所有行靠它串起来
阶段与原因	`Pause Full (G1 Evacuation Pause)`	`Pause` 即 STW；括号里是触发原因
堆变化	`4096M->4096M(4096M)`	前→后(总容量)，**回收量 = 前 − 后**
停顿	`2143.221ms`	应用线程被冻住的时间
CPU	`User=15.2s Sys=0.1s Real=2.14s`	`User/Real ≈ 7.1` 说明 8 个 GC 线程并行摊了 2.14 秒墙钟

最后一行很多人跳过，其实它是判"GC 线程有没有抢 CPU"的直接证据：`Real` 是墙钟停顿，`User` 是各 GC 线程 CPU 时间之和。`User/Real ≈ 并行线程数`；若 `User` 远小于 `Real`，说明 GC 线程在等 IO 或被 cgroup 限流（第 23 章 `cpu.stat` 里的 `nr_throttled`）。配置写法与版本区间：

```
# ✓ JDK 9+ 统一日志（JDK 17/21 标准写法），滚动保留 10×50M
-Xlog:gc*:file=/data/log/gc.log:time,uptime,level,tags:filecount=10,filesize=50m

# ✗ JDK 8 时代的写法，JDK 9 起已废弃
-XX:+PrintGCDetails -XX:+PrintGCDateStamps
```

`gc*` 里的 `*` 是通配标签，不是"只打 GC"；想少打一点可以用 `gc,heap`、`gc,pause` 精确组合。**日志必须常开**——JDK 9 起 `-Xlog` 的开销可忽略，出了事才开等于没有现场（第 24 章取证同理）。

`gc*` 是全开，生产上可以按目的收窄，少写几行不影响判据：

tag 组合	会打什么	用在什么时候
`gc*`	全部，含 `gc,heap`、`gc,cpu`、`gc,phases`	排障期、压测期
`gc,heap`	每次回收前后各代占用	常开，看基线是否抬升
`gc,pause`	只打 STW 停顿	只想做 P99 时间轴对齐时
`gc+heap=debug`	Region 级明细	查 Humongous / Evacuation Failure

```
# ✓ 常开最小集：堆变化 + 停顿，够算"回收量"和"基线"
-Xlog:gc,heap:file=/data/log/gc.log:time,uptime:filecount=10,filesize=50m
```

注意 tag 之间没有空格、`+` 表示加装饰器、`=` 后是 level；写错一个字符，日志会静默什么都不打，比没开更迷惑。**上线后用 `jcmd <pid> VM.flags | grep Xlog` 确认参数真的生效**，别只看启动脚本——这一条和 25.5″改完要用 `VM.flags` 验证″是同一件事。

G1 关键参数"什么时候动、动错会怎样"清单

G1 关键参数按″什么时候动、动错会怎样″排，别一次改一组：

参数	管什么	什么时候动	动错的后果
`MaxGCPauseMillis=200`	停顿软目标	P99 对停顿敏感，且日志确认停顿超标	设成 20ms，每次只敢收几个 Region，反而 `Evacuation Failure` → Full GC
`InitiatingHeapOccupancyPercent=45`	并发标记启动的堆占比	老年代涨太快、并发标记来不及	调太低，并发标记频繁抢 CPU
`G1ReservePercent=10`	预留 to-space	出现 `to-space exhausted`	调太高，可用堆变少
`G1HeapRegionSize`	Region 大小（1~32M，2 的幂）	Humongous 对象多	乱调会让大对象判定变化，需实测
`-Xmn`	固定年轻代	**G1 下不要动**	直接关掉自适应，`MaxGCPauseMillis` 永远达不到

前后数字：一次调参该记哪几列

25.6 给了"4096M->1200M、P99 降到 90ms"两个数字，但一次能进复盘的调参记录不止这两个。把"故障当时 / 止血后 / 根治后"三列并排，才知道哪一步真正起了作用：

指标	故障当时	止血后（禁显式 GC + 限流）	根治后（缓存加上限）
P50 / P99 / P999	25ms / 2100ms / 3800ms	25ms / 460ms / 900ms	24ms / 90ms / 160ms
Full GC 次数/小时	90	12	0
单次最长停顿	2143ms	380ms	Young GC 42ms
Full GC 后老年代基线	4096M（不降）	2800M	1200M
分配速率	1.8 GB/s	1.8 GB/s	0.6 GB/s
本地缓存 size	无限增长	无限增长	稳定 ≤ 10,000

三列之间的差别正好对应 25.5 的三段：止血把"停 2 秒"压到"停 380 毫秒"，但**分配速率一行没变、缓存还在涨**——这说明只买了时间，没改病因；根治那一列 `1.8 → 0.6 GB/s` 才是真正让 Full GC 消失的动作。这也是为什么 25.6 说"全程没有调过一个 JVM 参数"：三列里唯一显著变化的 JVM 侧数字（Full GC 次数、基线）都是被代码改动带着变的。

记录格式也有讲究：**看 P99 不看平均值**（第 26 章会展开），看"Full GC 后基线"看是否阶梯抬升（第 24 章），看 `User/Real` 看 GC 线程有没有被限流（第 23 章）。一次调参只改一个变量、压测条件完全相同、指标前后对比——这三条是 JVM 调优素材里反复强调的闭环，缺一条，数字就没有可比性。

> 把这张表贴进 27.5 的复盘改进项里，"验证方式"一栏就有了具体数字：不是"GC 好转了"，而是"Full GC 次数从 90 降到 0、P99 从 2100ms 降到 90ms、连续观察一个业务高峰无回升"。**能被数字复核的改进项，才是能被验证的改进项。**

第二十六章　容量与压测：QPS 是怎么估出来的

本章要回答：大促要准备几台机器？"压测到 5000 QPS"意味着什么？

本章你能带走

- 一个估算公式（从业务量到机器数）
- 压测的正确做法：**找拐点，不是刷峰值**
- 一条经验：**系统应该跑在 30–50% 水位**

难度构成：本章主线【必读】；含【进阶】深挖 2 处；章末「延伸」小节放支线深挖与知识库指针。---

26.1　现场：压测报告很漂亮，大促还是挂了

大促前压测：单机 2000 QPS，10 台机器 = 20000 QPS，远超预估峰值 8000。

大促当天，8000 QPS 打进来，系统在 5000 就雪崩了。

三个原因，都在压测报告里没体现：

1. **压测的是单个接口，真实流量是混合的**——下单链路要调 5 个服务，每个服务都在放大流量
2. **压测时数据库是空的**，真实数据量下同一条 SQL 慢 5 倍
3. **压测是均匀流量**，真实流量有尖峰（10 秒内涌入 3000 QPS）

> **压测报告的"最大 QPS"是最没有意义的数字。** 有意义的数字是"在什么条件下、什么数据量、什么流量模型下，系统还能保持 P99 < 200ms"。

当时怎么误判的：第一反应是"压测本身没压到位，可能是压测机成了瓶颈"——查压测机 CPU 没到 70%、网卡也没打满，排除；又怀疑"是不是大促当天个别慢接口拖垮了整体"，翻单接口 RT 都在预期内，方向还是不对。中间走了一条弯路：怀疑"是不是 K8s 调度把 Pod 打散到弱节点、或跨可用区网络抖动"，逐节点 `kubectl top` 看了一圈全是正常的，这条线索纯属陪跑。真正扭转方向的是把真实大促流量模型拉出来对照压测报告——真实是"下单链路 5 个服务放大 + 10 秒尖峰 + 冷缓存"，而压测报告里全是单接口、均匀流量、热缓存。教训：压测报告里"最大 QPS"是最没意义的数字，有意义的是"在什么条件、什么数据量、什么模型下还能保持 P99"。

26.2　从业务量估 QPS

【进阶】基本公式

```
峰值 QPS = 日请求量 × 峰值系数 / 86400
```

`峰值系数` 是关键，不是 1：

业务类型	峰值系数	说明
内部管理系统	3–5	上班时间集中
电商（日常）	5–8	午休 + 晚间双峰
电商（大促）	**20–50**	整点开抢，几秒内涌入
社交/内容	5–10	晚间高峰
定时任务驱动	100+	整点触发

举例：日订单 100 万，峰值系数 8：

```
峰值 QPS = 1,000,000 × 8 / 86400 ≈ 93 QPS
```

看起来很小？但这是"下单"这一个操作。**一次下单请求内部会放大：**

```
下单 → 查商品(2次) + 查库存(1次) + 查优惠(1次) + 写订单(3次) + 发MQ(1次)
     = 下游 8 次调用
```

所以下游的 QPS 是 `93 × 8 ≈ 744`。**做容量规划要按"放大后的 QPS"算，不是按入口 QPS。**

【必读】二八法则的误用

"80% 的流量集中在 20% 的时间里"——这句话常被用来估算峰值：

```
峰值 QPS = 日请求量 × 0.8 / (86400 × 0.2) = 日请求量 / 21600
```

这个算法对"大促"是完全错的。 大促不是"20% 的时间"，是**"0.1% 的时间"**——9:00:00 到 9:00:30 的流量可能占全天的 30%。

大促的估算要用**"库存/商品数 × 抢购人数"** 反推，不能按日均值乘系数。

【必读】从 DAU 到峰值 QPS：把算式补全

26.2 的公式是 `峰值 QPS = 日请求量 × 峰值系数 / 86400`，其中"日请求量"其实还能再拆一层，落到业务给得出的数上：

```
日请求量 = DAU × 人均请求数
峰值 QPS = DAU × 人均请求数 × 峰值系数 / 86400
```

但大促不能这么算——整点开抢的流量不是"摊在一天里再乘个系数"，而是**集中在几十秒内**。更贴近的写法是把"系数"换成"集中占比 / 集中时长"：

```
峰值 QPS = DAU × 人均请求数 × 集中占比 / 集中秒数
```

给一组能直接对账的数字。某电商：DAU 200 万，人均每天 30 次请求，大促当天 30% 的请求集中在开抢后 30 秒内：

```
日请求量 = 2,000,000 × 30 = 6,000 万次
峰值 QPS = 6,000 万 × 0.30 / 30 = 600,000 QPS   ← 入口
```

用 26.2 里那条 `下单 → 下游 8 次调用` 的放大倍数：

```
下游总 QPS = 600,000 × 8 = 4,800,000 QPS
按 26.5 的健康容量单机 400 QPS、冗余 0.7：
需要实例数 = 600,000 / 400 / 0.7 ≈ 2143 台    ← 显然不能这么硬扛
```

最后这个数字本身就是结论：按入口 60 万 QPS 硬算，机器数是四位数，任何团队都不会这么买。所以大促真正的解法不是"估 QPS 然后堆机器"，而是第 22 章的秒杀库存前置、第 6 章的削峰、以及 26.3 的限流——**算式的价值不在算出机器数，在于证明"必须削峰"**。这也是 26.2 那句"做容量规划要按放大后的 QPS 算"的量化版：入口 60 万看着已经很大，放大后 480 万才是压到数据库连接池上的真实数字（第 4 章）。

一个反常识点：**DAU × 人均请求这两个数，通常比"日请求量"更容易从业务方拿到，也更不容易拍脑袋**。让运营报"明天多少 QPS"，多半得到一个整数；让他们报"多少活跃用户、每人点几次"，反而能追问出依据。

【必读】二八法则错在把"时间"当成均匀分布

26.2 给了二八算式 `峰值 QPS = 日请求量 / 21600`，把它和上面的大促数字放一起，误差立刻可见。仍用日请求量 6,000 万：

```
二八估法：6000 万 × 0.8 / (86400 × 0.2) = 6000 万 / 21600 ≈ 2778 QPS
集中估法：6000 万 × 0.30 / 30 = 600,000 QPS
两者相差 ≈ 216 倍
```

差的不是系数，是**对"时间"的假设**。二八法则假设 80% 的流量铺在 20% 的时间（约 4.8 小时）里，且在这 4.8 小时内均匀分布；大促是 30% 的流量挤在 30 秒（占全天的 0.035%）里。把"0.035% 的时间"代进"20% 的时间"的公式，分母大了近 600 倍，峰值自然被低估两个数量级。**二八法则不是错在 80/20 这两个数，是错在它默认流量在窗口内均匀**——这个默认在日常流量里勉强成立，在整点开抢里彻底失效。

判据也很简单，问一句"峰值窗口有多长"：

场景	峰值窗口	该用的估法
内部系统上班高峰	1~2 小时	日请求量 × 系数 / 86400，够用
日常双峰（午休+晚间）	各 1 小时	二八或峰值系数 5~8
整点开抢 / 秒杀	10~60 秒	**库存×人数反推，或 集中占比/集中秒数**
定时任务整点触发	秒级	按任务实例数×单次请求数直接算，别用日均

26.2 说"大促的估算要用库存/商品数 × 抢购人数反推"，补一个算式：`峰值 QPS = 抢购人数 / 抢购窗口秒数 × 人均提交次数`。10 万人抢、30 秒窗口、人均重试 2 次 → `100000/30×2 ≈ 6667 QPS` 的下单入口，再乘 8 倍放大给下游。**这种"从人数和窗口反推"的算式，比从 DAU 顺推更抗拍脑袋**，因为人数和秒数都是活动方案里写死的。

26.3　压测：找拐点，不是刷峰值

【必读】阶梯加压

```
第 1 轮：100 QPS，持续 5 分钟 → 记录 P99、错误率、CPU
第 2 轮：200 QPS，持续 5 分钟
第 3 轮：400 QPS ...
每一轮翻倍，直到 P99 超过目标 或 错误率 > 1%
```

你要找的是"拐点"——P99 开始非线性上升的那个点。它代表系统的真实容量。

```
QPS     P99       CPU    错误率
100     45ms      20%    0%
200     48ms      35%    0%
400     55ms      60%    0%
800     180ms     85%    0.2%     ← 拐点出现了
1600    1200ms    95%    8%       ← 已经崩了
```

这个系统的健康容量是 **400 QPS**，不是 1600。

【必读】压测必须满足的四个条件

条件	为什么	怎么做到
数据量接近生产	100 行和 1 亿行的同一条 SQL 差几十倍	影子库导入生产数据（脱敏）
流量模型接近真实	均匀流量比尖峰"好压"	加尖峰：10 秒内打 3000 QPS
链路完整	单接口压测测不出放大效应	全链路压测（打入口，走完整调用链）
读写成比例	纯读和纯写的瓶颈完全不同	按真实比例混合

"影子库 + 全链路压测"是唯一能反映真实的做法。 单接口压测只能用来做"容量标定"，不能用来判断"大促扛不扛得住"。

【必读】压测最容易漏的三件事

1. **缓存是热的**。压测前预热缓存，和压测时不预热，结果能差 5 倍——大促当天缓存是冷的
2. **连接池是满的**。压测时长时间运行，连接池已经建立；大促瞬间涌入时，建连本身要耗时
3. **日志是开着的**。压测往往关日志，生产是开的，日志 IO 会吃掉 10–20% 性能

【必读】拐点是三条曲线一起拐，不是一条

26.3 的阶梯加压表有 QPS / P99 / CPU / 错误率四列，但"拐点"真正的判据要看三条曲线**同时**发生变化，只看 RT 容易被客户端假象骗过去：

曲线	健康段	拐点判据	越过之后
吞吐（RPS）	随并发/速率线性增长	**不再线性增长，进入平台期**	加并发只加排队，不加吞吐
RT（P99）	平缓、P99/P50 < 3	**非线性上翘**，P99/P50 > 5	排队时间主导 RT
错误率	0	**> 0.1% 且随压力上升**	超时、拒绝、连接池耗尽

三条曲线要一起看，是因为它们各自都能单独"假拐"：吞吐平台期可能只是压测机自己满了（26.4 第一行）；P99 上翘可能只是 GC 停了一下（第 25 章）；错误率上升可能只是参数化脚本造出了 404。**只有"吞吐不再涨 + P99 非线性上翘 + 错误率破 0"同时出现，才是服务端真实容量上限。** 26.3 那张表里 800 QPS 那一档正好三条齐了：`180ms`（P99 从 55 跳到 180）、`0.2%`（错误率破 0）、CPU 85%（吞吐逼近极限）；1600 档 `1200ms / 8%` 已经是崩了，不是拐点。

还有一个必须说清的坑：**测 P99 要用恒定速率发压，不能用"尽力打满"**。wrk 默认模式是"发完立刻发下一个"，实际 RPS = 并发 / RT；服务一变慢，RPS 自动下降，延迟分布里混进了排队时间，这叫 Coordinated Omission，测出来的 P99 偏乐观。wrk2 用 `-R` 按固定 RPS 匀速发压，才模拟"真实流量恒定、服务变慢"：

```
# ✗ 尽力打满：服务变慢时客户端自动降速，P99 被排队污染
wrk -t8 -c200 -d60s --latency http://svc/order/list

# ✓ 恒定速率：每档固定 RPS，测出来的 P99 才有容量评估价值
for r in 100 200 400 800 1600; do
  echo "=== RPS $r ==="
  wrk2 -t8 -c200 -d180s -R$r --latency http://svc/order/list | tee "rps-$r.log"
  sleep 30     # 冷却，避免上一档排队污染下一档
done
```

每档之间那 30 秒冷却不是仪式：上一档把连接池和队列打满后，不冷却直接进下一档，会把"恢复期"算进新一档的 RT，拐点会提前出现在一档偏低的位置。

三条曲线还必须**按同一时间窗聚合**再比，否则会互相错位：吞吐常按 10 秒窗口统计，P99 按 1 分钟窗口统计，错误率按整轮累计，三者横轴不一样时，"同时拐"这个判据根本不成立。做法是压测脚本统一输出秒级桶，再用同一个桶算三条：

```
# ✓ 每档结束后从 wrk 日志抽同一批数据，别用三个不同窗口
grep 'Latency Distribution' rps-800.log
grep 'Requests/sec'         rps-800.log
grep 'Socket errors'        rps-800.log   # timeout / Non-2xx 非 0 即拐点信号
```

`Socket errors` 这一行尤其不能漏：`timeout` 或 `Non-2xx responses` 一旦非 0，说明已经有请求没能被正常服务，它比 P99 更早宣告"这一档越线了"。

【必读】最容易漏的三件事：数据热度、缓存预热、依赖方

26.3 列过"缓存是热的、连接池是满的、日志是开着的"三条，那是从**运行时状态**说的。换一个角度，从"压测数据和环境"看，最容易漏的其实是另外三件，而且它们各自能把结果抬高好几倍：

漏掉的	压测里什么样	生产里什么样	差多少	补法
数据热度	反复查同 100 个 ID，Buffer Pool 全命中	真实长尾分布，冷热混合，回表多	同一 SQL 可差 5 倍以上	影子库导生产脱敏数据；Feeder 按真实分布取 ID
缓存预热	压测跑久了缓存已热，命中率 99%	大促 9:00 冷启动，命中率从 0 爬升	回源 QPS 差 5~10 倍	专门加一轮"清缓存后 30 秒冲峰"
依赖方	第三方被 mock 或放开限频，永不失败	三方有限频、有超时、会抖动	错误率与 RT 完全不可比	依赖方按生产 SLA 注入延迟与限频

三条里**数据热度**最隐蔽：它不是"数据量不够"（那是 26.3 四条件里的第一条），而是"数据分布不对"。1 亿行表里压测只碰那 100 行，索引和页缓存全在内存，SQL 快得像是另一个系统。判据是压测中盯 `Buffer Pool hit rate`：若它贴着 100% 不动，这轮压测的 DB 结论作废。

```
-- ✗ 固定 ID：压出的是缓存命中率，不是接口性能
request = function()
  return wrk.format("GET", "/sku/10086")
end

-- ✓ 按真实长尾取 ID + 一定比例冷数据，把热度分布压出来
request = function()
  local id = math.random(1, 10000000)
  if math.random(1, 10) == 1 then id = math.random(1, 100000000) end
  return wrk.format("GET", "/sku/" .. id)
end
```

Gatling 侧的等价做法是用 Feeder 喂真实 ID 分布（`csv("sku-ids.csv").circular`），并在场景开头加一段"冷缓存冲峰"。**这三件事和 26.3 四条件是乘法关系，不是加法**：数据量对了但热度不对，仍然会高估容量；四条件全满足而依赖方被 mock，全链路结论依然不成立。第 23 章那个正则回溯也是同一类——参数值域没覆盖到最坏输入，压测就永远看不见它。

26.4　瓶颈在哪：四层排查

压测时 QPS 上不去，按这个顺序找：

层	指标	典型瓶颈
压测机	压测机自己的 CPU/网络	压测机先满了（最常见，也最尴尬）
入口	Nginx 连接数、`$request_time - $upstream_response_time`	连接池、keepalive（第四章）
应用	Tomcat 线程池、GC、CPU	线程池打满、Full GC（第九章、第二十五章）
下游	DB 连接池、慢 SQL、Redis	连接池耗尽、慢查询（第四章、第十三章）

先确认压测机不是瓶颈——这一条听起来很傻，但实际压测中 30% 的"系统上不去"是压测机自己 CPU 满了。

判断方法：压测机的 CPU 如果超过 70%，先加压测机。

26.5　容量水位：为什么是 30–50%

一个反直觉的原则：

> **系统的设计容量应该是"实际峰值 2–3 倍"，日常运行在 30–50% 水位。**

理由：

- 留出**突发余量**（大促、热点事件、爬虫）
- 留出**故障余量**（一台机器挂了，其余要能扛住）
- 留出**长尾余量**（P99 的请求需要更多资源）

所以"10 台机器刚好扛住 8000 QPS"是危险的配置——**一台挂掉就雪崩**。

正确的算法：

```
需要机器数 = 峰值 QPS / (单机健康容量) / (1 - 冗余比例)
          = 8000 / 400 / 0.7 ≈ 29 台
```

28 台不是浪费，是为"挂两台还能活"付的钱。

【必读】30–50% 水位：用排队论把理由写成算式

26.5 说"日常跑在 30–50% 水位"，给的三条理由（突发、故障、长尾余量）都对，但都能被一句"我们机器够，跑 70% 也没事"顶回来。把理由写成排队论的算式，就没得争了。

把单机看成一个 M/M/1 队列：到达率 `λ`、服务率 `μ`、利用率 `ρ = λ/μ`。平均响应时间：

```
W = 1 / (μ − λ) = (1/μ) / (1 − ρ) = W0 / (1 − ρ)
```

其中 `W0 = 1/μ` 是无排队时的纯服务时间。**关键是 `1/(1−ρ)` 这一项，它随 ρ 接近 1 而发散**：

利用率 ρ	W / W0	相对 30% 水位放大
0.30	1.43×	1.00×
0.50	2.00×	1.40×
0.70	3.33×	2.33×
0.80	5.00×	3.50×
0.90	10.0×	7.00×
0.95	20.0×	14.0×

ρ 从 0.5 涨到 0.9，QPS 只多了 80%，RT 却涨了 5 倍——**这就是 26.3 拐点"P99 非线性上翘"的数学来源，也是 25.2 里线程池排队那类毛刺的来源**。所谓 30–50% 水位，本质是给 `1/(1-ρ)` 留一段还近似线性的区间：ρ=0.5 时 RT 只是纯服务时间的 2 倍，ρ=0.8 时已是 5 倍，再往上一点抖动就会把 P99 拉到不可接受。

故障余量也能算。N 台机器均摊，挂 1 台后剩余利用率 `ρ' = ρ × N/(N-1)`：

```
N=10、ρ=0.5 → ρ'=0.556，还在可控区间
N=3 、ρ=0.5 → ρ'=0.75，W/W0 从 2.0 跳到 4.0
N=2 、ρ=0.5 → ρ'=1.00，队列发散，直接雪崩
```

这就是"10 台刚好扛住"危险、也是 26.5 算式里那个 0.7 的来历：它不是拍脑袋的冗余系数，是"挂一台之后 ρ 别越过 0.7"的约束。回到 `8000 / 400 / 0.7`：

```
8000 / 400 = 20 台（刚好打满，ρ=1.0）
20 / 0.7 ≈ 28.6 → 取 29 台，日常 ρ ≈ 0.7
挂 1 台后 ρ' = 0.7 × 29/28 ≈ 0.725，仍在线性区边缘
```

原文 26.5 结尾写"28 台不是浪费"，按算式 `20/0.7 = 28.57` 应向上取整取 **29 台**，这里以算式为准确掉那一位。多出来的一台，买的就是 `ρ'` 不越线。

26.6　监控：容量的先行指标

容量问题不是突然发生的，它有先行指标：

指标	含义	阈值
线程池 `queued`	开始排队了	> 0 持续 1 分钟
连接池 `pending`	拿不到连接了	> 0 持续 1 分钟
GC 频率	内存压力	Full GC > 0 / 小时
P99 / P50 比值	长尾开始出现	> 5 倍
CPU 使用率	最后才会动	> 70%

前三个比 CPU 早得多。 CPU 到 70% 时，系统已经在排队了；而 `queued > 0` 的时候，用户还能感觉到"有点慢"但没报障——**这是扩容的最后窗口**。

【进阶】先行指标：扩容的最后窗口有多长

26.6 的表列了五个先行指标，没说的是**它们之间的先后顺序，以及这段顺序给了你多长反应时间**。按一次真实容量爬坡，指标动起来的次序是固定的：

```
① 线程池 queued > 0 / 连接池 pending > 0     ← 开始排队，用户只觉得"有点慢"
② P99/P50 比值 > 5                           ← 长尾出现
③ 吞吐进入平台期，RT 非线性上翘              ← 26.3 的拐点
④ GC 频率上升（分配压力变大）                ← 第 25 章
⑤ CPU > 70%                                  ← 最后才动
```

CPU 是最后一个动的，不是第一个。 因为排队的线程大部分时间在等锁、等连接、等 IO，不占 CPU；等它们终于都开始算，系统早就过了拐点。这解释了一个反常识现象：`queued > 0` 时 CPU 常常才 40%，看起来"还很有余量"，其实扩容窗口已经开始关闭了。

窗口有多长，可以估：设当前到达率 `λ`、健康容量 `μ`、从 `queued>0` 到 `ρ=1` 的余量是 `μ-λ`，若流量以每秒 `g` 的速率增长，剩余时间 `t ≈ (μ-λ)/g`。举例 `μ=400 QPS`、当前 `λ=280 QPS`、大促前流量折算 `g≈1 QPS/s`：

```
t ≈ (400 − 280) / 1 = 120 秒
```

两分钟。 这就是为什么 26.6 说"`queued > 0` 是扩容的最后窗口"——它不是"还早"，是从这个信号到雪崩只剩一次灰度发布的时间。所以先行指标必须配**自动扩容或预扩容**，不能配"人看告警再决定"；第 27 章把这类"监控能不能提前到足够早"归为深层原因，也是同一个道理。

补齐 26.6 那张表里该带的阈值来源：

指标	阈值	阈值从哪来
`queued > 0` 持续 1 分钟	排队起点	26.3 拐点前一档
`pending > 0` 持续 1 分钟	连接池起点	第 4 章连接池容量
P99/P50 > 5	长尾起点	26.3 健康段的基线比值
Full GC > 0/小时	内存压力	第 25 章 GC 日志基线
CPU > 70%	**最后确认，不是触发**	压测平台期对应值

阈值不该拍脑袋，全部从"压测健康段最后那一档"反推——这也是 26.3 说"没有基线的压测等于没压"的另一半含义：**基线不只用来报容量，还用来定告警线**。

最后补一条和第 27 章的接口：先行指标要**能触发动作**，否则只是好看的曲线。`queued > 0` 的告警不该发给人，该直接触发 HPA 或预扩容工单——按上面算出的两分钟窗口，等人登录控制台再决定，窗口已经关了。把"指标 → 阈值 → 动作 → 验证"四项写进应急预案，是第 27 章"改进项必须可验证"在容量领域的版本：**一条没有绑定动作的告警，等于没有告警**。

26.7　自检清单

- [ ] 有 QPS 估算文档，写清了业务量、峰值系数、放大倍数
- [ ] 压测用的是生产级数据量（影子库），不是 100 行测试数据
- [ ] 压测包含尖峰模型，不只是均匀流量
- [ ] 做过全链路压测，不只是单接口
- [ ] 压测时开了日志、缓存是冷的（模拟真实）
- [ ] 知道每个服务的"健康容量"（拐点 QPS）
- [ ] 机器数 = 峰值 / 健康容量 / 0.7（留冗余）
- [ ] 日常水位在 30–50%
- [ ] 监控了 `queued` / `pending` / P99-P50 比值

26.8　一句话总结

> 容量规划的本质不是"能扛多少"，是**"在挂掉一部分之后还能扛多少"**。 只按 100% 健康算出来的容量，等于没有冗余。

【选读】延伸

- 知识库对应原理篇：12-性能与稳定性/压测与容量规划、12-性能与稳定性/性能指标与监控

第二十七章　一次故障复盘该怎么写

本章要回答：复盘写完，除了归档，还能留下什么？

本章你能带走

- 一份可直接套用的复盘模板（一页纸 + 五个部分）
- 一个区分：**触发因素 / 直接原因 / 深层原因**，混在一起就找不到改进点
- 一条判据：**改进项如果不能被验证，就等于没写**

难度构成：本章主线【必读】；含【进阶】深挖 0 处；章末「延伸」小节放支线深挖与知识库指针。---

27.1　现场：两份复盘

同一个故障，两份复盘。故障是第六章那个超卖：库存扣了，订单没写，多扣 347 件。

第一份（真实存在过的写法）：

> **故障描述**：11 月 8 日预售活动期间，出现库存超卖，影响约 300 单。 **原因分析**：开发同学在实现时对 Spring 事务机制理解不够深入，未使用正确的事务写法。 **改进措施**：1. 加强代码评审；2. 组织 Spring 事务培训；3. 提高开发人员质量意识。 **责任人**：张三

第二份：

> **影响**：9:04 – 10:31，共 347 次扣减无对应订单，涉及 312 个用户，潜在资损 0 元（已拦截发货），客诉 6 起。 **时间线**：见下（精确到分钟，含三次误判） **触发因素**：营销服务 9:00 起 RT 从 20ms 涨到 900ms（当日上线了新功能）。 **直接原因**：`deductStock` 通过 `this` 自调用，`@Transactional` 从未生效。 **深层原因**：① 团队缺少"事务边界"的评审项；② 监控只按 HTTP 状态码统计错误，业务失败不可见；③ 库存与订单无常量对账。 **改进项**：见下（6 项，含验证方式与 owner）

差别不在文笔，在**能不能据此行动**。

第一份的三条改进措施，没有一条可以被验证。"加强评审"——怎么算加强了？"组织培训"——培训完怎么知道有效？"提高意识"——这是一个无法证伪的句子。

而它的"原因分析"其实是一句责难：把系统性问题归到"某人不熟悉"。下次换一个人，同样的 bug 会再来一次——因为环评项、监控盲区、对账缺失，一个都没变。

【必读】两份复盘逐项对照：差在能不能据此行动

27.1 摆了两份复盘，说"差别不在文笔"。把这句话拆成可以逐格打勾的对照，才知道差在哪儿：

维度	第一份	第二份	能不能据此行动
影响	"影响约 300 单"	9:04–10:31、347 次扣减、312 个用户、6 起客诉、资损 0 元	第一份无法定优先级
时间线	无	精确到分钟，含三次判断与两次判断修正	第一份无法还原决策路径
原因	"对事务机制理解不够深入"	触发 / 直接 / 深层三层分开	第一份把系统问题归到个人
改进项	加强评审、组织培训、提高意识	6 项，含验证方式、Owner、期限	第一份三项全不可验证
责任人	张三（一个人）	每项一个 Owner + 期限	第一份是追责，不是分工
同类风险	未提	明确"发现一处，扫一遍全站"	第一份只修了这一个 bug

第一份那三条改进措施，用 27.5 的判据逐条量一下就露馅了：**"加强代码评审"没有验收动作，"组织培训"没有可观测的结果，"提高质量意识"是一个无法证伪的句子**。它们不是"写得不够细"，是**在语法上就不可能被完成**——你没法给"提高意识"打勾。

更关键的是它的"原因分析"。把"事务没生效"写成"某人不熟悉 Spring 事务"，等于把一次系统性失效压缩成一次个人失误；下次换个人写，`this.xxx()` 自调用、`final` 方法、`catch` 吞异常、`@Async` 叠加（第 6 章）还会再来一遍，因为**评审项、监控口径、对账机制一个都没变**。第二份的深层原因三条——无评审项、错误率只按 HTTP 状态码、无对账——每一条都指向一个能被改掉的机制。

> 一个反常识点：**复盘写得越像检讨，可行动的信息越少。** 检讨的终点是"谁负责"，复盘的终点是"下次在哪一步能拦住"。第 6 章那个 `isActualTransactionActive()` 之所以值钱，不是因为它高深，是因为它是个布尔值——**能回答"是/否"的东西，才能写进验证方式那一列**。

当时怎么误判的：写第一份复盘时，第一反应是"这是张三对 Spring 事务理解不深、质量意识不足"——直接把系统性失效压缩成个人失误，连查都没查就归了责。中间还走了一条死路：以为"加强代码评审、组织事务培训、提高质量意识"三条已经是合格改进项，逐项对着"能不能打勾验证"量一遍才发现它们语法上就不可能完成，全是愿望不是动作。真正扭转方向的是把原因拆成"触发 / 直接 / 深层"三层——深层原因落到了"CR 无事务边界检查项、错误率只按 HTTP 状态码、库存订单无对账"这些能被改掉的机制上。教训：复盘追流程不追人，能问"下次在哪一步拦住"的复盘才值钱，能问"谁负责"的只是在写检讨。

27.2　三层原因，必须分开

这是复盘里最容易混、也最影响改进质量的一步。

层次	定义	例子	对应的改进
触发因素	让问题从"存在"变成"发生"的那件事	营销服务当天变慢	通常**不需要改**，它只是时机
直接原因	代码/配置层面"哪里错了"	自调用导致事务未生效	修代码 + 加静态检查
深层原因	为什么这个错误**能上线、能运行 47 分钟没被发现**	无评审项、监控盲区、无对账	**这才是复盘的产出**

只写直接原因，改进项就只能是"改代码"——而代码永远会再写错。

第六章那个故障，如果只改代码（改成跨 Bean 调用），下次换个姿势（`final` 方法、`catch` 吞异常、`@Async` 叠加）还会中招。**真正的修复是"把这类错误的识别成本降到最低"**：CR 检查项、静态扫描、监控口径、对账。

一条实用的提问方式：

> **"如果下个月新来的同事写了同样的代码，我们能不能在它上线前拦住？"** 答不上来，说明深层原因没找到。

【必读】三层原因的判据：一句话该归哪一层

27.2 给了定义和例子，但真写的时候，最难的是"这句话该放哪层"。给一组判据问句，写完全文逐句过一遍：

层次	判据问句	答"是"就归这层	反例（容易放错）
触发因素	去掉它，这个 bug 还在不在？	"还在，只是没被触发"	把"营销服务变慢"写成根因——它只是时机
直接原因	改掉这一处代码，这次还会不会发生？	"这一次不会了"	只写"改代码"，就停在这儿
深层原因	换个人、换个写法，同样的错能不能再次上线且长时间没人发现？	"能"	把"没评审项"写成"开发不细心"

三个问句是有顺序的：**先确认"去掉触发因素问题仍在"**，才能把触发因素和直接原因分开；**再确认"改这一处只解决这一次"**，才能把直接原因和深层原因分开。27.1 那次超卖，若把"营销服务 9:00 RT 涨到 900ms"当根因，改进项就会变成"让营销服务别变慢"——可营销服务下次上新功能还会慢，而库存照样会多扣。

深层原因的标准写法也有一条判据：**它必须指向一个"能被改掉的机制"，而不是一种"人的状态"**。

> ✗ 深层原因：开发人员对事务边界不敏感、质量意识不足
> ✓ 深层原因：CR checklist 无「事务边界」项；
> 错误率只按 HTTP 200/5xx 统计，业务失败不可见（第 11 章）；
> 库存与订单无常量对账，错误只能靠客诉发现（第 6 章）

三条都能对应到 27.5 里的一条改进项，且每条都有验证方式。**写不出"对应哪条改进项"的深层原因，多半还没挖到位。**

27.3　时间线：写"我们以为"，不只写"发生了什么"

普通的时间线只记录系统事件：

```
14:22 商品服务 RT 上涨
14:24 订单服务大量超时
14:28 网关 502
14:35 定位到问题
14:40 修复上线
```

这种时间线没有复盘价值，因为它只记录了"系统做了什么"，没记录"人做了什么判断"。

有价值的时间线，每一段都带当时的判断和依据：

```
14:22  商品服务 RT 从 20ms → 800ms（监控无告警，未超过阈值）
14:24  订单服务出现 Read timed out（值班同学判断：下游抖动，等自愈）
14:29  超时率超过 10%，决定扩容订单服务（判断：容量不足）
14:31  扩容无效，P99 继续上涨（判断修正：不是容量问题）
14:35  查到 Feign 默认 1s 超时（判断：超时配置问题）
14:38  调整超时到 8s，情况恶化（判断再次修正：超时放大线程占用）
14:44  回滚超时配置 + 熔断，流量恢复
```

"判断"和"判断修正"是时间线里最值钱的部分，因为它是下一次排障的路线图。它同时暴露了两件事：

- 哪些判断是对的（可以固化成手册）
- 哪些判断是错的（需要补什么信息才能判断对）

没有"误判"的复盘，通常意味着有人在美化自己。

【必读】时间线模板：把「我们以为」单独立一列

27.3 给了两段示例，但没给能直接套用的表头。普通时间线只有"时间 / 事件"两列，缺的正是**人当时怎么想**。把「我们以为」和「依据」各自单独立一列，判断才有被复核的可能：

```
| 时间 | 系统事件 | 我们以为 | 依据 | 后来发现 | 判断修正 |
|---|---|---|---|---|---|
| 14:22 | 商品 RT 20→800ms | 下游抖动，等自愈 | 未超告警阈值 | 是超时配置被放大 | 应看线程占用而非 RT |
| 14:29 | 超时率 >10% | 容量不足 | 历史经验 | 扩容无效 | 不是容量问题 |
| 14:35 | 查到 Feign 默认 1s | 超时配置太小 | 配置面板 | 调大后更糟 | 超时放大了线程占用 |
| 14:44 | 回滚 + 熔断 | — | — | 流量恢复 | 根因是超时×重试（第 2 章） |
```

```
<!-- ✗ 两列时间线：只记系统做了什么，人做了什么判断全丢 -->
| 时间 | 事件 |
|---|---|
| 14:22 | 商品服务 RT 上涨 |
| 14:40 | 修复上线 |

<!-- ✓ 六列时间线：判断与判断修正都可复核 -->
| 时间 | 系统事件 | 我们以为 | 依据 | 后来发现 | 判断修正 |
```

六列里最值钱的是「依据」和「判断修正」两列：**依据**让当时的判断可被理解（不是"他们怎么这么蠢"，是"他们手上有这些信息"）；**判断修正**暴露的是"还缺什么信息才能判断对"，直接对应 27.5 里的监控类改进项。27.3 说"没有误判的复盘，通常意味着有人在美化自己"——六列模板让美化变难，因为「我们以为」和「后来发现」并排放着，删掉前者，时间线就缺一格。

27.4　影响面：量化到"钱"和"人"

"影响约 300 单"是不够的。要回答四个数：

```
影响时长：   9:04 – 10:31（87 分钟）
影响用户：   312 人（其中 6 人发起客诉）
影响业务：   347 次扣减无订单，3,000 单正常
资损：       0 元（已拦截发货）/ 或 12,400 元（需赔付）
```

为什么要算到钱：因为它决定了改进项的优先级。资损 0 元的事故和资损 50 万的事故，投入的资源完全不同。而"影响 300 单"这个说法，无法做这个判断。

还有一个常被忽略的维度：**同类风险的存量**。"这次是 A 接口，那 B、C 接口有没有同样的问题？"——**发现一处，就要扫一遍全站。** 这一条能防止"修了一个，漏了十个"。

【必读】影响面：把「钱」和「人」写成算式

27.4 说"影响约 300 单是不够的"。把四个数写成可复算的算式，改一个输入就能重算，也能和财务、客服对数：

```
① 影响人时 = 受影响用户数 × 平均阻断时长
            = 312 人 × (87 分钟 × 不可下单占比 0.6)
            ≈ 312 × 52 分钟 ≈ 270 人·小时

② 资损 = 需赔付订单数 × 客单价 × 赔付比例 + 履约成本
       = 347 × 265 元 × 0.10 + 0（已拦截发货）
       ≈ 9,196 元
   若未拦截发货：347 × 265 × 1.0 = 91,955 元（全额货值）

③ 客诉率 = 客诉数 / 受影响用户数 = 6 / 312 ≈ 1.9%

④ 错误预算消耗 = 故障时长 / 月度允许不可用时长
              = 87 分钟 / (30×24×60 × 0.001) = 87 / 43.2 ≈ 2.0 倍
```

为什么非要算到钱？因为改进项之间要排序。资损 0 元（已拦截）和资损 9 万（未拦截）是同一种故障的两个结局，投入的资源完全不同；而"影响 300 单"这个说法，既看不出拦截动作的价值，也算不出赔付敞口。**27.4 那句"为什么算到钱：因为它决定改进项的优先级"，落到实处就是上面这四行算式。**

补一个 27.4 提到但没展开的维度——**同类风险存量**也要量化：`扫描范围 = 全站 @Transactional 方法数`，命中 `this.xxx()` 自调用组合 N 处，逐个确认；这一条能防止"修了一个，漏了十个"。算式写进 27.5 的改进项验证方式里，就是"扫描出 N 处，N 处全部确认或修复"。

27.5　改进项：三类，且必须可验证

```
| # | 类型 | 改进项 | 验证方式 | Owner | 期限 |
|---|---|---|---|---|---|
| 1 | 止血 | 修复自调用，改跨 Bean 调用 | 灰度后对账 24h 无差异 | @张三 | 已完成 |
| 2 | 止血 | 库存与订单增加小时级对账 | 人为构造 1 条差异，告警能触发 | @李四 | 11-10 |
| 3 | 根治 | 全站扫描 `this.xxx()` 自调用 + `@Transactional` 组合 | 扫描出 N 处，逐个确认 | @张三 | 11-15 |
| 4 | 根治 | 所有 `@Transactional` 补 `rollbackFor` | 静态检查规则上线，CI 拦截 | @王五 | 11-20 |
| 5 | 预防 | 错误率统计改为按业务 `code` 而非 HTTP 状态码 | 人为抛异常，看错误率是否上涨 | @李四 | 11-12 |
| 6 | 预防 | CR 模板增加「事务边界」检查项 | 抽查 10 个 MR，全部填写 | @张三 | 11-09 |
```

"验证方式"这一列是关键。 它把"我们要加强 XX"变成"我们怎么知道加强了"。

三类的区别：

- **止血**：让当前问题不再发生（通常当天完成）
- **根治**：让"这一类"问题不再发生（需要改流程/工具）
- **预防**：让"下一类"问题能被更早发现（监控/对账/演练）

只有止血的复盘是不合格的，因为它是"修了这个 bug"，不是"减少了一类 bug"。

【必读】改进项三类都要可验证，以及五个反模式的真实写法

27.5 那张六行表已经把"验证方式"列出来了。把它抽象成一条判据：**每条改进项都要能回答"谁、在什么条件下、做什么动作、看到什么结果算完成"**。四种验证方式基本覆盖全部场景：

验证方式	适合哪类改进	可打勾的结果
对账 / 差异注入	止血（数据正确性）	人为构造 1 条差异，告警能触发
演练 / 故障注入	预防（降级、熔断）	注入依赖延迟 500ms，成功率降幅 <5%
CI / 静态检查拦截	根治（写法规约）	扫描出 N 处，CI 对违规写法构建失败
监控 / 告警复核	预防（可观测性）	人为抛异常，错误率曲线确实上涨

三类改进项各配一种主验证方式：**止血**用对账，**根治**用 CI 拦截，**系统性（预防）**用演练和监控。27.5 说"只有止血的复盘不合格"，补一句判据：**一张改进项表里如果四行全是"改代码 + 人工验证"，说明深层原因没挖到**。

> ✕ 加强代码评审、组织 Spring 事务培训、提高开发人员质量意识
> ✓ CR 模板增加「事务边界」检查项；抽查 10 个 MR，全部填写该栏
> ✓ 静态检查规则上线，@Transactional 缺 rollbackFor 时 CI 构建失败

差别就在一句：**前者是愿望，后者有动作、有样本数、有失败条件**。"抽查 10 个 MR 全部填写"能被证伪——抽到第 3 个没填就失败了，这才是可验证。

把这条判据交给机器：下面这段挂进 CI，改进项表格缺字段就构建失败。第 23 章的取证脚本、第 26 章的压测断言，到这里收束成同一件事——能被机器执行的规约，才不会在第三次事故时被人省略。

```
awk -F'|' '/^\| [0-9]+ / { n++; if ($5 ~ /^ *$/ || $6 ~ /^ *$/ || $7 ~ /^ *$/)
bad++ }
           END { printf "改进项 %d 条，缺字段 %d 条\n", n, bad; exit (bad>0) }'
review.md
```

最后把 27.6 五个反模式各抄一句"真实写法"，都出自第一份复盘那种语感，附一句改法：

反模式	一句真实写法	改成
归因到人	"张三对事务机制理解不深入"	"CR 无事务边界检查项，自调用写法能上线"
不可验证	"后续加强代码评审力度"	"抽查 10 个 MR，全部填写事务边界栏"
无 owner	"研发团队后续跟进"	"@李四，11-10 前完成"
只止血	"已修复该接口的自调用问题"	追加"全站扫描 N 处同类写法，逐个确认"
过度追责	"本次事故由当事人承担主要责任"	删除；改为"复盘不追人追流程"

27.6　五个反模式

反模式	例子	问题
归因到人	"某同学理解不深"	换个人还会犯，系统性缺陷没修
不可验证	"加强评审""提高意识"	无法判断是否完成
无 owner	"研发团队"	集体负责 = 无人负责
只止血	只改代码，不改流程	同类问题必然复发
过度追责	谁写的谁背锅	下次没人敢报故障，隐瞒比故障更贵

最后一条尤其重要：

> **复盘的目标是让系统更可靠，不是让人更小心。** 如果复盘的后果是"谁出事谁挨骂"，那么下一次的故障会被瞒到无法隐瞒为止。

27.7　一页纸模板

```
# 故障复盘：<一句话描述>

### 【必读】1. 影响
- 时间：YYYY-MM-DD HH:mm – HH:mm（共 N 分钟）
- 范围：N 个用户 / N 笔订单 / N 个接口
- 资损：N 元（已拦截 / 需赔付）
- 客诉：N 起

### 【必读】2. 时间线
| 时间 | 事件 | 当时的判断 |
|---|---|---|
| 09:04 | 监控显示 X | 判断为 Y，依据 Z |
| 09:31 | 判断被推翻，因为…… | |

### 【必读】3. 原因
- 触发因素：
- 直接原因：
- 深层原因：① ② ③

### 【必读】4. 改进项
| # | 类型 | 内容 | 验证方式 | Owner | 期限 |

### 【必读】5. 同类风险
- 全站扫描结果：
- 待处理项：
```

27.8　最后：这本书在讲什么

二十七章下来，那些最贵的故障其实只有五个母题：

一、你以为执行了，其实没执行。事务没开（第六章）、锁没生效（第八章）、切面被绕过、`@Transactional` 加在 `final` 方法上。**框架按契约工作了，只是那个契约不是你以为的那个。**

二、你以为传过去了，其实变形了。Long 变成 800（第三章）、时区差 8 小时、金额 `0.10` 变 `0.1`、Feign 丢参数。**跨边界的时候，类型系统和语义都会失效。**

三、你以为很快，其实在排队。连接池（第四章）、线程池（第九章）、行锁（第七章）、MQ 积压（第二十章）、Nginx upstream。**"慢"的绝大部分不是"算得慢"，是"等得慢"——而"等"不出现在任何下游的监控面板上。**

四、你以为处理一次，其实处理了两次（或零次）。消息重复（第十九章）、重试重复下单（第二章）、幂等缺失。**"至少一次"是异步世界的默认语义。**

五、你以为监控看得见，其实它瞎了。错误率 0 但客诉 40 单（第十一章）、`send success` 但消息丢了（第十八章）、`4096M->4096M` 的 Full GC（第二十五章）。**你的可观测性，只覆盖了你想到的那些维度。**

这五个母题的共同点：

> **系统没有报错，是你在用错误的假设读它。**

所以这本书能给你的，不是 27 个答案，是一套提问方式：

- 这个注解，真的执行了吗？我怎么证明？
- 这个值，跨过边界之后还是它吗？
- 这 3 秒，是算的还是等的？
- 这件事，会发生几次？
- 如果它错了，我多久能知道？

能问出这五个问题的人，不需要这本书。

【必读】五个母题与前 26 章的章号映射

27.8 那五个母题不是五句感慨，它们各自能落到具体章号上。把 1–26 章逐章归位，复习时按母题串，比按卷顺读更容易带走：

母题	原文点名的章	同母题的其余章
一、以为执行了，其实没执行	6（事务）、8（代理/初始化）、11（异常被吃）	3（字段没序列化）、12（索引没走）、15（缓存没更新）、21（顺序没保住）
二、以为传过去了，其实变形了	3（Long→800、金额、时区）	1（跨边界拆解）、10（可见性）、17（分片键）、22（扣减变形）
三、以为很快，其实在排队	4（连接池）、7（行锁）、9（线程池）、20（积压）、25（GC/STW）	1、2（超时重试）、5（限流）、13（慢 SQL）、14（锁等待）、26（排队论）
四、以为处理一次，其实两次或零次	2（重试重复下单）、18（消息丢失）、19（重复消费/幂等）	20（积压重试）、21（顺序重放）、22（超卖/少卖）
五、以为监控看得见，其实它瞎了	11（错误率 0 但客诉）、18（send success 但丢）、25（4096M->4096M）	23（只看到 CPU 高）、24（137 无异常栈）、26（CPU 最后才动）、27（监控盲区=深层原因）

逐章对照（主母题）：

```
1→三  2→四  3→二  4→三  5→三  6→一  7→三  8→一  9→三
10→二 11→五 12→一 13→三 14→三 15→一 16→三 17→二 18→四
19→四 20→四 21→二 22→四 23→五 24→五 25→三 26→三
```

能问出 27.8 那五个问题、并且知道该翻哪一章的人，确实不需要这本书了。但"知道翻哪一章"比"记住五个问题"多出来的，正是这张映射表。

【选读】延伸

- 知识库对应原理篇：17-常见Bug与解决方案/README